AI in Microscopy: A BioImaging Guide

Table of contents

Welcome 13

1 Preface 14
1.1 Textbook Outline . . . 14

I Getting Started with AI 17

2 AI in a nutshell 18
2.1 What is Artificial Intelligence? . . . 19
2.1.1 How deep learning learns . . . 21
2.2 Learning from examples . . . 22
2.2.1 Supervised learning: the answer is given . . . 23
2.2.2 Learning without labels . . . 24
2.2.3 Learning from consequences: reinforcement learning . . . 26
2.3 The Tasks . . . 27
2.3.1 Segmentation: turning pixels into objects . . . 27
2.3.2 Restoration: recovering what the microscope could not record . . . 28
2.3.3 Classification, detection, and tracking . . . 30
2.3.4 Generative models: sampling instead of averaging . . . 31
2.4 Why It Fails, and How you find out . . . 32
2.4.1 Overfitting and the discipline of splitting data . . . 32
2.4.2 Domain shift and shortcut learning . . . 33
2.4.3 The black box: a correct answer is not an explanation . . . 33
2.5 What comes next . . . 34
AI Disclosure . . . 35

3 Large Language Models and AI Agents 36
3.1 From Deep Learning to Foundation Models and Generative AI . . . 36
3.1.1 The Extrapolation Problem: Why Task-Specific Models Break Down . . . 37
3.1.2 The Rise of Generalist and Foundation Models . . . 40
3.1.3 The Complexity Challenge . . . 41
3.1.4 Summary: The Evolution of AI Models for Microscopy . . . 41
3.1.5 The Path Forward: From Generative AI to AI Agents . . . 43
3.2 Foundations of Large Language Models: An Intuitive Primer . . . 44
3.2.1 How LLMs Learn: Training on the World's Text . . . 44

3.2.2 The Transformer: A Revolution in Understanding Context 45
3.2.3 Fine-Tuning for Domain-Specific Tasks: An ImageJ Macro Example . . 45
3.2.4 Code Generation for Microscopy: Capabilities and Limits 46
3.3 From Text Generation to Action: Function Calling and Tool Use 46
3.3.1 How Function Calling Works 46
3.3.2 From Fragmented Tools to a Universal Protocol 47
3.3.3 From Chatbots to Agents: The Trust Escalation 48
3.4 Multimodal AI: Vision-Language Models for Microscopy 48
3.4.1 How VLMs Interpret Microscopy Images 49
3.4.2 Practical Capabilities and Current Limitations 50
3.5 AI Agents for Microscopy and Workflow Orchestration 51
3.5.1 What Are AI Agents? 51
3.5.2 How Agents Interact with the World 52
3.5.3 Use Cases for Bioimage Analysis 53
3.5.4 Concrete Examples: Omega and the BioImage.IO Chatbot 54
3.5.5 The Vision: Intent-Driven Autonomous Microscopy 55
3.5.6 Autonomous Microscopy: AI-Controlled Instruments 56
3.5.7 Coding Agents: AI with Terminal Access 57
3.6 On-Demand Software and the Changing Nature of Microscopy Tools 58
3.6.1 When On-Demand Software Makes Sense 58
3.6.2 The Reproducibility Question 59
3.6.3 A Shift in Researcher–Software Interaction 60
3.7 Practical Guide: Getting Started with AI Agents for Microscopy 60
3.7.1 Choosing and Installing a Coding Agent 61
3.7.2 Writing Effective Prompts 62
3.7.3 Walkthrough: Cell Cycle Classification with an AI Agent 62
3.8 Challenges, Implications, and the Road Ahead 64
3.8.1 The Hallucination Problem 64
3.8.2 Reproducibility and Non-Determinism 65
3.8.3 Bias and Data Governance 66
3.8.4 The Evolving Role of Bioimage Analysts 67
3.8.5 Taking Stock: What Works, What Is Emerging, What Remains Aspirational 68
3.8.6 Preparing for an AI-Enabled Future 69
3.9 Conclusion 69
AI Disclosure 70

4 Architectures and Loss Functions **71**
4.1 Neural network architectures 71
4.1.1 Linear layers (aka fully-connected layers or dense layers) 73
4.1.2 Convolutional layers 77
4.1.3 U-nets 83
4.1.4 Vision transformers 83
4.2 Loss functions 86

4.3 Training neural networks . . . 89
4.4 Additional resources . . . 92

II Image Acquisition **93**

5 Collecting Training Data **94**
5.1 Planning what to collect . . . 95
5.1.1 Defining the task and supervision . . . 95
5.1.2 Choosing representative samples . . . 96
5.1.3 Planning independent splits . . . 96
5.2 Collecting and preparing data . . . 97
5.2.1 Understanding the image measurements . . . 97
5.2.2 Checking images during acquisition . . . 97
5.2.3 Keeping data traceable . . . 98
5.2.4 Acquiring and generating targets . . . 99
5.2.5 Checking paired acquisitions . . . 100
5.2.6 Preserving data during preprocessing and storage . . . 101
5.2.7 Normalizing image intensities . . . 101
5.3 Checking dataset quality before training . . . 102
5.3.1 Inspecting images and loader outputs . . . 102
5.3.2 Running dataset-wide checks . . . 102
5.4 Dataset checklist . . . 104
AI Disclosure . . . 105

6 Image Restoration **106**
6.1 General Concepts in Image Restoration . . . 107
6.1.1 Image Degradation Model . . . 107
6.1.2 Image Restoration with Traditional Approaches . . . 108
6.1.3 Image Restoration with Deep Learning-Based Approaches . . . 111
6.2 Deep Learning-Based Techniques for Image Restoration . . . 112
6.2.1 Denoising . . . 112
6.2.2 Deconvolution . . . 114
6.2.3 Deaberration . . . 114
6.2.4 Resolution Enhancement . . . 116
6.3 Practical Guidelines for Image Restoration . . . 119
6.3.1 Considerations for Dataset Preparation . . . 119
6.3.2 Choosing a Network Architecture . . . 119
6.3.3 Training . . . 120
6.3.4 Validation and Deployment . . . 120
6.4 Limitations and Future Perspectives . . . 121
6.4.1 Caveats of AI-based image restoration . . . 121
6.4.2 Outlook for the Future . . . 122

7 Adding AI to Hardware **123**
7.1 Framing the Event Detection Landscape . . . 124
7.2 Foundations of Automated Object Detection in Smart Microscopy . . . 126
7.3 Deep Learning for Object and Event Detection . . . 132
7.3.1 Evolution of Object Detection . . . 133
7.3.2 Extending Object Detection to Events . . . 134
7.4 Building Object and Event Detection Models . . . 135
7.4.1 Ground Truth and Labeling . . . 135
7.4.2 Unique Challenges of Event Detection . . . 135
7.4.3 Training Models . . . 136
7.4.4 Refining and Optimizing Models . . . 137
7.5 Real-Time Event Detection in Smart Microscopy . . . 138
7.6 Challenges and Future Directions in Smart Microscopy . . . 144

III Image Analysis **146**

8 How to Find and Select a Tool **147**
8.1 Assessing your requirements for a tool (in a perfect world) . . . 147
8.1.1 Questions about your overall workflow . . . 149
8.1.2 Questions about your resources: time, expertise, and compute . . . 149
8.1.3 Questions about your input data . . . 150
8.2 Assessing your requirements for a tool (in the real world) . . . 152
8.3 How do I decide when my workflow is "good enough"? . . . 153
8.4 Choosing the kind of tool to use . . . 155
8.5 Finding tools and models . . . 159
8.5.1 Where to look for tools and models . . . 159
8.6 Case Studies . . . 160
8.6.1 Case Study 8.1 . . . 160
8.6.2 Case Study 8.2 . . . 164
8.7 Conclusion . . . 168

9 How to Train and Use Deep Learning Models in Microscopy **169**
9.1 Preparing for your project . . . 170
9.1.1 Defining your task and success criteria . . . 170
9.1.2 Evaluating alternatives: Is DL the right choice? . . . 171
9.2 Implementing a DL segmentation workflow . . . 173
9.2.1 Overview of a typical DL segmentation workflow . . . 173
9.2.2 Selecting a suitable DL approach . . . 174
9.2.3 Deciding whether to train a new model . . . 174
9.2.4 Preparing your dataset for training . . . 176
9.2.5 Training a segmentation model from scratch . . . 179
9.2.6 Fine-tuning pre-existing models . . . 182

9.2.7 Evaluating the performance of your model . . . 184
9.2.8 Deploying your model on new data . . . 187
9.2.9 Troubleshooting common problems . . . 187
9.3 Further considerations for DL segmentation . . . 191
9.3.1 Choosing the right tools for DL . . . 191
9.3.2 Managing computational resources . . . 192
9.3.3 Ensuring reproducibility in DL . . . 193
9.4 Summary and Outlook . . . 194

10 Output Quality **196**
10.1 Metrics and Losses . . . 196
10.2 What Metric to Pick? . . . 209
10.2.1 Cell Tracking Metrics . . . 210
10.3 Conclusion . . . 218
AI Disclosure . . . 219

11 Outlook **220**
11.1 Early successes: AI across biological imaging . . . 220
11.1.1 In situ cryo-Electron Tomography . . . 220
11.1.2 Connectomics . . . 221
11.1.3 Image restoration and virtual staining . . . 221
11.1.4 Imaging development . . . 222
11.1.5 Multimodal integration in large-scale experiments . . . 222
11.2 Why is now a good time to start with deep learning for microscopy? . . . 223
11.3 What's next? . . . 224
11.4 How to continue from here? . . . 225

Glossary **227**
A-posteriori Analysis . . . 227
AI agent . . . 227
Aberration . . . 227
Activation Function . . . 227
Activations . . . 228
Adaptive Optics . . . 228
Agentic Workflow . . . 228
Artificial intelligence . . . 228
Attribution map . . . 228
Autoencoder . . . 228
Auxiliary Variables . . . 229
Average Precision . . . 229
Backpropagation . . . 229
Balanced Dataset . . . 229
Batch . . . 229

Batch Effects . . . 230
Bayesian Optimization . . . 230
Bias Vector . . . 230
Binary Cross-Entropy . . . 230
Binary Segmentation . . . 230
Boosting algorithm . . . 231
Bounding region . . . 231
CARE (Content-aware image restoration) . . . 231
Capacity . . . 231
Centroid coordinate . . . 231
Channels . . . 231
Class . . . 232
Class label . . . 232
Classification . . . 232
Clustering . . . 232
Coding Agent . . . 232
Computer Vision . . . 232
Computing Cluster . . . 233
Confidence Score . . . 233
Context Window . . . 233
Contrastive loss . . . 233
Convolution . . . 233
Convolutional Neural Networks (CNNs) . . . 233
Coordinate transformation . . . 234
Cytoprofile . . . 234
Data Augmentation . . . 234
Data Leakage . . . 234
Data Provenance . . . 234
Data loader . . . 235
Decision tree . . . 235
Deconvolution . . . 235
Deep learning . . . 235
Denoising . . . 235
Detection . . . 235
Differentiable Function . . . 236
Dimensionality reduction . . . 236
Distance Transform . . . 236
Domain Randomization . . . 236
Domain shift . . . 236
Downsampling . . . 237
Effect Size . . . 237
Embedding Space . . . 237
Encoder-Decoder . . . 237

End-to-end learning . . . 237
Ensemble . . . 237
Epoch . . . 238
Event . . . 238
Event detection . . . 238
F1 Score . . . 238
False Negatives . . . 238
False Positives . . . 238
Feature vector . . . 239
Fiji . . . 239
Fine-Tuning . . . 239
Foundation Model . . . 239
Frequency Domain . . . 239
Function Calling . . . 239
Gaussian Process . . . 240
General Agency . . . 240
Generative Adversarial Networks (GANs) . . . 240
Generative Model . . . 240
Genetic Algorithms . . . 240
Gradient Descent . . . 241
Ground Truth . . . 241
Hallucinations . . . 241
Harmonic Mean . . . 241
Hausdorff Distance . . . 241
Human-in-the-loop . . . 242
Hyperparameters . . . 242
Image Classification . . . 242
Image Restoration . . . 242
Image Translation . . . 242
Instance Segmentation . . . 242
IoU . . . 243
Kernel . . . 243
Large Language Model (LLM) . . . 243
Learning Rate . . . 243
Learning Rate Schedule . . . 243
Linear Layer . . . 243
Localization . . . 244
Logprobs . . . 244
Long-short term memory (LSTM) . . . 244
Loss function . . . 244
Machine learning . . . 244
Manual Annotation . . . 244
Mean Squared Error . . . 245

Metadata . . . 245
Model Context Protocol (MCP) . . . 245
Model collapse . . . 245
Momentum . . . 245
Multilayer Perceptron . . . 246
N2N (Noise2Noise) . . . 246
N2S (Noise2Self) . . . 246
N2V (Noise2Void) . . . 246
NaN . . . 246
Natural Image . . . 246
Network Architecture . . . 247
Neural Network . . . 247
Neural Network Unit . . . 247
Nonlinear . . . 247
Nonlinear Problem . . . 247
Normalization . . . 247
Nyquist sampling . . . 248
OME-Zarr . . . 248
Object . . . 248
Object Detection . . . 248
Object storage . . . 248
Optimizer . . . 248
Overfitting . . . 249
Padding . . . 249
Panoptic Segmentation . . . 249
Patches . . . 249
Perceptron . . . 249
Perceptual Loss . . . 249
Pixel Classifiers . . . 250
Point Spread Function (PSF) . . . 250
Pooling Layer . . . 250
Pre-training . . . 250
Precision . . . 250
Probability Vector . . . 251
Pseudo-label . . . 251
Quality Control Metric . . . 251
RCAN (residual channel attention network) . . . 251
Random forest . . . 251
ReLU . . . 251
Real-time analysis . . . 252
Recall . . . 252
Receptive Field . . . 252
Recurrent neural networks (RNNs) . . . 252

Region of interest (ROI) . . . 252
Regularization . . . 252
Reinforcement learning . . . 253
Representation . . . 253
Representation collapse . . . 253
ResNet . . . 253
Resampling . . . 253
Residual Connection . . . 253
Resolution enhancement . . . 254
Retrieval-Augmented Generation . . . 254
Segmentation mask . . . 254
Self-attention . . . 254
Self-supervised learning . . . 254
Semantic Segmentation . . . 254
Semi-Supervised Learning . . . 255
Shallow decision trees . . . 255
Shape descriptors . . . 255
Shortcut learning . . . 255
Sigmoid Function . . . 255
Signal-to-noise ratio . . . 256
Silent error . . . 256
Single-shot detection . . . 256
Skills Atrophy . . . 256
Skip Connection . . . 256
Smart microscopy . . . 256
Spatial Domain . . . 257
Star-convex Polygon . . . 257
Stochastic Gradient Descent (SGD) . . . 257
Stride . . . 257
Structural Similarity Index (SSIM) . . . 257
Supervised learning . . . 258
Support vector machine (SVM) . . . 258
Tensor . . . 258
Test Data . . . 258
Texture . . . 258
Tokens . . . 259
Tool Definitions . . . 259
Tracking . . . 259
Training Data . . . 259
Training Target . . . 259
Transfer Learning . . . 260
Transformer Models . . . 260
Transposed Convolution . . . 260

True Negatives . . . 260
True Positives . . . 261
U-Net . . . 261
UUID . . . 261
Unsupervised learning . . . 261
Upsampling . . . 261
Validation Data . . . 261
Virtual Machine . . . 262
Vision-Language Model (VLM) . . . 262
Visual Question Answering . . . 262
Weakly Supervised Learning . . . 262
Wiener Filter . . . 262
Zernike Modes . . . 263

References **264**

Appendices **295**

A 3D-RCAN for Image Restoration **295**
A.1 Brief Introduction to the Model . . . 295
A.1.1 Residual Learning . . . 296
A.1.2 Channel Attention Mechanism . . . 296
A.1.3 3D Adaptation for Image Volumes . . . 296
A.1.4 Channel Reduction for Efficient 3D Processing . . . 296
A.2 Overview of the Code and Data . . . 297
A.2.1 System Requirements . . . 297
A.2.2 Installation of Dependencies . . . 298
A.2.3 Quick Setup Steps . . . 298
A.2.4 Dataset . . . 299
A.2.5 Training . . . 299
A.2.6 Applying the Model . . . 300
A.3 Denoising Tutorial . . . 301
A.3.1 Data Preparation . . . 301
A.3.2 Training a Denoising Model . . . 302
A.3.3 Applying the Denoising Model . . . 303
A.3.4 Further Denoising Guidance . . . 304
A.4 Deconvolution Tutorial . . . 305
A.4.1 Data Preparation . . . 305
A.4.2 Training a Deconvolution Model . . . 307
A.4.3 Applying the Deconvolution Model . . . 308
A.4.4 Further Deconvolution Guidance . . . 309

A.5 Aberration Correction Tutorial 309
A.5.1 Data Preparation 309
A.5.2 Training an Aberration Correction Model 310
A.5.3 Applying the Aberration Correction Model 311
A.5.4 Further Aberration Correction Guidance 312
A.6 Resolution Enhancement Tutorial 313
A.6.1 Data Preparation 313
A.6.2 Training a Resolution Enhancement Model 314
A.6.3 Applying the Resolution Enhancement Model 315
A.6.4 Further Resolution Enhancement Guidance 315

Welcome

Editors:
Teng-Leong Chew, Rachel M. Lee, Owen F. Puls
HHMI Janelia Research Campus

AI in Microscopy is an interactive online textbook intended to introduce biologists and microscopists to the fundamentals of artificial intelligence and machine learning.

The table of contents on the left can be used to navigate the book. Subsections of each chapter appear as a clickable menu on the right side of each chapter page. Key terms in each chapter are highlighted as links to the Glossary page. A free PDF of the textbook can be downloaded from the upper left menu, which also contains a toggle for using the website in dark mode.

The source code for this textbook is available on GitHub. All content is licensed under BSD 3-Clause, except where noted otherwise.

> **Caution**
>
> *AI in Microscopy* is currently under construction and some chapters are incomplete. This page will continue to be updated as the textbook progresses.

1 Preface

AI at Every Stage of the Microscopy Workflow

Teng-Leong Chew
HHMI Janelia Research Campus

The rapid advancement of microscopy, further fueled by parallel developments in related technologies such as probe chemistry, molecular biology, and image analysis methods, has propelled bioimaging to the forefront of life science studies [14]. In fact, many life scientists now fully expect to visualize intricate biological processes in the context of a whole, living organism — a prospect deemed impossible a mere decade ago.

This seemingly limitless possibility has greatly amplified the complexity of the questions that can be probed. Likewise, it demands a significant degree of forethought in the design of biological experiments [321, 173] to ensure that the maximal amount of biological information can be extracted from the acquired data. At the same time, the wealth of information contained in modern bioimage data can now often exceed the grasp of a human observer and thus require computational means to aid in data interpretation. In other words, the ever-expanding and multifaceted field of artificial intelligence (AI) can no longer be separated from bioimaging. Yet the absence of a comprehensive resource to systematically explore how AI can enhance bioimaging prevents many in the community from taking advantage of this indispensable technology. This textbook therefore aims to demystify AI in its many forms and outline proper use cases thereof to facilitate biological discovery. We have specifically contextualized the textbook for our two target audiences: microscopists and biologists. The scientists developing new AI and machine learning (ML) tools often fail to tailor explanations and tutorials for these users who will ultimately effectuate this discovery. We hope this textbook will bridge the gap.

1.1 Textbook Outline

As the chapters of this book will collectively demonstrate, AI now plays an indispensable role in the four main stages of the microscopy experimental workflow (Figure 1.1): Project

design, tool-building, execution of experiments, and data modeling. To address each part of this workflow, this textbook is divided into three distinct sections.

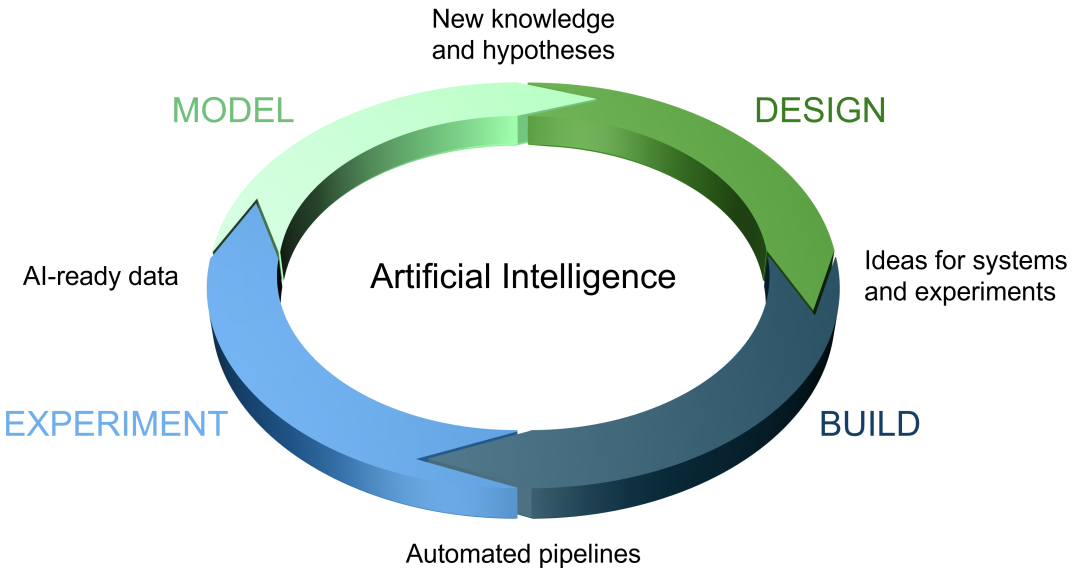


Figure 1.1: **AI impacts every aspect of the microscopy workflow.** The experimental process represents a cycle through four distinct phases: experimental design, constructing systems and workflows, testing hypotheses using these workflows, and analyzing their outputs to develop new models that inform new experiments. In the current age, AI holds the capacity to impact each phase of this cycle.

The first focuses on introducing the core concepts of AI, large language models (LLMs), and the computational architectures that underpin these technologies. Chapter 2 will introduce the basic principles behind artificial intelligence and ML as a gateway for those less familiar with this topic. In the experimental workflow, modern hypotheses can now drive a project design so complex that artificial intelligence may be required to conceptualize and build the dedicated microscopy tool needed to tackle the experimental challenge. As will be discussed in Chapter 3, this can be accomplished by using LLMs as assistants. Chapter 4 will further explain the construction of AI models.

The second section focuses on extending microscopy hardware with AI. To render a microscope "smart", i.e., for it to perform an experiment with minimal or no human intervention, ML software first needs to be trained on extensive and high-quality training data. The collection and design of training data will be covered in Chapter 5. Chapter 6 will discuss the implementation of AI and ML to enhance the capabilities of existing microscopes through resolution improvement and denoising. Likewise, AI and ML models can be designed to control event-driven image acquisition to visualize biological events that are rare and/or photosensitive (Chapter 7).

The third and final section will address the use of AI and ML to process and analyze microscopy data in order to extract biological information beyond what is achievable through human observation alone. Chapter 8 will focus on helping readers find and use available open-source tools. When those existing tools are still insufficient, Chapter 9 seeks to further widen the horizon by discussing the process of training new models for specific image processing and analysis tasks. Finally, Chapter 10 will address the question of "when is my model performing well enough?". This important step pinpoints potential weaknesses of any model, thereby placing results in meaningful context.

We conclude with Chapter 11, which summarizes the core concepts and aims to inspire readers to begin taking full advantage of the promise of the new era of AI and ML. To close the loop, we also hope that by equipping readers with the proper vocabulary, the book will facilitate more effective communication between biologists and AI tool developers.

Part I

Getting Started with AI

2 AI in a nutshell

An Introduction to Artificial Intelligence

Stephan Saalfeld
HHMI Janelia Research Campus

Empirical science relies on the analysis and interpretation of large amounts of diverse raw data. In the life sciences, microscopy has become one of the largest sources of such data. A single light-sheet experiment can produce terabytes of volumetric time-lapse data; high-content screening campaigns image millions of cells across thousands of conditions; electron microscopy reconstructs tissue at nanometer-scale resolution across volumes approaching a cubic millimeter, yielding more than a petabyte of images from a single sample [291, 310].

A human expert might trace a few hundred neurons or label several thousand cells, but biological questions increasingly demand analysis that is both orders of magnitude larger and more complete. Even where each individual decision is easy, the sheer number of decisions puts manual analysis out of reach.

Artificial Intelligence (AI) methods promise to address this challenge, and they offer three distinct things. The first is speed: where a trained annotator might outline the cells in a hundred images per day, a well-designed AI model does the same in seconds, and many copies of it can run in parallel. The second is consistency: human observers vary in their judgments and tire over long sessions, whereas a model applies the same criteria uniformly. The third is sensitivity to structure that escapes human perception. Given only transmitted-light images, for instance, a model can predict where fluorescently labeled organelles would appear, reading information from an unstained sample as if it had been stained [232].

But uniform is not the same as correct. Every AI method rests on two things: a computational structure that somebody designed and a set of examples it learned from. Choices on either side shape what the system does, and mistakes produce failures that are often distinctly non-human: systematic rather than sporadic, confident rather than hesitant, and capable of misleading a scientist in ways no tiring annotator would.

This chapter builds the vocabulary needed to make informed decisions: what terms like machine learning, deep learning, and supervised actually mean, what each family of methods asks of the user in return for what it does, and which failure modes each is prone to.

Two tasks recur here and throughout the book, because a great many microscopy applications reduce to one or the other: restoring images that the microscope could not record cleanly (Chapter 6), and segmenting them into the objects you want to measure (Chapter 9).

2.1 What is Artificial Intelligence?

The term Artificial Intelligence has been attached to systems as different as a chess engine, a spam filter, and a chatbot. For someone choosing a method, the most useful thing that separates them is where the division of labor falls: which parts of the solution a person designs, and which parts the machine derives from data. In the seventy years since the term was coined [206], that boundary has moved steadily in one direction—away from the designer and into the learned system. The three families below are stops along that path:

1. **Rule-based systems: you design everything.** The quantities to measure, the algorithm that acts on them, and every parameter value are chosen by you, and you adjust them for a new sample. This class includes conversational systems like ELIZA [336], expert systems for medical diagnosis like MYCIN [296], web search ranked by PageRank [38], the behavior of non-player characters in video games, image features designed to match views of the same scene across viewpoint changes [193], and the thresholding, morphological operations, and watershed transforms that carried quantitative microscopy for decades [97]. Nothing about such a system is mysterious: every decision follows from a known rule, so a failure can be traced to the rule that produced it and repaired by editing that rule. The price is that an expert has to anticipate every scenario in advance, as well as design and test the rules to address each and every scenario.

2. **Machine learning: you design the structure, and examples fix the parameters.** Rather than defining everything, you specify a family of functions with free parameters and let an optimization procedure choose the parameters that reproduce the expected outcome on a set of examples. Classic methods learn only the decision, on top of hand-designed features. Support vector machines [68], for example, learn to sort cells into classes from their measured size, shape, and brightness. Boosting trains a sequence of simple decision trees, each one correcting the errors its predecessors left behind [282, 88]. This boosting technique can be used to score cell phenotypes in CellProfiler Analyst (Tip 1) and, when paired with box filters that can be computed extremely fast, can make face detection real-time [320]. Random forests instead train many trees independently, each on a random subset of the examples, and let them vote [37]. They drive interactive pixel classification in tools like ilastik [25] (Chapter 8). Deep learning pushes the boundary one level deeper: the features themselves are learned, together with the decision that

uses them (Section 2.1.1). The principle was demonstrated on handwritten digits in the 1990s [170], but only in 2012, when labeled data and GPUs caught up, did it decisively outperform hand-designed pipelines on large-scale image recognition [162].

What stays on the designer's side is the architecture, the training objective, and the examples—annotated by hand, produced by a second measurement, as when one imaging modality is used to predict another [232], or supplied by the data itself, as when a model learns to reconstruct image regions that were masked out [115]. Interpretability erodes along the same step. As long as the features are engineered, you can easily ask which measurement drove a call, because the measurements have names: area, eccentricity, mean intensity in the second channel. Once the features are learned, they do not trivially decompose into something a person would recognize. In practice, you find out what such a model does by watching what it does, although a growing body of interpretability research recovers a good deal of its reasoning (Section 2.4.3).

3. **Foundation models: you use or adapt what somebody else has learned.** These are machine learning models too; what sets them apart is scale and how you use them. A foundation model is pre-trained on an enormous collection of images or text, usually for a task that needs no annotation because the data supplies its own answers: predict the next word of a sentence, or fill in a masked-out patch of an image [115]. To do this well, the model has to learn how such data are structured in general. What it retains is a general-purpose representation, not the answer to any particular question, and with enough data this representation even replaces assumptions that designers used to build into the architecture [78]. Adapting the model to a real task reuses all of this instead of rebuilding it, whether by fine-tuning on a modest set of labeled examples or by training it to follow instructions [233]. That is why a problem that would otherwise demand tens of thousands of annotations can often be solved with a few hundred. A single pre-training run serves many different tasks. Segment Anything (SAM), for example, outlines structures it was never explicitly taught: objects you point to with a few clicks or a box [158], mark once in a video to have them followed through every frame [257], or simply name in a short phrase [46].

   The catch is that a model can only absorb the structure of the data it was trained on. Because those data are typically vast, proprietary, or both, its biases and gaps are inherited unseen. This is what can make foundation models wrong in ways that are hard to anticipate. How these models are pre-trained, adapted, and potentially misapplied is the subject of Section 2.2.2 and Chapter 3.

The three examples we started with have each moved along this path. Chess engines went from the hand-written evaluation rules of Deep Blue to the learned position estimates of today's far stronger engines. Chatbots went from the scripted patterns of ELIZA to large language models. Spam filters went from keyword rules to learned word statistics and, increasingly, to large language models as well. The further along this path, the less of the solution you have to build—and the less of it you can inspect (Table 2.1).

Table 2.1: From left to right, more of the solution is derived from data, and it gets harder to find out why a system did what it did.

	Rule-based	**Machine learning**	**Foundation models**
You supply	features, algorithm, parameter values	model architecture, training objective, many examples; for classic methods also the features	a prompt, or a small set of examples for fine-tuning
Derived from data	nothing	the parameters; with deep learning, the features as well	the representation: what images or texts are like in general
Diagnosing a failure	find the rule that failed	test the model, check the training examples	observe its behavior; the pre-training data are usually out of reach

The table shows a trend, not a law: a thousand-tree ensemble is no more readable than a small network. What changes reliably is how much of a system's behavior you can anticipate without testing it. You know in advance that a threshold will break when the staining gets dimmer; a model that needed fewer examples gives no such warning. So prefer the earliest stop that solves your problem. Past that stop, trust has to be earned empirically, on ground truth the model never saw (Chapter 10). Much of what follows is about telling "works" from "mostly works".

2.1.1 How deep learning learns

A deep learning model is one large function that turns an image into the desired output: a class, a mask, a restored image. It is built by chaining many simple steps: weighted sums of pixel values, small filters slid across the image, and simple operations such as setting negative values to zero. The weights in these steps are the model's free parameters, often millions of them, and they start out random.

Training compares the model's output with the expected one. A loss function turns the difference into a single number, the error. Every step of the model is differentiable, so for each parameter one can compute how a tiny change would change the error. The chain rule of calculus, applied backwards from the error through all steps (backpropagation), does this for all parameters at once, and gradient descent nudges each one a little in the direction that lowers the error. Repeated over many examples until the error stops falling, this process turns random parameters into a working model [170]. Because the whole chain from pixels to output is trained together, the model learns its own features rather than relying on measurements a person designed.

Such models are called artificial neural networks by historical analogy: many simple units, each adding up weighted inputs and passing the result through a simple threshold-like response, wired into a network whose collective behavior is far richer than any unit's. "Deep" refers to the many layers of such units stacked on top of each other.

Differentiability is the only hard constraint, which leaves great freedom in how the steps are arranged. These architecture choices are where knowledge about the data re-enters a system that would otherwise have to learn everything from examples. Convolutional neural networks slide the same small filters across the whole image, building in the knowledge that a nucleus looks the same wherever it appears. Transformer architectures, originally developed for language, compare every part of the input with every other part, which helps when distant regions belong together—across a volume, or across the frames of a time-lapse. Chapter 4 explains architectures, loss functions, and the optimization procedures in detail.

The same freedom lets known physics be written into a model instead of learned from examples. If the point spread function of the microscope has been measured, the network does not have to rediscover how the microscope blurs: the known blur can be built into the architecture or the loss, and learning is spent only on what the optics do not explain [178, 343]. Such physics-informed models often need fewer training images, can be trained on images simulated from the optics, and produce intermediate results with a physical meaning, such as an estimate of the unblurred specimen, which makes them easier to check. Chapter 6 develops them for deconvolution and aberration correction.

2.2 Learning from examples

The promise of machine learning is that it can *infer the rules from examples*. Two conditions have to hold:

1. The model must be able to represent and express the rules.
2. The examples must be enough to pin them down.

Both fail quietly. A model that sees image patches too small to contain a whole cell cannot learn anything that depends on cell shape or size, however many patches it is shown. A model shown only healthy cells cannot learn to recognize a diseased one. And the more a system has to discover for itself, the more examples it needs: learning a decision on top of a few well-chosen measurements takes fewer cells than learning what to measure in the first place.

The approaches below differ in where the training signal comes from: a person who annotated the data, the structure of the data itself, or the consequences of an action. This determines how much annotation you have to provide and what kind of answer you get back, but not, by itself, how accurate that answer is. It is also a separate question from the division of labor in Section 2.1: a deep learning model can be trained with labels or without them, and clustering,

the classic method that needs no labels at all, works on hand-designed measurements as well as on learned ones.

2.2.1 Supervised learning: the answer is given

In supervised learning, every training image comes paired with the correct answer, the target, and the model learns to produce that answer from the image. The recipe does not care what the answer is: the same machinery classifies cells, outlines them, or predicts one channel from another. What changes is the form of the target, and in microscopy image analysis, that form varies a lot:

- **A category** for an entire image or for one cell: *treated* or *control*, *mitotic* or *interphase*.
- **A position:** a point per cell, or a box around it, saying where things are without saying where they end.
- **A class for every pixel**—nucleus, cytoplasm, background—which is what semantic segmentation asks for [265].
- **An identity for every object**, so that two touching nuclei are two things rather than one region. This is instance segmentation, and it is what counting and per-cell measurements need.
- **Another image:** a low-noise acquisition of the same field as the target for a noisy one [334], or a fluorescence channel as the target for the transmitted-light image it is predicted from [232].

The last case is still supervision, even though nobody annotated anything: the target came from a second measurement (Section 2.3.2).

The form of the target is the form of the answer you get back: a model trained on one class per image will never hand you cell boundaries, however many images it has seen. The target does not dictate how the model gets there, though. Instance segmentation in particular is rarely predicted directly: most methods predict something for every pixel, such as cell boundaries or the direction to each cell's center, and assemble objects from it in a second step (Section 2.3.1).

Supervision has the virtue of directness. What you want the model to do is what the loss function measures, so accuracy can be quantified on labeled examples the model never saw, and a failure can at least be named: these nuclei were merged, those were missed. None of the label-free methods below offer that.

The cost is that the targets have to be made, usually by hand, and that they are not merely training material: they *are* the definition of the task. Systematic annotator habits become model behavior: if your annotators consistently skip dim nuclei at the edge of the field, so will the model, silently and everywhere. Where two experts disagree, the model learns the average of a disagreement rather than a fact, which sets a ceiling on any accuracy you can later claim. Curating such a set is the subject of Chapter 5.

How much annotation is involved varies over orders of magnitude, and two rules of thumb cover most of it. Adapting a pre-trained model to your data takes far fewer labels than training one from scratch (Chapter 9). And label shapes differ enormously in price: one class per image or one click per cell is quick; a dense mask of a crowded field is the most expensive annotation in routine use. When a count is all you need, buy the label shape that gives you counts.

2.2.2 Learning without labels

Two families of methods need no annotation at all. Without a person supplying the right answer, both scale to as much data as a microscope can generate, but neither can tell you whether what it found means anything biologically.

Unsupervised learning is the older and stricter of the two: it receives the images, or measurements derived from them, and nothing else. It optimizes a property of the data itself rather than agreement with a target: how compactly the data can be grouped, or how well a compressed version reproduces the original. Dimensionality reduction condenses the hundreds of numbers measured per cell into two or three, so that a whole screen fits into a scatter plot [207]. Clustering sorts the cells into groups without being told what the groups mean. Together they are the standard tools of morphological profiling, which measures many features on every cell in every well and looks for structure in the result [36]. Compounds with indistinguishable profiles then become candidates for a shared mechanism.

The limitation is easy to state and easy to forget: the algorithm separates points in the feature space it was given, and nothing guarantees that this space aligns with a biological category. Change the features and the groups change with them. The assumptions have moved from the labels into the features. A cluster is a hypothesis about your data; it becomes a finding only when independent evidence confirms it.

Self-supervised learning goes one step further: it turns unlabeled data into a supervised problem. The usual recipe is to withhold part of the data and train the model to restore it, for example masked patches of an image [115] or masked words in a sentence [76]. Hiding single pixels of a noisy image and predicting them from their neighbors trains a denoiser that has never seen a clean image [163]. Movies are so redundant that a model still learns well when it has to fill in 90% or more of each clip [313].

The task sounds circular, but it cannot be solved by simple statistics. Filling a large hole convincingly requires some idea of what put the pixels there: that a membrane continues where it is hidden, that nuclei have characteristic shapes and textures, that illumination varies smoothly across the field. A network trained this way at sufficient scale comes to represent much of the structure that generated its images, without ever being told what that structure is.

A self-supervised denoiser is trained for its output: the restored pixel is the product, and the representation is only a means to it (Chapter 6). In pre-training it is the other way

around. The reconstruction is discarded, and the representation is what you keep. This also separates self-supervision from clustering: clustering hands you groups to interpret, whereas a representation is not yet an answer to anything. A second stage puts it to work. Fine-tuning on a few hundred annotated images can turn a generic visual representation into a segmentation model for your specimen, and semi-supervised training adds a large pool of unlabeled images to a small labeled set. Neither repeats the expensive pre-training. On ImageNet, a benchmark of more than a million labeled photographs, pre-training without labels and fine-tuning with 1% of them comes close to a network trained on all labels. That is a tenfold gain in label efficiency over earlier approaches [61].

Not every general-purpose model got there without labels. The original Segment Anything model was trained on roughly a billion masks in eleven million images, masks that annotators produced with the help of the model itself. A dedicated component reads the clicks, boxes, or rough masks you give it [158]. The model segments structures it was never explicitly taught, but only in the ways its designers built in. Either way, the training data sets the limit: a model absorbs the structure of the images it was shown, with their biases and artifacts, not the structure of the world. Versions fine-tuned on microscopy data exist [8, 238], but whether they help on your images has to be tested. Chapter 9 covers how to fine-tune, and Chapter 10 how to tell whether it worked.

Other recipes for self-supervision

Filling in masked-out data are only the most direct recipe. A second family learns embeddings that are insensitive to irrelevant variation but expressive enough to distinguishing meaningful classes. A contrastive loss pulls together the embeddings of two altered versions of the same image and pushes apart those of different images [60]; other methods only ask the network to agree with a slowly updated copy of itself [47]. Text models learn embeddings too, so that passages about the same topic land close together [76, 354].

The result can be more structured than the training task suggests: the image regions such a network attends to trace the outlines of objects, although nothing in the training asked for a segmentation [47]. Trained on large, carefully curated image collections, these networks learn features general enough that many tasks need only a simple classifier on top [230, 298].

A third recipe hides part of the input, as in reconstruction, but asks the network to predict the representation of the hidden part rather than its pixels, because fine detail is largely unpredictable and not worth reproducing [17].

Recipes that only reward agreement share a trap: describing every image the same way satisfies them perfectly and learns nothing. Most methods guard against this collapse with an assortment of engineering tricks; recent work replaces them with the single requirement that the embeddings be spread out evenly in all directions [15].

Clustering or segmentation? Let the question decide

Take one compound screen: a few thousand fields, three channels, a few hundred conditions. Two questions a biologist might reasonably ask of it need entirely different methods.

How many cells are dying in each condition? The category already exists—you can say what a dying cell looks like—and the answer is a number per condition, so every cell has to be found and judged: supervised detection or segmentation, then supervised classification. You provide annotations, and what you get back is a measurement, a count you can put an error bar on, because annotated images the model never saw tell you how many cells it missed or invented.

Are there phenotypes in this screen I have not thought of? Here there is nothing to supervise, because naming the classes is the result you are after. So you measure many features per cell and group the cells, or whole conditions, whose measurements look alike. Without annotation, you get a hypothesis: a set of conditions that behave similarly, which needs an independent experiment before it counts as a finding.

The rule generalizes. Supervised methods answer questions you can already phrase; label-free methods help you find the question. Many projects run the second kind of analysis once, to decide what is worth counting, and the first kind forever after.

The two also combine more often than they compete: the features are usually measured on cells that a segmentation model found first, so an exploratory analysis inherits every bias of that model—a reason to check the segmentation even when the result is only a hypothesis.

2.2.3 Learning from consequences: reinforcement learning

A third approach needs no examples at all. In reinforcement learning, an agent acts, the environment responds, and a *reward* scores how well that went; the agent adjusts its behavior to collect more reward, generating its own training data as it goes [214]. The reward can be sparse and arrive late—a game won twenty moves after the decisive one—which is what makes the approach both powerful and expensive.

Image analysis gives it little to work with, because there is no action to take on an image that has already been recorded: the pixels do not respond. Microscopy does offer one natural fit, and it is at the instrument. In closed-loop acquisition the model chooses where to look next, at what magnification, and for how long, and those choices have consequences that can be scored—an event captured or missed, a sample bleached or spared. Chapter 7 takes up event-driven imaging.

The term will also be familiar from large language models, which use reinforcement learning from human feedback. This is the stage in which a language model is tuned toward responses that people rated highly [233]. The mechanics are the same, with a learned stand-in for human preference supplying the reward (Chapter 3).

2.3 The Tasks

The approaches above sort methods by where their training signal comes from; the tasks below sort them by what comes out.

2.3.1 Segmentation: turning pixels into objects

Segmentation converts an image into a list of things you can measure. Counts, sizes, per-cell intensities, colocalization, morphology—almost every quantitative claim made from a microscopy image passes through it, which is why the rest of this book keeps returning to it.

The literature uses three terms, not always consistently. Semantic segmentation assigns each pixel a class without separating objects of the same class, instance segmentation gives each object its own identity (Section 2.2.1), and panoptic segmentation does both at once. Tip 6 works the distinction through on a real image.

What makes an annotation expensive is not the number of pixels but how precisely the shape has to be drawn. Semantic segmentation is forgiving: a pixel classifier can be trained from a few brush strokes per class on a handful of images, as interactive tools such as ilastik have done since before deep learning [25]. Instance segmentation is not, because every object needs its own identity, and most methods need each boundary drawn accurately—slow, careful work rather than a lot of clicking.

How much of that care is genuinely necessary depends on the method rather than the task. A network that predicts an embedding per pixel needs the pixels of one object to agree, not its boundary to be right, so in principle sparse labels should suffice, even though published implementations often use dense ones [167, 95]. A network that only has to place a box needs no outlines at all, which is why annotating for counting is cheap. What cannot be negotiated is the number of dimensions: a 3D annotation has to stay consistent from section to section, so it is one decision carried through a volume rather than one drawing per slice, and adding time compounds it again.

How a segmentation is produced is rarely what the label suggests. Object identity has no canonical output: which integer a given cell receives is arbitrary, and swapping two of them changes nothing about the answer, so there is nothing stable for a per-pixel classifier to predict. Almost every method therefore predicts a quantity that *is* well defined per pixel and assembles objects from it in a second step.

> A deeper understanding of what segmentation methods actually predict
>
> Methods differ mainly in what the network is asked to output per pixel, and each choice implies its own way of recovering objects.
> **Boundaries and affinities.** The network predicts, for each pair of neighboring pixels,

whether they belong to the same object [315]. Objects are then grown by watershed, or by agglomerative merging. Including pairs that are farther apart than one pixel helps detect thin gaps and provides evidence that two pixels belong to *different* objects; the mutex watershed uses that evidence to separate objects without seed points or a threshold [89, 339].

A description of the shape the pixel belongs to. StarDist predicts the distance from each pixel to the object's edge in a fixed set of directions and keeps the best non-overlapping polygons [287]. Local shape descriptors predict summary statistics of the surrounding object—its size, elongation, and offset to its center—as an additional training target that markedly improves the affinities predicted alongside [293]. PatchPerPix goes furthest and predicts an entire small binary mask of the object around each pixel, then assembles objects from the patches that agree [197].

A direction toward the object's center. Cellpose predicts, for every pixel, the direction toward the center of its cell. Following these directions makes the pixels of one cell converge on a single point, and those that arrive together become an object [305]. Nothing here assumes a shape, which is why it copes with irregular and elongated cells.

An embedding per pixel. The network learns a vector that should be identical for pixels of the same object and different across objects, and the objects fall out by clustering [167, 95].

One object at a time. Flood-filling networks start from a seed and iteratively extend a single mask until it stops growing, then move on to the next seed [142]. This sidesteps the identity problem rather than solving it, and it has proven itself where objects are enormous and branching, as in reconstructing neurons from electron microscopy.

The grouping step deserves as much attention as the network. It is where most false splits and false merges are made, and where most of the parameters a user still has to set actually live. It is also the knob that trades the two errors against each other, and which one you want depends on the question. A false split, or one cell cut in two, is usually easier to see and quicker to repair than a false merge, especially in objects with complicated shapes. Moreover, a merge quietly averages two cells into one measurement.

Chapter 4 covers the architectures behind these predictions, Chapter 8 how to find a tool that already makes them, Chapter 9 how to train or fine-tune one on your own data, and Chapter 10 how to tell whether it worked.

2.3.2 Restoration: recovering what the microscope could not record

Every image is a degraded observation. The microscope blurs the specimen by its point spread function, the detector records a finite number of photons and so returns a noisy count, and the pixel grid is often coarser than the finest detail you would like to resolve. Restoration estimates the underlying signal from what was recorded. This is an ill-posed inverse problem, because many different specimens are consistent with the same recorded image.

Supervised restoration fits the template of Section 2.2.1 exactly, with an image as the target instead of a label. The kind of target image defines the task: a high-signal acquisition of the same field for denoising, a sharper one for deconvolution and aberration correction, a higher-resolution or differently acquired one for resolution enhancement. Chapter 6 develops all of these. Restoration can also use a different channel entirely as a target for modality translation, where a fluorescence signal is predicted from transmitted light [232].

The reason to care is not that the images look better. Noise, blur, and limited resolution are all paid for in photons, and photons are what a living sample cannot spare. A denoiser that recovers a usable image from a tenth of the illumination converts directly into a tenth of the phototoxicity, or ten times the frame rate, or a time-lapse that runs ten times longer before the specimen gives out. Restoration is an acquisition technique that happens to run after acquisition.

One caveat matters enormously: restoration is not measurement. Wherever the recorded data carry too little information, the estimate comes from the training set rather than from the specimen. Section 2.3.4 names the mechanism and Chapter 6 collects the consequences.

Denoising often does not need a clean reference: self-supervision (Section 2.2.2) can train a denoiser on noisy images alone.

Denoising without a clean image

Deep-learning-based denoising methods rest on one asymmetry: the signal in an image is predictable from its surroundings, and the noise, by definition, is not. Put a network in a position where it can only reproduce the predictable part, and it denoises.

The direct approach needs a pair of images of the same field, one noisy and one with high signal, and learns the map between them [334]. Acquiring that pair is easy on a fixed sample and often impossible on a living one, which moves and changes between exposures.

Noise2Noise drops the need for a high-signal image. Two independent noisy exposures of the same field are enough, because the network cannot predict the particular noise in the target from the noise in the input; the best it can do is the signal the two share [174].

Noise2Void drops the need for a pair. A single noisy image suffices: hide one pixel, ask the network to predict it from its neighbors, repeat [163]. The network never sees the value it is trying to reproduce, so it cannot copy the noise. What it does require is that the noise be independent between pixels—true of photon shot noise, false of line artifacts such as scanning stripes, where the fix is to hide whole lines instead of single pixels [39].

A noisy image is consistent with many clean ones, and a network trained to minimize the average squared error returns their blend. Diversity denoising instead learns the distribution of clean images consistent with the input and draws samples from it, making the ambiguity visible rather than smoothing it away [245]. It is a generative model in the sense of Section 2.3.4.

Similar networks can also undo a deliberate mix. If two labeled structures are imaged

into a single channel, a network trained on separately acquired examples of each can pull them apart again, so fewer channels have to be acquired in the first place [11].
One assumption is shared by all of them: whatever cannot be predicted is treated as noise. Structure that is genuinely rare and genuinely unpredictable is therefore exactly what they are most likely to erase.
Chapter 6 discusses these approaches in more detail.

2.3.3 Classification, detection, and tracking

Classification attaches one label to an image, or to an object already found in it: cell type, cell cycle stage, treated or control, healthy or diseased. Image-level labels are the cheapest annotation there is, which makes classification a quick way to find out whether a signal is present in your data at all.

Detection locates each object by a point or a box, but not its exact outline. That annotation is dramatically simpler than a detailed boundary mask, and the output is sufficient whenever the answer is a count, a position, or a trigger to act on (Chapter 7).

Tracking links objects across frames, which is where much of the interesting biology is—division, migration, differentiation, events that a single timepoint cannot show. It combines two problems: finding the objects in each frame, which is segmentation or detection as above, and association, deciding which object in this frame is which object in the next.

Cell division makes it harder than plain matching, because one object legitimately becomes two, so the answer is a lineage tree rather than a set of paths—and a dividing cell rounds up and changes appearance precisely when the association is most delicate. Association errors also compound, since a single swap corrupts every measurement downstream of it in that lineage. A method can therefore score well on the fraction of correct links and still recover almost no complete tracks [204], which is why tracking has no single accuracy number (Chapter 10).

How tracking is actually done

Most trackers in use separate the two problems: segment or detect each frame independently, then link. The separation is convenient, because either part can be replaced independently. It is also the structure that most standard benchmarks assume [204].
The classical linker has not been displaced. Linear assignment solves frame-to-frame matching as an optimal assignment problem, then makes a second global pass that stitches track fragments together and repairs gaps, merges, and splits [143]. This is the main engine inside TrackMate, the tool most microscopists will actually open, which now accepts modern segmentation models as its detector [82]. An important finding from a decade of benchmarking is that segmentation quality dominates: a simple linker on good detections beats a sophisticated linker on poor ones [204].

Deep learning has changed the linker in three ways. It can learn the association itself, with a transformer scoring candidate matches from position and appearance across a window of frames, which works across bacteria, cell culture, and particles without per-dataset tuning [90]. It can fold detection and linking into one network, predicting for each pixel the offset to where its object was in the previous frame [191]. Or it can supply many candidate segmentations, from several models and parameter settings, and leave it to temporal consistency to choose among them, which is what makes terabyte-scale embryo recordings tractable [35]. Video foundation models are an obvious candidate as well: SAM 2 carries a mask through a video without any retraining [257], and microscopy time-lapses are videos too.
What remains hard is well documented. Pre-trained trackers generalize poorly to morphologies and modalities they have not seen; densely annotated, fully tracked datasets are scarce, which limits what more supervision can fix; and recovery of complete lineages lags well behind the local link-accuracy scores that are usually reported [204].

Rare-event detection uses these tasks to find the one cell in ten thousand that does something interesting. A model can search vast amounts of recorded data for such events, and if it runs fast enough, it can steer the microscope during acquisition: record the event in detail while it happens, and spare the sample and the storage during the hours in which nothing happens (Chapter 7). The two kinds of error weigh very differently. A false negative is a lost event, and when events are this rare, every one counts. A false positive costs less, but even a low rate adds up: one false alarm per thousand cells buries each real event under ten false ones. That is acceptable, because the detector does not have to be the last word. Tuned to miss as little as possible, it shrinks the pile enough that a slower, more accurate method, or a person, can sort through what remains.

2.3.4 Generative models: sampling instead of averaging

A model trained to minimize the average squared error returns the mean of all outcomes consistent with its input. Where the input is ambiguous, that mean is a blend, which is why strongly denoised images can look slightly waxy. A generative model is instead trained to draw *samples* from the distribution of possible outcomes. Each sample carries the texture and edge contrast of real data, and running the model twice on the same input yields two different answers. No single sample represents the whole distribution, though: it is one plausible answer, not the best estimate.

Four families are in current use. Variational autoencoders compress an image into a small code and decode it back; because the code is a distribution rather than a single point, sampling it yields different plausible reconstructions [157]. Adversarial training pits the generator against a second network that tries to tell generated images from real ones [98]. Diffusion models learn to reverse a gradual corruption by noise, and generate by denoising pure noise in many small

steps [125]. Flow matching learns a more direct path from noise to data, which takes fewer steps [183].

The risk follows from what sampling means: a generative model is built to produce output indistinguishable from real data, whether or not the input supports the particular detail it draws—a vesicle that is sharp, plausible, correctly textured, and never measured. This inverts the usual relation between image quality and trust: the restored image is *more* convincing than the raw data, so judging it by eye selects for exactly the failures you need to catch [173]. A structure that appears only after restoration must never be quantified.

2.4 Why It Fails, and How you find out

2.4.1 Overfitting and the discipline of splitting data

It is easy to say what kind of label, field of view, or resolution a task requires, and much harder to know whether a set of examples is *sufficient*, because nobody knows in advance which rules the model has to infer. Worse, the model may find a shortcut that works perfectly on the training examples and fails on the intended task. The only way to find out is to measure performance on a separate test dataset that was never used for training.

There is a second problem: gradient descent keeps driving the training loss lower, eventually by memorizing the training examples rather than learning rules that generalize. Because no training set fully represents the real world, this overfitting sets in if training runs long enough and the model sees each example too often. So when do you stop?

The answer is a validation dataset: examples withheld from training but used to monitor performance along the way. When performance on them stops improving, training stops.

The validation set must stay separate from the test set. Validation examples are not used for gradient descent, but they still shape the final model: you pick the training iteration, architecture, or hyperparameters that do best on them, so the model can overfit to the validation set too. Only the test set, used once at the very end, gives an unbiased estimate of performance on new data from the same source.

Both splits only work if the withheld examples are genuinely separate. Neighboring patches from the same stack are nearly identical, so splitting patches at random scatters near-copies across all three sets, and the score that comes back measures memory rather than performance. Chapter 5 covers how to build and split a dataset in practice, Chapter 10 what to measure on it.

2.4.2 Domain shift and shortcut learning

A model can pass both checks and still do worse on images acquired a few months later. Training and test data came from the same experiment; the new images did not. This is domain shift. The causes are mundane: a different objective, a realigned laser, a new batch of mounting medium, the same cell line at a higher passage number. The result is rarely outright failure but a slow degradation: slightly worse masks, a few more missed cells, a shifted effect size.

Microscopy is more exposed to this than most of computer vision. A model trained on hundreds of millions of photographs has already seen most of the variation it will meet. A microscopy model often knows only one laboratory's protocol on one instrument, and that protocol changes routinely, sometimes without anyone noticing.

The same narrowness allows a second failure. One experiment's images cannot separate the regularities of the biology from those of the protocol. If something incidental correlates with the label, gradient descent will use it, because it optimizes the loss and not the biology. This is shortcut learning [91]. The outer wells of a plate lose more medium to evaporation, and their cells look different [43], so a classifier whose controls sit in column 1 may be reading well position rather than phenotype. Shortcuts need not come from the sample at all. Training crops that are mostly background teach a segmentation model to predict background everywhere, which minimizes the loss and finds nothing (Section 8.6.2).

Neither failure shows up as a poor validation score, because the validation data share the protocol, and therefore the confounds, of the training data. For example, in one case, it was easier to train a model to identify the hospital and department that acquired an image than one that would successfully classify pneumonia in images from other hospitals [349].

Split at the level of whatever will vary in practice: whole plates, whole sessions, whole cell lines, not random crops from a common pool. That does not prevent domain shift, but it makes it visible before you rely on the model. The rest belongs elsewhere: a training set that spans the variation you expect, and augmentation to extend it (Chapter 5); fine-tuning when a published model does not transfer (Chapter 9); and appropriate measures of success (Chapter 10). A reported accuracy describes the data it was measured on, and says little about yours [165].

2.4.3 The black box: a correct answer is not an explanation

Nothing inside a trained network is hidden: every weight and every intermediate value can be printed. But the resulting features do not trivially decompose into interpretable quantities (Section 2.1). That is what the black box is: not secrecy, but the absence of any description of the model short enough to reason with.

This has two important consequences. The first is that behavior cannot be predicted by inspection. For a rule-based pipeline, you can read off where it will fail, because the rules are written down; for a network, the failures of Section 2.4.1 and Section 2.4.2 can only be found

by measurement. The second is that a correct prediction carries no reason with it. A classifier may separate treated cells from controls on every plate for something other than the phenotype, and no held-out test will reveal it, because the prediction is not wrong. That the network distinguishes the conditions is a measurement; that the treatment changes mitochondrial morphology is a claim the network never made.

The confidence score the model reports is not a check on this. It comes out of the same computation as the prediction, and current networks routinely report confidence above their measured accuracy [104], and most severely under domain shift, exactly when a warning would be most useful [237]. Whether a given score can be trusted is itself measurable (Chapter 10).

What remains is evaluation on data like yours, and interpretability methods that have themselves been tested. A common tool is an attribution map, a per-pixel image of how much each part of the input contributed to the output; several widely used variants produce nearly the same map after the network's weights are randomized, so they were never reporting on the trained model at all [3]. Methods that pass such checks can deliver. A network trained on electron micrographs identifies which of six neurotransmitters a *Drosophila* synapse releases in 87% of cases. Ordinary attribution maps did not reveal what it relied on, but comparing each synapse with a generated counterfactual, the same synapse made to look like another transmitter type, isolated ultrastructural differences a person could name and nobody had described before [79].

A network nobody can explain can still be a good instrument, provided it has been characterized like one.

2.5 What comes next

A model can be put to two uses. In *measurement*, you already know what to count and the model counts it. It is part of the instrument, and its output is a number in your results [321]. In *exploration*, you do not yet know what to look for, and the output is a suggestion about what to measure next. The two are not separate methods: the same segmentation model can count the phenotype you asked for and then reveal one you did not. What matters is saying which you were doing.

Using a model as an instrument means characterizing it like one: an accuracy, the metric it was computed with, and the data it was measured on. The last is the easiest to leave out and the most important (Section 2.4.2). The model, its version, whether you fine-tuned it and on what, and the score on held-out data belong in the methods, as specifically as an objective or an exposure time.

The rest of the book takes up what this chapter has only named. Chapter 3 covers the models whose output is language, Chapter 4 architectures and loss functions, and Chapter 5 the examples everything depends on. Chapter 6 treats restoration in depth, and Chapter 7 models that act on the specimen instead of reporting on it. Chapter 8 helps you find an existing tool,

Chapter 9 shows how to train or fine-tune one, Chapter 10 how to measure whether it worked, and Chapter 11 where exploration is already producing new biology.

AI Disclosure

AI language models and agents (Anthropic's Claude Code using Sonnet 5.5, Opus 4.6 and 5.5, and Fable 5.1) were used for the preparation of this chapter. They were used to brainstorm, discuss, draft, and edit prose, to check consistency across sections, glossaries, and other chapters. All content was reviewed, revised, and verified by the author, who takes full responsibility for the accuracy and scientific integrity of the final text. Selection and curation of references was done by the author.

3 Large Language Models and AI Agents

From Deep Learning to Autonomous AI-Powered Bioimage Analysis

Wei Ouyang & Hanzhao Zhang
SciLifeLab | KTH Royal Institute of Technology

Over the past decade, the way we analyze microscopy data has changed markedly. What began with specialized deep learning models for individual tasks — segmentation, denoising, classification — has evolved into a broader landscape where artificial intelligence (AI) systems can assist with experiment planning, code generation, instrument control, and workflow adaptation. This chapter traces that evolution and equips microscopists with the knowledge to navigate it.

We begin by examining the limitations of traditional deep learning and the rise of foundation models that generalize across imaging modalities. We then introduce Large Language Models and explain how they work at an intuitive level (Section 3.2), before exploring how function calling transformed them from text generators into action-taking agents (Section 3.3). The chapter covers vision-language models for microscopy (Section 3.4) and the growing ecosystem of AI agents that can orchestrate complex workflows, control instruments, and generate software on demand (Section 3.5, Section 3.6). A hands-on practical guide (Section 3.7) provides concrete workflows for getting started, followed by an honest discussion of challenges, implications for the profession, and an outlook on where the field is headed (Section 3.8).

Throughout, we maintain a critical perspective: these tools offer genuine capabilities, but they also hallucinate, carry biases, and cannot replace the domain expertise of trained microscopists. Understanding both the power and the limitations is essential for responsible adoption.

3.1 From Deep Learning to Foundation Models and Generative AI

The application of AI to microscopy has changed rapidly. The modern era began with convolutional neural networks (CNNs), which improved how we process biological images. These supervised models were trained to perform specific tasks: classifying images, segmenting structures, denoising acquisitions, or translating between imaging modalities. Architectures like

ResNet (for image classification) and U-Net (for segmentation and image-to-image translation) became foundational not only in bioimaging but across computer vision more broadly. See Chapter 4 for a detailed discussion of these architectures.

3.1.1 The Extrapolation Problem: Why Task-Specific Models Break Down

These early deep learning approaches showed impressive capabilities within their training domains, but they carry a fundamental and often underappreciated limitation: neural networks are sophisticated pattern-matching systems that work reliably only when processing inputs similar to their training data (see Chapter 5). They are poor at extrapolation. A network trained to segment fluorescent nuclei in one cell line, imaged on one microscope, at one magnification, may fail — sometimes silently — when applied to a different cell type, a different microscope, or even a different staining protocol. This brittleness has been documented systematically: standard neural networks fail at tasks as simple as arithmetic when tested outside their training range [314], and the problem compounds for the complex, high-dimensional data typical of microscopy.

To illustrate this, we conducted a simple experiment (see the Python code) where we trained a multi-layer perceptron to learn addition using numbers in the range [-1,1]. When tested on progressively larger ranges, the model's performance degrades severely — with errors increasing by approximately 7,700× from the training range (Figure 3.1).

Figure 3.2 demonstrates the same problem with direct relevance to microscopy. When we apply a standard ResNet50 model trained on ImageNet to a natural photograph, it correctly identifies a cat with reasonable confidence. But when presented with a fluorescence microscopy image showing TRAP1 protein localization in human cells, the same model produces confident yet completely irrelevant predictions — "jellyfish" (36.2%), "velvet" (13.1%), "coral reef" (8.3%). The model has never encountered microscopy data, so it maps unfamiliar visual patterns onto the closest categories it knows.

A researcher unfamiliar with the model's training background might trust these confident predictions, potentially misinterpreting biological data. The 36.2% confidence in "jellyfish" illustrates a general failure mode: neural networks can produce authoritative-sounding but scientifically meaningless or misleading outputs when applied outside their intended domain. Stretching these models beyond their training distribution can result in hallucinated objects, irrelevant labels assigned with deceptively high confidence, or artifacts that mislead biological interpretation.

This does not mean that ImageNet-pretrained models are useless for microscopy — in practice, most deep learning approaches in bioimaging start from ImageNet-pretrained weights and then fine-tune on domain-specific data. But even after fine-tuning, the base model retains learned features from its original training. When the model encounters microscopy images with visual patterns that happen to resemble natural objects — jellyfish-like fluorescent textures, coral-like

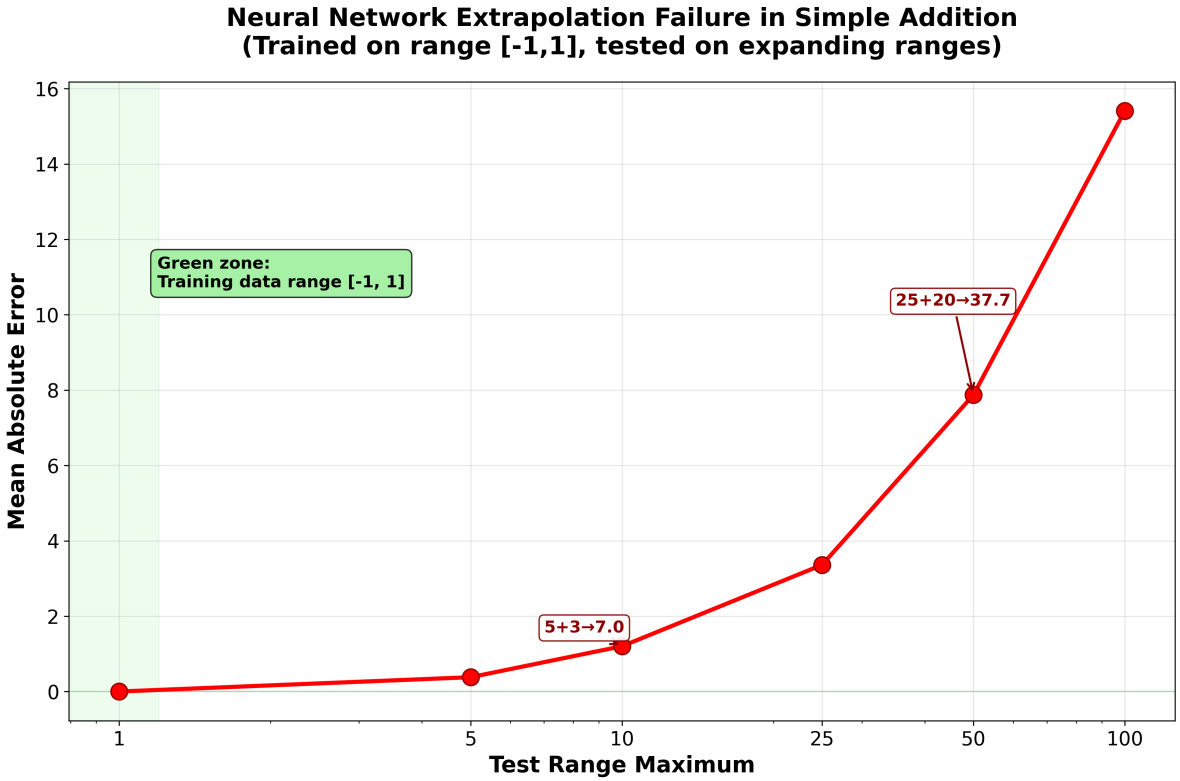


Figure 3.1: Neural network extrapolation failure in simple addition. A standard MLP trained on the range [-1,1] shows severe performance degradation on larger number ranges, despite addition being a simple operation that should generalize perfectly. This illustrates the core limitation of neural networks as pattern-matching systems that fail outside their training distribution.

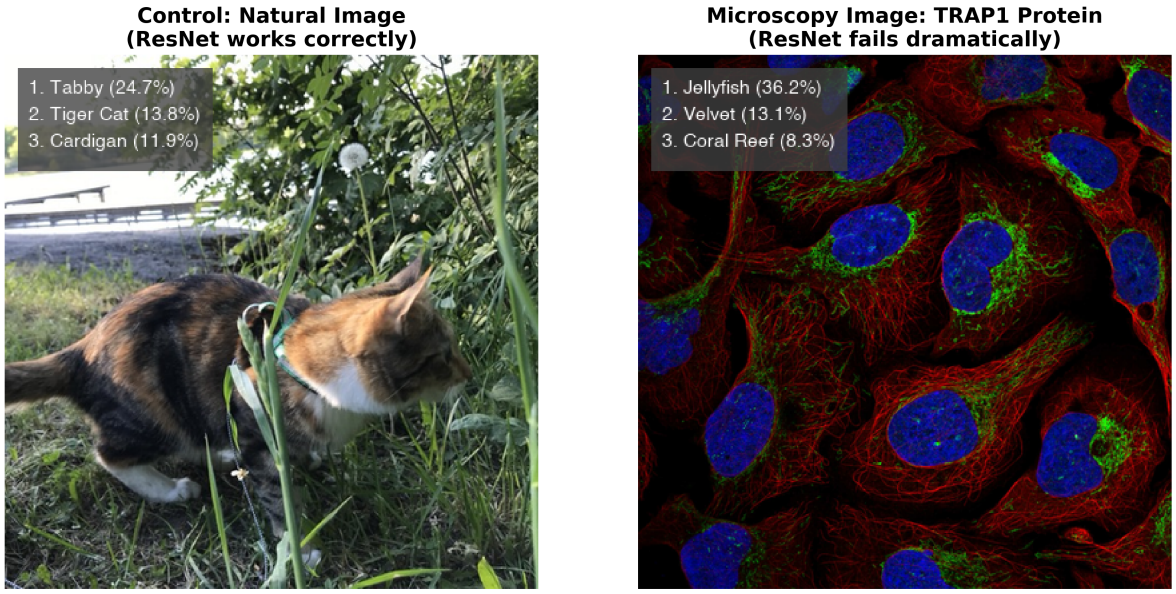


Figure 3.2: Domain mismatch between natural and microscopy images. A ResNet50 model trained on ImageNet correctly classifies a cat photograph (left) but produces confident, irrelevant predictions for a TRAP1 protein immunofluorescence image (right). Image credit: Human Protein Atlas (https://www.proteinatlas.org/ENSG00000126602-TRAP1).

tissue structures — these embedded representations can influence predictions, potentially introducing systematic biases.

3.1.2 The Rise of Generalist and Foundation Models

A major trend in bioimage analysis has been the shift from narrow, task-specific models toward **generalist** and foundation models that work across diverse imaging conditions without retraining.

Cellpose pioneered this approach for cell segmentation. By training on a diverse dataset spanning fluorescence, brightfield, and even non-microscopy images with similar visual characteristics, Cellpose achieved strong generalization across cell types and imaging modalities that previous models required separate training for. Cellpose 3 [304] further advanced this by introducing one-click image restoration that enables segmentation even of degraded images — noisy, blurry, or undersampled data that would defeat earlier models.

Segment Anything Model (SAM), released by Meta in 2023, advanced the field of promptable segmentation further. Trained on over one billion masks from 11 million images, SAM demonstrated that a single model could segment essentially any object in any image through simple prompts (points, boxes, or text). Despite being trained entirely on natural images, SAM showed unexpected capability on out-of-domain scientific images, sparking a wave of microscopy-specific adaptations:

- **micro-SAM** [8] fine-tuned SAM specifically for light and electron microscopy, extending it to support 3D segmentation and cell tracking. micro-SAM provides interactive segmentation tools through a napari plugin, making foundation model capabilities directly accessible to microscopists.
- **CellSAM** [202] combined SAM with a custom cell detection module (CellFinder) to achieve expert-level segmentation performance across mammalian cells, yeast, and bacteria — all with a single unified model.
- **Cellpose-SAM** [238] adapted SAM's pretrained transformer backbone to the Cellpose framework, achieving what the authors describe as superhuman generalization for cellular segmentation across diverse imaging conditions.

The key insight is that models trained on vast, diverse datasets can learn transferable visual features that generalize well to microscopy. A model that has processed billions of natural images learns representations of edges, textures, boundaries, and object structure that transfer effectively, even to fluorescent cells it has never encountered. This is the same principle underlying transfer learning, but operating at a much larger scale. See Section 9.2.6 for more information.

While these generalist models have made deep learning safer and more accessible, they remain largely focused on segmentation — a relatively well-defined computational task. More complex microscopy challenges such as classification across diverse experimental conditions, image

restoration, spectral unmixing, or artificial labeling still require task-specific approaches. The gap between what generalist models can do and what microscopy research demands is precisely the space that Large Language Models and AI agents are beginning to fill.

3.1.3 The Complexity Challenge

Microscopy presents complexity well beyond segmentation. Researchers routinely work with diverse labeling schemes, varying numbers of channels, different spatial scales, temporal dimensions, and countless combinations of experimental conditions. These multifaceted combinations make it difficult to build generalist models for broader tasks, and as a result, the vast majority of models remain highly task- and data-specific. This proliferation has driven community efforts such as the BioImage Model Zoo (https://bioimage.io) — a collaborative initiative to facilitate sharing of AI models across input image types and architectures, providing essential infrastructure for the community (Figure 3.3).

Yet the growing ecosystem has also revealed an accessibility paradox. Model developers struggle with diverse training data requirements, validation protocols, and framework compatibility, while the burden of creating user-friendly interfaces often exceeds available time. Users face the opposite problem: an overwhelming array of model choices without clear guidance on which to select, complex software installations, and limited technical expertise to validate whether a model works on their specific data. As highlighted by Ouyang et al. [234], the traditional desktop-centric paradigm — where analysis occurs in isolation on a single workstation — creates barriers to collaboration and reproducibility. While AI capabilities have advanced considerably, the practical barriers to using these tools have in many cases increased, widening the gap between AI potential and real-world adoption in biological research.

Modern microscopy data volumes amplify these challenges further. Terabytes of high-resolution images from time-lapse experiments, multi-channel fluorescence imaging, and high-content screening assays require sophisticated visualization and extensive manual interaction. Many important patterns, subtle phenotypes, or rare events are simply missed due to practical limitations of human-driven exploration. As following sections will highlight, agentic AI represents a potentially transformative tool to overcome some of these limitations.

3.1.4 Summary: The Evolution of AI Models for Microscopy

To orient readers, Table 3.1 summarizes the key model types discussed in this chapter — from traditional convolutional neural networks to modern AI agents — along with their inputs, outputs, and example tools.

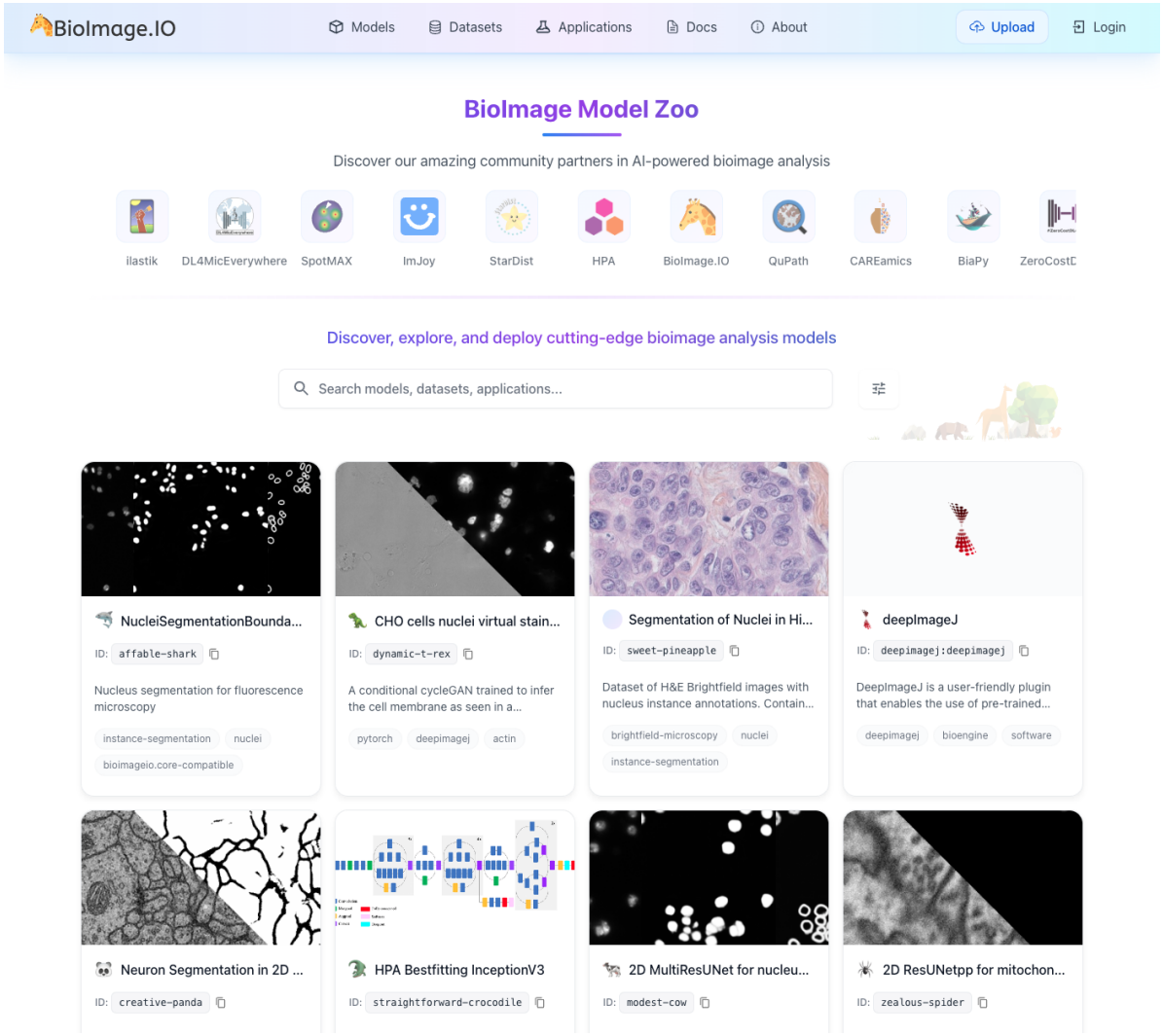


Figure 3.3: The BioImage Model Zoo serves as a centralized repository promoting FAIR (Findable, Accessible, Interoperable, and Reusable) principles for AI models in bioimage analysis. The platform integrates with popular tools including ilastik, Fiji/ImageJ, QuPath, napari, and others, allowing researchers to access AI models through familiar interfaces.

Table 3.1: Evolution of AI model types relevant to microscopy, from task-specific deep learning to general-purpose AI agents.

Model Type	Input	Output	Task Type	Example Tools
CNNs	Images	Labels	Classification	ResNet
U-Net variants	Images	Masks, restored images	Segmentation, restoration	U-Net, Cellpose
LLMs	Text	Text, code	Natural language, code generation	ChatGPT, Claude
VLMs	Images + Text	Text, annotations	Visual Q&A, classification, description	GPT-4o/5, Gemini, Claude
AI Agents	Files + Prompts	Code, results, workflows	Tool orchestration, autonomous analysis	Claude Code, Gemini CLI, Codex, Omega, BioImage.IO Chatbot

3.1.5 The Path Forward: From Generative AI to AI Agents

Large language Models (LLMs) — such as those from OpenAI, Google, and Anthropic — interact through natural language rather than specialized interfaces. They can generate code, provide explanations, and execute actions through conversation, lowering many of the technical barriers that have limited AI adoption in microscopy.

A critical development was the introduction of function calling (also called tool use) — the ability for LLMs to invoke external tools, APIs (application programming interface), and software through structured interfaces rather than generating plain text. First introduced by OpenAI in June 2023, function calling transformed LLMs from text generators into systems that can execute code, query databases, control instruments, and interact with scientific software. Combined with standardized protocols like Anthropic's Model Context Protocol (MCP, announced November 2024), this created a general-purpose interface layer connecting AI reasoning to existing scientific tools.

With tool-calling capabilities, LLMs can operate as AI agents — systems that plan actions, execute them, observe outcomes, and adapt iteratively. For bioimaging, this means AI agents can help select analysis methods, configure parameters, generate custom code, and troubleshoot failures through natural language interactions. Multimodal language models can also interpret microscopy images directly, combining visual understanding with code generation in a single interface. However, as we discuss throughout this chapter, these capabilities come with important limitations requiring careful human oversight.

The following sections explore each of these developments: how LLMs work (Section 3.2), how function calling enables tool use (Section 3.3), how vision-language models handle microscopy images (Section 3.4), and how AI agents are reshaping experimental workflows (Section 3.5, Section 3.6).

3.2 Foundations of Large Language Models: An Intuitive Primer

At its core, a Large Language Model (LLM) like ChatGPT, Gemini, or Claude is a text prediction engine operating at enormous scale. Its fundamental task is: given a sequence of text, predict the most probable next word. The autocomplete feature on your phone does something similar, but an LLM applies this principle with billions of parameters trained on a substantial fraction of human-written text. By repeatedly predicting the next word, an LLM can generate fluent, coherent paragraphs — and in the process, exhibit capabilities that go well beyond simple text completion, including code generation, logical reasoning, and scientific explanation.

3.2.1 How LLMs Learn: Training on the World's Text

The "Large" in LLM refers to two aspects: the size of the neural network (with billions of adjustable parameters) and the volume of text it learns from. Training involves feeding the model a massive dataset comprising a significant portion of the public internet, digital books, scientific articles, and code repositories.

The training objective is conceptually straightforward: given a sequence of text, the model learns to predict the next word. For example, given the partial sentence "The scientist placed the sample on the microscope," the model learns that "stage" is a likely next word. Before training, predictions are essentially random, but through billions of examples, the model gradually learns which words tend to follow which, adjusting its internal connections to improve predictions (see Chapter 4 for more details on the mechanics of training).

To process text, the model first breaks sentences into tokens — chunks that may be whole words or subword pieces. For instance, "microscope" might be a single token, while "fluorescence" might be split into "fluor" + "escence." Each token is converted into a numerical vector through a process called embedding, creating a mathematical representation that captures meaning and context. These vectors pass through layers of the neural network, where the model learns how words relate to each other. After training on billions of sentences, the model develops a rich statistical map of language — the relationships between words, phrases, and concepts.

3.2.2 The Transformer: A Revolution in Understanding Context

The critical breakthrough enabling modern LLMs is the transformer architecture, introduced by Vaswani et al [317]. in 2017. Before this innovation, language models had very short "memory" and would lose track of context in longer text. The transformer introduced a mechanism called self-attention, which allows the model to weigh the importance of every word in the input simultaneously, regardless of position. When processing the sentence "The *cell* was stained with DAPI, which binds to DNA in the *nucleus*," the attention mechanism enables the model to connect "nucleus" to "cell," maintaining long-range context and generating coherent, relevant responses.

3.2.3 Fine-Tuning for Domain-Specific Tasks: An ImageJ Macro Example

While the scale of training models like ChatGPT or Claude seems daunting, the underlying principles are accessible. To illustrate how LLMs can be adapted for domain-specific code generation, consider how one might fine-tune a small language model to generate ImageJ macros — simple scripts that automate repetitive tasks in Fiji/ImageJ, one of the most widely used tools in microscopy.

The process begins with **data collection**: gathering domain-specific examples, such as ImageJ macro files (`.ijm`) from public GitHub repositories. Even a modest collection — say, 28 macro files totaling about 82,000 characters — provides enough material for a small model to learn the syntax and patterns of the ImageJ macro language.

Rather than training from scratch, this approach uses transfer learning: starting with a small, pre-trained model (such as SmolLM2 [23] with 135 million parameters — orders of magnitude smaller than commercial frontier models) and fine-tuning it on microscopy-specific code. During fine-tuning, the model learns to predict the next token in ImageJ macro sequences, absorbing the syntax, common function calls, and typical workflow patterns. On a modern laptop, this process can complete in 10–20 minutes without specialized hardware.

After fine-tuning, such a model can generate plausible ImageJ macro code from natural language prompts. When asked to "Create a macro to threshold an image," it might produce something like:

```
setThreshold(50, 255);
run("Convert to Mask");
run("Analyze Particles...", "size=20-Infinity");
```

This small-scale example illustrates the same principle behind commercial LLMs: learn statistical patterns of language from large amounts of text, then specialize through fine-tuning. The difference is one of scale — frontier models train on orders of magnitude more data with vastly larger networks — but the core mechanism is the same.

3.2.4 Code Generation for Microscopy: Capabilities and Limits

For bioimage analysis, LLM code generation is most valuable when it bridges the gap between a researcher who understands what analysis they need and the programming required to implement it. A microscopist who knows they need to measure nuclear fluorescence intensity across treatment conditions can describe this to an LLM and receive a working Python script — without memorizing the syntax for loading TIFF files or calling scikit-image functions. This lowers the barrier to computational analysis, particularly for researchers who understand the biology but lack extensive programming backgrounds.

How well does this actually work? A rigorous benchmark by Haase et. al. [109] evaluated 18 LLMs on 57 bioimage analysis coding tasks with unit tests. The best-performing model (Claude 3.5 Sonnet) achieved a 58% pass rate, followed by GPT-4o at 50%. Notably, some tasks — including deconvolution, surface measurement, and voxel size reading — could not be solved by any model tested. A broader evaluation across 293 biomedical data science tasks found that standalone LLMs achieve under 40% accuracy, but an iterative agent approach — where the system executes code, observes errors, and refines its solution — raised accuracy to 74% [329]. These benchmark numbers will change as models improve, but they provide a realistic snapshot: AI-generated code is a useful starting point, not a finished product.

AI-assisted code generation has not reached the point where users without any programming understanding can reliably build complex analysis pipelines. Users still need foundational computational thinking — not necessarily memorized syntax, but the ability to decompose problems into discrete steps, recognize when an output looks wrong, and evaluate whether an approach is appropriate for their data. AI-assisted programming lowers the barrier to entry; it does not eliminate the need for scientific judgment about what the code should do.

3.3 From Text Generation to Action: Function Calling and Tool Use

A critical limitation of early LLMs was that they could only produce text. A researcher could ask ChatGPT how to segment nuclei in a DAPI image, and the model would generate a Python script — but the researcher still had to copy that code, set up the right software environment, install dependencies, debug errors, and run it themselves. The model was knowledgeable but unable to act. **Function calling** (also known as tool use) circumvented this barrier. Instead of merely describing what a user should do, an LLM can now invoke external tools, pass structured parameters, receive results, and reason about what to do next.

3.3.1 How Function Calling Works

In a function-calling workflow, the LLM is provided with a set of tool definitions — structured descriptions of available functions, their parameters, and expected inputs. When a user makes a

request, the model reasons about which tools are needed, generates structured calls with correct parameters, receives results, and decides whether to take further action or report findings.

Consider a concrete example. A microscopist asks: "Measure the average fluorescence intensity in the green channel of my image." Without function calling, the LLM would write a Python script and ask the user to run it. With function calling, the model directly invokes analysis tools in sequence:

```
{"tool": "load_image", "params": {"path": "experiment_01.tif"}}
{"tool": "extract_channel", "params": {"image": "...", "channel": "green"}}
{"tool": "measure_intensity", "params": {"image": "...", "metric": "mean"}}
```

Each call returns a result that the model inspects before deciding on the next step. If the image fails to load — perhaps the file format is unexpected — the model can read the error, try a different loading function, and continue. This structured, machine-readable interface between AI reasoning and tool execution is fundamentally different from generating a script and hoping the user can run it correctly.

3.3.2 From Fragmented Tools to a Universal Protocol

Function calling became an industry standard quickly — by early 2024, every major LLM provider supported structured tool invocation. Two subsequent developments made it considerably more powerful for scientific applications.

First, **multimodal function calling** (introduced with GPT-4o in May 2024) allowed a model to examine an image and call analysis tools in a single interaction. A researcher could upload a confocal image and ask "segment the nuclei," and the model would both interpret the image visually and invoke the appropriate segmentation tool — combining perception with action.

Second, a key practical challenge was **fragmentation**: every tool provider defined its own interface format, making it difficult to connect a single agent to diverse software. Anthropic's **Model Context Protocol (MCP)**, announced in November 2024, addressed this by providing an open standard for connecting AI agents to external tools and data sources. MCP gives any AI model a universal way to discover and use any compatible tool — analogous to how USB standardized device connectivity. For microscopy, MCP means that a single agent could potentially access a microscope control system, an image analysis library, a literature database, and a model repository through the same protocol. By late 2025, MCP had been adopted as an industry-wide standard, with its donation to the Linux Foundation and support from major AI providers.

Together, these developments transformed function calling from a feature of individual chatbots into the backbone of AI agent systems. Current frontier models combine large context windows with sophisticated tool use, enabling agents to work with entire experimental datasets in a single session. The practical impact for microscopy is explored in Section 3.5.

3.3.3 From Chatbots to Agents: The Trust Escalation

Function calling created a spectrum of increasing autonomy. At one end sits **assisted mode**, where the AI suggests tool calls but a human approves each one — useful when learning a new workflow or working with unfamiliar data. In the middle is **supervised mode**, where routine actions (loading an image, running a standard segmentation) execute automatically while consequential ones (deleting data, modifying acquisition settings) require confirmation. At the far end is **autonomous mode**, where the AI plans and executes multi-step workflows with minimal human intervention, checking in only for high-stakes decisions.

This progression mirrors how we delegate tasks to human colleagues — starting with close supervision and gradually extending trust as competence is demonstrated. In microscopy, the boundaries matter: an AI agent might autonomously run routine segmentation, but it should pause before modifying acquisition parameters on an expensive experiment, before discarding images that might contain rare biological events, or before committing to an irreversible analysis method. Deciding where to draw these boundaries is a design decision that each laboratory must make based on the stakes of the experiment.

> **Function Calling in Practice**
>
> When you ask a modern AI agent to "segment the nuclei in my DAPI image and count them," the agent does not just write a script. It directly calls a segmentation tool (e.g., Cellpose or StarDist[287]), passes your image, retrieves the segmentation mask, calls a counting function, and returns the result with a visualization. Each step is a structured tool call with typed parameters — not free-form text generation. If any step fails, the agent reads the error and adapts its approach.

3.4 Multimodal AI: Vision-Language Models for Microscopy

Biological research is inherently multimodal. A microscopy experiment produces not just images but also metadata about the sample preparation, the imaging conditions, the fluorescent labels used, and the biological question being asked. Traditional image analysis pipelines discard most of this context: a segmentation algorithm processes pixel intensities without knowing what protein is being labeled or what drug treatment was applied. Vision-language models (VLMs) — models that process both images and text within a single architecture — offer a different approach. By accepting natural language alongside visual input, they can incorporate experimental context into their analysis, at least in principle.

As of early 2026, all major LLM providers offer VLM capabilities (OpenAI's GPT-5 series, Anthropic's Claude 4.5/4.6, Google's Gemini 3 Pro, and open-source models such as Meta's Llama 4). These models accept images as input alongside text prompts and can generate text

descriptions, answer questions about visual content, and — when combined with function calling (Section 3.3) — invoke analysis tools based on what they see.

3.4.1 How VLMs Interpret Microscopy Images

The difference between a VLM and a traditional CNN becomes clear when both encounter unfamiliar data. As demonstrated in Section 3.1, a ResNet50 model trained on ImageNet confidently misclassified a fluorescence microscopy image of TRAP1 protein as "jellyfish" — the model had no mechanism to recognize that it was looking at something outside its training categories. Figure 3.4 shows what happens when a VLM encounters the same image. Rather than forcing the image into a predefined label set, the VLM recognized it as fluorescence microscopy and described the cellular structures in biological terms, acknowledging the limits of what it could determine from the image alone.

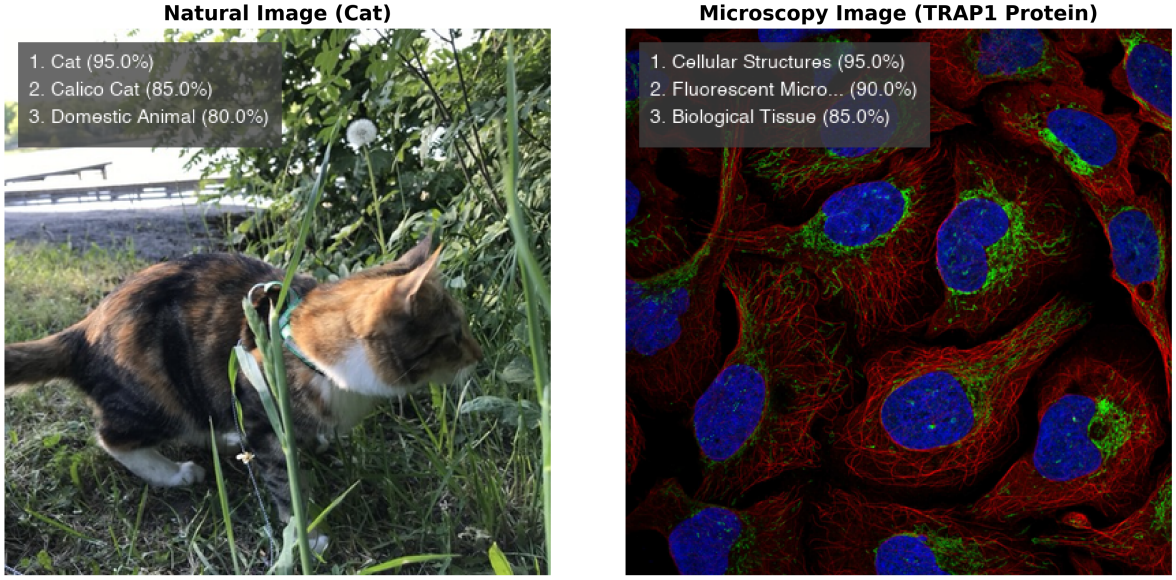


Figure 3.4: VLM generalization comparison using the same cat and microscopy images from Figure 3.2. Unlike the CNN, the VLM provides contextually appropriate descriptions for the microscopy image, recognizing it as fluorescence microscopy with biologically meaningful classifications rather than confident misidentifications.

This difference stems from how the models are built. A CNN trained for classification must assign one of its predefined labels to every input — it has no way to say "I don't recognize this." A VLM, by contrast, generates open-ended text responses and can express uncertainty, describe what it observes, and qualify its interpretations. When combined with token-level log-probability outputs (logprobs), VLMs can provide rough indicators of their own confidence

— though these reflect the model's internal state rather than calibrated probabilities, and should not be interpreted as rigorous uncertainty estimates.

VLMs can also perform Visual Question Answering (VQA) on biological images. When asked to analyze the TRAP1 protein image in detail, a modern VLM described the imaging modality, identified the likely fluorescent channels (blue DAPI nuclear staining, red cytoskeletal structures, green organelle labeling), and offered interpretations about intracellular organization. This contextual analysis from a general-purpose model — one not specifically trained on microscopy data — goes beyond what a classifier can provide. However, these interpretations should be treated as informed hypotheses, not established facts: VLMs can produce plausible-sounding but incorrect biological descriptions, particularly for specialized imaging techniques or unusual sample preparations. Early benchmarks suggest competitive performance on some biomedical image classification tasks in zero-shot settings, though results vary considerably across imaging modalities and systematic evaluations on microscopy-specific data remain limited.

3.4.2 Practical Capabilities and Current Limitations

The combination of visual understanding and language generation enables a workflow pattern that was impossible with text-only models: **vision-guided programming**. A researcher can upload a microscopy image, ask the VLM to suggest an appropriate analysis strategy, have it generate the corresponding code, execute that code, show the VLM the output, and iterate based on visual inspection of the results. This tight feedback loop between seeing and coding is particularly valuable for microscopy, where visual inspection of intermediate results — checking whether a segmentation mask aligns with cell boundaries, whether background subtraction was adequate, whether a threshold captures the structures of interest — is a routine part of quality control.

An important convergence is emerging between VLMs and the foundation models for segmentation discussed in Section 3.1. Models like CellSAM, micro-SAM, and Cellpose-SAM provide validated, high-performance segmentation across diverse imaging modalities. VLMs contribute broad visual understanding and natural language reasoning. When connected through function calling (Section 3.3), these capabilities combine: a researcher describes their goal ("segment all cells and measure their nuclear-to-cytoplasmic ratio"), the VLM interprets the intent, selects and invokes the appropriate segmentation model, and explains the results. The precision comes from the specialized tool; the flexibility comes from the language interface (see also Chapter 8 for guidance on selecting analysis tools, and Chapter 9 for model training approaches).

Despite these capabilities, several practical constraints limit where VLMs are useful today. Processing a single high-resolution microscopy image through a frontier VLM can cost several cents in API fees — acceptable for exploratory analysis of a few images, but prohibitive for high-content screening experiments involving thousands of images. Inference is slow (seconds per image rather than milliseconds), and the computational requirements carry a considerable environmental footprint. Most importantly, the quality of VLM outputs on specialized scientific

imagery, while often useful, is inconsistent: a VLM might correctly identify imaging modalities and fluorescent channels in one image, then misinterpret a phase-contrast image as brightfield in the next.

These constraints position VLMs most effectively in human-in-the-loop roles: initial data exploration, annotation quality control, generating training labels for specialized models, and helping researchers develop analysis strategies. In these applications, the cost per image is acceptable, speed is not critical, and expert oversight is already part of the workflow. For production analysis at scale, dedicated pipelines built on specialized models remain the practical choice — but VLMs are increasingly useful for designing those pipelines in the first place.

3.5 AI Agents for Microscopy and Workflow Orchestration

LLMs and VLMs provide powerful individual capabilities — generating text, writing code, interpreting images — but each interaction is essentially one-shot: the user asks, the model responds. **AI agents** extend these models into systems that can take actions (running code, calling APIs, controlling hardware), observe the results, and adapt their approach iteratively. Where a language model answers a question, an agent works through a problem step by step until it reaches a solution — or recognizes that it cannot.

3.5.1 What Are AI Agents?

At its core, an AI agent is a system built around a language model that operates in a loop: it **observes** the current state (an image, a dataset, an error message), **reasons** about what to do next, **acts** by invoking tools or executing code, and then **observes** the outcome — repeating until the task is complete. This observe-reason-act cycle, formalized in frameworks like ReAct [344], is what distinguishes an agent from a chatbot.

An important question is how agents handle uncertainty — when should they recognize that they cannot answer a query with sufficient confidence? Unlike traditional software that either succeeds or throws an error, language models can produce plausible-sounding but incorrect responses without any internal "failure signal." Current agents address this through several mechanisms: well-designed system prompts that instruct the model to express uncertainty rather than guess, tool calls that ground responses in verifiable data (e.g., retrieving documentation rather than reciting from memory), and confidence calibration through chain-of-thought reasoning where the model explicitly weighs evidence before responding. In practice, agents are more reliable when they can verify claims through tool use — for instance, running code to check whether a function exists rather than asserting that it does. For microscopy users, this means that agent responses backed by executed code or retrieved documentation are generally more trustworthy than ungrounded assertions about biology or software capabilities. When an agent says "I'm not sure" or asks for clarification, this is often a better outcome than a confident but incorrect answer.

The evolution of agent design reflects a shift in philosophy. Early agents followed agentic workflows — hand-designed sequences of tool calls with human-specified decision points. A developer would define: "first call the segmentation tool, then if the count is above 100 switch to algorithm B." These workflows were effective but rigid: every decision point had to be anticipated in advance.

Current practice favors general agency: giving agents access to broad capabilities — a terminal, a file system, APIs, analysis libraries — and letting them determine the right sequence of actions for each task. Rather than pre-scripting workflows, the agent receives a goal ("segment and count the nuclei in this dataset") and works out what tools to use, in what order, handling errors as they arise. This shift from prescriptive scripts to goal-directed problem-solving is the key conceptual development.

Three design principles have proven important for building effective agents, each with direct relevance to microscopy:

Harness engineering refers to the scaffolding that wraps around the language model — the system that manages tool definitions, permissions, error recovery, and the overall agent loop. For microscopy, a well-designed harness determines whether an agent can only call pre-validated segmentation functions (safe but limited) or can install arbitrary packages and run custom code (powerful but requiring oversight). A key lesson from early deployments is that the quality of this scaffolding — how tools are presented, how errors are handled, what safety boundaries are enforced — often matters more than the choice of underlying language model.

Context engineering is the discipline of providing the right information at each step of the agent's reasoning. An agent with a million-token context window could, in theory, ingest everything, but irrelevant information degrades performance. Effective context engineering involves dynamically loading relevant documentation, previous results, and tool descriptions while filtering out noise. Retrieval-Augmented Generation (RAG) is one common approach — injecting relevant external knowledge, such as analysis documentation or published protocols, into the agent's context at the moment it is needed.

Memory systems allow agents to retain information across sessions. Short-term memory (the conversation context) lets an agent build on previous steps within a session. Long-term memory (persistent storage) enables agents to remember, for instance, which segmentation parameters work well on a particular lab's imaging setup, or that a user's file naming convention encodes the treatment condition in the third field.

3.5.2 How Agents Interact with the World

The practical power of an agent depends on what it can access and how. Three complementary approaches have emerged, each with distinct strengths for microscopy.

Structured tool integration through protocols like MCP (Section 3.3) and function calling provides typed, validated interfaces between agents and specific tools. An MCP server for a

microscope might expose functions like `acquire_image(channel, exposure, z_stack)` and `move_stage(x, y)`. Because the agent can only invoke pre-defined operations, this approach is well-suited to instrument control, where safety and predictability are paramount.

Terminal and API access is more powerful and increasingly common. When an agent has access to a command line — as provided by terminal-based coding agents — it can install packages, read documentation, write and execute arbitrary code, and compose workflows by combining tools. This is the approach driving rapid improvements in software engineering benchmarks: the agent works like a developer at a terminal, using whatever tools the task demands rather than being limited to a pre-defined set. For microscopy researchers, this means an agent that can install Cellpose, write a Python analysis script, run it, inspect the output, and iterate — all through natural language conversation.

Graphical interface automation extends agent capabilities to GUI-based software. Computer use capabilities (e.g., Anthropic's Claude computer use, announced October 2024) allow agents to see a screen, move a cursor, click buttons, and type text — operating software the way a human would. For microscopy, this opens the possibility of controlling proprietary acquisition software or navigating Fiji/ImageJ menus even when no programmatic API is available. Performance on GUI automation benchmarks is improving but remains well below human reliability on complex multi-step tasks.

The trend across all three approaches is toward broader access with safety guardrails. Modern agents operate within permission systems that ask for confirmation before destructive actions, sandbox execution environments to prevent damage to host systems, and provide human-in-the-loop checkpoints for consequential decisions. For microscopy, where instruments are expensive and samples can be irreplaceable, these guardrails are not optional — an agent should never modify acquisition parameters on a running experiment without explicit human authorization.

> **The Coding Agent Ecosystem (2026)**
>
> Several production-grade coding agents are available as of early 2026. Terminal-based agents — **Claude Code** (Anthropic), **Gemini CLI** (Google, open-source with a generous free tier), and **OpenAI Codex** (open-source) — run in the developer's command line. IDE-integrated agents like **Cursor** and **GitHub Copilot** (agent mode) embed in code editors. **Devin** (Cognition Labs) provides fully sandboxed development environments. All support MCP and function calling; the choice often comes down to cost, interface preference, and which model best suits the task. See Section 3.7 for a hands-on tutorial.

3.5.3 Use Cases for Bioimage Analysis

AI agents are well-suited to microscopy because they combine the ability to reason about biological problems with the ability to execute computational solutions. Several patterns of use

have emerged:

Conversational image analysis. A researcher describes a multi-step analysis in natural language — "segment the nuclei in this image, measure their areas, and compare the size distributions across treatment conditions" — and the agent breaks this into tool calls, executes them, and presents results with visualizations. The key advantage over a static pipeline is that the researcher can inspect intermediate results and redirect the analysis conversationally: "the segmentation is merging touching nuclei — try a smaller cell diameter."

Automated workflow construction. Agents can build analysis pipelines dynamically by selecting appropriate tools, configuring parameters, and chaining operations based on image content and user goals — adapting the workflow as intermediate results come in.

On-demand software generation. Agents can create custom scripts, GUIs, and analysis tools tailored to specific experimental needs — a capability transformative enough to warrant its own discussion (see Section 3.6).

Autonomous error handling. When a pipeline step fails, agents can diagnose the problem by analyzing error messages, searching documentation, and implementing fixes without human intervention. For researchers without extensive programming experience, this considerably reduces the barrier to using computational tools.

Intelligent data exploration. For large microscopy datasets — where a researcher might otherwise spend days scrolling through images — an agent can cross-reference experimental metadata, identify relevant subsets, and surface patterns or rare events that would be impractical to find by manual inspection.

3.5.4 Concrete Examples: Omega and the BioImage.IO Chatbot

Two published systems illustrate what AI agents can accomplish in bioimage analysis today, and where the practical limits lie.

Omega, published in 2024 [268], is a napari plugin that enables conversational image analysis. Users describe tasks in natural language — "segment the nuclei" or "create a depth-coded projection of this z-stack" — and Omega generates and executes the corresponding code within the napari viewer. Through its LiteMind library, Omega connects to multiple LLM providers (OpenAI, Anthropic, Google), allowing it to visually interpret the contents of the napari viewer and make context-dependent decisions about analysis approaches. The system can also generate custom widgets on demand — for instance, a slider-based tool for filtering segmented objects by area. Omega demonstrates that conversational agents can meaningfully lower the barrier to interactive image analysis, though the quality of results depends on how precisely the user describes their intent and on the underlying LLM's ability to generate correct code.

BioImage.IO Chatbot, also published in 2024 [175], takes a different approach. Built on Retrieval-Augmented Generation (RAG), the chatbot draws on a curated knowledge base

of community-contributed documentation from tools including ImageJ, deepImageJ, and the bio.tools registry. Rather than generating analysis code from scratch, it retrieves and synthesizes relevant information from existing documentation, helping users navigate the complex landscape of bioimage analysis tools. The chatbot can also execute Python code in the browser, run models from the BioImage Model Zoo (such as Cellpose), and generate statistical reports. Its extension mechanism, built on the Hypha distributed computing framework, allows developers to serve additional capabilities — including hardware control and GPU-accelerated computation — making it a platform for community-driven agent development rather than a standalone tool.

Both systems are early examples of a paradigm that is evolving rapidly. Neither replaces the need for domain expertise: the researcher must still judge whether a segmentation result makes biological sense, whether the analysis approach is appropriate, and whether the conclusions are supported.

> ⚠ Agents Require Validation
>
> An agent may produce code that runs without errors but implements an inappropriate analysis method or makes incorrect biological assumptions. Segmentation that looks reasonable at first glance may systematically miss a subpopulation of cells. Always inspect intermediate results visually, test on known data when possible, and verify that the analysis logic matches your scientific intent. See Section 3.8 for a full discussion.

3.5.5 The Vision: Intent-Driven Autonomous Microscopy

The trajectory of these developments points toward what we call **intent-driven microscopy**: the researcher describes *what* they want to discover, and the AI system determines *how* to achieve it — planning the experiment, controlling the instrument, acquiring images, analyzing results, and adapting in real-time (Figure 3.5).

Consider a concrete scenario: a researcher tells the system *"Find all cells undergoing mitosis in my sample and capture high-resolution z-stacks of each."* An intent-driven system would plan a scanning strategy (low-magnification overview followed by targeted imaging), acquire overview images, run real-time cell classification to identify mitotic cells, switch to high magnification for targeted z-stack acquisition, and deliver the dataset with metadata and quality metrics. Several of these building blocks exist today in isolation — the scientific and engineering challenge lies in integrating them into systems that are reliable, general-purpose, and safe enough for routine use.

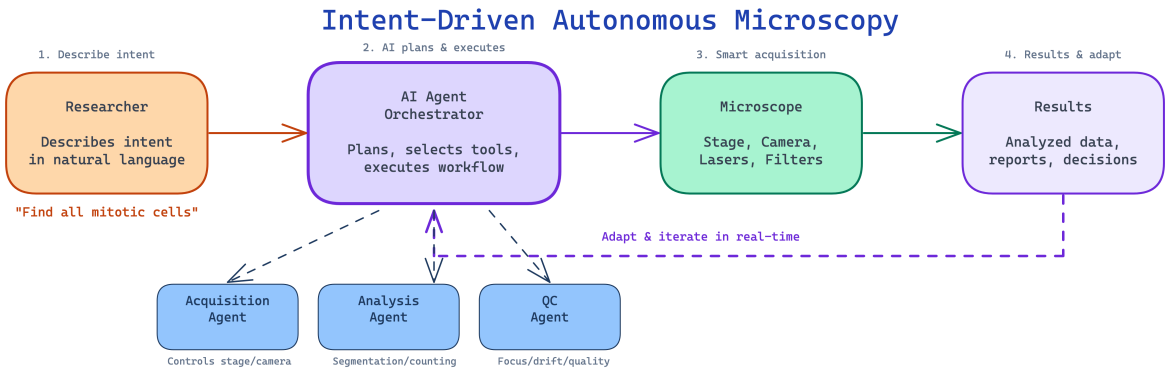


Figure 3.5: The vision of intent-driven autonomous microscopy. The researcher describes their experimental goal in natural language. An AI agent orchestrator plans the workflow, coordinating specialized sub-agents for acquisition, analysis, and quality control. The system controls the microscope hardware, collects and analyzes data, and feeds results back to adapt the experiment in real time — creating a closed-loop system that operates with minimal human intervention.

3.5.6 Autonomous Microscopy: AI-Controlled Instruments

The most ambitious application of AI agents in microscopy is direct instrument control — systems that not only analyze images but plan and execute acquisition protocols (see also Chapter 7). Several recent systems illustrate different points on the spectrum from AI-assisted to AI-driven microscopy.

EIMS (Embodied Intelligence Microscope System), described in a 2025 preprint [130], reconceptualizes the microscope as an autonomous agent. A high-level LLM acts as a deliberative planner, interpreting experimental intent (e.g., "capture mitotic events in this sample") and decomposing it into subgoals executed by specialized control modules. The system reportedly achieves generalization on multi-step protocols without task-specific training, including automated capture of rare spatiotemporal events such as cell division.

AILA (Artificially Intelligent Lab Assistant), published in 2025 [200], applies LLM-based agents to atomic force microscopy. Using specialized agents coordinated by a central planner, AILA was benchmarked on 100 real laboratory tasks (AFMBench). A central finding was sobering: even frontier LLMs struggle with basic experimental tasks, and general language fluency does not translate into experimental competence. Multi-agent architectures significantly outperformed single-agent designs, and the authors raised important safety concerns about autonomous instrument operation — what they termed "sleepwalking," where an agent takes confident but incorrect actions on expensive equipment.

Two further systems illustrate the broader trend toward AI-controlled microscopy, though they use specialized machine learning rather than LLM-based agents. **SmartEM**, published in

2026 [211], integrates machine learning directly into the electron microscope acquisition loop, implementing a "foveated" strategy: a quick low-resolution survey pass identifies regions of interest, followed by high-resolution rescanning of only those regions, achieving up to 7-fold acceleration of connectomic brain imaging. **pySTED**, published in 2024 [28], demonstrates that reinforcement learning agents trained in realistic simulation can transfer successfully to real STED super-resolution microscopes without additional fine-tuning, bridging the simulation-to-reality gap for microscopy control.

These examples span a spectrum from LLM-based planning (EIMS, AILA) to specialized ML-driven acquisition (SmartEM, pySTED). Collectively, they suggest that AI-controlled microscopy is moving from proof-of-concept toward practical deployment — though most systems remain validated only in narrow domains, and the gap between demonstrating a capability in one laboratory and deploying it reliably across diverse instruments is considerable.

3.5.7 Coding Agents: AI with Terminal Access

A particularly consequential development has been the emergence of coding agents — AI systems that operate directly in development environments with access to terminals, file systems, and web browsers. These agents go well beyond earlier LLM assistants that could only suggest code snippets.

Claude Code (Anthropic) is a CLI-based agent that can read and edit files across entire codebases, run shell commands, execute tests, interact with version control, and orchestrate complex multi-step tasks — all while maintaining context across long sessions with million-token context windows. **Devin** (Cognition Labs) operates in a sandboxed development environment with its own code editor, terminal, and web browser. **GitHub Copilot** has evolved from inline code suggestions to agent mode, where it can plan changes across repositories, edit multiple files, and iterate on feedback. **Cursor** and **Windsurf** represent a new category of AI-native code editors with deeply integrated agent capabilities.

The pace of improvement has been notable. On the SWE-bench Verified benchmark [146] — which tests agents on real-world GitHub issues requiring understanding codebases, diagnosing bugs, and implementing fixes — success rates rose from under 5% in early 2024 to over 70% by late 2025. These benchmarks measure general software engineering, not scientific applications specifically, but the underlying capabilities — reading code, debugging errors, composing multi-step solutions — are directly relevant to microscopy analysis pipelines.

For microscopy researchers, the practical significance is that these agents can install analysis packages, write and debug Python scripts for napari or ImageJ, run analysis pipelines, and iterate on results — all through natural language conversation in a terminal. The walkthrough in Section 3.7 demonstrates this workflow concretely.

3.6 On-Demand Software and the Changing Nature of Microscopy Tools

For decades, microscopy analysis has depended on durable software products — applications like CellProfiler, Fiji/ImageJ, and napari that represent years of development effort, extensive documentation, and sustained community maintenance. These tools are general-purpose by design: they must serve a broad range of users and experimental scenarios to justify the investment required to build and maintain them. The trade-off is that researchers must learn to navigate complex interfaces, adapt their thinking to the software's workflow logic, and work within design decisions made for a broad user base rather than their specific experiment.

AI agents are beginning to change this equation. Instead of learning to operate a general-purpose application, a researcher describes what they need — "create a tool that lets me draw regions on my confocal image, measure mean fluorescence intensity in each region across all channels, and export the results as a CSV" — and the agent generates a purpose-built script or application, typically within minutes for straightforward tasks. The result does exactly what was requested and requires no training to use. If the requirements change, the agent generates a new version rather than patching the old one.

3.6.1 When On-Demand Software Makes Sense

Consider a concrete example. A researcher performing calcium imaging needs to load .nd2 files from a specific acquisition system, extract time-series fluorescence traces from user-drawn regions of interest, apply baseline correction using the F0 method appropriate for their calcium indicator, detect peaks above a threshold, and export both the individual traces and summary statistics. No single existing tool performs exactly this combination of steps in the way the experiment requires. Traditionally, the researcher would either write a custom script (requiring programming expertise), adapt a general-purpose tool (requiring time to learn the interface and work around its assumptions), or ask a bioimage analyst to build something bespoke (requiring their availability and understanding of the experiment).

With a coding agent, the researcher describes these requirements conversationally and receives a working Python script — built on established libraries like scikit-image and matplotlib — that they can run immediately. If the following week's experiment uses a different calcium indicator requiring different baseline correction, the researcher generates a new script rather than modifying the old one. The entire interaction, for a task of this complexity, might take fifteen to thirty minutes, including testing and refinement.

This approach is most valuable for tasks that are too specific for general-purpose tools but too routine to justify dedicated software development: custom visualization dashboards for a particular multi-dimensional dataset, batch-processing scripts tailored to a lab's file naming conventions, or quick analysis prototypes for exploring whether an approach is worth pursuing. For complex analysis pipelines involving multiple interdependent steps, novel algorithms, or

large-scale processing, the generation process is neither instantaneous nor guaranteed to work on the first attempt — the researcher must understand the analysis well enough to evaluate whether the generated code is correct.

3.6.2 The Reproducibility Question

On-demand software raises a legitimate concern for scientific practice: if an analysis tool is generated and then discarded, how does another laboratory reproduce the analysis?

The concern is real but addressable with existing practices. The generated code is ordinary Python, R, or ImageJ macro — it can be inspected, version-controlled, and shared like any other analysis script. Researchers using this approach should save the generated code alongside their data, record the prompt and model used to produce it, and include the script in their supplementary materials. Importantly, on-demand tools typically compose and configure established, validated libraries — scikit-image, Cellpose, napari — rather than implementing algorithms from scratch. The generated code calls the same functions that a human programmer would use; the agent handles the boilerplate of loading files, configuring parameters, and creating visualizations.

A subtler concern is long-term reproducibility. If a finding is questioned years later, the analysis must still be runnable. Version-pinning dependencies (recording the exact library versions used), including the generated code in a repository, and documenting any manual steps taken during the analysis all help — the same practices that good computational science has always required, but that are easily neglected when the code feels disposable.

There is also a risk of lost institutional knowledge. When a lab builds durable software, the development process creates shared understanding of the analysis logic — what assumptions were made, what edge cases were handled, why certain parameters were chosen. When software is generated on demand and used once, this knowledge may not accumulate. Laboratories adopting this approach should consider maintaining a shared repository of validated analysis scripts, annotated with the experimental context in which they were used.

Best Practices: What to Record When Using an AI Model

When using AI-generated code or analysis in your research, document the following for reproducibility:

- **Model identity**: the specific model name and version (e.g., "Claude Sonnet 4.5, accessed 2026-03-15")
- **Prompts**: the exact prompts or conversation used to generate the code — save the full session log if possible
- **Generated code**: the complete script as produced, before any manual edits, stored alongside the final version

- **Dependencies**: all library versions used (`pip freeze` or `conda list`), ideally captured in a `requirements.txt` or `environment.yml`
- **Manual modifications**: any changes you made to the AI-generated output, and why
- **Validation steps**: how you verified the output — test datasets used, visual inspections performed, comparisons with known results

3.6.3 A Shift in Researcher–Software Interaction

The broader implication is a change in the relationship between microscopists and their computational tools. Traditional software requires the researcher to learn the tool's interface and adapt their workflow to its logic. On-demand software generation inverts this: the researcher describes the goal, and the software adapts to the researcher. This means that researchers who understand their biology deeply but lack extensive programming training can access computational analysis that was previously gated by programming skill — a meaningful expansion of who can do quantitative microscopy.

At the same time, this approach sits *on top of* the established software ecosystem, not in place of it. When an agent generates a segmentation script, it calls Cellpose or StarDist underneath. When it builds a visualization, it uses matplotlib or napari. The durable, community-maintained tools become the validated building blocks that agents compose; they become more valuable in an agent-driven world, not less. Software that exposes programmable APIs and command-line interfaces will thrive because agents can call it. Closed, GUI-only tools that cannot be invoked programmatically face a harder path — not because they are technically inferior, but because agents cannot use what they cannot call.

A useful analogy: autofocus did not eliminate the need to understand optics — it freed microscopists to focus on experimental design rather than manual focusing. Similarly, AI agents will not eliminate ImageJ, but they will reduce the need to memorize its menu hierarchy or hand-write batch-processing macros.

How far this shift will go — and where the limits of AI-generated software lie in terms of reliability, complexity, and scientific rigor — remains to be determined through practical experience.

3.7 Practical Guide: Getting Started with AI Agents for Microscopy

Most microscopists encounter AI agents for the first time through web-based chatbots — typing a question and reading a response. That interaction model is useful for learning concepts or troubleshooting errors, but it cannot access your files, run code on your data, or iterate on results. This section walks through a more powerful approach: using a terminal-based coding

agent that operates directly in your data directory, writes and executes analysis scripts, installs packages as needed, and refines its work based on your feedback.

3.7.1 Choosing and Installing a Coding Agent

Several terminal-based coding agents are available as of early 2026. The choice between them depends mainly on cost tolerance, institutional preferences, and which model you find most effective for your tasks — the workflow pattern is the same across all of them.

Table 3.2: Terminal-based coding agents available as of early 2026. Pricing and capabilities change frequently; check provider documentation for current details.

Agent	Provider	Linux/Mac Install command	Free tier	Context window
Gemini CLI	Google	`npm install -g @google/gemini-cli`	Generous (1,000 req/day, as of early 2026)	1M tokens
Claude Code	Anthropic	`npm install -g @anthropic-ai/claude-code`	Limited	1M tokens
OpenAI Codex	OpenAI	`npm install -g @openai/codex`	Limited	Varies

For this walkthrough, any of these agents will work. To install and start Gemini CLI as an example (it offers a generous free tier at the time of writing):

```
npm install -g @google/gemini-cli
cd /path/to/your/microscopy-data
gemini
```

On first run, the agent will prompt you to authenticate. Once set up, it reads the files in your working directory and responds to natural language requests — writing Python scripts, executing shell commands, installing packages, and iterating on results.

> ⚠ Data Privacy and AI Services
>
> When you use a cloud-based AI agent, your prompts and — in some cases — your files are sent to external servers. Before uploading unpublished data, patient-derived images, or proprietary experimental details, check your institution's data governance policies and the provider's data retention terms. See Section 3.8 for a fuller discussion.

> **⚠ Always Verify AI-Generated Code**
>
> Never run AI-generated analysis on important data without understanding what the code does. Read through generated scripts before execution, test on a small data subset first, inspect intermediate results visually, and compare with known ground truth when available. AI agents can call non-existent functions, use incorrect parameters, or implement inappropriate methods — and these errors may not produce obvious failures.

3.7.2 Writing Effective Prompts

The quality of an agent's output depends heavily on how clearly you describe your goal. A few principles help:

- **Be specific about your data.** Rather than "analyze my images," say "I have 16-bit TIFF files with three channels: DAPI (blue), phalloidin-488 (green), and MitoTracker Red."
- **State your biological question.** "I want to measure whether mitochondrial fragmentation increases after drug treatment" gives the agent far more context than "process these images."
- **Name the tools you prefer.** If you want Cellpose rather than a threshold-based approach, say so. The agent will default to whatever it deems easiest, which may not be the most appropriate method.
- **Ask to see intermediate results.** "Show me the segmentation overlay before proceeding" catches errors early.
- **Describe failures concretely.** "The segmentation is merging touching cells — can you try the Cellpose 'cyto3' model with a smaller diameter?" is more useful than "it didn't work."

3.7.3 Walkthrough: Cell Cycle Classification with an AI Agent

This walkthrough demonstrates a realistic, multi-step analysis. The scenario: you have multi-channel fluorescence images of FUCCI-labeled U2OS cells and want to classify cell cycle phases. FUCCI (Fluorescent Ubiquitination-based Cell Cycle Indicator [276]) uses two fluorescent reporters whose expression is regulated by the cell cycle: mKO2-hCdt1 (red) accumulates during G1 phase, while mAG-hGeminin (green) accumulates during S/G2/M. Cells transitioning through early S phase co-express both reporters, appearing yellow. U2OS is a human osteosarcoma cell line commonly used in cell cycle studies.

Step 1: Orient the agent. Start the agent in your data directory and describe what you have:

```
> I have fluorescence images of FUCCI-labeled U2OS cells with three
  channels saved as separate .bmp files: brightfield, 488nm (green,
  mAG-hGeminin), and 561nm (red, mKO2-hCdt1). I want to segment
  individual cells, measure per-cell fluorescence in both channels,
  and classify cell cycle phase. Can you examine the files here and
  outline an analysis plan?
```

The agent will list your files, check image dimensions and bit depth, and propose a step-by-step plan — explaining the biology and the computational steps needed. This is the key advantage over a web chatbot: the agent works with your actual data, not a hypothetical example.

Step 2: Visualize the data. Before any analysis, inspect what you are working with:

```
> Load the brightfield, green, and red channel images for one field
  of view. Display them side by side with appropriate contrast.
  Save the figure as overview.png.
```

The agent writes a script, runs it, and produces the figure. If the contrast is wrong — "the green channel looks saturated" — describe the problem and the agent will adjust.

Step 3: Segment cells. Ask the agent to identify individual cells:

```
> Use Cellpose to segment cells from the brightfield image. Install
  cellpose if needed. Show me the segmentation outlines overlaid on
  the brightfield image so I can check the quality.
```

Inspect the overlay carefully. Are touching cells correctly separated? Are small debris objects being counted as cells? If not, describe the problem: "Cells near the image border are being cut off — can you exclude objects touching the edge?" The agent modifies the code and re-runs.

Step 4: Measure and classify. With validated segmentation masks, quantify fluorescence:

```
> For each segmented cell, measure mean intensity in the green and
  red channels. Classify cells as: G1 (high red, low green),
  S-phase (co-expressing both - yellow), or G2/M (high green, low
  red). Create a red-vs-green scatter plot colored by phase, and
  export the per-cell data as a CSV.
```

If the classification boundaries do not match your data — perhaps because your FUCCI reporter expression levels differ from typical values — ask the agent to adapt: "The fixed thresholds are misclassifying transitional cells. Can you try k-means clustering with three clusters on

the log-transformed red/green ratio instead?" The agent revises its approach based on your biological knowledge.

Step 5: Validate visually. The final and most important step:

```
> Overlay each cell's classification (red=G1, yellow=S, green=G2/M)
  on the brightfield image. Save as validation_overlay.png.
```

Compare the overlay against what you see in the fluorescence channels. Do the red-labeled cells correspond to visibly red cells in the mKO2 channel? Are the yellow (S-phase) cells genuinely co-expressing both reporters? This visual validation is the human expert's irreplaceable contribution. No amount of computational sophistication substitutes for a biologist confirming that the output makes biological sense.

> **Beyond This Walkthrough**
>
> The same workflow pattern — orient, visualize, analyze, validate — applies to any microscopy analysis task. Researchers have used coding agents to batch-process time-lapse data, build custom measurement pipelines, generate publication-quality figures, and automate file format conversions. The BioImage.IO Chatbot and Omega for napari, discussed in Section 3.5, offer complementary approaches: web-based guidance and in-viewer analysis, respectively.

3.8 Challenges, Implications, and the Road Ahead

The capabilities described in this chapter are real, but they carry limitations that researchers must understand before incorporating AI agents into their work. Scientific research demands accuracy, reproducibility, and transparency — standards that current AI systems cannot independently meet. The broader ecosystem of AI tools also introduces practical fragility: scripts break when dependencies update, hosted APIs change without notice, and plugins go unmaintained. This section examines these challenges and considers what they mean for microscopy practice, the bioimage analysis profession, and the scientific enterprise.

3.8.1 The Hallucination Problem

The most consequential limitation of current AI systems is their tendency to hallucinate — to generate confident, fluent, and entirely fabricated information. Unlike a human colleague who typically hedges when uncertain, language models produce invented content with the same surface characteristics as verified facts, making the two difficult to distinguish without independent checking.

Hallucination rates vary by task and model. A study evaluating LLMs on systematic review tasks found that a large fraction of AI-generated scientific references were fabricated — ranging from roughly 30% for the strongest models to over 90% for weaker ones [54]. These figures apply specifically to reference generation, a task where models are particularly prone to invention; hallucination rates for other tasks differ and are less systematically characterized.

In microscopy, hallucinations take several forms that deserve specific attention. Models routinely produce realistic-looking but entirely fictional scientific citations — complete with plausible author names, journal titles, and page numbers for papers that do not exist. When asked for experimental protocols, models can produce detailed, professional-sounding procedures that are scientifically wrong — suggesting fluorophore pairs whose excitation/emission spectra overlap completely, or recommending laser powers that would damage living samples within seconds. They may assert with confidence that specific proteins localize to particular compartments, or that certain treatments produce established phenotypes, without basis in the literature. And they frequently describe software features, function parameters, or capabilities that do not exist in the actual tools — an agent might generate code calling a Cellpose function with a parameter that was never implemented, producing a runtime error, or worse, silently defaulting to unexpected behavior.

The root cause is architectural: language models generate text by predicting statistically likely continuations, not by retrieving verified facts. A plausible-sounding citation is, from the model's perspective, simply a likely sequence of tokens. Retrieval-augmented generation (RAG) partially mitigates this by grounding model outputs in retrieved documents, but does not eliminate hallucination — models can still misinterpret, selectively quote, or confabulate beyond what retrieved sources support.

3.8.2 Reproducibility and Non-Determinism

AI-generated analysis introduces reproducibility challenges distinct from those in traditional computational work. The same natural language prompt given to the same model at different times can produce different analysis scripts — using different libraries, different algorithmic approaches, or different parameter values. Even when both scripts are correct, the results may differ in ways that are difficult to trace.

This non-determinism is not a bug — it is built into how LLMs work. Rather than computing a single deterministic answer, the model produces a ranked list of candidate next words and then *samples* from that list. A setting called **temperature** controls how much randomness is involved: low temperature makes the model stick closely to its top predictions, while higher values allow more variety. Additional settings like **top-k** and **top-p sampling** further control which candidates are considered. Because of this sampling process, the same prompt can produce different outputs on every run — much like rolling weighted dice. Even when randomness is turned down to its minimum and a fixed random seed is set, exact reproducibility is not guaranteed: the underlying math involves floating-point operations on GPUs that can

produce slightly different results depending on the hardware, parallelization, and even system load. Finally, commercial API providers routinely update models behind the same name — "GPT-4o" in January may behave differently from "GPT-4o" in June — adding another layer of variability invisible to the user.

These sources of non-determinism can be partially controlled but not eliminated. Most API providers offer temperature and seed parameters that reduce output variability. Researchers who need reproducibility should save the exact generated code (not just the prompt), pin library versions, and record the model name, version, and parameters used — treating the AI interaction as they would any other step in their methods. Where possible, using specific model version identifiers (e.g., `gpt-4o-2024-08-06` rather than `gpt-4o`) provides a more stable target, though even versioned endpoints may eventually be deprecated.

A subtler issue is false precision. AI-generated code may report cellular measurements to many decimal places — six-digit fluorescence intensities, sub-pixel centroid coordinates — creating an impression of precision that the underlying measurement does not support. Experienced analysts routinely round to appropriate significant figures and propagate uncertainty estimates; AI-generated pipelines rarely do so unless explicitly instructed.

3.8.3 Bias and Data Governance

The training data underlying language models carries systematic biases that affect their recommendations for microscopy. Published scientific literature skews toward positive results, well-funded institutions, and commercially dominant platforms. Documentation for expensive commercial microscopy systems is more extensively represented online than information about open-source or custom-built equipment, which can bias AI suggestions toward costly solutions — a concern for laboratories in resource-limited settings.

Privacy and data governance present practical challenges for any laboratory working with sensitive material. When using cloud-based AI agents, prompts and uploaded files are transmitted to external servers. Laboratories working with patient-derived samples, clinical trial data, or proprietary imaging techniques must consider whether sending this data to third-party services complies with institutional review board approvals, HIPAA, GDPR, or other applicable regulations. Local model deployments address these concerns but require institutional infrastructure that may not be available everywhere.

The economic costs of cloud-based AI processing also merit attention. API charges for processing a single high-resolution image through a frontier VLM are modest (a few cents), but high-content screening experiments involving tens of thousands of images can accumulate considerable costs. The energy consumption of large-scale AI inference also raises environmental questions that the scientific community is beginning to address.

⚠ The Validation Imperative

Every AI-generated protocol, analysis pipeline, and factual claim must be independently verified before use in research. AI tools augment human expertise; they do not replace it. When AI is used in a study, document the specific models, prompts, and validation steps in your methods section.

3.8.4 The Evolving Role of Bioimage Analysts

If AI agents can generate analysis pipelines, write code, and operate microscopes, what becomes of the bioimage analyst?

The role remains essential — but it is evolving. As AI makes advanced analysis accessible to more researchers, the demand for expert guidance on validation, interpretation, and experimental design is likely to increase rather than decrease. The job description is shifting: less "the person who runs CellProfiler" and more "the person who evaluates whether an AI-generated pipeline produces biologically meaningful results, recognizes when a segmentation has failed in subtle ways, and designs experiments amenable to rigorous quantitative analysis."

Microscopists bring domain knowledge that current AI systems lack entirely. They recognize sample preparation artifacts, notice when photobleaching is compromising a time-lapse, understand that a bright ring around a nucleus is an optical aberration rather than a biological structure, and judge whether a segmentation result is consistent with known cell morphology. The AILA benchmarking study [200] made this point explicitly: even frontier LLMs "struggle with basic experimental tasks." Fluent language generation does not confer experimental competence.

The real risk is not job displacement but skills atrophy — a gradual erosion of the community's ability to evaluate AI outputs. Consider a graduate student who uses an AI agent to build every analysis pipeline during their PhD, never learning how watershed segmentation works or why intensity thresholds behave differently on 8-bit versus 16-bit images. Five years later, as a postdoc, that researcher reviews a paper where the AI-generated segmentation has subtly failed — merging adjacent G1 and S-phase nuclei in a FUCCI experiment because the model was trained on well-separated cells. Without foundational understanding of how segmentation algorithms handle touching objects, this error is invisible. The researcher approves the analysis, and the flawed quantification enters the published literature.

Every field that has adopted automation faces this question. Aviation provides a well-studied parallel: as cockpit automation advanced, training programs had to be redesigned to ensure pilots could recognize and respond to automation failures — precisely because those failures became rarer and harder to detect when they did occur.

How Should the Field Prepare?

1. **Teach AI literacy alongside microscopy fundamentals** — understanding what an algorithm does is as important as knowing how to invoke it
2. **Develop community validation standards** for AI-generated analyses, building on efforts like the BioImage Model Zoo's metadata requirements
3. **Maintain human-in-the-loop checkpoints** for experimental decisions with irreversible consequences
4. **Preserve hands-on training** — a researcher who has never manually drawn a segmentation boundary cannot evaluate whether an automated one is correct

3.8.5 Taking Stock: What Works, What Is Emerging, What Remains Aspirational

The pace of development has been rapid, and it is worth distinguishing what the field can do today from where enthusiasm or marketing might place it.

Capabilities that are validated and in routine use. LLM-assisted code generation for analysis scripts works reliably when paired with human validation — agents produce functional Python pipelines, ImageJ macros, and napari scripts that accelerate development. Conversational AI assistants are effective for learning concepts, troubleshooting errors, navigating documentation, and drafting protocols (with verification). VLM-based image interpretation provides useful starting points for biological analysis. AI-assisted literature search is already part of many researchers' daily workflows.

Capabilities demonstrated in research but not yet routine. Autonomous microscope control has been demonstrated by EIMS [130] and SmartEM [211], showing that AI can plan and execute imaging protocols — though these systems have been validated only in narrow settings. AI agents that orchestrate multi-tool analysis pipelines function in frameworks like Omega and the BioImage.IO Chatbot but lack systematic benchmarking across diverse tasks. On-demand software generation Section 3.6 shows promise for changing how researchers interact with analysis tools. Multi-agent coordination of biomedical research workflows is being explored [176], and LLM-driven automation of complex experimental protocols has been demonstrated for specific applications such as CRISPR gene editing [253]. Closed-loop adaptive imaging experiments are feasible in specialized domains.

Capabilities that remain aspirational. Fully autonomous discovery — where AI formulates hypotheses, designs experiments, collects data, and iterates toward insight without human intervention — has been attempted in narrow settings but remains far from routine in biology. Systems that genuinely replace domain expertise in experimental judgment, perform causal reasoning from image data alone, or continuously learn and improve during deployment are active research topics, not available tools.

The trajectory of improvement has been faster than most observers expected, and capabilities are migrating between tiers at an accelerating rate. But honest assessment requires acknowledging what remains genuinely hard. Causal reasoning, scientific creativity, and the judgment that experienced researchers exercise intuitively — sensing that a result is "too clean," recognizing that an artifact resembles a biological signal — are not capabilities that current AI systems possess. These tools excel at pattern recognition and workflow orchestration. They do not perform the lateral thinking that drives scientific discovery.

3.8.6 Preparing for an AI-Enabled Future

As these capabilities mature, the microscopy community faces practical decisions that will shape whether AI tools are adopted responsibly and equitably.

Training programs must evolve. AI literacy — understanding what these tools can and cannot do, how to evaluate their outputs, and when human judgment must override — belongs alongside optical physics, sample preparation, and experimental design in microscopy curricula. At the same time, foundational training must not be displaced: the ability to understand *why* a segmentation algorithm works, not merely how to invoke it through an agent, remains essential for catching errors.

Infrastructure investments matter. Open-source tools, open protocols like MCP, and community-driven resources like the BioImage Model Zoo and AI4Life ensure that AI capabilities remain accessible across institutions with different levels of funding — preventing a widening gap between well-resourced and under-resourced laboratories.

Publication and funding policies need updating. Journals and funding agencies should establish clear expectations for documenting AI tool usage in methods sections: which models were used, what prompts were given, how outputs were validated. These standards should emerge through community consultation, recognizing that both the technology and best practices are evolving rapidly.

3.9 Conclusion

This chapter has traced a shift in what AI can do for microscopists — from task-specific deep learning models that segment cells or denoise images within narrow training distributions, through foundation models that generalize across imaging modalities, to Large Language Models and AI agents that change the nature of the interaction itself. Where earlier tools required researchers to learn specialized software interfaces and write analysis code, LLMs and agents allow them to describe their intent in natural language and have the system handle tool selection, code generation, execution, and iteration.

This is a genuine change in how microscopy analysis can be conducted, but it is not the change that breathless commentary sometimes suggests. AI agents do not understand biology. They

do not design experiments. They cannot judge whether a segmentation result makes sense given what is known about the sample. The AILA benchmarking study [200] found that even frontier LLMs "struggle with basic experimental tasks" — a finding that should temper expectations about autonomous AI-driven research. What these tools do well is orchestrate existing computational resources, lower barriers to accessing validated analysis methods, and accelerate the cycle of writing code, testing it, and refining it. They are powerful amplifiers of human expertise, not replacements for it.

The practical value depends on how these tools are used. A researcher who treats AI-generated output as a draft to be evaluated — inspecting segmentation overlays, verifying citations, questioning parameter choices — gains a capable assistant. A researcher who accepts outputs uncritically risks introducing systematic errors that are harder to detect precisely because they are embedded in professional-looking code and fluent prose. The validation practices discussed throughout this chapter are not optional safeguards; they are integral to responsible use.

Several open questions will shape how this field develops. Can the community establish validation standards for AI-generated analyses that are practical enough to be widely adopted? Will training programs evolve to teach AI literacy alongside microscopy fundamentals, or will foundational skills erode as automation makes them seem unnecessary? Will open protocols and community infrastructure keep these capabilities accessible to laboratories worldwide, or will they concentrate in well-funded institutions? These are not technical questions — they are choices the community is making now through its investments, curricula, and publication standards.

For readers looking to go deeper: Chapter 4 introduces the neural network architectures underlying the models discussed here, Chapter 9 provides guidance on training and fine-tuning models for specific tasks, and Chapter 10 offers frameworks for evaluating whether AI-generated analysis outputs meet the quality standards that scientific research demands.

AI Disclosure

AI language models (Anthropic's Claude, specifically Claude Opus 4.6 and Claude Sonnet 4.5) were used during the preparation of this chapter. Their roles included literature search, drafting and revising prose, generating code examples, and producing demonstration figures. All content was reviewed, revised, and verified by the human authors, who take full responsibility for the accuracy and scientific integrity of the final text. Factual claims were checked against primary sources; all cited references were verified to exist and to support the claims attributed to them (verification completed March 2026). The demonstration scripts in `3-llms-assets/` were authored with AI assistance and tested by the authors. Given the rapid pace of AI development, specific model versions, benchmark results, and tool capabilities described in this chapter may have changed since publication.

4 Architectures and Loss Functions

Carsen Stringer & Marius Pachitariu
HHMI Janelia Research Campus

The problems faced by biologists differ in multiple ways from traditional computer vision problems. For example, object detection in computer vision typically involves only a few objects per scene, e.g., determining the approximate locations of cars or pedestrians in a self-driving car application. On the other hand, biologists may need to detect hundreds or thousands of cells in a single image, often with precise segmentations of cell boundaries. Because of these differences, the best algorithms for biological problems might not be the same as those used in standard computer vision. In this chapter, we introduce and describe several neural network architectures and loss functions that are especially useful for identifying and outlining objects in images (i.e., segmentation) for biological applications. In subsequent chapters, these concepts will be applied to other biological problems besides segmentation. For a deeper dive into computer vision topics we recommend several resources in Section 4.4.

4.1 Neural network architectures

In computer vision, multiple neural network architectures were designed for various visual tasks. These architectures take as input an image, which often has multiple channels—in the case of natural images there are three input channels for red/green/blue (RGB). The neural networks process the image and output various quantities depending on the task. For example, in the case of object recognition, the networks output a single probability vector to indicate the likelihood that each possible object (cat, dog, etc.) is in the image. As an example, we illustrate AlexNet [162], the first neural network to achieve high performance at object classification. This neural network takes as input an RGB image, applies filters to the image, downsamples the image (reducing the size of the image), and then repeats this process several times, in the end producing an object category label.

In the case of biological problems, however, it is often necessary for the output to be the same size as the input—pixel-level labels—rather than a single object category label. For example, for cell classification, each pixel in the output may represent the cell class of the pixel. A common architecture for this is a U-Net[265], which applies filters and downsamples like AlexNet, and then additionally applies filters with upsampling, in the end producing an output at the same pixel resolution as the input.

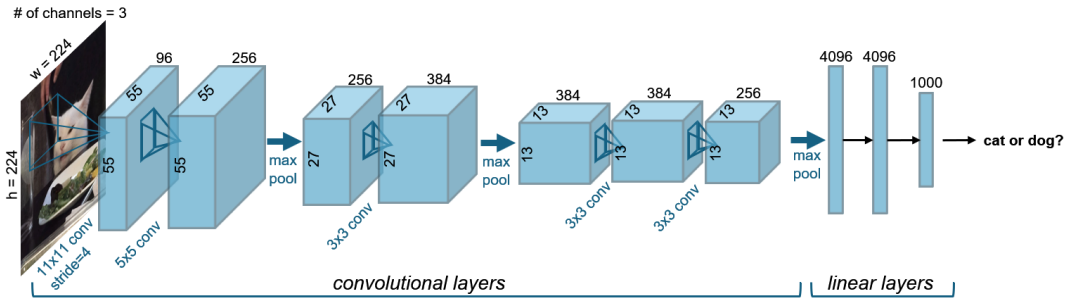


Figure 4.1: AlexNet architecture, adapted from [162]. This early neural network first demonstrated the capabilities of deep learning when trained on large datasets.

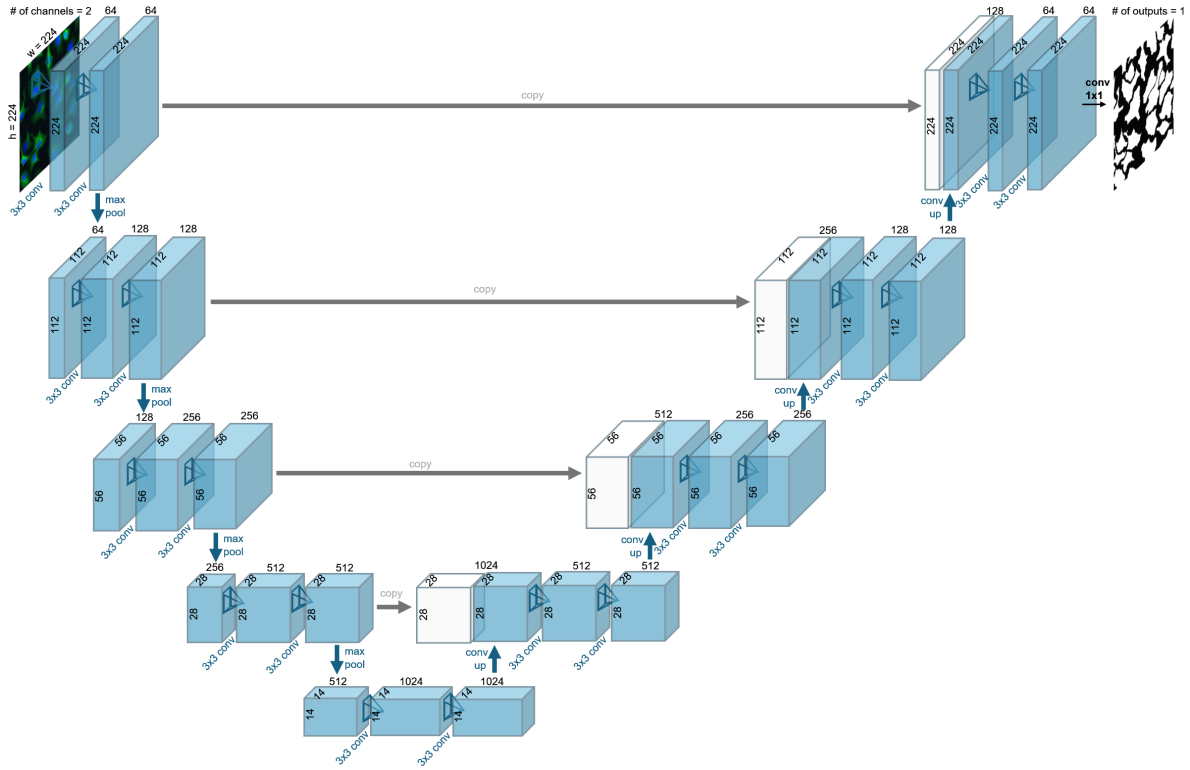


Figure 4.2: U-net architecture, adapted from [265] and described further in Section 4.1.3. This neural network was revolutionary for biological analysis, in part due to the image-sized outputs and dense skip connections between layers on the downsampling and upsampling passes.

4.1.1 Linear layers (aka fully-connected layers or dense layers)

Early research focused on the computational properties of perceptrons, defined as linear weighted sums of inputs, followed by a nonlinear activation function [270]. In other words, a vector of features ($\vec{x}$) is the input: they are multiplied by a weight ($\vec{w}$) specific to each feature , summed together, and then put through a nonlinear function. An example of a feature set may be a summary of cell-based quantifications in an image—e.g., how many cells are oblong or the fraction of cells with specific RNA expression—and the output may represent the phenotype of the cells in the image—e.g., cancerous or non-cancerous.

A neural network layer consists of a collection of such perceptrons, each with their own input weights; when these vectors of weights for each perceptron are put together, they create a weights matrix W. Another term often added to these computations is a bias value, and there is one term for each output, creating a vector $\vec{b}$. This additive term provides flexibility for the layer to produce non-zero outputs, even when the inputs are zero. This is summarized by Equation 4.1, where m is the number of inputs and n is the number of outputs.

$$W = \begin{bmatrix} W_{11} & \cdots & W_{1m} \\ \vdots & \ddots & \vdots \\ W_{n1} & \cdots & W_{nm} \end{bmatrix}, \quad \vec{b} = \begin{bmatrix} b_1 \\ \vdots \\ b_n \end{bmatrix}, \tag{4.1}$$

The output of the linear layer is given by Equation 4.2.

$$\vec{y} = W\vec{x} + \vec{b} = [\sum_{j=1}^{m} W_{ij}x_j + b_i]_{i=1}^{n}. \tag{4.2}$$

Each of the elements in $\vec{x}$ and $\vec{y}$ is also called a neural network unit. The values of these elements, which are dependent on the specific inputs to the layer, are called activations.

The weights matrix allows any combination of inputs to be used to create each output. This is why these layers are also called "fully-connected", because there is a weight value between each input and output feature. They can also be referred to as "dense" layers because the connectivity is not sparse.

The activation function (f) is the nonlinearity applied to each output of the linear layer, such as a ReLU nonlinearity which sets the minimum value of the output to zero (Equation 4.3). Including a nonlinearity allows the network to compute more complicated functions of the input than would be possible with a simple linear model [69]. In practice, this is essential—when processing microscopy data, we are performing a complex, nonlinear transformation of the pixel intensities to variables like cell type.

$$y = f(z) = \begin{cases} z, & \text{if } z \geq 0 \\ 0, & \text{otherwise} \end{cases} \tag{4.3}$$

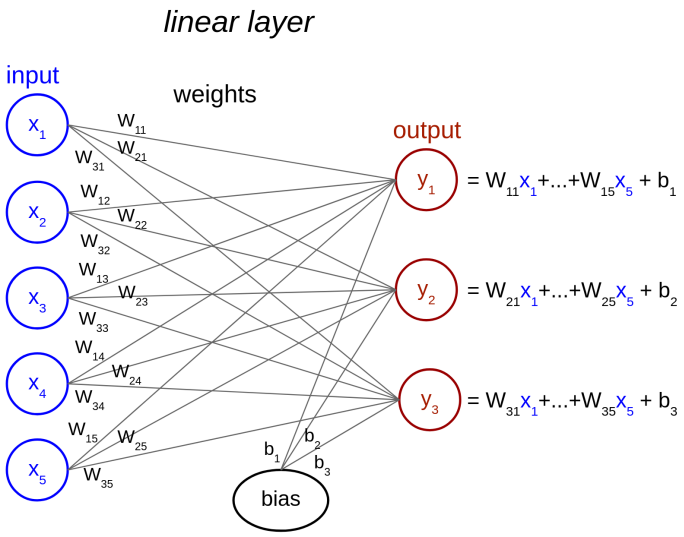


Figure 4.3: Illustration of a linear layer with five input features and three output features.

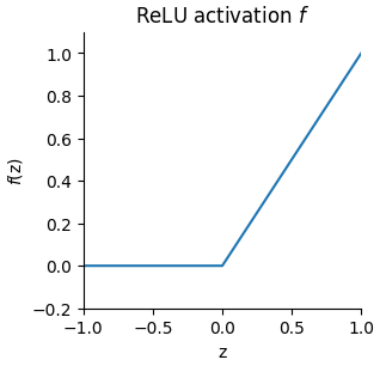


Figure 4.4: ReLU nonlinearity. This is the most-used activation function in neural networks, sometimes with small modifications.

A sequence of such layers that are applied in series, each on the output of the previous one, is called a multilayer perceptron (MLP) (see Figure 4.5). This is the simplest example of a deep neural network, and we illustrate the code below.

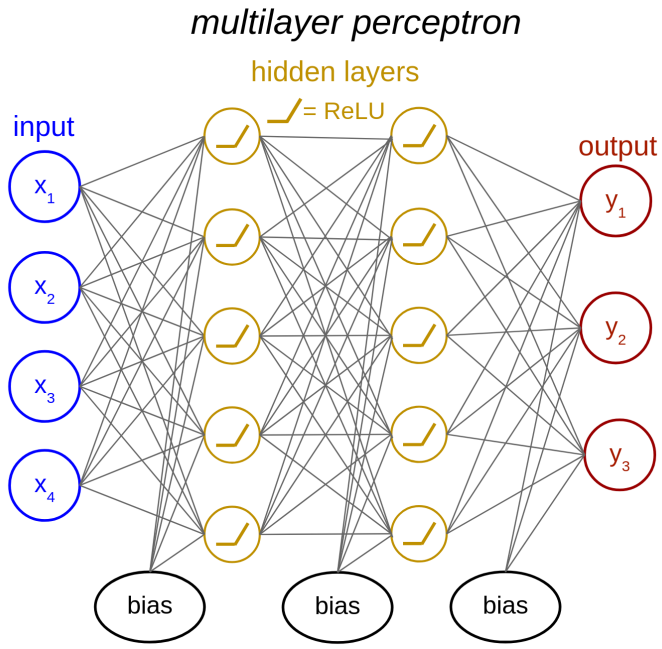


Figure 4.5: Illustration of a multilayer perceptron (MLP) with two hidden layers.

For images, we note that dense linear layers are not efficient in the number of parameters and computations they perform. To see why, consider a modest-sized input image of 100 by 100 pixels with 3 channels. In this case, there are 30,000 input values. If the output of the layer is the same size as the input, then there are 30,000 output values. The weights matrix then has size 30,000 by 30,000 and the bias vector has length 30,000. This is almost a billion parameters to fit, and would lead to overfitting even with a large amount of training data.

Multi-Layer Perceptron for a 100 x 100 pixel image

Here we show a code example for a multi-layer perceptron with 30,000 input values, a hidden layer with 30,000 units, and an output layer with one output. The code is run on an example of 8 input images, and outputs the sizes of the input, output, weights matrix (W), and bias vector (b).

```
import torch
from torch import nn
from torch.nn import functional as F
```

```
class MLP(nn.Module):
    """ Network with one hidden layer

    Args:
        n_inputs (int): number of input units
        n_hidden (int): number of units in hidden layer

    Attributes:
        in_layer (nn.Linear): weights and biases of input layer
        out_layer (nn.Linear): weights and biases of output layer

    """

    def __init__(self, n_inputs, n_hidden):
        # needed to invoke the properties of the parent class nn.Module
        super().__init__()
        # input units --> hidden units
        self.in_layer = nn.Linear(n_inputs, n_hidden)
        # hidden units --> output
        self.out_layer = nn.Linear(n_hidden, 1)

    def forward(self, X):
        """ Input images and output label (e.g. 0 for cat / 1 for dog)

        Args:
        X (torch.Tensor): input image (flattened), must be of
            length n_inputs. Can also be a tensor of shape n_images
            x n_inputs, containing n_images of image vectors

        Returns:
        torch.Tensor: network outputs for each input provided in X of
        length n_images.

        """
        z = self.in_layer(X)  # hidden representation
        z = F.relu(z)
        y = self.out_layer(z)
        return y

# declare network
net = MLP(n_inputs=30000, n_hidden=30000)
```

```
print('shape of W in first layer: ', net.in_layer.weight.data.shape)
print('shape of b in first layer: ', net.in_layer.bias.data.shape)

# define input images
n_images = 8
X = torch.zeros((n_images, 30000))

# evaluate network
net.eval()
with torch.no_grad():
    y = net(X)

print('input shape: ', X.shape)
print('output shape: ', y.shape)
```

```
shape of W in first layer:  torch.Size([30000, 30000])
shape of b in first layer:  torch.Size([30000])
input shape:  torch.Size([8, 30000])
output shape:  torch.Size([8, 1])
```

4.1.2 Convolutional layers

Instead of linear layers, convolutional layers are often used in vision tasks, as a parameter-efficient alternative [172]. A convolutional layer slides a small two-dimensional filter across the input image, computing a weighted sum of the input at each image position using the filter weights. It also includes the addition of a vector of bias terms. For 2D image processing, we use two-dimensional convolutions, but one-/three-dimensional convolutions may be used for 1D/3D data respectively. Here is an illustration of a 2D convolution:

> **Convolution vs Filtering**
>
> Note that "convolution" and "filtering" are often conflated in neural network terminology, whereas they refer to distinct operations in the signal processing and mathematical literature.

The 2D convolutional filter W is often also called a kernel, and the size of the filter is called the *kernel size*. To preserve the size of the input, the kernel size needs to be odd and the input needs to be padded with zeros around the edges of the image. This padding size should be the floor of half the kernel size, e.g., if the kernel size is 3 the padding is required to be 1, as shown in Figure 4.7. Additionally, the stride parameter—the number of pixels between each application of the filter—is set to one in this example. A stride of 2 would mean skipping over

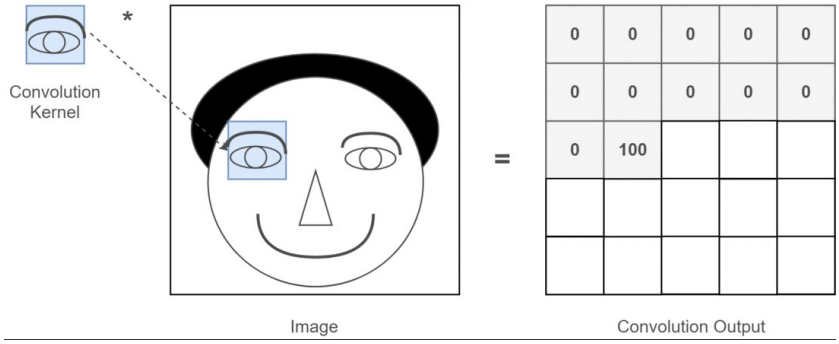


Figure 4.6: Toy illustration of convolutional operation from Towards Data Science. In practice, convolutional kernels are more complicated than simple template detectors.

every other input position both vertically and horizontally. In most applications, especially with small kernel sizes, a stride of 1 is used for convolution.

The number of input and output images is called channels, similar to the three red/green/blue channels for RGB images. Figure 4.7 is an example with a single input and a single output channel, but in general a 2D convolutional layer has multiple input and output channels. Each output channel is the result of a 2D convolutional kernel applied to the input, with weights for each input feature.

A convolutional layer operates under two main assumptions: 1) the computation only requires local features that are within the spatial extent of the filter operation; and 2) it is not necessary to perform different computations at different positions in the image, and thus the same filter operation can be convolutionally applied across all positions in the image. When these assumptions are acceptable, a convolutional layer can reduce the number of parameters substantially compared to linear layers. We note that for certain microscopy tasks, the amount of spatial context required may vary—for example determining the boundary of a tissue may require a larger context than a task like nuclear segmentation.

Taking our example from above, let's estimate the number of parameters with filters (i.e. kernels) of size 3 by 9 by 9 pixels, where 3 is the number of input channels and 9 is the size in pixel space (xy), i.e., the "kernel size". If we define the layer to have 6 output channels, this requires 6 of these kernels, resulting in 1458 parameters in the kernels, along with 6 bias terms, resulting in 1464 parameters in total. As you can see, the number of parameters is now independent of the size (in pixels) of the input, and this is a dramatic reduction from the nearly 1 billion parameters for the dense linear layer example above.

Convolutional Network Example Code

Here we show a code example for a convolutional layer with 3 input channels, 6 output channels and a kernel size of 9.

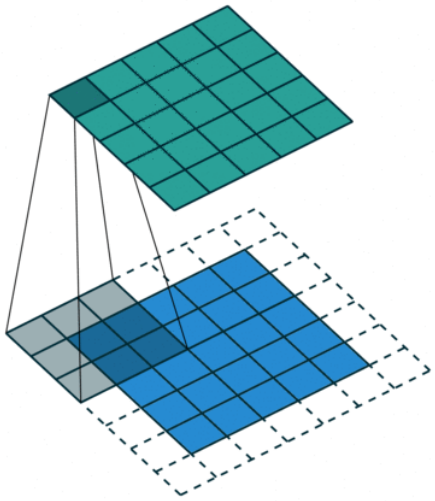


Figure 4.7: Convolutional operation with padding from conv_arithmetic. This illustrates the creation of a single channel output from a single channel input, where the input is in blue, the filter is gray, and the output is in green. In practice, both inputs and outputs have multiple channels and all combinations of input and output need to be calculated and summed accordingly.

```
class ConvolutionalLayer(nn.Module):
    """Deep network with one convolutional layer
        Attributes: conv (nn.Conv2d): convolutional layer
    """
    def __init__(self, c_in=3, c_out=6, K=9):
        """Initialize layer

        Args:
            c_in: number of input stimulus channels
            c_out: number of output convolutional channels
            K: size of each convolutional filter

        """
        super().__init__()
        self.conv = nn.Conv2d(c_in, c_out, kernel_size=K,
                              padding=K//2, stride=1)

    def forward(self, X):
        """Run images through convolutional layer
```

```
        Args:
            X (torch.Tensor): n_images x c_in x h x w tensor with stimuli

        Returns:
            (torch.Tensor): n_images x c_out x h x w tensor with
            convolutional layer unit activations.

        """
        Y = self.conv(X)  # output of convolutional layer

        return Y


# declare layer
layer = ConvolutionalLayer(c_in=3, c_out=6, K=9)
print('shape of filter W: ', layer.conv.weight.data.shape)
print('shape of b: ', layer.conv.bias.data.shape)

# define input images
n_images = 8
X = torch.zeros((n_images, 3, 100, 100)) # n_images x c_in x h x w

# evaluate network
layer.eval()
with torch.no_grad():
    y = layer(X)

print('input shape: ', X.shape)
print('output shape: ', y.shape)
```

```
shape of filter W:  torch.Size([6, 3, 9, 9])
shape of b:  torch.Size([6])
input shape:  torch.Size([8, 3, 100, 100])
output shape:  torch.Size([8, 6, 100, 100])
```

We define the receptive field size of a convolutional layer as the total spatial extent of pixels that influence the activations of the layer. For a single convolutional layer, the activations have a receptive field size equivalent to the kernel size: each activation only receives information from pixels within the kernel window. However, objects in images are often larger than the kernel size. To increase the amount of spatial information used for computation, pooling layers are introduced between convolutional layers. A pooling layer performs operations in sliding

windows across the image, just like the convolutional layer, but in this case a maximum or average operation is computed within the window, independently for each input channel (rather than the application of a filter). To reduce the size of the image, this convolutional operation is applied with a "stride": to downsample the image by a factor of two, we may use a pooling size of 2 with a stride also set to 2, like in the example below.

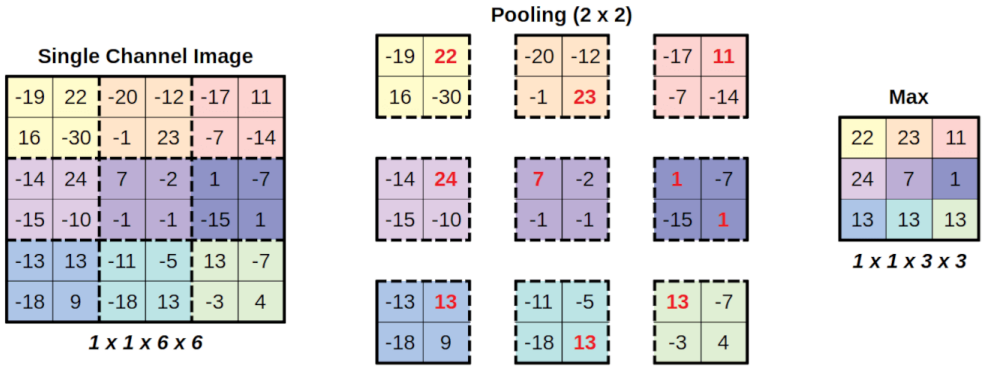


Figure 4.8: Illustration of max-pooling with a kernel size of 2 and stride of 2, from PyTorch-StepByStep.

Max-Pooling Example Code

This example code implements max-pooling with a kernel size of 2 and a stride of 2.

```
# max pooling operation
pool_layer = nn.MaxPool2d(kernel_size=2, stride=2)
y = pool_layer(X)

print('input shape: ', X.shape)
print('output shape: ', y.shape)
```

```
input shape:  torch.Size([8, 3, 100, 100])
output shape:  torch.Size([8, 3, 50, 50])
```

If we apply several convolutional and pooling layers, we end up with an output that is smaller in resolution than the input. In Figure 4.9, we show an example of such a network, that has been trained to classify the handwritten digits of the MNIST dataset [171]. The activations of the units in the network across layers are visualized:

Having an output that is downsampled relative to the input is appropriate for a task like object recognition, where the output is the class of the image, like cat or dog. In the case of biological images, this could be a global image label of a cancerous or non-cancerous phenotype [301].

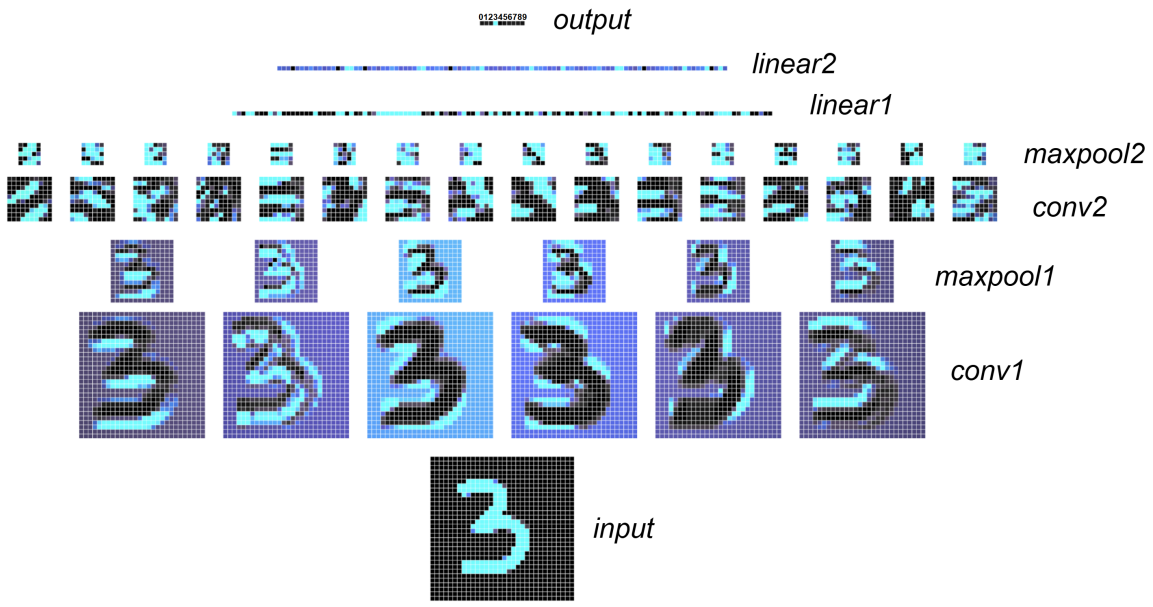


Figure 4.9: Activations of each layer of a small, illustrative network. Lighter blue indicates a higher activation for each unit. The final output layer represents the probability of the input image corresponding to a value between 0 and 9. In this example, the network correctly identifies the input as the number 3. To see this in action, check out the interactive demonstration by Adam Harley: https://adamharley.com/nn_vis/cnn/2d.html

However, for pixel-specific classification or segmentation, we require the output to be the same size as the input. This is where u-nets come in.

4.1.3 U-nets

U-nets were introduced by Ronneberger, Fischer, and Brox [265] and are one example of a convolutional neural network (CNN) (Figure 4.2). They share some similarities with feature pyramid networks [181], but u-nets are more frequently used in biological applications, so we will focus on them. U-nets have what is called an encoder-decoder structure, like an autoencoder [123]. The "encoder" consists of the convolutional layers and pooling layers (downsampling), and is the first half of the network. The "decoder" consists of convolutional layers and upsampling or strided transposed convolution layers ("conv-transpose"). This is the second half of the network, which returns an output at the same spatial resolution as the input.

In the case of an autoencoder [123], the network is trained to reproduce the input, and the output of the encoder is defined to be lower-dimensional than the input. The output of the encoder is then a compressed version of the inputs that can be used to visualize the data. We can also visualize the output of the encoder of a u-net, although u-nets are usually not trained to predict their inputs. Instead, u-nets are often trained to predict pixel-level cell probabilities, with pixels inside cells defined as one and pixels outside cells defined as zero.

Downsampling results in a loss of fine spatial information. To recover this information, in the u-net the output of the convolutional layers in the encoder is concatenated with the activations from the decoder at each spatial scale using skip connections ("copy" operation in Figure 4.2). This preserves the higher resolution details, which is important for precise segmentations and pixel-wise predictions.

Conventional u-nets [265] have two convolutional layers per spatial scale and a small kernel size of 3 in each layer. Each of these sets of convolutional layers per spatial scale is often referred to as a block. The downsampling after each block is often set to a factor of 2. Because the kernel size is small, the only way to have large receptive field sizes is through several downsampling blocks. If we want the network to learn complicated tasks across many diverse images, then we need it to have a large capacity. This can be achieved by adding more weights to the network, for example by increasing the number of channels in the convolutional layers and/or by adding more convolutional layers in each block and/or by increasing the number of downsampling and upsampling stages [305].

4.1.4 Vision transformers

Vision transformers are modern architectures that are replacing convolutional networks in many applications. They are not as parameter-efficient as convolutional neural networks. For

example, the Cellpose segmentation u-net has 6.6 million parameters while ViT-H ("vision-transformer-huge") has 632 million parameters [78]. They are still much more efficient than dense linear layers due to special architecture choices (see below), and they introduce a new type of operation called self-attention. Transformers avoid overfitting this large set of parameters through training on very large amounts of data. Even though they have many more parameters than standard convolutional networks, they are not too much slower because most of the operations within the transformer are matrix multiplications which are fast on newer GPUs with tensor cores (learn more in this explainer from TechRadar). With more parameters, they have a larger capacity than standard convolutional neural networks to learn from large training datasets.

The vision transformer first divides the input image into patches, e.g., 16 by 16 pixels each. In the first layer of the transformer the patches are transformed into the embedding space, using a linear operation that is often implemented using strided convolutions. This embedding space is generally several times larger than the number of pixels in the patch; for example the embedding space in ViT-H is 1280. If an image is 512 by 512 pixels, then it has 32 by 32 (=1024) patches, so the patch embeddings are of dimensionality 1024 patches by 1280 feature dimensions in the case of ViT-H.

These patch embeddings are input to the transformer encoder, which consists of many blocks (L). Each transformer block has a self-attention block and a multi-layer perceptron (MLP) block. The input patches to each self-attention block are multiplied by three sets of learned feature vectors, which create the queries (Q), keys (K), and values (V) input to the self-attention block. The queries and keys are multiplied across patches, and a normalization operation is applied—this is the attention matrix, where each entry (i, j) is the interaction between patches i and j. Because this matrix is across all patches in the image, this operation enables sharing of information across the entire image. The attention matrix is then multiplied with the values matrix, and the result of this operation, which is of size equal to the number of patches by the number of features, is fed into the MLP. This MLP acts on the feature dimension, applying the same operation to each patch, and often consists of two layers, with a nonlinearity in the hidden layer only. Residual connections are added to both the attention and MLP operations, in order to speed up training.

Vision transformers were first trained on image classification tasks but have been extended for various visual tasks like segmentation. A notable example is the Segment Anything model (SAM) [158]. This model was originally trained and designed for natural image datasets, but has since been adapted for cellular segmentation by multiple groups [8, 202]. The Segment Anything model is considered to be a foundation model: it can generalize well to new images and be used for a variety of visual tasks in addition to segmentation.

SAM is trained as a promptable model, meaning a user can specify an object for segmentation with a click or a bounding box and the model will produce a predicted object in that region. However, we often want automated segmentations in which we do not have to click on everything in the image. Thus, multiple groups have adapted the SAM decoder to predict auxiliary variables, which we will discuss next as we learn about loss functions. Other innovations in transformers

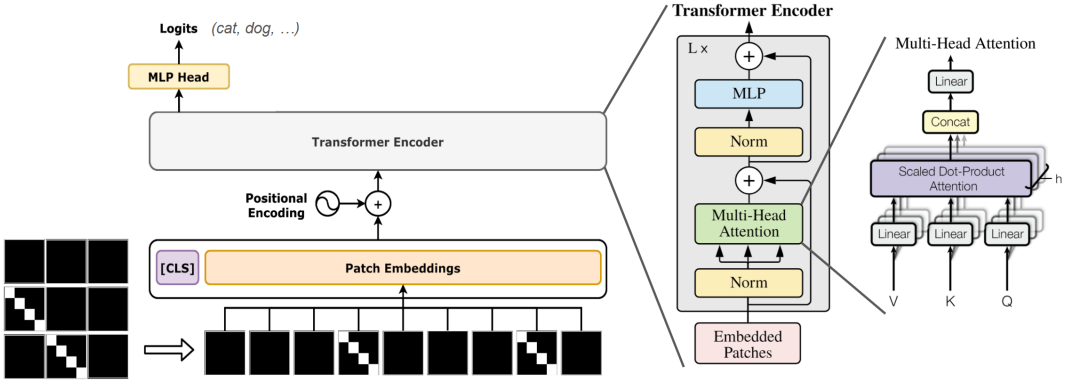


Figure 4.10: Vision transformer architecture. Originally invented for language modeling [317], transformers were then adapted for image processing [78]. The critical innovation in transformers is the self-attention block (left illustration from PyTorchStepByStep).

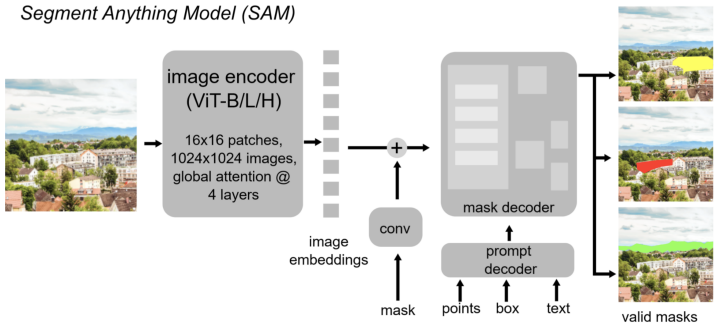


Figure 4.11: Segment Anything model [158]; illustration from [238]. This architecture differs from a standard transformer in the decoding module, which is necessary for outputting segmentations of images. This model requires inputs in the form of point clicks, box outlines or textual instructions.

have adapted the architecture to be more suited to the spatial layout of images, such as the Swin transformer [188], which has also been adapted for biological applications [353, 1].

4.2 Loss functions

In a standard image classification network, the output is a vector with length equal to the number of classes in the task. Each of the entries in this vector represents the predicted probability of the class, and the predicted label for the image is chosen as the index of the vector with the largest entry. For each training image we have a ground-truth label for the class. How closely the network matches the label is the loss, which is defined as a function between the vector output of the network and the ground-truth label. A lower loss means we matched the ground-truth data better. The gradients of the network are computed automatically via back-propagation, and an optimizer is specified to modify the parameters in order to minimize the loss (described more below).

For classification, we are evaluating predicted probabilities, so we need to take the output from the network and convert it to a probability that sums to one across classes. For this, a softmax (or its cousin the sigmoid) operation is performed, defined as $p_c(x) = e^{x_c}/(\sum_{i=1}^{C} e^{x_i})$. The cross-entropy loss is a standard loss function for classification, and for comparing probability distributions generally. This loss maximizes the predicted probability of the true class, achieving its minimum value of 0 when the probability of the true class is predicted to be one (Equation 4.4).

$$\ell(x, y) = -\sum_{c=1}^{C} \log[p_c(x)]y_c. \tag{4.4}$$

In segmentation and biological classification tasks, as mentioned before, the output is often the same size as the input in pixels and the loss is computed per-pixel. There will thus be multiple outputs of this size, each one corresponding to a class, like the entries in the vector for overall image classification. In the original u-net paper, the loss was defined using two classes, "not cell" as zeros and "cell" as ones [265].

In PyTorch, the sigmoid and cross-entropy loss are combined into a single function; for two-class prediction the function is `nn.BCEWithLogitsLoss`, and for multi-class prediction the function is `nn.CrossEntropyLoss`.

> **Cell/Not-Cell Prediction**
>
> Here is example code for cell/not-cell prediction with one convolutional layer.

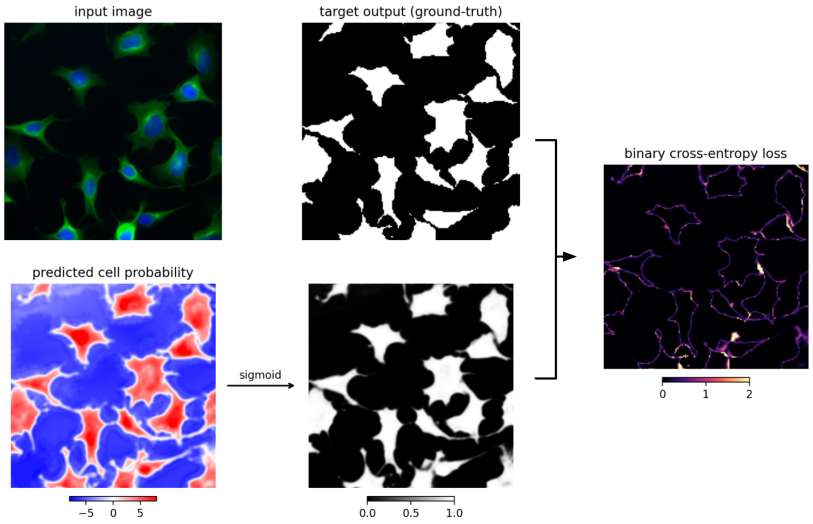


Figure 4.12: An image (upper left) is input to a u-net trained to predict cell/not cell pixels. The output of the network is the predicted cell probability image (bottom left). A sigmoid is applied to the cell probability (middle bottom), and compared to the ground-truth cell probabilities (middle top). The comparison is performed using the binary cross-entropy loss per-pixel (right).

```
# declare layer
layer = ConvolutionalLayer(c_in=3, c_out=1, K=9)
print('shape of filter W: ', layer.conv.weight.data.shape)
print('shape of b: ', layer.conv.bias.data.shape)

# define input images
n_images = 8
X = torch.zeros((n_images, 3, 50, 50))
# define random cell probabilities
y_true = (torch.randn((n_images, 1, 50, 50)) > 0.5).float()

# define loss function
loss_fn = nn.BCEWithLogitsLoss()

# compute network prediction and loss
y = layer(X)

print('input shape: ', X.shape)
print('output shape: ', y.shape)
print('target shape: ', y_true.shape)

# evaluate loss function
loss = loss_fn(y, y_true)

print('loss = ', loss.item())
```

```
shape of filter W:  torch.Size([1, 3, 9, 9])
shape of b:  torch.Size([1])
input shape:  torch.Size([8, 3, 50, 50])
output shape:  torch.Size([8, 1, 50, 50])
target shape:  torch.Size([8, 1, 50, 50])
loss =  0.6866452693939209
```

To create segmentations for each cell, a threshold is defined on the cell probability and any pixels above the threshold that are connected to each other are formed into objects. This threshold is defined using a validation set—images that are not used for training or testing—to help ensure the threshold generalizes to the held-out test images. The predicted segmentations with this loss function often contain several merges, because cells can often touch each other and the connected components of the image will combine the touching cells into single components.

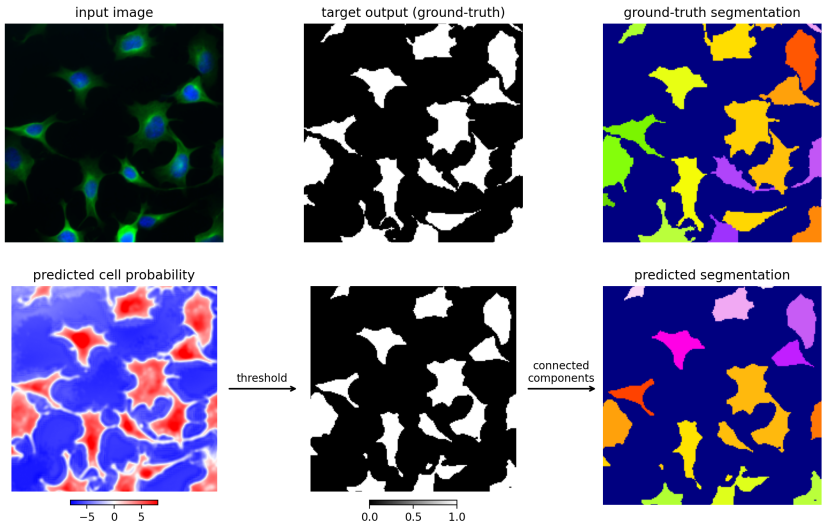


Figure 4.13: Segmentation with pixel-wise cell/not cell predictions.

To better train the network to discriminate boundaries, a boundary pixel class can be introduced, creating a three-class prediction loss function. This can improve segmentation performance, but the segmentations can still contain merges. This is illustrated in the tutorial notebook.

The training signals for the network so far have consisted only of class labels. However, it can be helpful to ask the network to predict more complex information about the segmentation. For example, the distance to the boundary can be used to provide context about the local shape of the object [117]. The distance-to-boundary is computed for each pixel in each object, and the network is trained to predict this, in addition to a multi-class loss like cell/not-cell. Distance-to-boundary is not a class but a continuous variable. In this case, the loss function often used is a mean-squared error between the ground-truth ($\tilde{Y}$) and the predicted values (Y), as shown in Equation 4.5.

$$\ell_{\text{MSE}} = \sum_{ij} (Y_{ij} - \tilde{Y}_{ij})^2. \tag{4.5}$$

StarDist[287] extends this approach by computing the distance-to-boundary along multiple fixed rays emanating from each pixel in an object, which can then be used for reconstruction of convex objects [287]. Cellpose creates a different type of representation from gradients indicating the direction towards the cell center, and tracks the gradients to reconstruct the segmented objects from the fixed points of the dynamics [305]. This representation helps prevent merges by having strong differences in gradients in boundary areas between two objects, and is sufficiently flexible to be used for non-convex cellular shapes. Other examples of auxiliary variables include local shape descriptors (LSDs), which include other global features like the size of the object, which can be useful for electron microscopy segmentation of neurons with long processes [293].

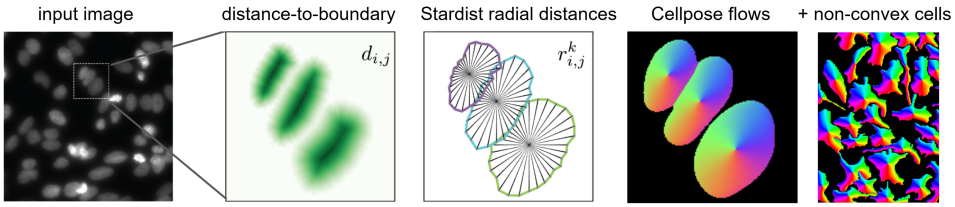


Figure 4.14: Segmentation networks are often trained to predict an auxiliary variable. For example, they may predict the distance from the cell boundary, radial distances (as in StarDist [287]), or gradient flows (as in CellPose [305]). The calculation of gradient flows is useful for non-convex cells (right panel).

4.3 Training neural networks

Now that we have defined a loss function, we want to minimize the loss ℓ by updating the weights in the network. For a problem like segmentation, we will need images with ground-truth segmentation labels—this labeling can be done using tools like Ilastik, ImageJ, Paintera, or Napari. Once ground-truth labeling is performed on some images, training can be attempted.

We will want to use most of the ground-truth labeled images for training, making up a training set, and leave a small subset (like 10%) for testing the performance of the network. For some algorithms, we may also need a validation set for setting post-processing parameters like the cell probability threshold, in which case we can reserve 10-15% of the training set images for validation. In other words, we use the validation set to determine the threshold on the cell probabilities that produces the best segmentations on the ground-truth labels. See Chapter 5 for more details on this.

During optimization, we run the training images through the network, compute the loss, and compute the weight updates that minimize the loss via gradient descent. Gradient descent computes the gradient of the loss with respect to each weight. Moving the weights in the negative direction of the gradient reduces the loss for the given images or data points over which the loss is computed. These weight updates are scaled by the learning rate, α (Equation 4.6).

$$\begin{aligned} \vec{v}_t &= \alpha \, dL(\vec{w}_t)/d\vec{w}_t \\ \vec{w}_t &= \vec{w}_{t-1} - \vec{v}_t \end{aligned} \tag{4.6}$$

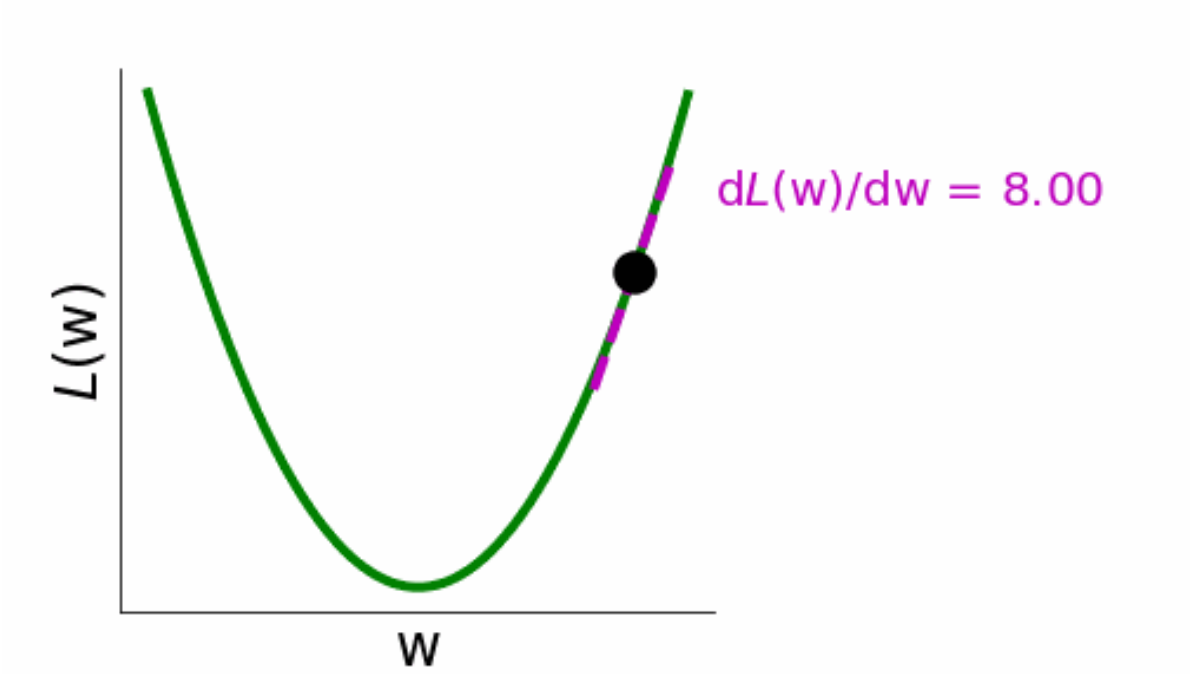


Figure 4.15: Gradient descent illustration. Image from Neuromatch.

We could compute the loss and gradients over all images in the training set, but this would take too long, so in practice the loss is computed in batches of a few to a few hundred images – the number of images in a batch is called the *batch size*. The optimization algorithm for updating the weights in batches is called stochastic gradient descent (SGD). This is often faster than full-dataset gradient descent because it updates the parameters many times on a single pass through the training set (called an epoch). Also, the stochasticity induced by the random sampling step in SGD effectively adds some noise in the search for a good minimum of the loss function, which may be useful for avoiding local minima.

It can also be beneficial to include momentum to speed up training, with some value β between zero and one. Momentum pushes weight updates along the same direction they have been updating in the past. The updated version of $\vec{v}$ in this case is shown in Equation 4.7, where in

standard SGD, $\beta = 0$ (i.e. no momentum). In other words, past weight updates slowly decay over optimization steps (at rate β), and new weight updates are added with size α.

$$\vec{v}_t = \beta \vec{v}_{t-1} + \alpha \frac{dL(\vec{w}_t)}{d\vec{w}_t} \tag{4.7}$$

Different weights in the network may have differently scaled gradients, and thus a single learning rate may not work well. The Adam optimizer uses a moving average of both the first and second moments of the gradient for rescaling the weight updates while including a momentum term [156]. This optimizer works better than standard SGD in many cases, and requires less fine-tuning to find good hyperparameter values. In addition to using an optimizer like Adam, it may be helpful to use a learning rate schedule that reduces the learning rate towards the end of training to enable smaller steps for fine-tuning the final weights [192]. Sometimes a validation set is used to re-instantiate the best weights, as evaluated on the validation set, before a decrease in the learning rate [246].

During fitting it is important to monitor the training loss and the validation loss. With an appropriate learning rate that is not too large, the training loss should always decrease. The loss on held-out validation data should also ideally decrease over training. If not, then the network is overfitting to the training set: the weights are becoming specifically tuned for the training set examples and no longer generalize to held-out data.

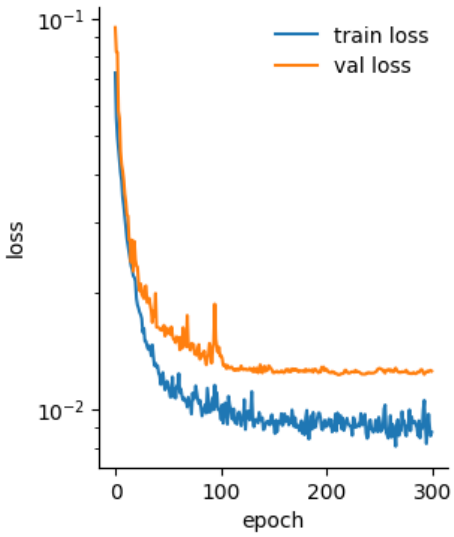


Figure 4.16: Example training loss and validation loss across epochs.

To avoid overfitting to the training set, various weight regularization strategies are used. Most commonly in computer vision problems, weight decay will be used for regularization, which is closely related to L2 regularization. This operation reduces the weights by a small fraction λ at each optimization step (Equation 4.8).

$$\vec{w}_t = \vec{w}_{t-1} - \alpha\lambda\vec{w}_{t-1} - \vec{v}_t. \tag{4.8}$$

This fraction is proportional to the size of the weight, so large weights require stronger gradient sizes to maintain their large size. This means that weight changes that are spurious, for example because a single training batch happened to generate them, will later be undone by the weight regularization returning the weights towards zero.

Other forms of regularization include drop-out, in which a random subset of activations is dropped in linear layers [124], or entire layers or blocks are randomly dropped [131], for example in transformer architectures [158]. Often, normalization layers are used after each layer to normalize the activations, with batch normalization used for convolutional layers and layer normalization used in transformers [138, 13]. This normalization reduces the likelihood of vanishing and exploding gradients, and helps to regularize the weight values.

Additionally, data augmentation reduces the likelihood of overfitting (see Chapter 5 for more information on these topics). There is also more detail about training neural networks in Chapter 9.

In summary, we hope this chapter serves as a brief introduction to architectures, loss functions, and optimization techniques used for problems in the bioimaging community.

4.4 Additional resources

There is a tutorial for exploring these concepts available as a Jupyter notebook.

> **Resources for Continued Learning**
>
> For more details and exercises for these concepts we recommend the following resources on deep learning:
>
> - Course: Neuromatch Academy Deep Learning Course
> - Tutorial: Learn PyTorch for Deep Learning, Computer Vision
> - Multiple Tutorials: PyTorch
> - Demonstration: CNN Explainer. This includes discussion of activations across layers, and interactive visualizations for padding and stride [330].
> - Illustration: How momentum works by Gabriel Goh

Part II

Image Acquisition

5 Collecting Training Data

Image Collection and Considerations

Martin Weigert
Center for Scalable Data Analytics and Artificial Intelligence (ScaDS.AI), Technische Universität Dresden

Imagine training a denoising model from paired fluorescence images with low and high signal-to-noise ratios. Training proceeds normally and the loss decreases, yet fine structures in the restored images look blurred. You later discover that the sample moved between acquisitions, misaligning the input and target. Each image looked plausible in isolation, but the pair was inconsistent. Or consider a segmentation model whose normalization parameters (discussed in Section 5.2.7) were estimated from the training images. Later images acquired with a different detector gain are ten times brighter and fall outside the training range, leading to poor segmentations despite looking familiar in an automatically scaled display. A decreasing training loss alone does not show whether input-target pairs are aligned or whether later images match the training conditions. Model performance depends strongly on accurate and representative training examples [215, 111], yet misaligned targets and mismatches between training and later images can remain hidden unless you inspect the data. These failures are examples of silent data bugs.

Understanding how microscope images are formed and acquired helps you recognize and explain such data bugs. The optics, detector, acquisition settings, and specimen preparation determine which structures are visible and how their intensities are recorded, so knowing their effects helps you distinguish biological variation from artifacts and acquisition differences. The practical goal is to **become one with the data**: inspect it often enough that unusual images, artifacts, and mismatched pairs become apparent. The chapter begins with planning data collection and independent splits. It then follows the data through acquisition and preparation to checks of the images and targets that reach the model.

5.1 Planning what to collect

5.1.1 Defining the task and supervision

Fluorescence images of nuclei can be used to classify cells by phenotype or to outline individual nuclei, but these tasks require different training targets. During training, predictions are compared with these targets to calculate the training error. For example, phenotype labels identify the class of each cell, whereas segmentation masks describe the extent of each nucleus. Before collecting images, you therefore need to specify both the intended output and the labels or reference measurements needed for training, as illustrated for common tasks in Table 5.1.

Table 5.1: Task-specific training-data requirements

Task	Intended output	Typical training data
Classification	Class assigned to a cell, image, or specimen	Images or image regions paired with labels
Segmentation	Pixel classes or individually outlined objects	Images paired with class or instance masks for full supervision, or sparse annotations such as points or scribbles for weak supervision
Image restoration and translation	Image with less noise, blur, or scattering, or with a predicted staining pattern	Matched images of the same specimen, such as short- and long-exposure images, low- and high-NA images, or unstained and stained images
Tracking	Object identities and links across time, including divisions where relevant	Image sequences with annotated object positions or masks and links between time points

Even after choosing the task, the supervision method can change what you need to acquire. For denoising (discussed in detail in Chapter 6), a higher-signal reference must show the same structures as the noisy input, but acquiring it may introduce motion blur or bleaching. Two short, noisy exposures can instead serve as input and target, provided they capture the same underlying signal with independent noise [174]. When even paired acquisitions are not possible, self-supervised learning offers an alternative using individual noisy images. For example, Noise2Void predicts hidden pixels from their surroundings, using their original noisy values as training targets. This relies on spatially independent noise and predictable image structure [163]. Choose the supervision method before acquisition, because a missing reference channel or repeated exposure may be impossible to obtain later.

5.1.2 Choosing representative samples

Changes in staining, illumination, or detector settings can make images of the same biological structures look different and a model may fail on later acquisitions if its training data do not cover these differences. Such a systematic difference in the distribution of images, labels, or their relationship between training and later data is called domain shift. The bottom row of Figure 5.1 illustrates an extreme example: a denoising model trained on nuclei is applied to membrane images and fails to preserve their thin boundaries. Another example is classifying control and treated samples that were stained using different protocols. Such a systematic influence of staining on image appearance, called a batch effect, may let the model distinguish the groups by staining rather than by the biology [91]. If the staining protocol changes again in later acquisitions, the same batch effect can also cause domain shift.

To address these problems, collect independent specimens from each condition you plan to analyze, rather than relying on many images from one experiment. Sampling different regions within each specimen captures variation in structure, object density, and signal level, but cannot show how these features change between experiments. Including control and treated samples in the same experiments and acquisition batches then helps separate biological differences from technical effects [27]. Having relevant variation in the training data provides a form of insurance against failures on later experiments, so filling gaps in coverage should take priority over acquiring more similar images. Rotations, flips, intensity changes, or simulated noise can introduce variation through data augmentation, provided they are physically and biologically plausible for the task. Spatial transformations must be identical for paired inputs and targets. Although augmentation can increase the nominal training-set size, it creates transformed versions of existing measurements and does not add independent specimens or experiments.

5.1.3 Planning independent splits

If images from the same embryo appear in both the training and test data, a model may perform well on the test images without handling a new embryo equally well. Planning the split before acquisition helps ensure that evaluation includes specimens the model has not encountered during training. As explained in Chapter 4, training data are used to fit the model, while validation data help choose settings and decide when to stop training. Because these choices already use the validation results, a separate test set is needed for the final evaluation. For a model intended for new specimens, keep all images from each specimen in one split, including crops, neighboring frames, and augmented copies. Otherwise, related examples can cross the split and produce an overly optimistic evaluation, a form of data leakage. If the model must also handle new experiments or acquisition batches, reserve complete experiments or batches for testing. Allow for these held-out samples when planning data collection, so that enough independent specimens remain for both training and evaluation.

5.2 Collecting and preparing data

5.2.1 Understanding the image measurements

A microscope is a physical measurement device whose optical properties, detector response, and pixel size are often well characterized. Knowing these parameters helps relate pixel values to properties of the specimen, but doing so requires understanding how the optics and detector affect the measured signal. For example, a fluorescence image can become brighter because the specimen contains more fluorophores or because the exposure time or detector gain has increased. Similarly, whether two nearby structures appear distinct depends on optical resolution and spatial sampling. Understanding these relationships helps you distinguish biological differences from acquisition effects when selecting training images. Table 5.2 summarizes how common acquisition factors affect training images.

Table 5.2: Acquisition factors and their effects on microscopy images

Factor	Effect on the image
Optical resolution/Numerical Aperture	Optical blur limits detail, pixel spacing determines sampling
Intensity and dynamic range	Illumination and detector settings affect intensities, saturation clips bright signals
Noise and background	Noise introduces fluctuations, background can obscure structures
Exposure and laser power	Longer exposures can reduce noise but create blur, illumination can cause bleaching and phototoxicity [166]

For moving specimens, a longer exposure may reduce noise while blurring the boundaries you want the model to identify. In that case, shorter, noisier acquisitions may provide more useful training images. Acquisition settings should therefore preserve the structures or changes needed for the task, even when another setting produces a cleaner-looking image.

5.2.2 Checking images during acquisition

Use the acquisition software's saturation indicator to check whether bright structures are clipped. Automatic contrast adjustment can make an image look well exposed even when intensity differences in those regions have been lost. If structures needed for the analysis are saturated, adjust the acquisition settings before collecting more images. Check that the pixel size is small enough to sample the detail resolved by the optics (Nyquist criterion) [150]. Pixel size also determines how large those structures appear to the model. For example, doubling the pixel size makes the same cell span half as many pixels across, which can reduce performance

unless the model has been designed to handle this variation. Verify the pixel size in the acquisition metadata, since the objective alone does not determine it. In 3D images, the axial spacing is often larger than the lateral pixel size. This matters for augmentation because rotations that exchange the z-axis with x or y also exchange differently sampled directions. For anisotropic stacks with equal x and y spacing, 90° rotations within the xy plane preserve the sampling geometry, provided the biological orientation permits them. Differences in axial spacing between training and later acquisitions can also change how structures appear across slices, even when the lateral pixel size is unchanged. Record the spacing along each axis so that later processing can account for these differences. For example, stacks acquired with a larger axial step can be resampled along z to match the training spacing before prediction, or training augmentations can rescale the z spacing such that the model is robust to them.

5.2.3 Keeping data traceable

If cells occupy fewer pixels in one image than another, acquisition metadata helps you check whether the pixel size changed. Each image should therefore remain linked to its specimen and acquisition settings, including the channels, exposure, and pixel size [165]. Resampling can also change how many pixels a cell occupies, so acquisition settings alone may not explain the difference. The record therefore needs to include the source image and the methods and settings used to produce each processed image or target. This history is called data provenance. Acquisition settings can often be stored directly in the image metadata, while a versioned table or short README can document the subsequent processing. For annotations, this history includes the criteria used, who reviewed them, and which version belongs to the dataset. If a mask was annotated on a cropped or resampled image, the annotated region and coordinate transformation are also needed to relate it to the source. Keeping these links makes it possible to trace a corrected annotation to the datasets that need updating. Shared identifiers in filenames help maintain these links, but pairing files by their sorted positions can silently break them (Warning 1).

⚠ Warning 1: Sorting can silently misalign inputs and targets

Include a unique identifier in filenames, such as a UUID or a timestamp combined with a sequence number, and share it between corresponding images and targets. To keep corresponding files in the same sort order, place the identifier consistently and zero-pad sequence numbers, as in `20300517T143052_0001_image.ome.tif`. Separating the fields with underscores or hyphens also simplifies scripted access. Without fixed-width numbers, sorting images and masks separately can produce different orders. For example,

```
file1.tif      file1_mask.tif
file11.tif     file11_mask.tif
```

becomes

```
file1.tif      file11_mask.tif
file11.tif     file1_mask.tif
```

Pairing these sorted lists mismatches both targets. Match images and targets by their shared identifier, such as `file1`, rather than their positions in the lists, and check that every expected pair is present. This also prevents a missing file from shifting all subsequent pairs.

5.2.4 Acquiring and generating targets

For segmentation, carefully reviewed manual annotations are often the best available ground truth, a reference accepted as accurate for a particular task. However, annotating every object manually becomes time-consuming as datasets grow, particularly for 3D images and time-lapse sequences. To reduce this effort, you can start from masks proposed by traditional image processing or pretrained models and correct them, as described in Section 9.2.4.3. Both manual and assisted annotation require clear criteria for ambiguous cases, such as faint nuclei or dividing cells, so that objects are annotated and counted consistently. Having a second person independently annotate a small subset can help identify criteria that need clarification. Because errors in model-generated labels, often called pseudo-labels, can be easy to overlook, review the suggested masks using the same criteria as manual annotations.

For denoising, you can acquire a short-exposure input and a longer-exposure reference from the same field of view, or construct the reference by averaging repeated exposures. Acquiring the reference takes additional time and light exposure, which may increase bleaching, motion blur, or phototoxicity. Noise2Noise avoids the high-signal reference by using two noisy acquisitions instead [174]. Choose the acquisition sequence before collecting the dataset, and record the exposure settings, acquisition order, and interval between images so that the pairs can be checked afterward.

An additional imaging channel can make target generation largely automatic when it reveals structures that are difficult to annotate in the intended input. For example, nuclei that are hard to annotate in brightfield may be easier to segment in fluorescence using a nuclear marker such as H2B-GFP. If you acquire both channels together, you can automatically segment the fluorescence images and use the resulting masks as targets for the corresponding brightfield inputs. You can then review and correct these masks instead of manually annotating each nucleus in brightfield, reducing the manual work needed to generate training pairs. In histopathology, a tissue section can similarly be restained with an antibody that labels a certain structure (e.g., epithelium) and then reimaged. After the two images are registered, simple thresholding of the antibody image can then generate masks for the original tissue image [27]. Likewise, if the intended output is a fluorescence image rather than a segmentation mask (i.e., image translation), the acquired fluorescence image itself can serve as the target [63, 232]. In

all these cases, the additional image or channel provides targets during training, while the trained model takes only the original image as input.

5.2.5 Checking paired acquisitions

When targets come from paired acquisitions, corresponding structures need to occupy matching positions if training assumes alignment. Sequential acquisitions can provide valid pairs if the relevant structures remain unchanged between exposures, but sample motion, focus drift, and channel offsets can break spatial correspondence, while bleaching can change the recorded signal. Although pair identifiers and recorded transformations help trace which images were paired and how they were processed, they do not show whether the structures still match. You therefore need to inspect overlays to check alignment and compare intensities to identify signal changes. Misalignment can affect the prediction even when training appears to proceed normally. For example, the model trained with shifted membrane targets in Figure 5.1 produces blurred predictions despite a decreasing training loss.

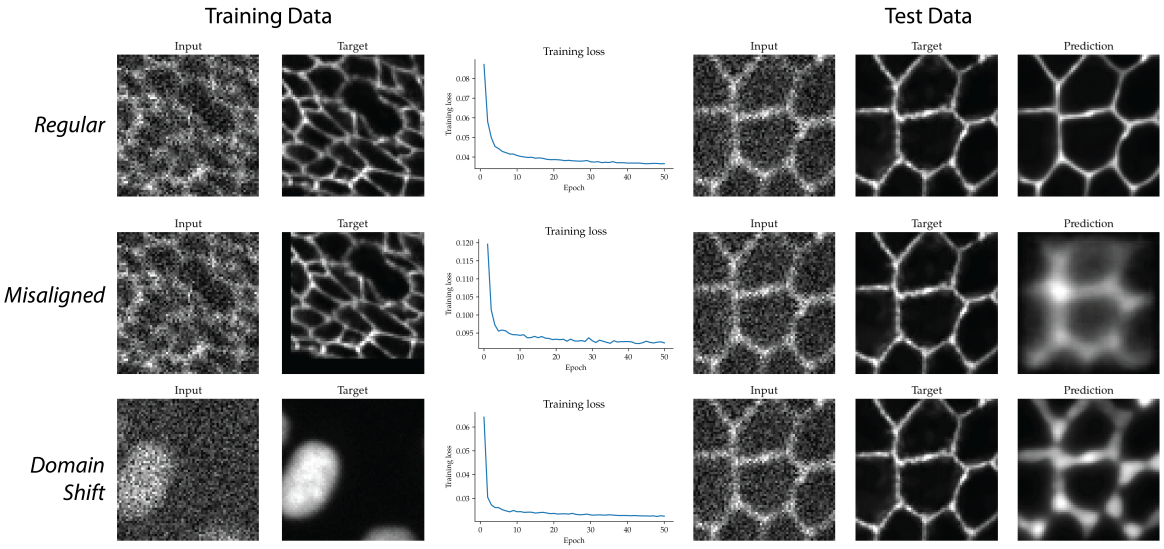


Figure 5.1: **Examples of training-data mismatches in a denoising task.** Rows compare aligned membrane pairs (top), spatially shifted membrane targets (middle), and train/test domain shift (bottom). Columns show representative training pairs (left), loss curves (middle), and predictions (right) for the same membrane test image.

Even when input and target are aligned, the target may contain details that the input does not resolve. For example, a higher-resolution reference may separate two nearby structures that appear merged in a blurred input. A model may infer their separation from patterns learned during training, but the predicted detail can be unsupported by the acquired image, an error often described as a hallucination [22]. Because plausible detail in the output may not match

the real specimen, use a small pilot dataset to compare predictions with independent reference measurements before collecting many such pairs. Check whether the agreement is sufficient for the intended biological analysis, such as counting the two structures separately.

5.2.6 Preserving data during preprocessing and storage

Cropping an image without applying the corresponding crop to its target breaks their alignment. Spatial preprocessing therefore needs to preserve input-target correspondence, and resampling requires updating the pixel size metadata. Keep the original measurements and record processing steps with their order and parameters so that you can trace unexpected changes. Segmentation masks need particular care because their values identify classes or objects rather than intensities. For example, interpolating betwen object labels 4 and 7 can produce values such as 5 and 6, assigning pixels to unrelated object identities. Use nearest-neighbor interpolation for these masks, and compare images and targets before and after processing to check for shifted boundaries, swapped axes, clipping, or lost objects.

Storage can also change the data if compression is lossy. Saving an image as JPEG changes pixel values and introduces artifacts, while saving a segmentation mask this way can corrupt object labels and background values. JPEG is therefore suitable for thumbnails, but source measurements and targets should be stored losslessly [150]. Formats such as OME-Zarr can also retain axes, channels, and pixel size metadata. OME-Zarr stores arrays in chunks, allowing tools to read selected regions of large local or remote datasets without loading the entire image [216]. Check that the tools you use preserve both the array values and the metadata during conversion.

5.2.7 Normalizing image intensities

Images acquired with different exposures or detector settings can span different intensity ranges, so normalization rescales their values before they are passed to a model. However, a few hot pixels can set the maximum intensity of an image, so scaling between its minimum and maximum can compress the remaining intensities into a narrow range. Percentile-based normalization reduces this sensitivity and is commonly used in microscopy workflows [334, 287]. It maps selected lower and upper percentiles, for example the 1st and 99.8th, to 0 and 1 without clipping values outside that interval. Values below 0 or above 1 therefore retain intensity differences at the extremes.

Computing percentiles separately for each image can reduce brightness differences caused by illumination or detector settings, but it can also remove differences caused by biology. For segmentation, this may be acceptable when the intended output is an object mask. You can then use that mask to measure fluorescence on the original or appropriately calibrated image. If the model itself needs to use brightness differences between images, consider a shared scaling rule. Estimate any dataset-wide parameters from the training split alone and apply them

unchanged to validation, test, and later images. For multichannel data, normalize each channel independently by default, unless intensity ratios between channels carry information that the model needs to retain (e.g., for natural RGB images). Record the percentile settings and the images and channels over which parameters are computed so that training and later prediction use the same procedure.

5.3 Checking dataset quality before training

5.3.1 Inspecting images and loader outputs

An image may look correct on disk while the model receives a shifted target or swapped channels. Such mismatches can arise in the data loader, which turns stored images and targets into training tensors through operations such as cropping, normalization, augmentation, and type conversion. For example, applying a spatial augmentation only to the image breaks its alignment with the target, while converting labels to an integer type too small for the largest label id can change object identities. To detect these errors, compare the raw input, overlaid target, processed image, and loader output. Saving a few input-target pairs at the start of each run, together with their shapes, data types, and value ranges, makes this comparison easier.

A repeated time point can look plausible when viewed in isolation yet disrupt a cell track. For volumetric time-lapse data, therefore, inspect neighboring time points as well as individual volumes. Consecutive-frame differences, timestamps, or hashes can flag possible frozen frames, which you can then examine alongside neighboring time points and acquisition metadata to distinguish repeated data from a stationary specimen. If the loaded samples appear correct, fitting a small, manually verified training subset can expose problems that remain after visual inspection. Failure can point to target, loader, or configuration errors, although successful fitting (see Section 9.2.5.2) does not establish generalization. During this small run, also check whether data loading leaves the model waiting for batches. For datasets that do not fit in memory, efficient access to the chunks described in Section 5.2.6 can reduce this waiting time.

5.3.2 Running dataset-wide checks

Inspection of a few examples cannot show whether every file can be used by the training pipeline, so run scripted checks across the collection. These checks should confirm that image-target pairs are complete and that shapes, data types, and value ranges match the pipeline's requirements. For segmentation masks, this includes the expected background value and whether labels must be contiguous. A mask exported as an RGB visualization, for example, may contain three color channels instead of object identifiers, while merging all instance labels into one foreground class removes the distinction between objects. Flag such encoding problems for review, along with nonfinite values such as NaN or infinity and suspiciously black frames. For targets that

pass these file checks, summaries of object counts, sizes, track lengths, and annotation density can help locate unusual examples for closer inspection.

Even when every image-target pair is valid, a biological condition may occur in only one acquisition batch or have very few examples (i.e., an unbalanced training dataset). To identify these gaps, count independent specimens and experiments across the biological and acquisition conditions you planned to cover. For example, cross-tabulating biological conditions against preparation days and acquisition batches reveals conditions confined to a single batch. Within these groups, count rare classes and empty images separately [27], since pooled counts can hide gaps in particular conditions or batches. Alongside class frequencies, record the foreground fraction per image to distinguish rare classes from images in which most pixels are background. These counts reveal gaps within the dataset, but do not show whether its images resemble those you will later analyze. In histopathology, for example, stain color can vary between laboratories and scanners [139] so comparing brightness, background noise, and object sizes in pixels and physical units with the training images is important. For example, if cells of similar physical size occupy twice as many pixels in one collection, check whether the recorded pixel size accounts for the change. Differences you expect in the intended application should also be represented in the test set, including difficult examples [165]. If you also want to test behavior beyond that use, keep inputs such as severely corrupted images or specimens outside the intended population in a separate stress set and report its results separately.

Previews and reports help you locate examples that need closer inspection in large collections. For example, thumbnails, orthogonal slices, or downsampled movies provide overviews from which you can select regions for closer examination in an interactive viewer. To generate these overviews for data on a cluster or object store, scripts can run close to the data and return reports, projections, and flagged ids without transferring the full images. Keep each result linked to its source image, target, and metadata so that you can investigate the flag. The image dimensions, dataset size, and storage location determine which tools in Table 5.3 are suitable for this inspection.

Table 5.3: Example tools for dataset inspection and quality control

Tool	Typical data or workflow	Useful checks
Fiji/ImageJ [285]	Local 2D, multichannel, z-stack, or time-lapse images	Intensities, histograms, overlays, and pair alignment
napari [4]	N-dimensional images, labels, points, shapes, or tracks	Layer comparison and 3D or time navigation
QuPath [16]	Histopathology and large 2D slides	Stains, tissue regions, annotations, metadata, and slide heterogeneity
TrackMate [82] and Mastodon [94]	Time-lapse tracking and lineage data	Detections, divisions, gaps, links, and lineage consistency

Tool	Typical data or workflow	Useful checks
Vizarr/Viv [201]	Remote multiscale OME-Zarr or Zarr images	Browser-based channel and spatial inspection
Neuroglancer [100] and webKnossos [281]	Large multiscale 3D volumes, especially electron microscopy	Volume navigation, segmentation, annotation, and proofreading
Python check scripts	Dataset-wide automated checks	Dimensions, ranges, pairing, duplicates, frozen frames, and stratified summaries

Coding agents can help coordinate check scripts and viewers if the tools provide callable operations with readable outputs (see Section 3.5.7). With this access, an agent can first run inexpensive checks across the collection, then use their results to decide which previews or detailed checks to request. Combining metadata, summary statistics, and previews can help identify cases where several observations together suggest a problem that none would identify alone. To reduce the risk of overlooking such cases, design the initial checks to prioritize recall, meaning that few actual problems are missed. The agent can then investigate flagged cases with additional tools and report the reasons for each flag, together with any checks it could not complete.

5.4 Dataset checklist

Dataset checklist

- include specimen types, treatments, and imaging conditions you expect to analyze, including rare and difficult cases
- split (for training, validation, and test) by independent specimens, experiments, or batches, keeping related images together
- check targets for errors and inconsistent annotations
- check image-target pairing and alignment in loader outputs, then fit a small verified subset
- record normalization settings and estimate shared parameters from training data alone
- save images and targets losslessly, with their metadata, processing steps, split assignments, and inspection results

If a model later underperforms, these records help you investigate whether problems in the data contributed. The next chapters show what such datasets enable (Chapter 6, Chapter 7), how to select tools and train models (Chapter 8, Chapter 9), and how to judge whether the outputs are good enough for your analysis (Chapter 10).

AI Disclosure

AI language models (Claude Opus 5, Codex/GPT 5.6 Sol) were used during the preparation of this chapter. Their roles included revising prose and assisting in writing the training scripts for the training-data mismatch figure. All content was reviewed, revised, and verified by the human author, who takes full responsibility for the accuracy and scientific integrity of the final text. Selection of references was the work of the human author.

6 Image Restoration

Using Artificial Intelligence for Image Restoration in Fluorescence Microscopy

Yue Li[1], Jun Zhu[1], Mingzhe Wei[1], Hari Shroff[2], & Min Guo[1]
[1]Zhejiang University, [2]HHMI Janelia Research Campus

Fluorescence microscopy is central to biological discovery, enabling the visualization of cellular and subcellular structures with exquisite detail. From observing dynamic intracellular processes to mapping entire tissues, microscopy is indispensable for understanding biological systems [341]. However, the quality ofy images is often compromised due to intrinsic limitations such as noise, optical aberrations, diffraction, and limited signal [283]. These factors hinder analysis and interpretation, particularly when studying fine biological structures or dynamic processes. For example, by quantifying the intensity of fluorescence signals, researchers infer the abundance or expression levels of specific molecules (e.g., proteins tagged with a genetically expressed marker). Using image segmentation, researchers can analyze the size, shape, distribution, and number of specific objects within a defined region [8]. By tracking the movement of fluorescently labeled molecules or cells over time, researchers can study processes including endocytosis, intracellular transport, and cellular signaling [275]. In these scenarios, noise and low signal-to-noise ratio (SNR) can make it difficult to distinguish between the actual signal and background, and spatial blurring causes fluorescence signal to spread out, confounding the ability to accurately assign pixels to specific regions or structures.

Many of these problems can be overcome with suitable hardware or by using advanced imaging techniques. To improve spatial resolution, super-resolution techniques — stimulated emission depletion (STED), structured illumination microscopy (SIM), or single-molecule localization microscopy (SMLM) — may be employed [284]. To suppress noise and maximize the collection of useful signal, highly sensitive detectors, such as cooled charge-coupled devices (CCDs) or complementary metal-oxide-semiconductor (CMOS) sensors, can be used. To correct optical distortions, advanced microscopes integrate adaptive optics (AO) [144, 112], which use real-time feedback to dynamically adjust the focus and compensate for aberrations. However, these technologies often come with high cost, require complex operation, or need extensive maintenance. Additionally, improving one attribute of the image (e.g., spatial resolution) often results in a compromise in another (e.g., temporal resolution), requiring careful consideration of the specific needs of a study and the resources available [297].

Another way to address these limitations is to develop image restoration techniques that enhance the quality of microscopy data, often without expensive instrumentation. Traditionally, these techniques relied on mathematical algorithms and physical models of the imaging process. However, recent advances in artificial intelligence (AI), particularly deep learning (DL), have transformed image restoration by enabling data-driven approaches that offer improved performance and flexibility. Manufacturers have increasingly integrated image restoration technologies as an essential component of their products to ensure their users can obtain clearer images directly from the microscope, e.g., Leica THUNDER Imager, Olympus cellSens, Andor iQ, ZEISS arivis, and Nikon NIS-Elements.

In this chapter, we provide a practical overview of image restoration for applications in fluorescence microscopy. We begin by introducing the general concept of image restoration, then describe traditional and deep learning-based approaches. Next, we review key technologies and advances by categorizing restoration into four major areas: denoising, deconvolution, deaberration, and resolution enhancement. Finally, we provide practical guidelines for implementing image restoration, including step-by-step workflows and test datasets for interested readers to practice applying these methods.

6.1 General Concepts in Image Restoration

Image restoration is the application of mathematical and computational techniques aimed at improving the quality of an image by reversing or reducing the effects of degradation that may occur during the imaging process. These degradations stem from noise, blur, aberrations, and other artifacts. The goal of the restoration is to recover or estimate the latent (true) image from the degraded raw data. Image restoration has the potential not only to improve the precision of biological analyses but also to expand the capabilities of existing microscopes, enabling researchers using basic hardware to achieve advanced imaging quality.

6.1.1 Image Degradation Model

Understanding the types of distortions and the causes behind them is the first step in image restoration. This could involve a mathematical model that characterizes how an image is corrupted, using the concept that no real-world imaging system can capture an object perfectly. Instead, the imaging system or sample introduces imperfections, which distort the image. In fluorescence microscopy, a general image degradation model can be written as:

$$i = o \otimes f(\phi) + n \tag{6.1}$$

where i is the image acquired by the microscope; o is the intensity distribution of the biological sample; $f(\phi)$ is the point spread function (PSF) of the system; ϕ is the phase aberration or wavefront distortion (when $\phi = 0$, the wavefront is 'flat' or aberration-free and f is the ideal

PSF predicted by theory); $\otimes$ is the convolution operator; and n models noise contamination. Two common types of noise are Gaussian noise and Poisson noise. Gaussian noise is caused by random fluctuations in the image due to imperfections in the camera sensor or electronic interference. Poisson noise arises from the random nature of photon detection, meaning that the number of photons detected can vary from one measurement to the next, especially at low light levels. Equation 6.1 is described schematically in Figure 6.1.

6.1.2 Image Restoration with Traditional Approaches

Traditional microscopy image restoration techniques use mathematical models of the imaging model and statistical analyses that attempt to reverse or mitigate image degradations, and/or additional microscope hardware. We find it helpful to categorize image restoration into four main areas: denoising, deconvolution, deaberration, and resolution enhancement.

6.1.2.1 Denoising

Traditional denoising methods try to remove noise while preserving important features such as edges and fine structures. Many denoising algorithms assume Gaussian noise for computational tractability. Gaussian noise removal can be achieved by filtering-based methods in the spatial or frequency domain, such as Gaussian filtering, mean filtering, wavelet filtering, bilateral filtering [318], and non-local BM3D [73]. Simple linear filters are easy to implement, but they cause a loss in high-frequency information. Complex denoising methods require careful parameter design and are usually computationally intensive. In many cases, the Gaussian noise model provides a good approximation, but Poisson noise is also a key source of noise for fluorescence microscopy given the quantized nature of fluorescence emission, especially under low signal conditions when detector noise is minimal. One method for dealing with Poisson noise [355] directly incorporates its statistics, e.g. the PURE-LET method [177]. Alternatively, a non-linear variance-stabilizing transformation (VST) [198] can be used to convert the Poisson denoising problem into a Gaussian denoising problem. There are several free and open-source resources available for microscopy image denoising, including filters and the PureDeNoise [195] plugin in ImageJ/Fiji, and the scikit-image library in Python [322].

6.1.2.2 Deconvolution

Deconvolution is the process of reversing optical blur introduced by the PSF of the microscope ($f(\phi)$ in Figure 6.1) and is most often implemented without considering optical aberrations. It improves effective contrast and resolution by accounting for such blur and reassigning the relevant signal to its most likely location, given the blurring and noise models. Traditional deconvolution methods include frequency-domain algorithms like naive inverse filtering and Wiener filtering[338, 97], optimization-based algorithms like Tikhonov regularization [312], and iterative deconvolution methods like the Tikhonov-Miller algorithm [212], fast iterative

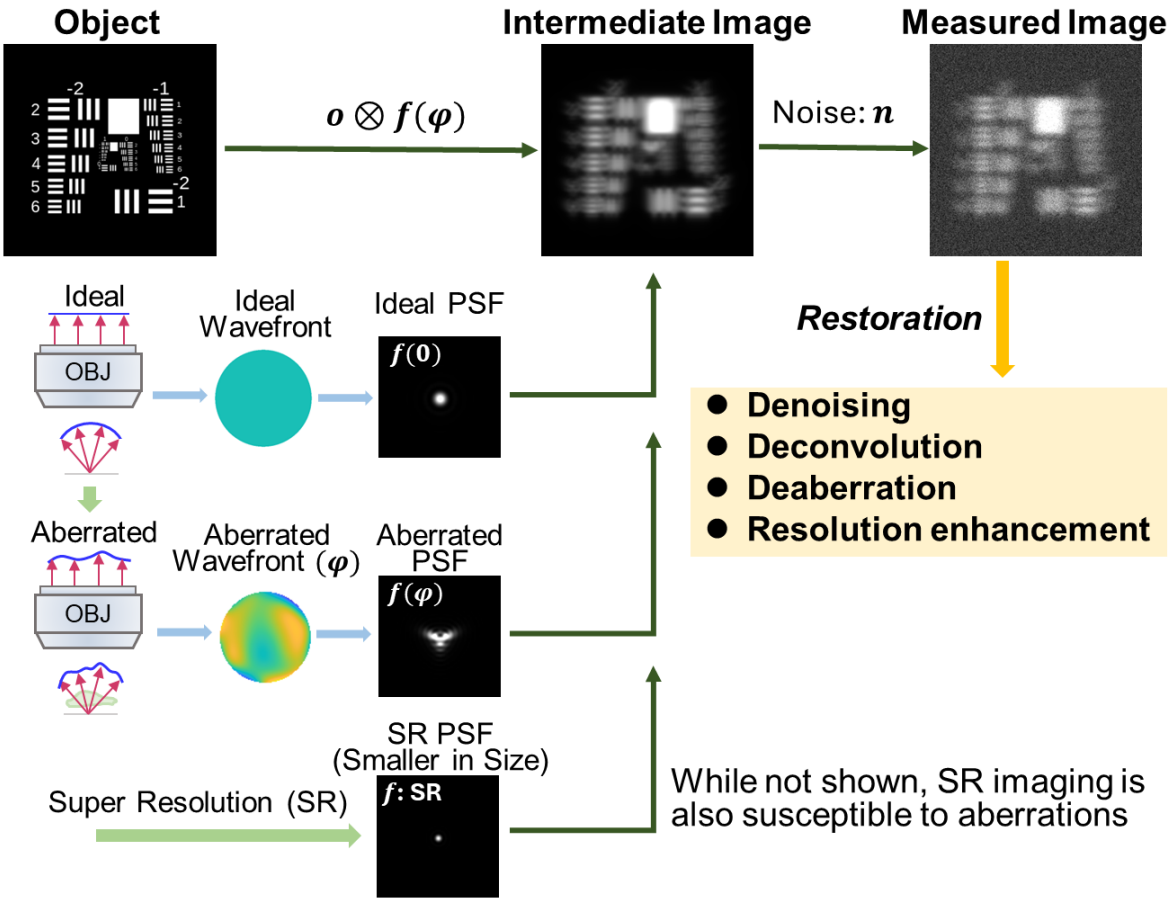


Figure 6.1: **Schematic of image degradation model for fluorescence microscopy.** Object information passes through the imaging system with degradation due to blurring from the PSF (here denoted by the convolution kernel f) and noise. In an ideal case, there is no distortion, and the wavefront associated with a point source maintains a flat phase at the back focal plane of the microscope objective. In any real situation, wavefront distortion due to optical imperfections or heterogeneity within the biological sample results in an aberrated wavefront (indicated by ϕ) and PSF, which results in additional blurring. If spatial resolution is enhanced, e.g., by super-resolution methods (SR), the PSF becomes smaller, resulting in sharper images. Super-resolution imaging is also (typically more) prone to aberrations than conventional diffraction-limited imaging, and noise is inherent to any imaging process.

soft-thresholding algorithm [20], and Richardson-Lucy deconvolution[194, 263]. Traditional deconvolution methods can require significant computational resources, especially for large datasets with complex blurring functions, and also can amplify noise [279, 99, 106]. Commercial deconvolution software includes Huygens, DeltaVison Deconvolution, and AutoQuant. There are also open-source deconvolution plugins integrated into Fiji [285] (e.g., DeconvolutionLab2 [273]) or Medical Image Processing, Analysis, and Visualization (MIPAV) [19] programs.

6.1.2.3 Deaberration

Aberrations refer to the optical imperfections or distortions that occur during image acquisition, degrading the quality of the captured image. Such aberrations can arise due to optical path length differences introduced anywhere in the imaging path, including instrument misalignment, optical imperfections, or differences in refractive index between the heterogeneous and refractile sample, immersion media, and/or objective immersion oil. Aberrations significantly alter the wavefront of light. When the wavefront is distorted, the light rays that are focused by the microscope do not converge as they ideally should. This distortion can lead to various image quality issues, such as loss of resolution and distortion of fine features. Adaptive optics [144, 32] can mitigate aberrations by measuring wavefront distortions and subsequently compensating for them using a deformable mirror or other optical elements. However, implementing AO is nontrivial, often requiring additional control algorithms and new hardware, adding considerable expense to the underlying microscope.

6.1.2.4 Resolution Enhancement

Optical super-resolution techniques, like STED, PALM, SIM, etc., bypass the diffraction limit by utilizing on/off fluorophore state transitions [119], sophisticated hardware designs, and/or image reconstruction algorithms [275, 319, 340]. Alternatively, physical expansion of the sample can be used with conventional microscopes [55, 331]. While effective, all these methods present some tradeoff for the gain in spatial resolution, including increased acquisition time, additional illumination dose, specially designed probes, or more complex instrumentation [316, 56].

6.1.2.5 Drawbacks

Although traditional image restoration algorithms are used extensively in all these categories, they suffer from several drawbacks. For example, many methods require careful and manual parameter tuning, and they often perform poorly in challenging conditions, such as in the presence of high noise, low contrast, defocus, or complex background signal. This is often because they are based on idealized assumptions (Equation 6.1), which are often not met in practice. Another limitation of traditional restoration methods is that they are generally 'content-unaware' and do not use sample-specific prior information (e.g., shape, size, intensity distributions). On the one hand, this means that traditional algorithms generalize well; on the

other hand, they are not as performant as newer deep learning methods. Deep learning methods can learn a task such as denoising from the data themselves or provide a sample-specific prior [111], and thus can outperform traditional methods in many cases.

6.1.3 Image Restoration with Deep Learning-Based Approaches

AI, particularly deep learning, has achieved remarkable progress in recent years, leading to breakthroughs in image restoration tasks. Deep learning uses artificial neural networks with multiple layers to model and learn complex patterns in data through end-to-end training, eliminating the need for handcrafted features or manual parameter tuning (see Chapter 4 for more information). By leveraging large datasets and computational power, deep learning can capture non-linear relationships and subtle details within image data, making it particularly well-suited for image restoration. Techniques such as convolutional neural networks (CNNs), generative adversarial networks (GANs), autoencoders, and transfer learning have been shown to tackle applications in denoising, deblurring, super-resolution, and deaberration. Recent advances, such as content-aware restoration (CARE) [334], residual channel attention networks (RCANs) [58], and deep Fourier-based models [250], have demonstrated improvements in image quality while reducing phototoxicity and photobleaching during acquisition. These models offer several advantages over more traditional methods of image restoration:

- **Automatic Feature Learning**: Traditional image restoration methods rely heavily on manually designed features (e.g., regularization selection) and parameter tuning, which often require fine adjustments for different types of images. Deep learning models, on the other hand, can automatically learn features from large datasets, eliminating the need for manual intervention. This significantly reduces the complexity of parameter tuning and makes the models more adaptable to complex samples or tasks.
- **Strong Non-Linear Modeling Capabilities**: Traditional algorithms are often based on linear image degradation models (Equation 6.1), which ignore sample-specific information and perform poorly in the presence of complex image distortions or noise. However, recovering the object structure from the acquired image is an inherently ill-posed and non-linear problem due to noise and blurring. Deep learning uses non-linear activation functions (ReLU, sigmoid) and hierarchical layers to approximate complex relationships in the image data. Thus, deep learning can better model the global context of an image, allowing for more accurate restoration of image details under challenging and suboptimal imaging conditions. This can extend the use of existing hardware beyond its original purpose for new biological questions that were previously inaccessible.
- **Efficient Handling of Complex Scenes**: Deep learning models are typically trained on large datasets and are capable of handling complex scenes (e.g., images contaminated with noise and aberrations [128]) and large-scale data (e.g., GB- or TB-scale time-lapse data). Once a model is trained, it can automatically adapt to new input data with

similar acquisition parameters, which makes it much more efficient for large-scale image restoration tasks.

As deep learning models continue to evolve and integrate with advanced microscopy hardware and large datasets, they are expected to further push the boundaries of biological imaging and discovery. In the next section, we will introduce the application of deep learning methods in different image restoration tasks.

6.2 Deep Learning-Based Techniques for Image Restoration

Following the convention from Section 6.1.2, we group AI-based restoration into four broad categories.

6.2.1 Denoising

Traditional denoising techniques struggle to preserve fine details while removing noise [250, 351, 70]. Deep learning-based methods, on the other hand, can learn to predict what fine details look like even in the presence of noise. Such methods can be generally divided into supervised learning and self-supervised learning (Figure 6.2). See Chapter 2, Chapter 5, and Chapter 9 for more information on different kinds of learning.

Supervised learning methods require a large number of paired datasets of noisy and clean images for training. These datasets can be obtained by tuning illumination intensity and/or exposure time during image acquisition: when imaging with high intensity or longer exposure time, high-SNR clean images can be collected; otherwise, noisy images are collected. Training a supervised deep learning method on these data results in a mapping between noisy and clean images. When sufficient training data are available, these methods can provide very high-quality restoration on noisy data unseen by the network in the training process. Typical denoising networks include U-Net-based CARE [334] and attention-based RCAN [58]. However, paired datasets can be difficult to obtain in some microscopy settings (e.g., live-cell imaging).

Self-supervised learning methods do not require paired noisy and clean images. Instead, only noisy images are needed for training. They exploit the inherent structure of the data, learning to predict parts of the image using information gleaned from other parts. Typical self-supervised methods include Noise2Void (N2V) [163] and Noise2Self (N2S) [18]. The key idea underlying N2V and N2S is that pixels from clean images are often highly correlated, i.e., nearby pixels usually look similar and follow predictable patterns. For example, neighboring pixels could show smooth gradients in image texture. However, many noise sources are uncorrelated in space, particularly across large areas. Thus, by selecting a suitable mechanism, noise and signal can be separated effectively. In N2V, missing pixels are randomly masked in the noisy input image and the network is then trained to predict the value of the missing pixel using the surrounding noisy pixels. In N2S, instead of masking the noisy image entirely, local patches

are used, and the network is trained to predict the value of the noisy pixel from its neighbors. While self-supervised methods can be effective, they often do not perform as well as supervised methods when the sample or noise distribution is complex. These methods nevertheless are quite useful when paired noisy-clean images are difficult or impossible to obtain.

Noise2Noise (N2N) [174] represents another kind of self-supervised training. It trains the model using two noisy images (A and B), where both images contain the same underlying clean signal. Essentially, it learns to map noisy image A to noisy image B. Since both images share the same clean signal beneath the noise, the network can learn to recover the underlying clean image. This method still relies on paired data as supervision, but the supervision signal comes from another noisy image rather than a traditional clean image.

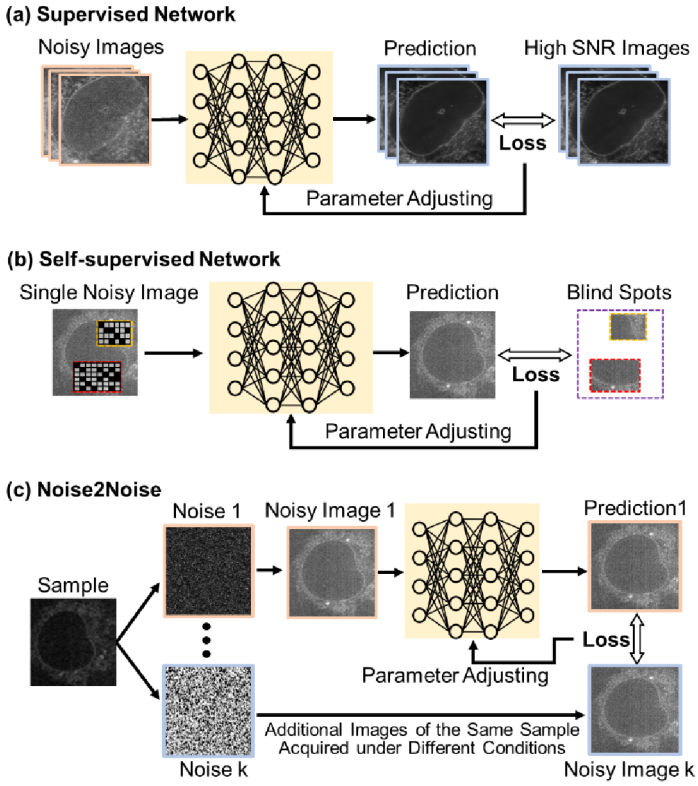


Figure 6.2: **Different denoising methods for microscopy images.** (a) Supervised denoising methods[334, 58], train a neural network using paired noisy and high-SNR images. (b) In self-supervised methods[163, 18], a portion of the image is masked (blind spots or black pixels in yellow and red subregions) for training, removing the need for explicit high-SNR images. (c) The Noise2Noise network Lehtinen et al. [174] is another form of supervised training, whereby the network is trained with a set of noisy images of the same underlying sample.

6.2.2 Deconvolution

Deconvolution has long relied on iterative algorithms like the Richardson-Lucy method. Recent advances in deep learning have revolutionized this task, offering high speed and, in some cases, even more accurate predictions than traditional methods [178]. Deep learning deconvolution methods can be divided into two categories: purely data-driven learning and physics-informed learning (Figure 6.3).

In the data-driven approach, training is conducted similarly to supervised denoising methods. The raw images are the low-quality blurred acquisitions; the high-quality reference can be the results of traditional restoration (e.g., multi-view jointly deconvolved results, or high-SNR deconvolved results). Models like CARE, RCAN, and DenseDeconNet [106] can be used for deconvolution. Once trained, such data-driven deconvolution generally enables more rapid deconvolution than traditional methods. However, these methods are content-aware and depend heavily on high-quality paired training data. This dependence may introduce artifacts and affect how well the network generalizes.

Physics-informed methods integrate domain knowledge — such as the microscope's PSF and image formation models — into the network architecture or loss functions. For example, the Richardson-Lucy Network (RLN) [178] embeds Richardson-Lucy iterations within a convolutional network, creating a hybrid method that leverages classical knowledge and data-driven deep learning. MultiWienerNet uses multiple differentiable Wiener filters paired with a convolutional neural network, exploiting the knowledge of the system's spatially varying PSF to quickly perform 2D and 3D reconstruction, achieving good results on spatially varying deconvolution tasks [343]. Physics-informed methods can also improve interpretability and reliability and may reduce the demand for experimental training data by leveraging synthetic data generated from optical models.

6.2.3 Deaberration

Deaberration addresses image degradation caused by optical distortions due to refractive index mismatches in biological samples or imperfections in the optical system. These aberrations degrade image quality, particularly in thick or scattering samples. Recent advances in deep learning have introduced powerful alternatives to traditional AO for both explicit wavefront estimation and the prediction of cleaner images in which aberrations are suppressed (Figure 6.4).

Initial efforts focused on estimating the distorted wavefronts with AI, followed by explicit wavefront correction with hardware (e.g., a deformable mirror or spatial light modulator) to obtain a clean image. The wavefront aberration can be decomposed into a sum of Zernike polynomials. The problem of wavefront estimation then becomes inferring the amplitudes of different Zernike polynomials [274]. This approach combines the strengths of traditional AO with artificial intelligence [153, 152, 84]. Following the concept of deconvolution, researchers

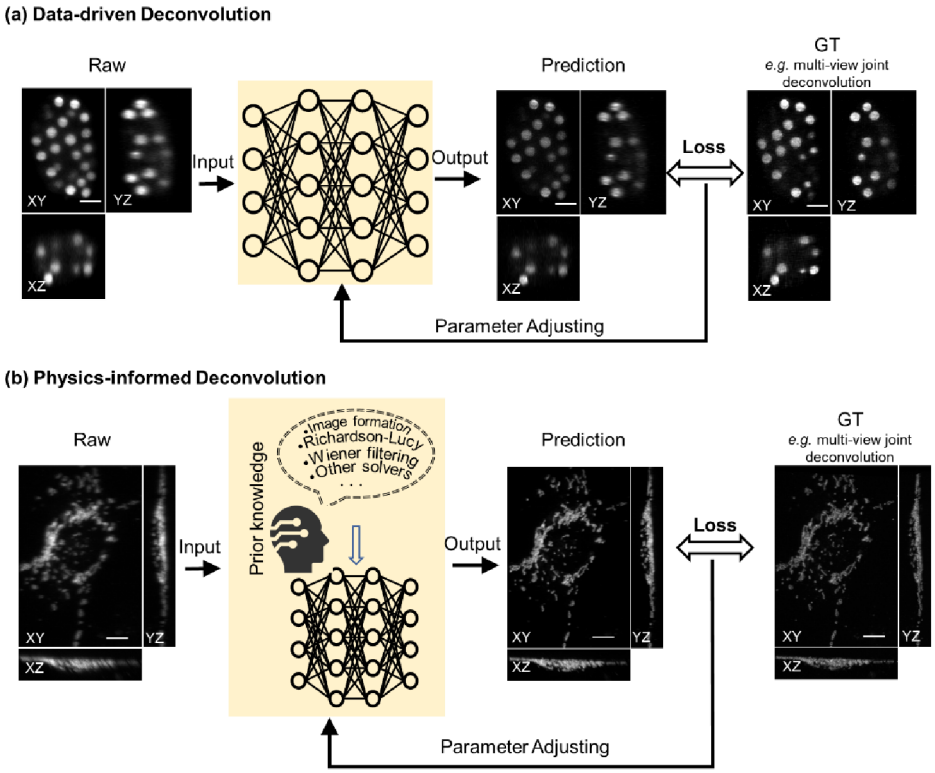


Figure 6.3: **Different Deep Learning-Based Methods for Deconvolution in Microscopy Images.** (a) Data-driven based deconvolution methods[106, 334], where a neural network is trained using only a large-set of paired raw data and ground truth (GT, e.g., joint deconvolution of multiview data) . (b) Physics-informed deconvolution[178, 343], whereby a neural network is trained using not only paired datasets but also incorporatesprior physical knowledge, such as embedding the image formation function in the loss function and/or the Richardson-Lucy framework or Wiener filtering in the network architecture.

have also used deep learning to predict the aberrated wavefronts or blurring kernels (aberrated PSFs), also combining this information with post-processing deconvolution algorithms for image restoration[359, 249].

In contrast to explicit wavefront prediction, other efforts have trained neural networks directly on paired aberrated and corrected images [105, 129]. These models are designed to learn the mapping between aberrated and corrected images, enabling aberration correction as a purely computational post-processing step. This approach eliminates the need for hardware-based wavefront sensing or correction, making it a practical and scalable solution for many microscopy applications. More recently, a unified deep learning framework for simultaneous denoising and deaberration in fluorescence microscopy has also been developed [128]. By addressing both noise and aberrations in a single model, this approach simplifies the image restoration pipeline while delivering high-quality results, highlighting the potential of multitask models to streamline microscopy workflows.

6.2.4 Resolution Enhancement

The spatial resolution of optical microscopy is fundamentally limited by the diffraction of light, a barrier described by Abbe's law. Deep learning methods can learn to predict what a higher-resolution image might look like, given lower-resolution input data (Figure 6.5). Such methods use artificial neural networks to infer high-resolution (HR) details from low-resolution (LR) fluorescence images, which can offer a sharper image without the need for super-resolution microscopes. Cross-modality super-resolution (CMSR) refers to a class of techniques that use information from different imaging modalities to enhance resolution. Models such as CNNs and GANs are trained to learn the mapping between these modalities. For example, researchers can use super-resolution (e.g., STED) images to enhance diffraction-limited (e.g., confocal) images [325], or use models trained on expansion microscopy data to predict super-resolution images from diffraction-limited inputs. Fourier-based approaches (e.g., DFCAN [250]) leverage the frequency content differences across distinct features in the Fourier domain to enable the networks to learn the hierarchical representations of high-frequency information efficiently. While powerful, we note that these resolution enhancement methods offer a prediction at best and cannot truly retrieve higher resolution information absent in the low-resolution input data.

Another resolution enhancement task is the prediction of isotropic resolution from blurry axial views. The majority of fluorescence microscopy techniques, including most widefield, confocal, and light-sheet microscopes, suffer from mismatched resolution in the lateral and axial directions (i.e., resolution anisotropy), which severely deteriorates the quality, reconstruction, and analysis of 3D images. Deep learning models can be trained to predict and fill in the missing information in the z-direction using the higher lateral resolution view and knowledge of the anisotropic PSF, effectively making the resolution more uniform across the entire image[242, 228].

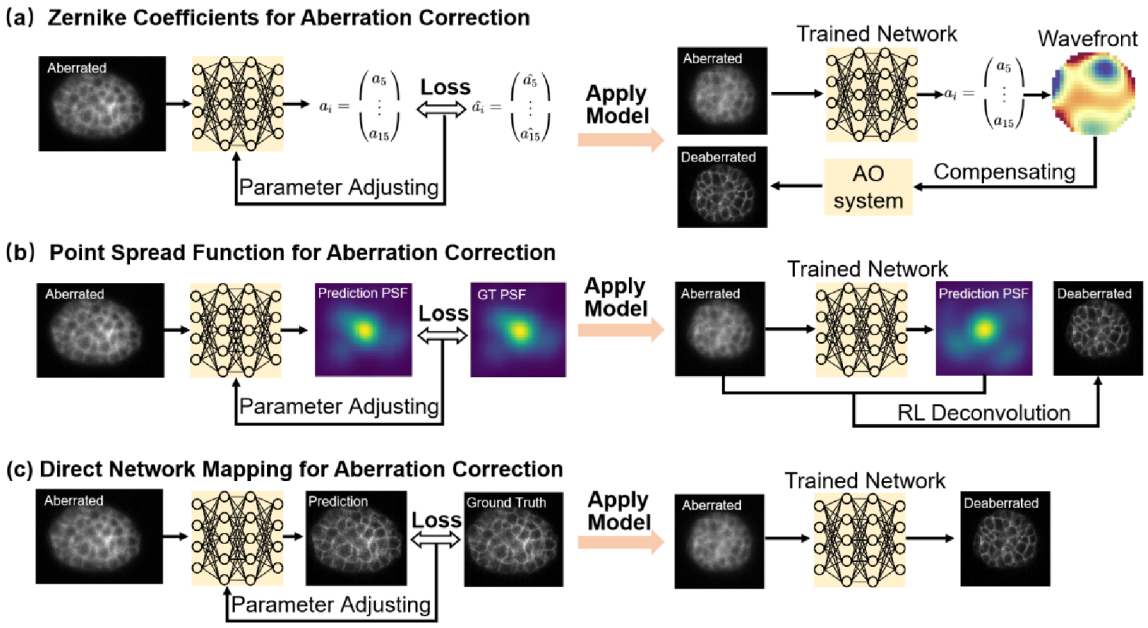


Figure 6.4: **Different Deep Learning-Based Methods for Aberration Correction in Microscopy Images.** (a) A method where the network is configured to learn the distorted wavefront (using a sparse representation of Zernike coefficients) [153, 152, 84]. The trained network is then used to predict wavefront distortions (in terms of Zernike coefficients) for a new dataset, followed by hardware-based correction using an adaptive optics (AO) system (e.g., a deformable mirror). (b) A method where the network is configured to learn the aberrated point spread function (PSF) [359, 249]. The trained network is then used to predict the aberrated PSF corresponding to a new dataset, and a second step of image deconvolution is applied with the predicted PSF to remove aberrations from raw images. (c) A method where a neural network learns the mapping between aberrated and corrected images directly for post-processing aberration correction [105, 129]. The trained network is then used to directly predict the deaberrated image corresponding to a new dataset.

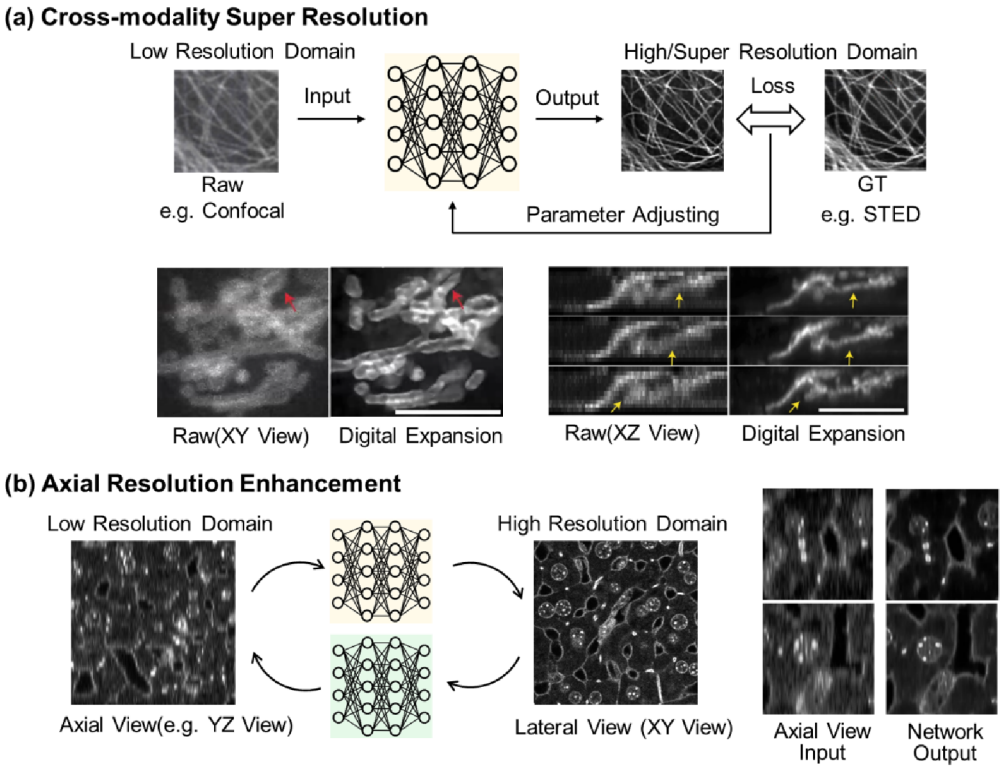


Figure 6.5: **Schematic of resolution enhancement with cross-domain networks.** (a) Cross-modality super resolution[58, 325]. Top row: the method takes diffraction-limited data (e.g., confocal microscopy data) as input and super-resolved data (e.g., STED microscopy data or expansion microscopy data) as ground truth. By training a neural network with paired datasets, the network parameters are updated based on the loss between the network's output and the ground truth, thereby achieving a mapping from the low-resolution domain to the high-resolution domain. Bottom row: similar cross-modality learning can also be adapted using expansion microscopy as the ground truth. The red and yellow arrows highlight improvements in lateral and axial slices due to the restoration process. By 'digital expansion', we mean the super-resolution prediction based on expansion microscopy ground truth. (b) Axial resolution enhancement [242, 228]. The method uses datasets composed of low-resolution axial slices (sometimes synthesized from higher-resolution lateral views Weigert et al. [334]) and high-resolution lateral slices, employing a one-step [242] or two-step [228] framework to learn the mapping from the low-resolution domain to the high-resolution domain, thus achieving axial resolution enhancement for fluorescence microscopy 3D stack data.

6.3 Practical Guidelines for Image Restoration

For researchers new to deep learning-based image restoration, successfully applying these techniques requires careful preparation and a clear understanding of basic concepts. Here we briefly describe practical guidelines to help you get started, from data preparation to model deployment. A step-by-step manual for different image restoration tasks (i.e., denoising, deconvolution, deaberration, and resolution enhancement) using RCAN is provided in Appendix A.

6.3.1 Considerations for Dataset Preparation

Microscope Compatibility: Ensure images are captured with consistent settings (e.g., exposure time, magnification). Common sources include confocal, widefield, light-sheet, or super-resolution microscopes.

Paired vs. Unpaired Data: Paired data (low-quality/high-quality pairs) is required for supervised learning. In contrast, unpaired data are typically used for self-supervised methods (e.g., Noise2Noise). For unsupervised methods, collecting degraded images with varying conditions (e.g., different noise levels) helps the model generalize better across different conditions.

Training Data Size: Depending on the size of the acquired data, start with at least dozens of datasets of size 128×128 for 2D and 128×128×128 for 3D. Augmentation with rotations, flips, and noise injections can expand your training dataset (see Chapter 5 for details on dataset preparation).

6.3.2 Choosing a Network Architecture

The choice of network architecture can play an important role when performing image restoration (see also Chapter 4). Here are some popular architectures used for this purpose:

- **Convolutional Neural Networks (CNNs)**: CNNs are widely used because they are highly effective at capturing spatial features in images. For example, you can use U-Net [265], a popular CNN architecture designed for image segmentation that is also well-suited for restoration tasks.
- **Generative Adversarial Networks (GANs)**: GANs can be used for tasks like image generation or synthesis (e.g., resolution enhancement), where the goal is to generate realistic images from degraded inputs. GANs consist of two networks: a generator (which creates images) and a discriminator (which evaluates the quality of the generated images).
- **Autoencoders**: These are unsupervised learning models that can be used for denoising, inpainting, and compression. Autoencoders consist of an encoder and a decoder that map input data into a lower-dimensional representation and then reconstruct it.

Choosing established networks (e.g., CARE, Noise2Void), or even starting with pre-trained models, will lead to faster deployment. See Chapter 8 for information on finding existing toos.

6.3.3 Training

Once you have your data, hardware, and software environment and have chosen a model architecture, it's time to train the model (see Chapter 9). Training involves feeding the model the degraded images and their corresponding high-quality ground truth and then updating the model weights to minimize the difference between the restored and the ground truth images.

Loss Functions: Common loss functions for image restoration include mean-squared error (MSE), structural similarity index (SSIM), and perceptual loss, which helps preserve perceptually important image details.

Optimization: Use optimizers like Adam [156] or Stochastic Gradient Descent (SGD) to update the model's parameters during training.

Training deep learning models can take a significant amount of time, depending on the size of the dataset, the complexity of the model, and the hardware you use. You can use techniques like early stopping [145] to avoid overfitting and ensure the model is trained for an optimal number of epochs [278, 213].

6.3.4 Validation and Deployment

After training, it's important to evaluate your model's performance (see Chapter 10). This can be done by testing it on a separate test set that wasn't used during training. See Chapter 5 for more information on how different datasets are used for model training and evaluation.

Common evaluation metrics for image restoration include peak signal-to-noise ratio (PSNR), SSIM, and visual quality assessment. These metrics help assess how close the restored images are to the ground truth in terms of quality and structure. Perform biological validation (e.g., expert evaluation, cross-validation with alternative techniques, or repeated experiments to evaluate uncertainty) to check that restored images are free of artifacts.

If the initial results are not satisfactory, you may need to fine-tune your model. This can involve: adjusting the model architecture (e.g., adding more layers or changing the number of filters), using a different loss function, and fine-tuning hyperparameters like the learning rate (see Chapter 9).

Once your model has been trained and evaluated, the next step is deploying it. Depending on the use case, you might deploy it for real-time image restoration or integrate it into a larger image processing pipeline.

6.4 Limitations and Future Perspectives

The integration of AI into fluorescence microscopy image restoration has opened new avenues for biological discovery. However, as the field evolves, critical challenges and opportunities must be navigated carefully. Below, we outline key failure modes, caveats, and emerging trends, including the role of transformers and large models, to guide future research.

6.4.1 Caveats of AI-based image restoration

While AI has made remarkable strides in fluorescence image restoration, there are still failure modes that researchers must be aware of. One potential issue is overfitting, where a model becomes too specialized to the training data and struggles to generalize to unseen data. This can result in poor restoration performance on images that differ from the dataset used for training.

Additionally, the loss of fine details [250, 289] and artifact generation [355, 186] remain concerns. Restored images may exhibit blurring, distorted textures, or unrealistic artifacts [33], especially in high-resolution regions or when attempting to restore challenging features (e.g., dense and overlapping structures [251] or highly degraded structures [128]). For example, while doing confocal-to-STED microscopy restoration with RCAN, certain microtubules evident in the STED images remain unresolved in the RCAN result; and the RCAN prediction for nuclear pores revealed slight differences in pore placement relative to STED ground truth [58]. When denoising fly wing data with CARE, multiple independently trained models produced clearly dissimilar results [334]. During denoising, conflicts between restoring local details and enhancing global smoothing can arise [358, 231]. Due to AI's inability to achieve a global understanding of semantic information, stitching artifacts may arise during large-scale data processing involving tiling operations [241], resulting in inconsistent intensity, geometry, and textures. These artifacts may not always be immediately noticeable but can significantly affect subsequent analysis and interpretation. Ensuring algorithm robustness across varied datasets and imaging conditions is key to minimizing these issues.

Deep learning models often operate in a "black-box" manner. This means that while these models can produce useful predictions, the reasoning behind their predictions is not easily interpretable by humans. This lack of transparency and interpretability means researchers might not know exactly how or why certain features in an image are being restored in specific ways. This issue can undermine trust in the model's results and limit its acceptance in critical applications.

AI-based image restoration techniques are highly dependent on the quality of training data. For fluorescence images, obtaining high-quality ground truth data can be challenging, especially when using highly specific imaging conditions or performing imaging on rare biological samples. The availability and variability of the resulting data can limit the model's ability to generalize to diverse datasets.

Deep learning models can be computationally expensive and require powerful hardware for training and inference. This can be a significant barrier for some research labs or organizations with limited access to high-performance computing resources. Researchers need to consider these limitations when planning AI-based restoration projects and balance tradeoffs between model complexity, data availability, and computational requirements.

6.4.2 Outlook for the Future

With advances in transfer learning and self-supervised learning, AI models are likely to become more efficient, requiring less annotated data and computational power. As datasets grow and become more diverse, the dependency on highly specialized ground truth data will be reduced. Moreover, the integration of AI with microscopy platforms in real time will enhance the ability to process images on the fly, providing immediate feedback to researchers and enabling dynamic imaging of live cells and tissues.

Newer methods of AI will be developed. Transformer models and large-scale models have shown remarkable success in other domains like natural language processing and computer vision. Transformer architectures, known for their ability to capture long-range dependencies and global context, could significantly improve image restoration tasks by better handling complex, large-scale image structures. Large models—trained on massive datasets—are likely to offer even greater performance, able to generalize across different imaging modalities and restoration tasks. As computational resources continue to expand and more sophisticated models are developed, we can expect these methods to further push the boundaries of image restoration, achieving even finer levels of detail and more accurate reconstruction.

In conclusion, the future of AI in fluorescence microscopy image restoration is bright. While challenges such as interpretability, data quality, overfitting, and computational demands remain, the field is poised for rapid advances with new architectures and training techniques.

7 Adding AI to Hardware

An Introduction to Smart Microscopy

Victoria Augoustides[1], Giorgio Tortarolo[2], Alara Kiris[2], Suliana Manley[2], & Wesley Legant[1]
[1]University of North Carolina at Chapel Hill, [2]École Polytechnique Fédérale de Lausanne
Victoria Augoustides, Giorgio Tortarolo, and Alara Kiris contributed equally to this chapter. Suliana Manley and Wesley Legant are co-corresponding authors.

Microscopy experiments answer biological questions by measuring structures and dynamic processes in living systems. To collect useful data, the microscopist must decide where, when, how fast, and how long to image based on features and processes in the specimen. Traditionally, the microscopist identifies regions of interest (ROIs), monitors sample behavior, and adjusts acquisition settings using domain expertise and visual inspection. While effective for small-scale experiments, this manual workflow limits throughput, introduces operator-dependent variability, and can miss rare or rapidly occurring biological phenomena.

These challenges motivate the development of smart microscopy systems that can automatically recognize biologically relevant features and respond in real time. Automated decision-making can increase acquisition speed, improve reproducibility, and enable experiments that would be impractical for a human operator to perform manually. Recent advances in machine learning have substantially expanded the range of biological signatures that can be recognized during image acquisition, bringing microscopy closer to a paradigm in which imaging systems actively direct an experiment's trajectory rather than simply record data.

In this chapter, we introduce the concepts, technologies, and challenges involved in developing AI to augment microscopy experiments in real-time. Our goals are to:

- Define biological objects and events as targets for image acquisition and analysis
- Describe the evolution of automated object detection from classical image processing to modern machine learning
- Explain the unique data, labeling, and modeling considerations associated with event detection and real-time performance
- Evaluate the opportunities, limitations, and reliability of AI-driven microscopy workflows

7.1 Framing the Event Detection Landscape

For an imaging system to recognize and respond to biological activity, it must first distinguish between objects and events:

- An object is an identifiable feature located within a two-dimensional image or three-dimensional volume, such as a cell, organelle, or protein assembly. Objects are primarily defined by spatial characteristics including size, shape, intensity, and location.
- An event, by contrast, represents a change or transition that unfolds over time with distinct start and end points. Examples include cell division, mitochondrial fission, receptor clustering, or transient calcium signaling.

Whereas object detection relies primarily on spatial information, event detection requires both spatial and temporal information because the defining characteristic of an event is its evolution over time (Figure 7.1). This distinction has important consequences for image acquisition. To detect an object, the microscope must resolve the object's relevant spatial features. To detect an event, the microscope must additionally sample the event at sufficient temporal resolution and for sufficient duration to capture its dynamics.

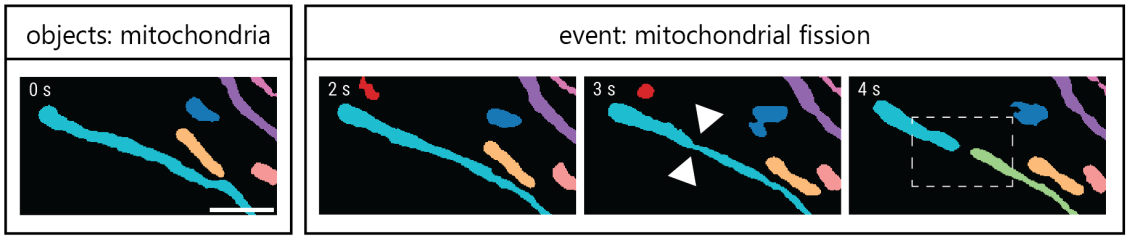


Figure 7.1: **Objects versus events, illustrated with mitochondria.** *Left*: objects, individual mitochondria, are identified and color-coded. *Right*: an event, fission of the cyan mitochondrion. Arrowheads (3 s) mark the constriction preceding scission; the dashed box (4 s) highlights the event site. Capturing objects requires sufficient spatial resolution; capturing events additionally requires sufficient temporal resolution and duration to resolve their dynamics. Scale bar, 2 µm. Figure credit: raw image acquired by Tatjana Kleele.

The spatial scales relevant to event detection span many orders of magnitude, ranging from millimeter-scale tissue remodeling, through tens-of-micrometer whole-cell movements, down to nanometer-scale reorganization of protein complexes. Temporal scales are similarly diverse, extending from days for embryonic development, to hours for processes such as cell division and differentiation, and down to milliseconds for rapid phenomena such as calcium transients, ion-channel activity, and single-molecule receptor binding. Importantly, many biological events encompass multiple spatial and temporal scales. For example, molecular-scale signaling events may occur rapidly and subsequently give rise to slower changes in cell morphology (Figure 7.2).

Such complexity underscores the advantages of tailoring image acquisition parameters to the biological process under investigation, a central objective of smart microscopy.

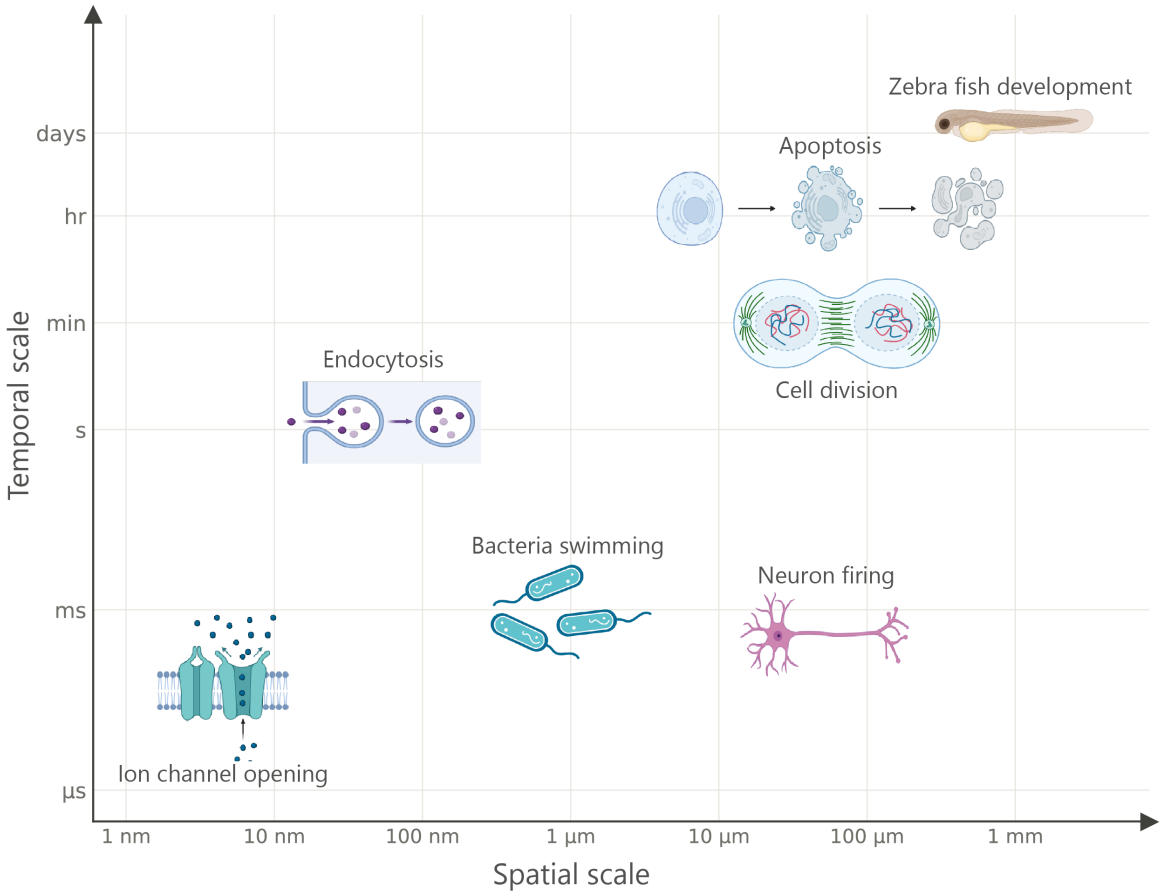


Figure 7.2: **Spatial and temporal scales of biological events.** Processes are positioned by characteristic feature size (nm–mm) and timescale (µs–days), spanning ion-channel opening, endocytosis, cell division, apoptosis, and zebrafish development. Created with BioRender. Figure credit: inspired by Fig. 2 of Daetwyler & Fiolka [71].

Once the relevant spatiotemporal scales of an event have been identified, an appropriate microscopy modality can be selected. Different imaging methods provide distinct trade-offs between spatial resolution, temporal resolution, imaging depth, and phototoxicity. Widefield, spinning-disk confocal, and light-sheet microscopy generally offer high temporal resolution suitable for capturing fast dynamics, while providing varying levels of volumetric imaging capability and moderate spatial resolution. In contrast, super-resolution techniques such as stimulated emission depletion (STED) microscopy achieve exceptional spatial detail, often at the expense of acquisition speed and increased phototoxicity [341]. Label-free imaging approaches, although not discussed in detail here, provide complementary contrast based on

intrinsic sample properties while minimizing photobleaching and phototoxicity [290].

Regardless of the imaging modality used, biological events must produce detectable and consistent changes within the acquired data. Some events manifest as morphological transformations, including cell migration, membrane remodeling, organelle reorganization, or changes in protein-complex architecture. Other events are more readily observed using fluorescent biosensors that report biochemical states such as enzyme activity, ion concentrations, metabolite levels, or membrane potential. These processes can be measured through image-derived signals including changes in fluorescence intensity or fluorescence lifetime. In each case, the measured signal serves as a proxy that provides an indirect but interpretable readout of the underlying biological event.

By selecting imaging strategies that align with the spatial and temporal scales of interest, researchers can ensure that the essential features of biological objects and events are faithfully represented in the acquired data. The next challenge is to determine when and where specific events occur. In the following sections, we discuss how this challenge is addressed through computational approaches to object and event detection.

7.2 Foundations of Automated Object Detection in Smart Microscopy

Before modern deep-learning methods, object detection in microscopy was typically performed using a combination of image-processing and machine-learning operations. Although the specific algorithms varied, most approaches followed a common workflow: first, objects were isolated from the background [210], second, quantitative features were extracted from each object, and third, these features were used to classify objects into biologically meaningful categories [48, 25].

The goal of an object detection algorithm is to identify objects within an image and describe their spatial location and extent. Figure 7.3 A illustrates representative biological objects that vary widely in their spatial distribution, size, and morphology, including nuclei, cytoplasmic RNA and nucleoli, mitochondria, and other cellular structures. Object detection pipelines generally characterize each detected object by reporting:

1. the centroid coordinates of each object,
2. the coordinates of a bounding region, and/or
3. a class label describing the object's identity [225].

The first step in many classical detection pipelines is image segmentation, in which pixels are separated into objects and background (Figure 7.3 B). A common approach is thresholding, in which pixels above a certain intensity are assigned to the foreground and all others to the background, producing a binary mask. Intensity alone is rarely sufficient to cleanly separate objects, so additional morphological operations (e.g., erosion, dilation, or watershed) can then

be applied to refine object boundaries. These operations, among others, can help distinguish neighboring objects that may otherwise appear to be merged. For an extended discussion of object segmentation, please see Chapter 9.

Once objects have been segmented, quantitative measurements can be extracted from the individual objects in the masked raw image or from the mask itself, often using tools such as CellProfiler [48]. Common features include intensity, area, perimeter, shape descriptors, texture measurements, and spatial relationships to surrounding structures [132]. Together, these measurements are aggregated into a **feature vector** that provides a numerical description of each object (Figure 7.3 B).

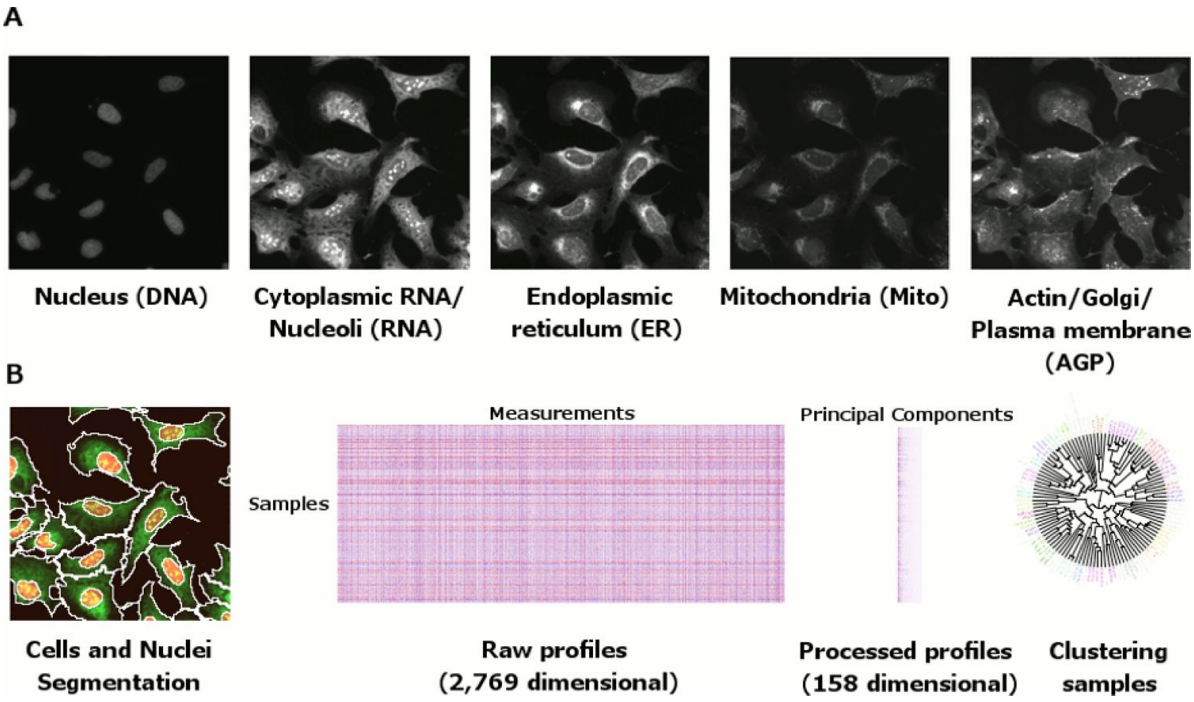


Figure 7.3: **The process of creating a feature vector from image data.** (A) Representative fluorescence images are acquired illustrating five different cellular compartments and structures: nucleus (DNA), cytoplasmic RNA/nucleoli (RNA), endoplasmic reticulum (ER), mitochondria, and the actin, Golgi apparatus, and plasma membrane (AGP). (B) Following image acquisition, individual cells and nuclei are segmented to define cellular boundaries, enabling extraction of quantitative features of the objects in (A). These measurements are assembled into high-dimensional morphological profiles, which can be processed using dimensionality reduction and clustering methods to compare cellular responses across experimental conditions and support downstream biological inference. Reproduced from Fig. 1 of Rohban, et al. [264] under a CC BY license.

The final stage is classification, in which feature vectors are assigned to biologically mean-

ingful categories [31, 222, 62]. In simple applications, classification can be performed using manually defined rules. For example, objects exceeding a particular size threshold might be classified differently from smaller objects. However, manually setting cutoffs for each feature becomes increasingly difficult as the number of measured features grows. To address this challenge, machine-learning methods were adopted to learn classification rules directly from data. Algorithms, such as perceptrons [266], support vector machines (SVMs) [68], and other classifiers [348], learn decision boundaries that separate groups of feature vectors within a multidimensional feature space. For example, a classifier may learn to distinguish normal cells from cancer cells based on combinations of morphological and intensity-based measurements in feature vectors (Figure 7.4). Compared with manually-specified rules, these learned decision boundaries can more effectively exploit high-dimensional feature sets and often generalize better to new data [29].

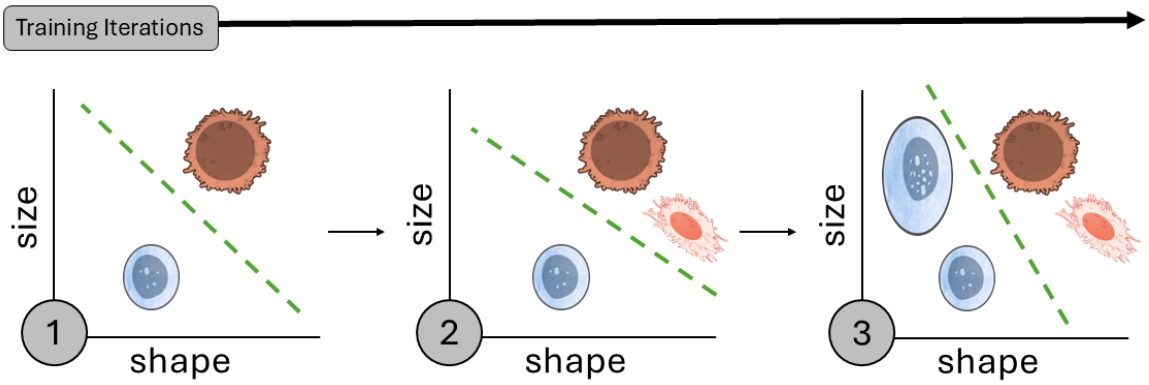


Figure 7.4: **Schematic of supervised learning with a classifier** As additional labeled examples of normal (blue) and cancer (orange) cells are introduced during new training iterations, the classifier adjusts its decision boundary to improve class separation. Inspired by Rosenblatt Rosenblatt [266]. Cell illustrations from NIAID NIH BioArt Source (items 358, 508, 376, and 82 at https://bioart.niaid.nih.gov/bioart/)

The incorporation of machine learning into classical object detection pipelines marked an important transition from automated image processing to automated decision-making. The initial use for machine-learning classifiers was primarily to categorize objects during post-acquisition analysis [148, 292, 74]. As computational capabilities advanced, these same classification approaches began to inform microscope control in real time, enabling imaging systems to react to the objects they detected [196, 6, 217].

CellProfiler Analyst (Tip 1) exemplifies the first stage of this progression. The platform automated segmentation, feature extraction, and classification, allowing large populations of cells to be quantitatively characterized using high-dimensional cytoprofiles. Machine learning improved the accuracy and scalability of object classification, but the resulting decisions remained confined to downstream data analysis after images had been acquired.

MicroPilot (Tip 2) illustrates the next stage, in which object detection was integrated directly into the acquisition process [66]. Rather than simply classifying objects after imaging, *MicroPilot* used detection results to guide data acquisition in real time. In a benchmark study, the system autonomously identified and imaged 232 mitotic cells during four nights of unattended operation. Achieving a comparable dataset through manual screening would have required more than a month of effort by an experienced microscopist. Tip 2 demonstrated how automated object detection could move beyond analysis to actively control image acquisition, a key step toward modern smart microscopy.

A common feature of both case studies, and of most object detection approaches of the time, was that machine-learning algorithms operated and learned from feature vectors rather than image pixels. Consequently, their performance depended on the quality of the upstream segmentation and feature-extraction steps, processes that may discard biologically relevant information present in the native image structure. Furthermore, object localization and classification were treated as separate stages, reinforcing a distinction between identifying where an object is and determining what it is.

Advances in model architectures and computational power eventually enabled end-to-end learning, in which object localization and classification could be optimized jointly [93]. By learning directly from image data, these approaches allowed spatial and semantic information to be captured simultaneously. Combined with the emergence of larger imaging datasets, these developments greatly expanded the capabilities of object detection and laid the foundation for modern deep-learning approaches, discussed in the next section.

Tip 1: Cell Profiler Analyst

Semi-Automation using ML for Post-Acquisition "Cytoprofiling"
CellProfiler [48] is a post-acquisition software for automated image segmentation, feature extraction, and downstream analysis of biological images. While there are many different versions of CellProfiler, this study presented an example of a semi-automated machine learning approach to object detection using CellProfiler Analyst [148, 74, 149]. Feature vectors, here called cytoprofiles, were automatically extracted from raw image data during the detection step (Figure 7.5 A). Next, a human identified phenotypes of interest from the data that the software used to create classification rules for scoring cytoprofiles. To automate classification, the user trained a supervised machine learning model (specifically a boosting algorithm applied to an ensemble of shallow decision trees [88]) for each imaging experiment (Figure 7.5 B). Once trained, the model could be applied to feature vectors from new experiments to score wells in high throughput experiments (Figure 7.5 C). Using this approach, the authors processed 40,000 fluorescence images capturing 8.3 million cells from which they detected and classified 14 distinct phenotypes, such as diverse distributions of actin and cell cycle states (Figure 7.5 D).

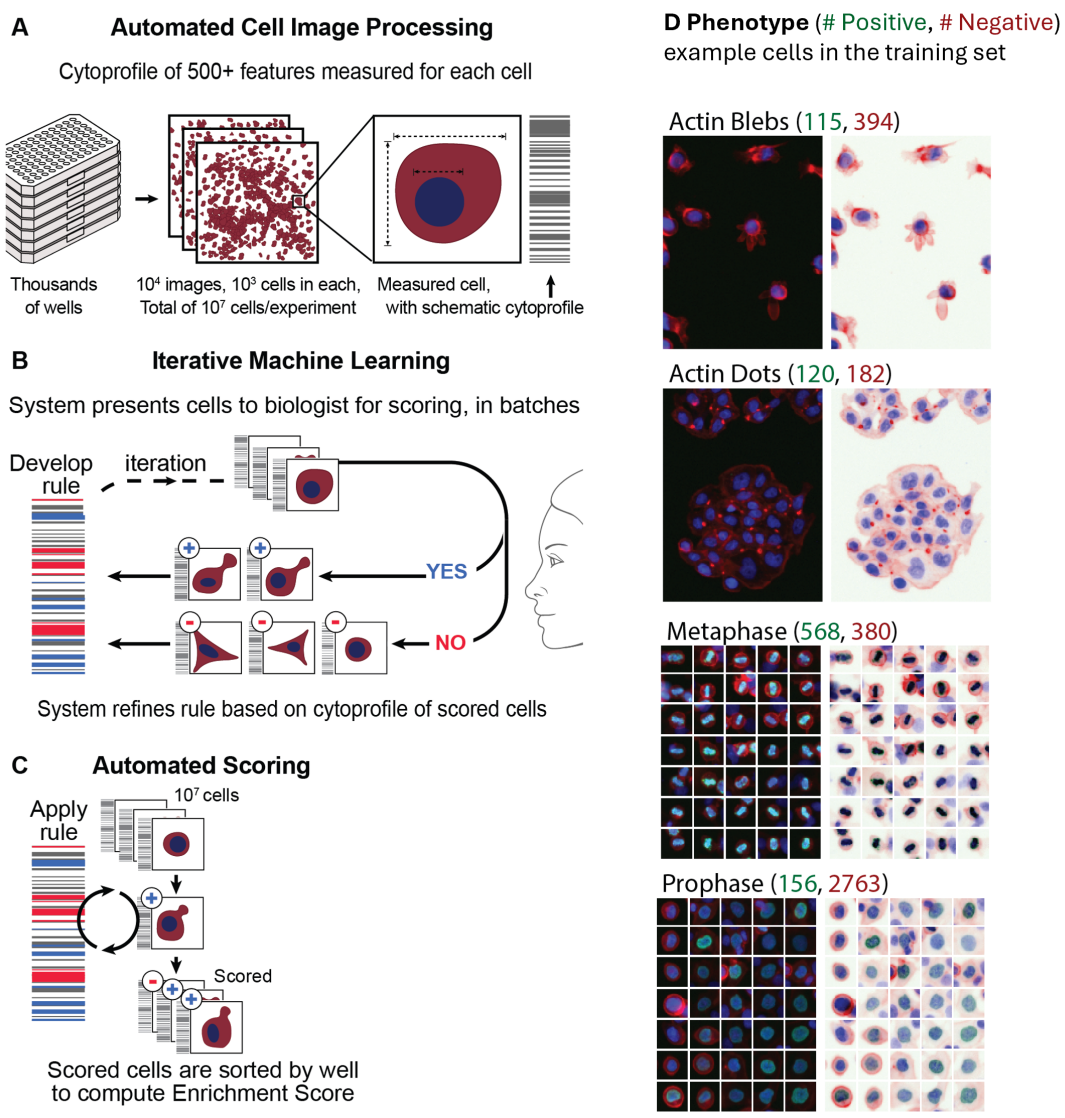


Figure 7.5: **Overview of the semi-automated CellProfiler Analyst workflow for high-content image analysis** (A) Following automated image segmentation and feature extraction in CellProfiler, each detected cell is represented by a high-dimensional feature vector, a cytoprofile. (B) A user interactively labels example cells, which are used to train a supervised machine learning classifier to recognize phenotypes of interest. (C) The trained classifier is then applied to score cells across large imaging datasets. (D) Representative examples of automatically classified phenotypes, including actin organization and mitotic cell-cycle states. Reproduced from Jones, et al. [149] with permission from PNAS: Panels A–C are from Fig. 1, and panel D is from Figs. 3 and 4.

Tip 2: MicroPilot

Integration of Object Detection with Automated Assays
Rather than analyzing data after imaging was complete, MicroPilot used ML models to classify structures from low-resolution preview scans in real time (Figure 7.6 A). First, the low-resolution preview scan was segmented, and a feature vector was generated (Figure 7.6 B). Once a structure of interest was classified by the ML model, the software automatically triggered a predefined imaging assay (Figure 7.6 B). This could range from acquiring high-resolution time-lapse videos to initiating complex laser-based perturbations, such as photobleaching or protein ablation, all without user intervention.

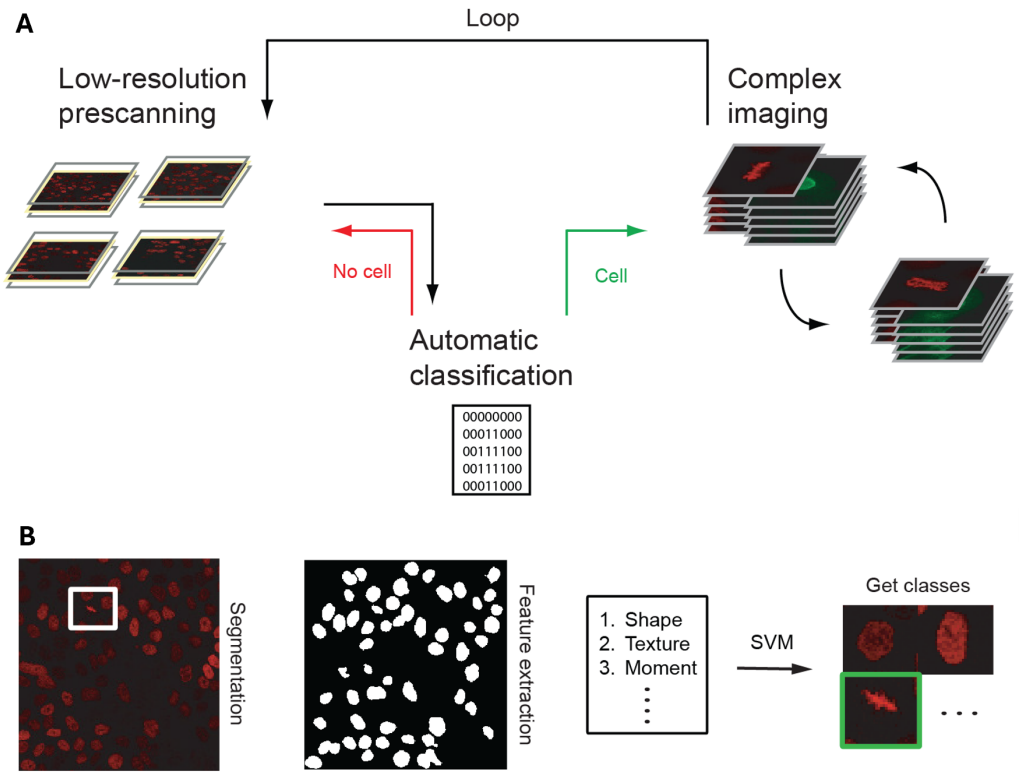


Figure 7.6: **Machine learning-guided autonomous microscopy using MicroPilot** (A) Overview of the MicroPilot workflow: Low-resolution prescans are automatically analyzed to detect and classify cells of interest. When a target phenotype is identified, the software autonomously triggers a predefined imaging assay, such as high-resolution imaging or laser-based perturbation, before returning to the prescreening loop. (B) *Left*: A low-resolution preview image is segmented to identify individual cells. *Middle*: Quantitative morphological features are extracted from each segmented object. *Right*: A support vector machine (SVM) classifier assigns each cell to a phenotype class. These classifications are then used to make real-time decisions that control subsequent microscope acquisition. Reproduced from Fig. 1 of Conrad et al. Conrad et al. [66] reproduced with permission from SNCSC.

7.3 Deep Learning for Object and Event Detection

The transition toward end-to-end learning began with networks capable of learning features directly from images, starting with convolutional neural networks (CNNs) and more recently

featuring transformer architectures. Although CNNs were originally developed in the 1980s, their widespread adoption was catalyzed by the ImageNet Large Scale Visual Recognition Challenge in 2010, which introduced a large, standardized dataset and robust evaluation metrics [75, 271].

Early datasets like *ImageNet* typically included a single centered object per image, and over time, they evolved to include multiple objects per image. Therefore, models such as *AlexNet* [162] and *ResNet* [113] were first only applied to single-object classification and localization tasks (Figure 7.7 A). Later, these frameworks were extended to classify and localize many objects to perform object detection.

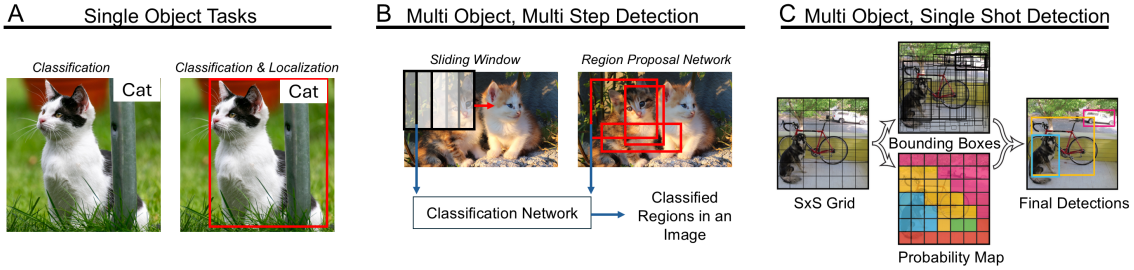


Figure 7.7: **Progression of object detection architectures** (A) Single object tasks such as image classification and localization. (B) Multi-object, multi-step detection pipelines, including sliding-window and region-proposal approaches, which separate object localization from classification. (C) Multi-object, single-shot detectors that jointly predict object locations, confidence scores, and class labels in a single end-to-end model. Figure credit: Panels A-B images sourced from Pexels under CC0; Panel C was reprinted from Fig. 2 of [259] with permission from IEEE Proceedings.

7.3.1 Evolution of Object Detection

Multi-object, multi-step detection comprises three key steps: candidate region proposal, classification, and refining classified regions. The simplest way to profile an image for candidates is to iteratively scan the image with a sliding window [306, 267, 320, 65], and then evaluate each window with a classifier (Figure 7.7 B). A more efficient approach came from the *R-CNN* series of models (2014-2017) that instead proposed regions in an image for classification (Figure 7.7 B) [93, 92, 260].

Sliding window and region-based models were succeeded by single-shot detection approaches, where detection and classification occur together in a single forward pass (Figure 7.7 C). Some of the most widely used algorithms are the *You Only Look Once* (YOLO) models, which treat object localization and classification as an optimization problem across a fixed grid [259, 258, 30, 311, 147]. Although YOLO performs classification and localization jointly, the rate limiting steps of YOLO models occur in post-processing the proposed bounding boxes [323].

These CNN-based models are well-suited for detecting small, well-separated features, such as puncta, nuclei, or other localized signals in biological images. However, CNNs can struggle to capture broader spatial relationships such as multicellular phenotypes within a tissue. Transformer-based models [317], including natural language processing models, process entire inputs at once via self-attention. For images, the *Vision Transformer* (ViT) [78], can process an entire image at once unlike CNNs. Similarly to extending early classification networks, detection models like *DEtection TRansformer* (DETR) [45] and *RT-DETR* [357] adapted the transformer architecture for object detection allowing these models to infer complex spatial relationships without relying on region-proposals, grids, or post-processing steps.

> **Transformers**
>
> Due to the complexity of transformers, we direct the interested reader to the following video resources:
>
> - Transformer Neural Networks, ChatGPT's foundation, Clearly Explained!!!
> - Vision Transformers - The big picture of how and why it works so well

7.3.2 Extending Object Detection to Events

Most object detection models, whether region-based, one-shot, or transformer-based, are designed to operate on individual two-dimensional (2D) images, treating each frame independently. In contrast, events unfold over time and require models that can reason across input sequences of images, such as a 2D image + time (2D+t) stack, rather than individual frames. Broadly, there are several strategies for detecting events from sequential image formats:

1. Decoupled Object Detection and Temporal Linking: Apply an ML object detector to each frame of a time stack independently, then use post-processing or a feedback loop to link detections over time into events [294].
2. Sequential Detection and Temporal ML Model: Detect objects in each frame and incorporate a sequence model (such as a recurrent neural network or long short-term memory) to capture temporal dependencies and classify dynamic transitions [126, 302].
3. Simultaneous Spatiotemporal Detection: Feed multiple frames of a 2D+t series into the model simultaneously allowing the network to directly output temporally aware predictions such as 2D+t bounding boxes or event labels [227].

It can be challenging to adapt ML approaches that are optimized around natural image data which is fundamentally different from microscopy images [209]. Biological signals, especially fluorescent signals, are often very sparse against a homogeneous background. Additionally, biological processes are often rare, asynchronous, and non-repetitive, unlike typical videos such as cars moving through traffic. In the next section, we delineate considerations for implementing event detection from other ML-based image algorithms in microscopy.

7.4 Building Object and Event Detection Models

7.4.1 Ground Truth and Labeling

The performance of a machine-learning model depends fundamentally on the quality of the examples used during training. Before a model can learn to detect biological objects or events, the experimental objective must be translated into a set of labeled examples, commonly referred to as ground truth. These annotations define the outputs that the model should predict from the input imaging data, such as the location of an object, its identity, the occurrence of an event, or the timing of a biological transition. For more information on designing model outputs, see Chapter 2 and Chapter 4.

In object detection tasks, labels typically describe the spatial position and identity of structures of interest. For example, annotations may indicate the location of a centrosome, or a cell or organelle boundary. Event detection introduces an additional temporal dimension: annotations must specify when an event begins and ends in addition to identifying what and where the object is. For example, labeling mitochondrial fission requires determining the precise frame at which a single mitochondrion divides into two daughter structures (Figure 7.1) [196, 302].

Creating ground-truth annotations is often the most time-consuming stage of model development. High-quality datasets may require hundreds or thousands of manually annotated examples, and it's important that within that volume, there is sufficient phenotypic diversity. For object detection, the annotator needs to capture diverse object appearances and experimental conditions. In contrast, event detection is often limited by the scarcity of events themselves and so the challenge is frequently obtaining sufficient examples of rare events. Biological imaging further complicates creating labeled data because annotations frequently require unbiased expert interpretation. Annotators must distinguish signal from noise, recognize subtle morphological features, and apply consistent criteria across diverse experimental conditions.

Furthermore, biological ground truth is rarely absolute and variability in the ground truth is multifaceted. Object boundaries may be diffuse, and event definitions may vary between experts. Differences in imaging conditions, sample preparation, fluorophore expression, and biological variability can further reduce annotation consistency. As a result, training datasets often contain a degree of uncertainty that ultimately influences both model performance and evaluation.

7.4.2 Unique Challenges of Event Detection

Many of these general challenges are amplified in event detection. Because events unfold over time, annotators must identify not only whether an event occurred, but also its precise temporal boundaries. This requirement introduces ambiguity when biological transitions are gradual or poorly defined. For example, in mitochondrial fission, multiple experts may disagree on the exact timestep when constriction becomes visible or when membrane scission occurs. Temporal

sampling also plays a critical role; if imaging is performed too slowly, key transitions may be missed entirely.

Event detection datasets can exhibit severe class imbalance because biologically relevant events may occupy only a small fraction of the recorded frames. In addition, event annotations frequently depend on information distributed across multiple imaging channels or modalities, requiring annotators to interpret spatial, temporal, and molecular context simultaneously. These challenges make event labeling substantially more demanding than object labeling and often motivate the use of interactive annotation tools [48, 25, 300, 16], weak supervision [360], or human-in-the-loop workflows [101] to improve efficiency and consistency.

7.4.3 Training Models

Once a ground-truth dataset has been established, the next step is to train a model that can accurately predict object locations, identities, or event occurrences (see also Chapter 4 and Chapter 9). Training proceeds as an iterative loop: the model makes a prediction on the image, compares that prediction to the corresponding ground truth annotation and adjusts its internal parameters to minimize the discrepancy between them. These updates are performed through backpropagation [270], an algorithm that calculates how much each individual parameter of the model contributes to the prediction error and updates the value(s) accordingly. Model optimization is guided by one or more loss functions, which quantify prediction errors in tasks such as object localization, classification, or object presence (Table 7.1).

To evaluate how well the model generalizes beyond its training data, the ground truth dataset pool is typically divided into separate training, validation, and test sets. The training set is used to optimize the model parameters, while the validation set is used periodically after each pass through the training dataset to assess performance on unseen data. Validation results can help guide decisions such as when to stop training or how to adjust model settings between training runs. Because these decisions are influenced by the validation dataset's performance, a third, independent test set is reserved for a final evaluation. Model performance is then evaluated using metrics such as precision and recall that measure how well the predictions match ground-truth labels on the test data [29].

Table 7.1: Examples of loss functions in object detection models

Type	Description	Examples
Localization Losses	Measure how close the predicted bounding box is to the ground truth box	• L1 Loss • L2 Loss • IoU Loss [347, 261]
Classification Losses	Classify each detected object into the correct class	• Cross Entropy Loss • Focal Loss [182]

Type	Description	Examples
Objectness Losses	Predict whether an object exists in a box	• Binary Cross Entropy • Focal Binary Cross Entropy

Model development typically proceeds in two phases: exploration and optimization. During the exploration phase, the goal is to determine whether the model can learn to solve the biological problem using the available data. Researchers often begin with a small representative dataset and a well-established architecture, such as *U-Net* [265], *Mask R-CNN* [114], or *YOLO* [259]. Short training runs on limited datasets can reveal whether the model converges, produces biologically meaningful predictions, or exhibits obvious signs of overfitting, where the model fits the training data but is unable to generalize to an unseen dataset. These early experiments help identify potential obstacles before substantial effort is invested in optimization.

7.4.4 Refining and Optimizing Models

Once feasibility has been established, attention shifts toward improving model performance and robustness. In many cases, the most effective strategy is to expand and improve the annotated dataset, as model performance is fundamentally constrained by the quality and diversity of the training examples. Additional annotations, refined labels, and data collected under different experimental conditions can all improve generalization. When annotated data are scarce, simulated datasets, data augmentation, or pre-trained models may also be used to bootstrap learning and reduce annotation requirements.

As model development progresses, increasingly sophisticated architectures may be introduced. Three-dimensional convolutional networks can better exploit volumetric image data, while recurrent or transformer-based architectures can incorporate temporal information required for event detection. These approaches enable models to capture complex spatial and temporal relationships that are difficult to represent using classical object detection pipelines.

Model optimization is often constrained by computational resources. Training deep-learning models on high-resolution, volumetric, or time-lapse microscopy datasets can require hours or days of computation, limiting the speed at which new hypotheses can be tested. Consequently, model development is typically an iterative process in which improvements to data quality, model architecture, and training strategy are evaluated over multiple training cycles. As datasets and models increase in scale, researchers may transition from single-workstation training to high-performance computing or cloud-based resources capable of supporting large-scale experiments.

Ultimately, successful model development depends on balancing biological relevance, annotation quality, model complexity, and computational cost. These considerations become particularly important for event detection, where temporal dynamics, class imbalance, and annotation

uncertainty introduce additional challenges beyond those encountered in conventional object detection tasks.

7.5 Real-Time Event Detection in Smart Microscopy

In smart microscopy workflows, event detection can be performed either after image acquisition or during image acquisition. These two approaches, often referred to as *a-posteriori* and *real-time* analysis respectively, differ not only in when inference occurs but also in how detection influences the experiment itself.

In a-posteriori analysis, events are identified after the complete dataset has been acquired and stored. This approach allows computationally intensive algorithms to be applied without constraints imposed by the instrument's acquisition speed and is well suited for retrospective analysis, hypothesis generation, and machine learning model development. However, because detection occurs after the experiment is complete, the results cannot influence image acquisition or experimental conditions.

Real-time analysis, in contrast, processes image data as it is acquired. Rather than serving solely as a downstream analysis tool, event detection becomes part of the imaging workflow itself. This approach is particularly valuable when phototoxicity, storage requirements, or instrument throughput limit the feasibility of continuous high-resolution imaging. Real-time inference has strict time constraints and therefore requires efficient algorithms capable of producing reliable predictions from incomplete and continuously evolving datasets.

The principal advantage of real-time event detection is that it enables adaptive microscopy. By identifying biologically relevant events as they occur, imaging parameters can be adjusted dynamically to focus acquisition effort on the most informative regions, objects, or time periods. This strategy reduces unnecessary sample exposure, minimizes photobleaching and photodamage, and decreases the collection of uninformative data. As a result, experiments can often be performed for longer durations while generating smaller, more information-rich datasets.

Hybrid event-driven acquisition (Tip 3) illustrates the benefits of adaptive microscopy for capturing rare and transient events while minimizing unnecessary fluorescence exposure [302]. This framework monitored samples using a low-phototoxicity imaging modality and selectively triggered fluorescence imaging only when an event was detected, substantially increasing the number of cells that survived long-term imaging compared with continuous fluorescence acquisition.

Real-time detection also enables closed-loop experimentation. Detected events can trigger downstream actions such as object tracking, high-resolution imaging, experimental perturbations, or automated classification. More broadly, the microscope can use incoming observations to guide future acquisition decisions, transforming image acquisition from a passive recording

process into an active, information-driven measurement strategy. This capability is central to the concept of smart microscopy.

Real-time event detection remains challenging because models must balance detection accuracy against computational latency, often while operating under limited hardware resources and in the presence of noisy biological data. Strategies that perform well for one imaging modality or biological system may not generalize readily to others, making robust deployment difficult. Furthermore, uncertainty estimates are rarely incorporated into acquisition decisions, despite their potential value for reducing false positives and missed events. Addressing these challenges will require continued advances in machine learning, microscope control software, and hardware–software integration.

Automated Early Generalizing Protein Aggregation ONset (Tip 4) addresses two important barriers to broader adoption of real-time smart microscopy: computational efficiency and generalizability. The authors showed that deep learning models could maintain high event detection accuracy using substantially fewer temporal and spatial inputs, making real-time inference feasible on more modest computational hardware. They also demonstrated that the framework generalized across different microscope platforms and imaging modalities without transfer learning, illustrating how event-driven microscopy can become more accessible beyond highly specialized experimental systems.

Together, these developments and challenges illustrate the evolution of event detection from a retrospective analytical tool to a mechanism for real-time experimental control. By enabling microscopes to respond autonomously to biological processes as they unfold, event-driven microscopy represents a major step toward intelligent and adaptive imaging systems.

Tip 3: Hybrid Event Driven Acquisition (EDA)

Smart hybrid microscopy for cell-friendly detection of rare events
Hybrid-EDA [302] is a real-time event detection and acquisition tool that addresses a fundamental trade-off in fluorescence microscopy: obtaining sufficient signal while minimizing photodamage caused by prolonged accumulation. This framework separates surveillance from event capture by monitoring the sample continuously under phase-contrast, a label-free imaging modality with minimal phototoxicity. When a neural network detects an event from subtle structural cues in the phase-contrast images, fluorescence imaging is triggered (Figure 7.8 A). This approach dramatically reduces phototoxic burden relative to continuous fluorescence surveillance (Figure 7.8 B).
The authors demonstrate this strategy by detecting mitochondrial division and organelle contacts. Mitochondrial division presents a detection challenge because phase contrast is a label-free modality that captures the whole cellular landscape alongside mitochondria, so the subtle structural cues preceding a fission event can easily be lost among unrelated features. To address this, the authors equipped a U-Net with temporal memory across frames, allowing it to track the evolution of a candidate site rather than evaluating each

frame in isolation. Hybrid-EDA captured fast, fragile physiological signals that continuous fluorescence imaging would struggle to resolve without accelerating photodamage. As one example, membrane potential dynamics were visualized at a site of mitochondrial constriction using the potentiometric dye TMRE (Figure 7.9).

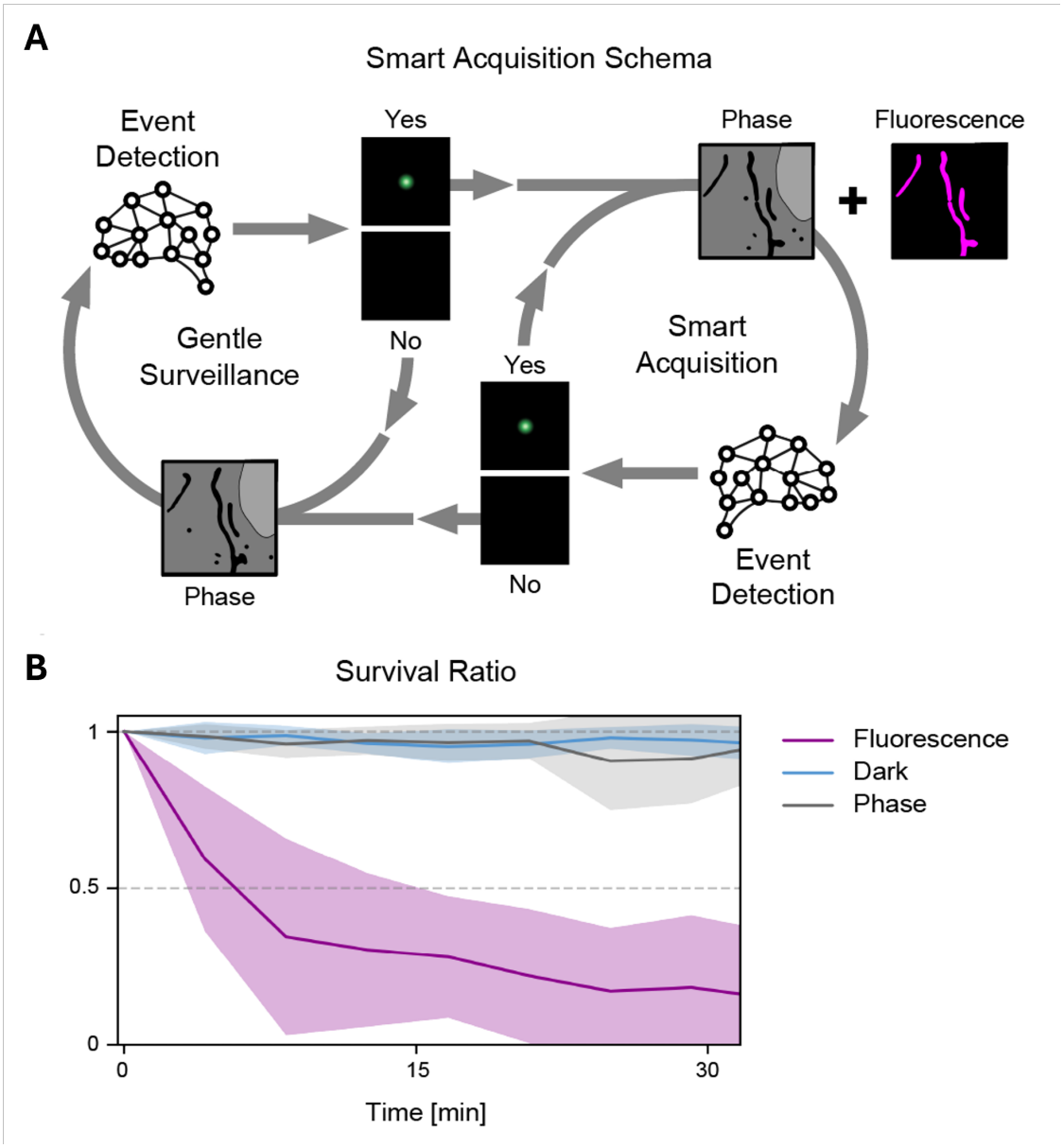


Figure 7.8: **Hybrid-EDA balances phototoxicity and event capture in live-cell imaging.** (A) Logic of hybrid-EDA: a neural network continuously evaluates phase-contrast frames and triggers a switch to combined phase/fluorescence imaging only when it detects an event, then reverts to phase-only monitoring. (B) Cell viability over ~30 minutes under continuous fluorescence, phase-contrast, and no illumination. Fluorescence causes rapid loss of viability, while phase-contrast remains comparable to the unilluminated control. Taken from Fig. 1 of Stepp, et al. [302] under a CC BY license.

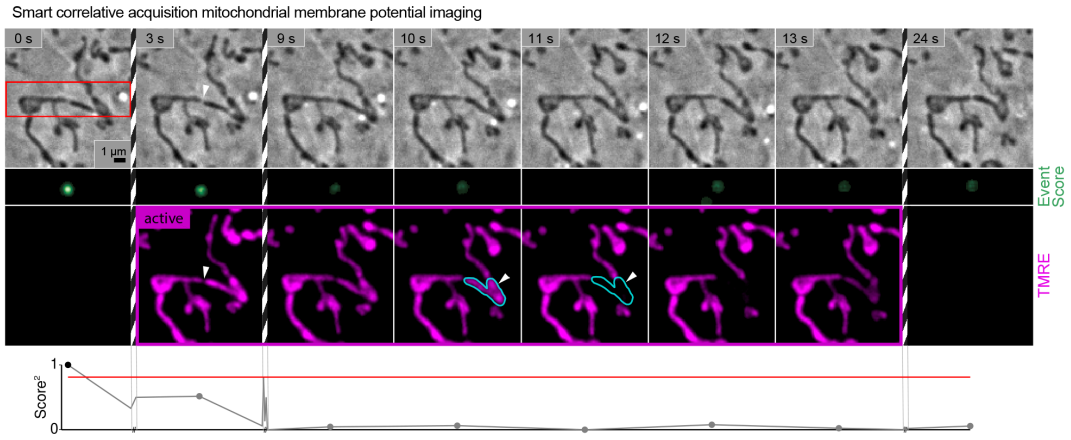


Figure 7.9: **Correlative imaging reveals a localized membrane potential drop at a mitochondrial constriction** *Top*: phase-contrast time series showing a constriction site (arrowhead) that precedes division. *Middle*: neural network event score, which exceeds threshold at the constriction and triggers fluorescence acquisition (active period, magenta). *Bottom*: TMRE fluorescence over the same period, showing a transient, spatially confined loss of membrane potential (outlined in cyan) that remains restricted to one side of the constriction before recovering. Reproduced from Fig. 3 of Stepp, et al. [302] under a CC BY license.

Tip 4: Automated Early Generalizing Protein Aggregation ONset (AEGON)

Performing longitudinal protein aggregation detection with AEGON

The AEGON framework [135] uses fast fluorescence imaging to identify candidate biological events for a slower and more experimentally demanding Brillouin microscopy assay, a scattering-based technique for measuring the mechanical properties of biological specimens [127]. The authors demonstrate the framework by identifying the onset of Httex1 protein aggregation in a well-characterized cellular model of Huntington's disease. Early detection of these events is a challenge for conventional Brillouin microscopy due to its limited throughput. First, the microscope rapidly acquires fluorescence images across a predefined grid of stage positions (Figure 7.10 A), and a deep learning model classifies each image as either an aggregation event or a non-event (Figure 7.10 B). Locations predicted to contain an event are then revisited using optimized Brillouin acquisition settings for longitudinal monitoring (Figure 7.10 C).

A major contribution of this work was demonstrating that accurate real-time event detection could be achieved with substantially less input data, reducing the computational demands of closed-loop microscopy. First, the authors trained models on 4D image stacks and systematically reduced the dimensionality of input data by testing fewer timepoints

(Figure 7.11 A). Next, they reduced the number of z-planes input to the ViViT model (Figure 7.11 B). With only a single time point and single z plane, it maintained 86% accuracy as compared to the 91% accuracy it achieved when provided with a single time point and eight z-planes. These results show that high-performance event detection can be maintained under a lower computational budget. The optimized model was then used for real-time event detection of fluorescently tagged Httex1 aggregation (Figure 7.11 C). This adaptive strategy enabled longitudinal monitoring of Httex1 aggregation using brightfield and Brillouin microscopy triggered after the initial detection event (Figure 7.11 D).

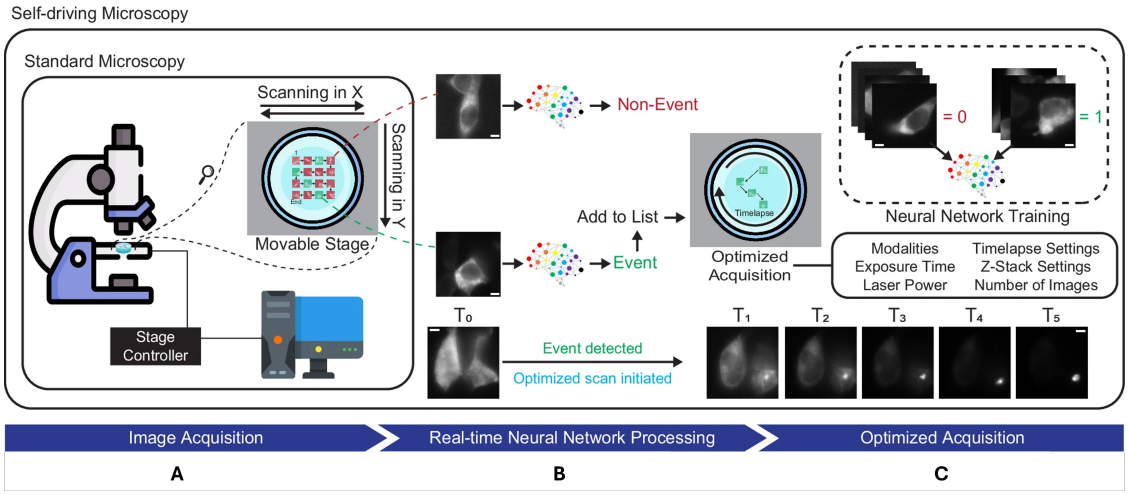


Figure 7.10: **Self-driving microscopy workflow for Automated Early Generalizing Protein Aggregation ONset (AEGON) detection** (A) The microscope performs low-cost imaging while scanning a spatial grid of stage positions. (B) Each acquired image is analyzed in real time by a previously trained neural network and classified as either an aggregation onset event or a non-event. Candidate event locations are added to a list and immediately trigger optimized acquisition. (C) The microscope automatically revisits detected events using optimized imaging parameters, including imaging modality, exposure time, laser power, z-stack settings, and timelapse frequency, to capture protein aggregation dynamics. Reproduced from Fig. 1 of Ibrahim, et. al [135] under a CC BY license.

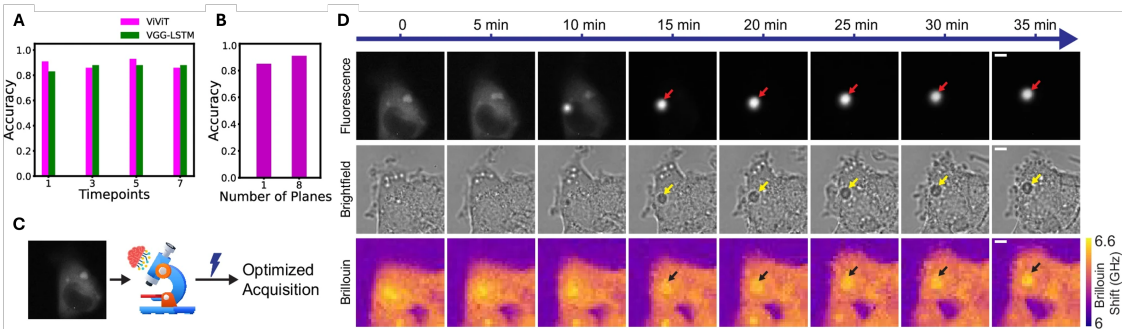


Figure 7.11: **Reducing the computational requirements of AEGON while preserving real-time event detection performance.** (A) Classification accuracy of the two deep learning architectures as the number of temporal frames used for training and prediction is varied (B) Classification accuracy of the ViViT model using a single time point with either one or eight z-planes (C) Illustration of the AEGON framework using real-time event detection to trigger optimized microscope acquisition. (D) Representative longitudinal imaging experiment. Following event detection (red arrow), the microscope automatically switches to optimized acquisition settings to monitor protein aggregation dynamics using fluorescence, brightfield, and Brillouin microscopy over time (yellow and black arrows). Reproduced from Fig. 3 of Ibrahim, et. al[135] under a CC BY license.

7.6 Challenges and Future Directions in Smart Microscopy

Despite its promise, smart microscopy remains subject to important limitations. The performance of object and event detection models depends strongly on the quality and representativeness of their training data. Models may fail when presented with unfamiliar biological conditions, imaging modalities, or rare phenotypes, and biases in annotated datasets can influence both detection outcomes and subsequent acquisition decisions. Furthermore, adaptive imaging strategies can alter experimental conditions themselves, for example, by exposing different samples to different cumulative light doses, requiring careful experimental controls and validation.

Practical challenges also remain. Event detection often requires large, expertly annotated datasets. Integrating machine-learning models with microscope hardware and control software introduces additional complexity, while increasingly capable microscopes generate data volumes that can be difficult to store, process, and interpret.

At the same time, the opportunities afforded by smart microscopy are substantial. By coupling biological event recognition directly to image acquisition, microscopes can focus on the most

informative regions, objects, and time periods while reducing data collection. Advanced instruments already combine multiple imaging modalities, adaptive acquisition strategies, or increasingly generalizable machine-learning models to detect a broader range of biological phenomena [71, 196, 294, 302, 135, 72]. More broadly, smart microscopy represents a shift from passive image collection toward active, information-driven experimentation, in which microscopes not only record biological processes but also respond to them in real time.

Part III

Image Analysis

8 How to Find and Select a Tool

Beth Cimini & Erin Weisbart
Imaging Platform, Broad Institute

We are now transitioning away from a discussion of hardware and into a discussion of software. More specifically, in this chapter, we focus on image analysis, particularly the human thought processes and decisions needed for selecting what software to use and figuring out how to appropriately and effectively use it. Amazing technological developments have occurred since the invention of microscopy almost half a millennium ago. Our current relationship to biological microscopy is profoundly shaped by the development of both quantitative microscopy and artificial intelligence in the mid-twentieth century. However, underlying the actual use of any technological development is a human (in our case, likely a biologist) selecting a specific technology to solve a specific problem.

In this chapter, we answer the questions of "how do you find new-to-you artificial intelligence (AI) models?" and "how do you assess whether a new-to-you model or tool will meet your needs?". In this chapter, we first help you assess your needs. We then introduce a way of categorizing tools and help you use your needs assessment to select the right tool category. We suggest several places where you can find AI models for bioimage analysis and describe how to assess how well tools in those locations meet your needs. Finally, we show a couple of case studies that fulfill different requirements.

8.1 Assessing your requirements for a tool (in a perfect world)

Before finding and selecting an AI tool, you first need to decide on the type of tool you need. Your tool selection will be influenced by a number of factors, and you should first honestly answer the questions that follow.

Tip 5: Workflows contain modular components

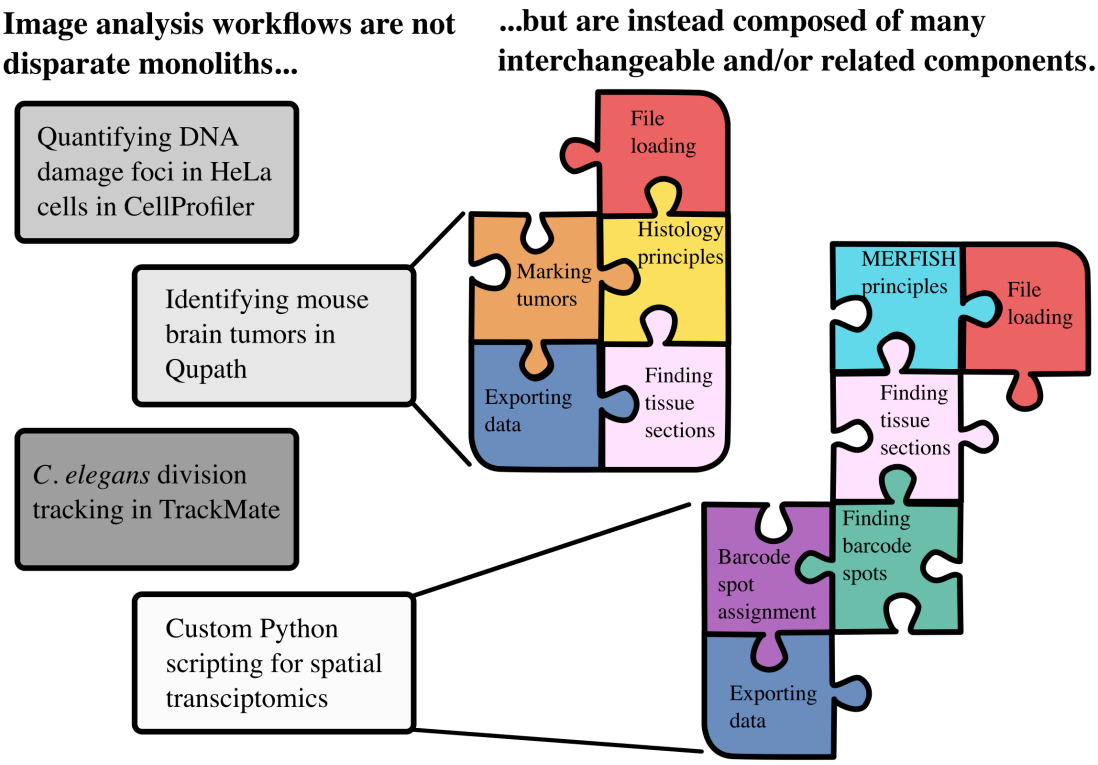


Figure 8.1: Adapted from Haase et al. [108].

When thinking of your workflow and how you might incorporate an AI tool, consider that a workflow (left) typically consists of multiple, modular subcomponents (right). You may be able to replace your whole workflow with an appropriate AI tool or you may be able to replace only a subcomponent while still keeping many of the original workflow subcomponents the same. Replacing a single subcomponent with an AI tool may force you to change surrounding subcomponents in your workflow - either with new tools or new configurations of existing tools - as the AI tool may require different inputs and produce different outputs.

Though you may have built a workflow across multiple software tools, it is worth considering whether a single software (or a reduced number of software programs) can run your workflow, and part of this consideration includes assessing how well your desired AI tool can run from within other software. Some examples of software that can be used to run other software include CellProfiler [303], Fiji [285], Bilayers [288], ZeroCostDL4Mic [51] or DL4MicEverywhere [121], and Jupyter notebooks [159].

8.1.1 Questions about your overall workflow

- **What is your desired output?**
  You must be able to concisely and specifically state what you would like as output from your workflow. Do you need to segment objects? If so, do you need semantic, instance, or panoptic segmentation (Tip 6)? See Chapter 4 and Chapter 9 for more information. Do you need measurements and, if so, should they be made on individual objects or on images?

- **How will you assess your outputs?**
  Will your assessment be qualitative or quantitative? Do you have ground truth? Do you have the expertise to assess output quality? (See Chapter 10 for more information.)
- **What is your desired "quality" of your outputs?**
  Are you expecting outputs that approach ground truth, or will "quick and dirty" be enough for you?
- **What does your ideal workflow look like?**
  Do you need all of the outputs to be produced in a single software, or are you comfortable using multiple pieces of software? If your images require preprocessing, does that need to happen in the same step? The same software?

Ground Truth

Ground truth can mean many things. The Broad Bioimage Benchmark Collection (BBBC) [190] provides a nice example of this. In the BBBC, all datasets include ground truth, but that ground truth can be one or many of counts, foreground/background, outlines of objects, biological labels, location, or bounding boxes.

8.1.2 Questions about your resources: time, expertise, and compute

- **How much time are you able to put into the task?**
  An honest assessment about the time you are willing to invest is critical at the beginning of any project . If you are in a rush, you'll probably want to select a method that is already familiar or that best matches your existing competencies.

- **What is your priority?**
  Determining what your priority is goes hand in hand with assessing the amount of time you can put into your task. Perhaps you're in a time crunch, and speed is the most important consideration for you. Perhaps you have a specific quality control (QC) metric and you need the tool that will give you the outputs that optimize this metric. Perhaps you've always wanted to gain experience with a particular class of tool, so you want to figure out if this is the right use case for trying it.

- **What is your level of computational domain expertise and comfort?**
  If you don't have a high level of computational comfort, do you have the time and motivation to expand your skillset by building new skills outside of your comfort zone? Do you have a computational collaborator and how much time – either teaching you or handling the data themselves – are they able to contribute?
  Computational domain expertise has two critical, but separable, components. The first is the ability to understand what you are doing thoroughly enough that you can design, analyze, and interpret your experiment. The other component is the ability to comfortably interact with software as a computationalist might (e.g., are you comfortable working from the command line, which is text only, or would you prefer a graphical user interface (GUI), where you can point and click?).
- **How do you like to learn things?**
  Are you a person who likes to jump in and just turn all the knobs and buttons to see what they do? Do you love to cozy up with a good written manual at the end of the day? Do video walkthroughs bring you joy or frustration? How important is it to you to be able to ask the tool creator questions directly? The type of documentation and instructional materials, as well as their depth, can vary greatly between tools.

- **What is your level of biological domain expertise?**
  Are you confident that you fully understand the task so you can assess how well your AI model is performing? Do you have a biologist collaborator and how much time are they able to contribute to designing the experiment and/or analyzing data? Do you understand what controls (e.g., single-color) and/or corrections (e.g., flat-field) you will need to make to be able to interpret your outputs?

- **What access do you have to compute resources?**
  Do you need to be able to run everything on your laptop, or do you have ready access to a high-performance cluster or scalable cloud compute? Do you have access to GPUs? Different tools have different compute requirements, especially as you scale them, and those requirements don't always scale linearly with data size.

8.1.3 Questions about your input data

- **How difficult are your images?**
  Perfect, clean data is the ideal input to any analysis. But that's not always the regime we're in. There are many sources of "difficult". A couple of examples and corresponding questions are below.
  - Do your images have debris or other technical artifacts such as out-of-focus regions? Do those artifacts need to be identified and removed? If identified, should the whole image be dropped, or do you need to keep non-artifact areas of the image?

  - Do you have metadata that needs to be associated with your images? Is that metadata organized? How is that metadata organized (e.g., in the file name, in a .csv file, in a picture of a handwritten lab notebook) and does it play nicely with the tool you would like to use?

  - Are your images in a friendly format? Are they in a proprietary file format? Are they in a file format that allows access to chunks?

- **How big is your data?**
  The larger data is, the harder it can be to work with. If your data is large, what dimensionality is big (e.g., many images, large individual file sizes, many channels)?

- **Do you need outputs in multiple dimensions?**
  If you have z-planes, do you need 3D objects, or can each plane be handled separately? If you have multiple timepoints, do you need objects tracked across timepoints, or can each timepoint be handled separately?

- **Do your images require preprocessing?**
  There are many different reasons that images might require preprocessing and some of those reasons may be a way to overcome concerns/technical challenges raised in other questions above. Some examples of preprocessing include stitching or cropping of images, denoising, background subtraction, or flat-field correction.

Tip 6: Segmentation methods

In computer vision, there are several discrete kinds of segmentation, although in the field of bioimaging we often refer to all forms of segmentation under the single blanket term. **Semantic segmentation** divides an image into classes. ilastik is an example of a popular image analysis software that performs semantic segmentation. **Instance segmentation** detects individual, specific objects within an image. CellProfiler is an example of a popular image analysis software that is most commonly used for instance segmentation. **Panoptic segmentation** is a combination of semantic segmentation and instance segmentation that separates an image into regions while also detecting individual object instances within those regions. Deep learning can be used for semantic, instance, or panoptic segmentation. Most classic image analysis methods are built around instance segmentation for cell-based images, though semantic segmentation is not uncommon for tissue/histology images.

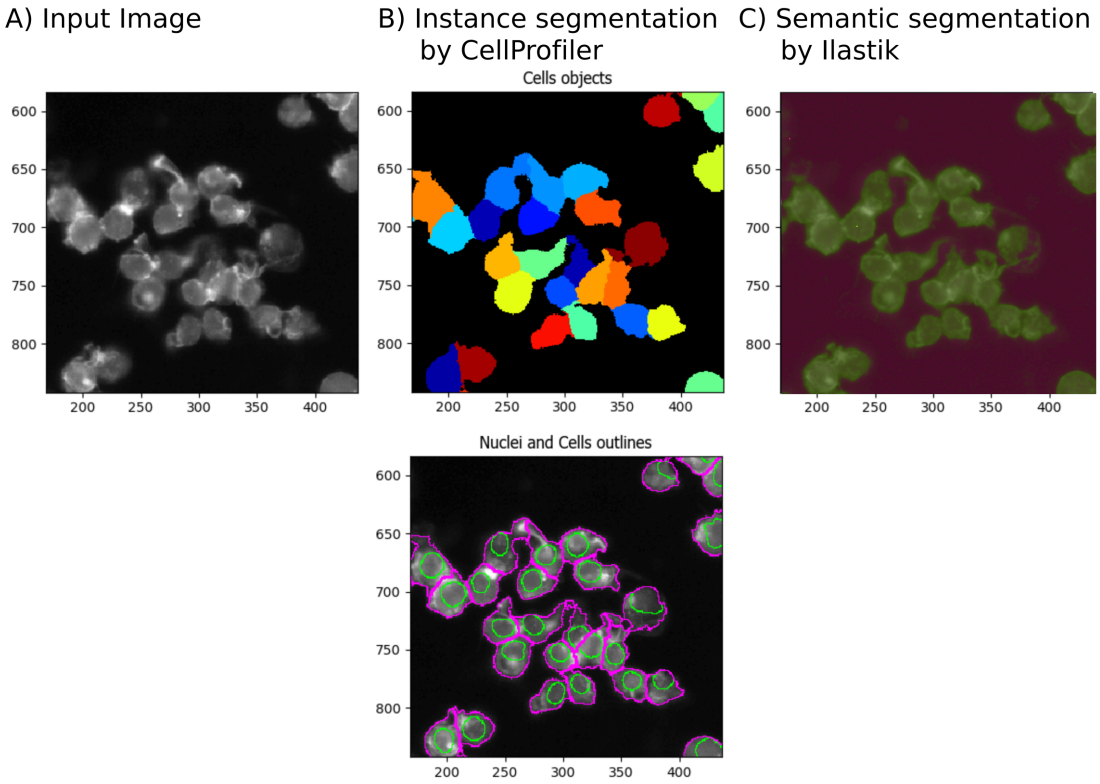


Figure 8.2: **Examples of different types of segmentations produced by CellProfiler and ilastik.** (A) Input image of Drosophila Kc167 cells, provided in the Example pipeline packaged with CellProfiler. (B) CellProfiler uses classic image processing to create instance segmentations. The individual objects identified are shown as individually colored masks in the upper panel. The lower image is the input image overlaid with nuclei objects outlined in green and cell objects outlined in pink. (C) ilastik uses pixel-based machine learning to create semantic segmentation. The image is the input image with green shaded areas in the class "Cells" and red shaded areas in the class "Background".

8.2 Assessing your requirements for a tool (in the real world)

In several previous surveys we have run, scientists have reported that they generally don't actually follow any thoughtful, rigorous process for deciding what tool they will use for a particular task. Instead, they sit down with a thing they know how to use and hope they can make it follow their use case [140, 299]. If they don't already have familiarity with a tool that

is appropriate for their task, they ask their labmates or maybe do a quick internet search. So know that if you found the section above overwhelming, you are not alone.

If you already have familiarity with a tool and it fits your newest use case (especially if your newest use case isn't *very* different from a previous use case where you've successfully used it), go ahead and stick with that tool. "If it ain't broke, don't fix it" is a common adage for a reason. But even for existing use cases, it's worth actively contemplating your priorities every so often—maybe your GPU cluster is filling up these days and is harder to access, or maybe after a new operating system upgrade, a tool that was working great is now randomly crashing, or maybe there simply is something new (or new-to-you) available that will make your life easier. In our experience, the best way to be sure you're best serving your scientific needs is simply to step back with some regularity and consider your big picture goals and whether your tool choices are serving those goals.

Once you have conducted a thorough needs assessment, it should *theoretically* be simple to determine whether a given tool or model is the right one for you. Unfortunately, no tool is likely to perfectly fit every single need that you have! You will likely be forced to break your needs into "must-haves" and "nice-to-haves", and rank tools accordingly. Make sure you consider not just how well the tool will perform your specific task, but also how well it will fit in your workflow (Tip 5).

8.3 How do I decide when my workflow is "good enough"?

Deciding when a workflow is "good enough" can be really hard! We generally factor in two major considerations when deciding what quality level we are aiming to achieve:

- **Is this a pilot or final data?**
  We generally hold pilot data to a lower quality standard than final or production data. We are looking for a proof of concept that our assay/method can work but understand that there are likely to be refinements and improvements at many points in the workflow that will increase our final quality. Additionally, during the piloting phase, there are likely to be changes introduced that alter the inputs to our workflow, so time spent beyond a certain point of refinement is lost with the adjustments required for the next pilot batch.
- **How big is the effect size we're looking for?**
  Phenotypic effect size has a large impact on necessary quality level. Simple, easily distinguished phenotypes (e.g. GFP, is either in the nucleus or cytoplasm) may reach statistical significance with many fewer individual measurements and using much simpler, cruder regions of interest (e.g., cell bounding boxes instead of careful cell margin segmentations) than more subtle phenotypes.

Tip 7: Research progress is rarely linear

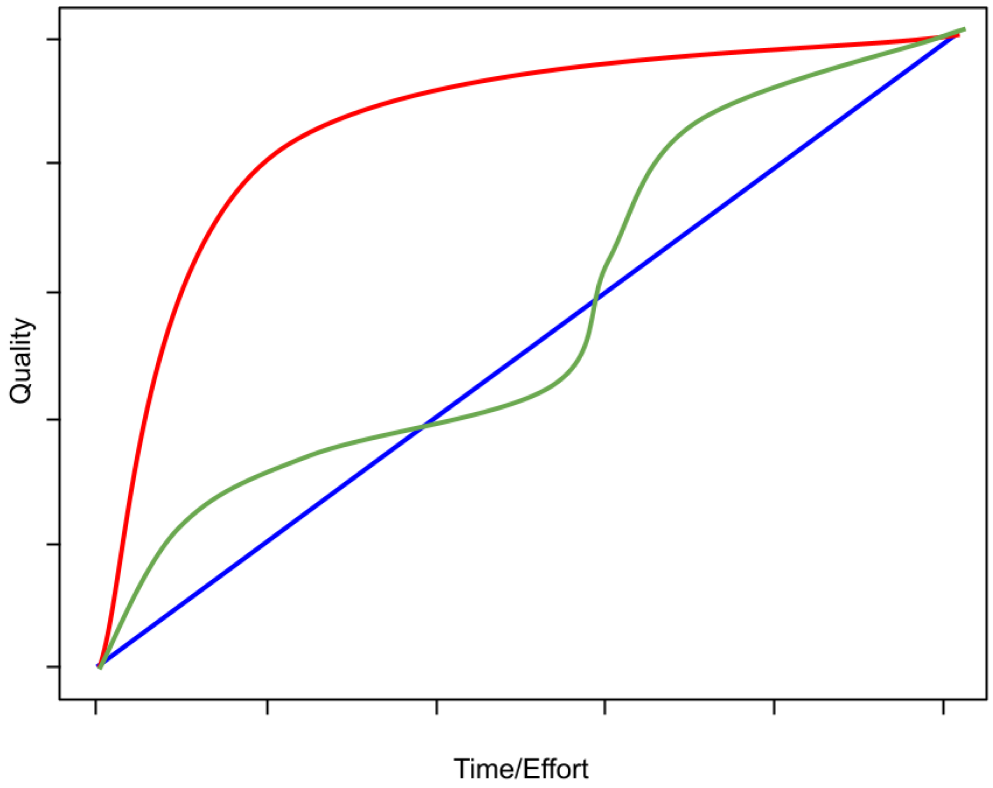


Figure 8.3: Research progress is rarely linear.

As scientists who care about the quality of our work, it's tempting to always try to maximize the accuracy of our analysis. But it's worth considering that "accuracy" vs "time spent" is often an asymptotic curve - you can keep pushing the accuracy higher and higher, but the gains may be marginal past a particular point. Time and effort spent on a project can have very different relationships with the output quality. One might hope to be in a regime where the amount of time spent is linearly correlated with the quality of a project's output (straight blue line), so that it is clear exactly how much time must be put in to get a desired product. However, research rarely proceeds in such a fashion. Instead, it progresses in unpredictable fits and starts (curvy green line) and/or it is in a state of diminishing returns (asymptotic red line) where the closer you get to "perfect", the longer it takes to improve the quality.

In [64] we detail and provide examples for a few key heuristics that we think about when

deciding whether a workflow is "good enough":

1. How close to accurate do I need to be to assess the percent or fold change I expect to see in this experiment?

2. How close to 100% accurate is it possible to get with my current workflow?

3. How important is this aspect of my experiment to my overall hypothesis?

4. How wrong is my current pipeline output?

5. What else could I do (e.g., develop a different approach/workflow, learn a new tool, generate new input data) in the time it will take me to make my pipeline maximally accurate?

8.4 Choosing the kind of tool to use

Let's consider the use case of instance segmentation (see Tip 6), which is often a critical step in image analysis workflows. We have defined six categories of segmentation methods, each of which has its valid use cases. Ordered by increasing levels of computation comfort required, they are:

- Manual annotation
- Classical image processing
- Pixel-based machine learning
- Pre-trained deep learning models
- Fine tuned deep learning models
- From-scratch deep learning models

We will not cover from-scratch deep learning models in this chapter. They are covered in Chapter 9.

It is worth noting that many of the tool suggestions below are not confined to a single "class". Many tools listed in the non-deep-learning categories allow you to run pre-trained deep learning models inside them (e.g., CellProfiler has a RunCellPose module, ImageJ and QuPath also contain the ability to run pixel classifiers). All deep learning models *can* be re-tuned with some effort (they may just not have a friendly interface for doing so). See Chapter 9 for more information on fine-tuning existing models.

Method category	Suggested light microscopy tools	Time required	Best if your priorities include	Biological expertise level required	Ability to handle *reasonable* technical noise and variability	Largest drawback
Manual Annotation: Hand-drawing the boundaries of all objects	ImageJ [285, 286], napari [223], see also this forum thread for 3D annotation	Highest	High accuracy/ outputs approaching ground truth	High	High	Inter- and even intra-operator manual annotations (e.g., after time or image rotation) variability. Irreproducibility. Cannot be automated.
Classical Image Processing: Combining mathematically well-defined operations such as thresholding and watershedding to create objects	CellProfiler [303], QuPath [16], Icy [53]	Medium	Needing to have a good sense of what the failure cases will be (e.g., when cells are too big, too small, too crowded, etc.); minimal computational footprint	High	Initially low, can be improved with experience	Most performant on very bright objects on dark backgrounds

Method category	Suggested light microscopy tools	Time required	Best if your priorities include	Biological expertise level required	Ability to handle *reasonable* technical noise and variability	Largest drawback
Pixel-based machine learning: Using properties of individual pixels to classify each pixel into a given set of "classes" (e.g., "true signal", "autofluorescence", and "background")	ilastik [25], Labkit [10]	Low to Medium	Identifying well-separated objects that are not bright-on-dark or are not the *only* bright-on-dark objects present	High	High if properly represented in the training set	Must be followed by Classical Image Processing if individual object masks are required

Method category	Suggested light microscopy tools	Time required	Best if your priorities include	Biological expertise level required	Ability to handle *reasonable* technical noise and variability	Largest drawback
Pre-trained deep learning models: Using a set of learned transformations (created by the developer) to define the positions and boundaries of objects	StarDist [287], InstanSeg [95]	Low	Maximum likelihood for good quality with minimal human time required	High	High if properly represented in the training set (which you may not know)	Unlikely to work if data similar to your data is not present in the training set. Can be hard to package into workflows.
Fine tuned deep learning models: Updating or adding to a pre-trained deep learning model with your data	Cellpose [239], µSAM [8]	Medium	Maximum likelihood for very good quality with less human time than manual annotation	High	High if properly represented in the training set	Most computationally intensive. Many available models are not provided in a non-computationalist-friendly format. Can be hard to package into workflows.

You may have noticed that all methods are listed as requiring a high level of biological expertise. It can be tempting for researchers without deep biological expertise to assume that they could just use a pre-trained or fine tuned deep learning model and call it a day. However, we would caution that, while they may require less of a biologist's time to tune at a first pass, it is still essential to involve a biologist with subject matter expertise to verify that the models are producing results that match known biology.

8.5 Finding tools and models

8.5.1 Where to look for tools and models

There are many places that one can look to find new (or new-to-them) tools and models. A few suggestions are as follows:

- BioImage Model Zoo [235] is a community-driven AI model repository that offers a variety of pre-trained AI models. All models are described by a common Model Resource Description File Specification, which simplifies using models from the BioImage Model Zoo with partner software such as ilastik [25], ImJoy [236], Fiji [285], deepImageJ [96] and ZeroCostDL4Mic [51].
- Bioimage Informatics Index (Biii) [350] is a community-curated index that includes software and helps researchers find tools whether their search is problem-based (e.g., "find nuclei in cells"), method-based (e.g., "active contour-based segmentation"), or tool-based (e.g., "CellProfiler").

- The Scientific Community Image Forum (forum.image.sc) [269] is a platform where users can pose bioimage software-related questions and receive feedback from a broad user community and maintainers from more than 75 software partners.

- Academic literature can include biology-focused research, where users describe applications, or computer science research, where the tools and models themselves are more likely to be described. Keeping an eye on preprint servers like arXiv (particularly in Computing Research Repository or Quantitative Biology sections) and bioRxiv can help you find the most cutting-edge developments.

BioImage Analysts

There is a whole discipline of people who specialize in figuring out workflows for analyzing microscopy images—their official titles often vary, but many think of themselves as *bioimage analysts*. If you want to find a local bioimage analyst, or even become one, you can check out the Global Bioimage Analysts' Society (GloBIAS), which holds events, coordinates trainings (including free help sessions), and hosts a database of bioimage

analysts and bioimage analysis trainers.

8.6 Case Studies

8.6.1 Case Study 8.1

8.6.1.1 Introduction

Elena is an image analyst who uses high-content microscopy as a method for exploring drug mechanism of action. She uses an automated microscope to acquire nine fields of view in each well of a 384-well plate and has just acquired a batch of 4 plates worth of images. She often performs the same assay, so she has a series of pipelines that she has nicely tuned for her usual experimental parameters. Elena has a new collaborator who would like to perform a similar screen in a different cell line, as the collaborator is interested in a different specific area of biology. Because this is the first time that Elena has performed this assay in a new cell line, she needs to carefully tune her existing pipelines so that she has high-quality segmentations of Nuclei and Cell objects.

8.6.1.2 Workflow

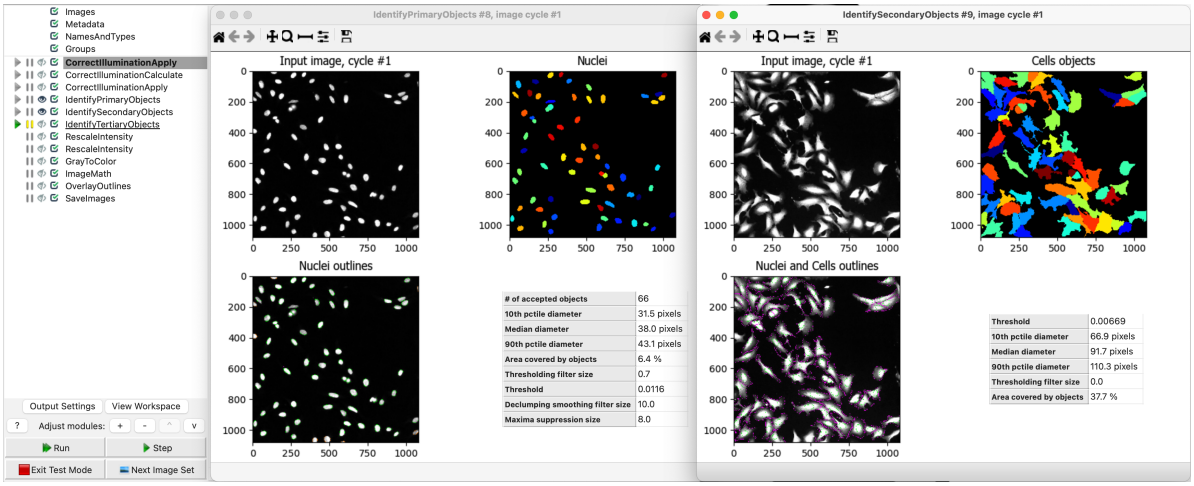


Figure 8.4: CellProfiler for nuclei and cell segmentation

Elena usually uses CellProfiler as an image analysis workflow tool, using classical image processing methods for object identification in her pipeline. Elena typically performs her screens in A549, U2OS, or HeLa cells, which are all considered easy to use for high-content

microscopy because they are relatively consistent in size, relatively round, and can grow both sparsely and confluently while remaining in a monolayer. She's quite used to making parameter changes to the IdentifyPrimaryObjects and IdentifySecondaryObjects modules to fine-tune her nuclei and cell segmentations, respectively. The image above (Figure 8.4) shows the modules in her CellProfiler pipeline (left) and the outputs of her IdentifyPrimaryObjects (center) and IdentifySecondaryObjects modules (right).

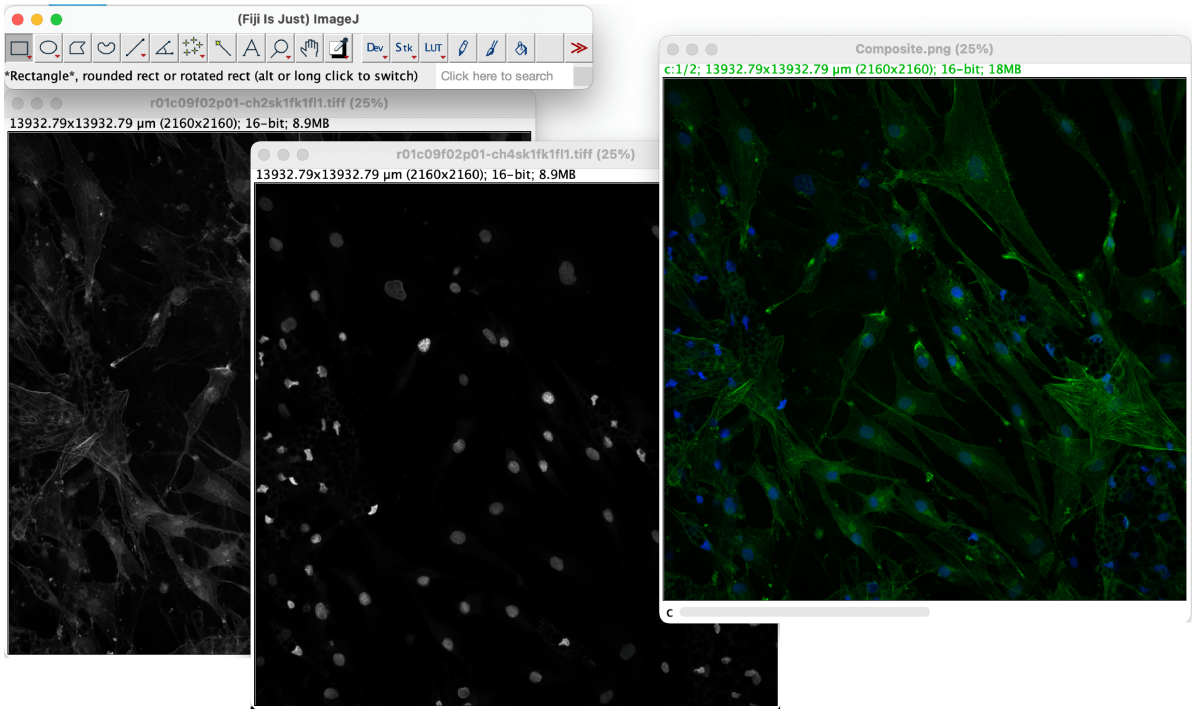


Figure 8.5: Fiji for compositing a new png image

Elena's collaborator has acquired images in an adipocyte cell line, and these cells are morphologically quite different from her typical screening cell lines. Dissatisfied with what she was able to produce using classical segmentation techniques in her standard CellProfiler pipeline, Elena decides to turn to an AI tool. She starts by opening Cellpose in its web console because she has heard from colleagues that it is both very good out of the box and allows a user to fine-tune it for their own data. She initially struggles to load one of her own images into the web tool but notices that the web page says "upload one PNG or JPG <10 MB". She also notices that the example images that are most similar to hers are in color, with a nuclear channel in blue and a cell marker in green. Elena's raw images are in .tif format with a single grayscale channel per image, so she uses Fiji to make a composite .png image. The image above (Figure 8.5) shows her single-channel .tif images opened with Fiji (left, center) and the composite .png image that she created (right).

Elena loads her new composite .png into the Cellpose web console and sees that it has identified

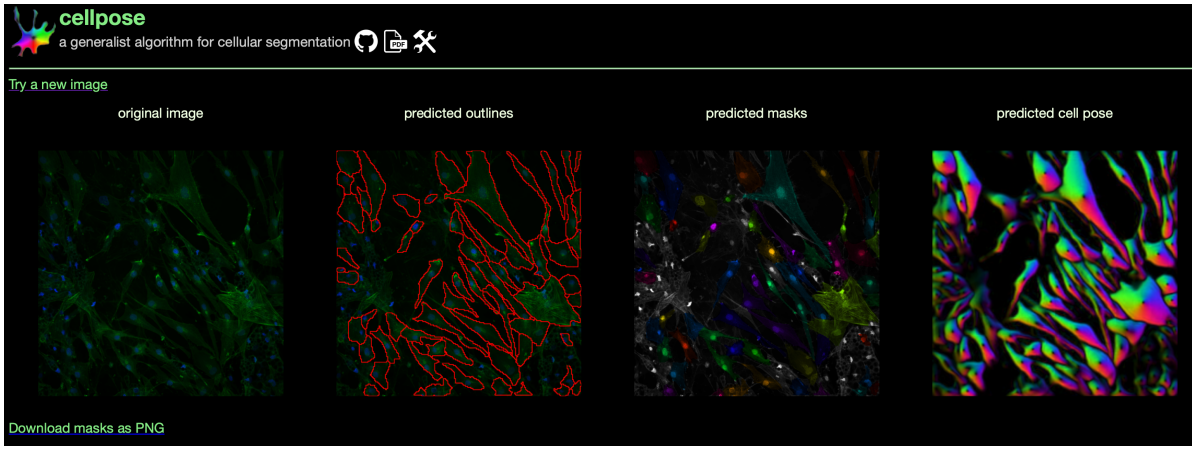


Figure 8.6: Cellpose web console

a number of reasonable cell objects but is not of the quality that she wants for use in her screen. However, Elena is excited to explore a new tool and has been actively building her computational skills so she decides to proceed with Cellpose and see if she can improve the outputs. The image above (Figure 8.6) shows her example image loaded into the Cellpose web console (far left) along with the predicted object outlines (center left), masks (center right), and flows (far right).

Elena takes advantage of the fact that Cellpose allows for fine-tuning of the pre-trained deep learning models it provides. She retrains Cellpose using a Colab notebook and sees that it now performs quite well on test images from her dataset. Because CellProfiler is also an image analysis workflow tool, Elena is able to insert a RunCellpose module in her original pipeline instead of the original secondary object identification module. This allows her to keep the rest of her image handling and analysis the same while using the newer, better-performing object identification that Cellpose provides. The image above (Figure 8.7) shows her new CellProfiler pipeline with the RunCellpose module pointing to her custom fine-tuned Cellpose model.

8.6.1.3 Conclusion

This case study starts with a scientist trying to use her standard classical image processing methods for object identification but finding that she gets better results with a new, challenging segmentation task by moving to an AI tool. It shows iterative improvements using the same tool in different ways and highlights some of the simple changes she needed to make to her overall workflow to use the new tool.

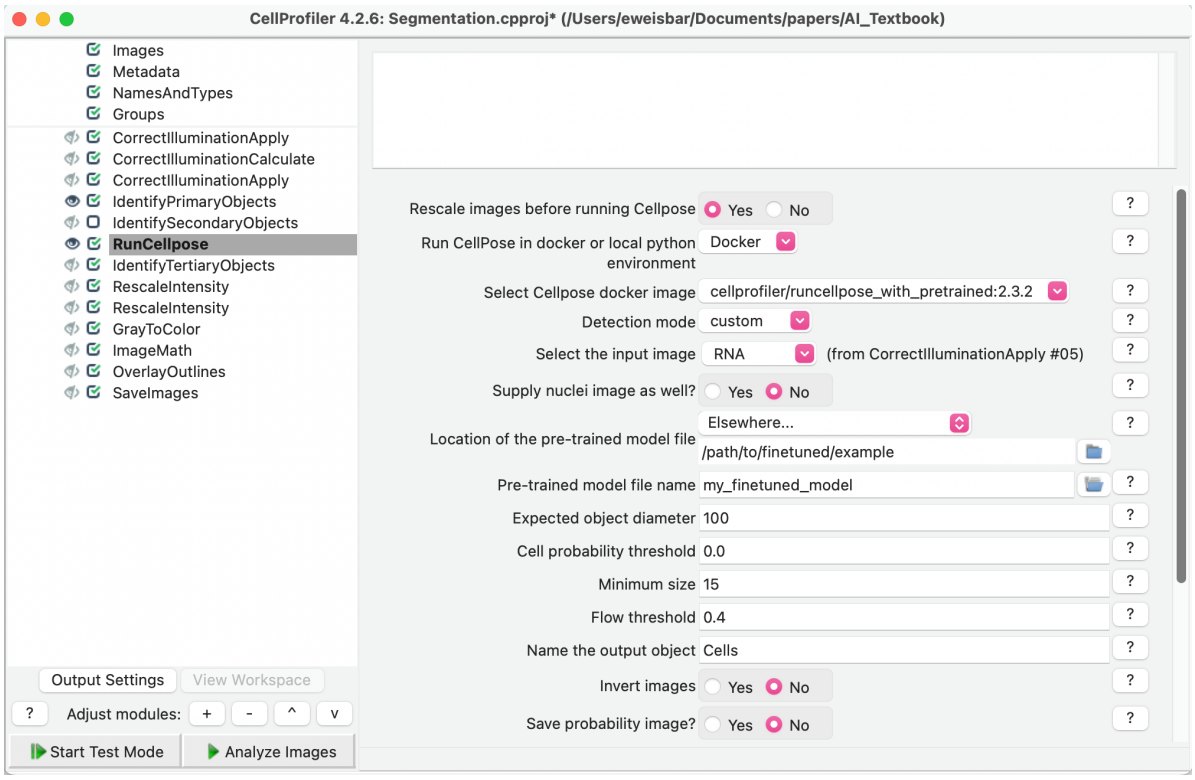


Figure 8.7: RunCellpose in CellProfiler

8.6.2 Case Study 8.2

8.6.2.1 Introduction

Esteban is an image analyst, working with imaging data acquired by some collaborators. The collaborators study zebrafish brains and sent Esteban fluorescent images taken in a large z-stack in live zebrafish. The zebrafish express a GFP-tagged protein in their microglia and the collaborators would like to be able to identify the microglia using an automated pipeline. Ultimately, the researchers aim to ask biological questions such as where the microglia are within the brain and whether they are close to other cell types (labeled in other fluorescent channels). The task is complicated because there is significant autofluorescence in the skin that they do not want to be erroneously identified as microglia. The images also have technical challenges because of how they were acquired: in live 3D images, as you go deeper in the tissue the signal tends to get dimmer. This makes using normal computer vision segmentation quite hard.

8.6.2.2 Workflow

To help orient Esteban to the image data, he was given a spreadsheet of x,y,z coordinates of microglia that the collaborators manually annotated. Esteban starts by opening the images and their point annotations in napari to orient himself to the data. The image above (Figure 8.8) shows an example image opened in napari with red circles around the point annotations.

Esteban then loads µSAM in napari to use as an annotation tool so that he could use the point annotations to make object masks. He tries using automatic segmentation, but it didn't work on his images. Instead, he uses µSAM-assisted segmentation, manually providing positive prompts, and gets out reasonable segmentations of the microglia in the image. The image above (Figure 8.9) is quite similar to the previous image but also has the µSAM segmentations in color overlays.

Though Esteban has three-dimensional data, the segmentations he is creating are 2D so he confirms that he is happy with his final masks by examining the image and masks in a 3D viewer. The image above (Figure 8.10) shows a three-dimensional view of his data that contains all of the image slices and annotations viewed together.

Now that Esteban has both images and masks, he calls Cellpose in Python and retrains the cyto model with his annotations (Figure 8.11). He runs the retrained model on a new image (left image) and finds that the cell probability prediction seems to identify microglia and autofluorescence (center image), but, unfortunately, the gradient prediction is not finding any cell objects (right image). Esteban's training images are quite sparse (the vast majority of the image is background), so the Cellpose model learned that it can be mostly correct by deciding that *everything* is background.

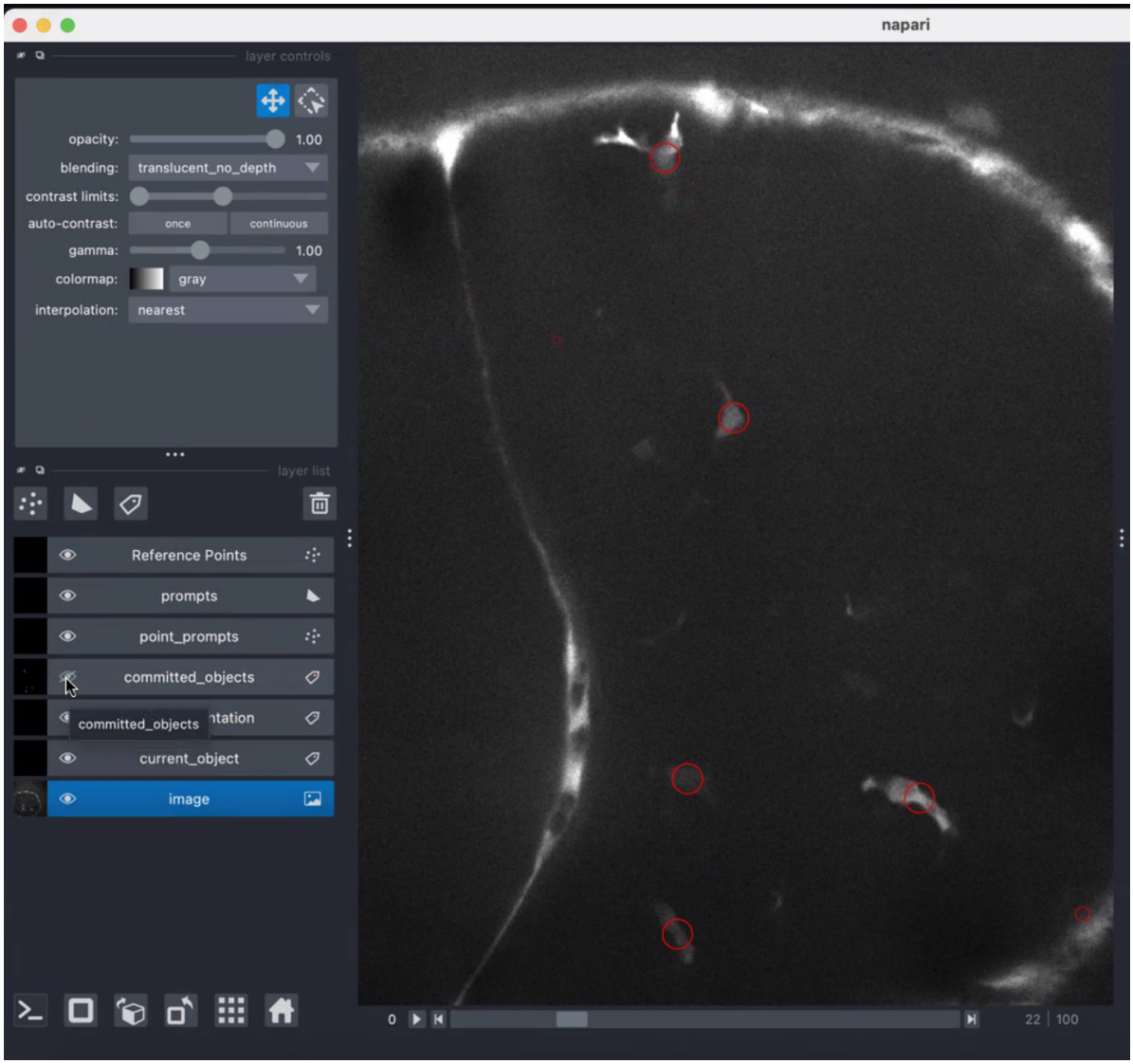


Figure 8.8: Point Annotations in napari

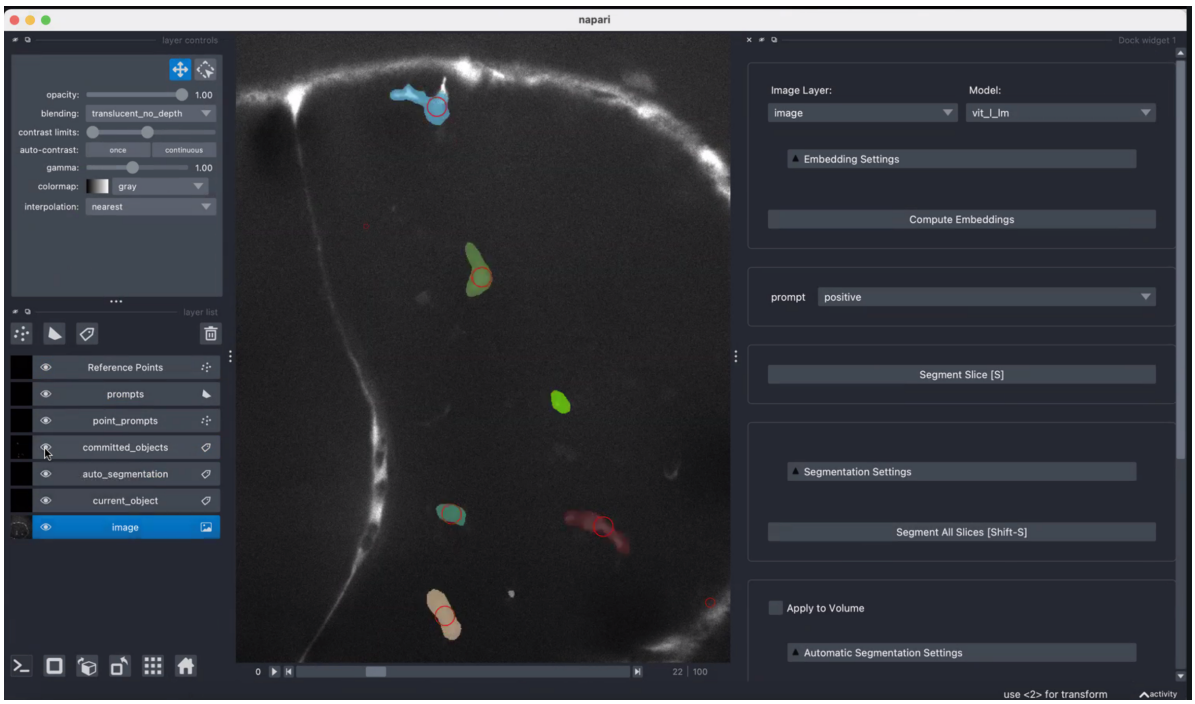


Figure 8.9: Guided Annotations in napari with SAM

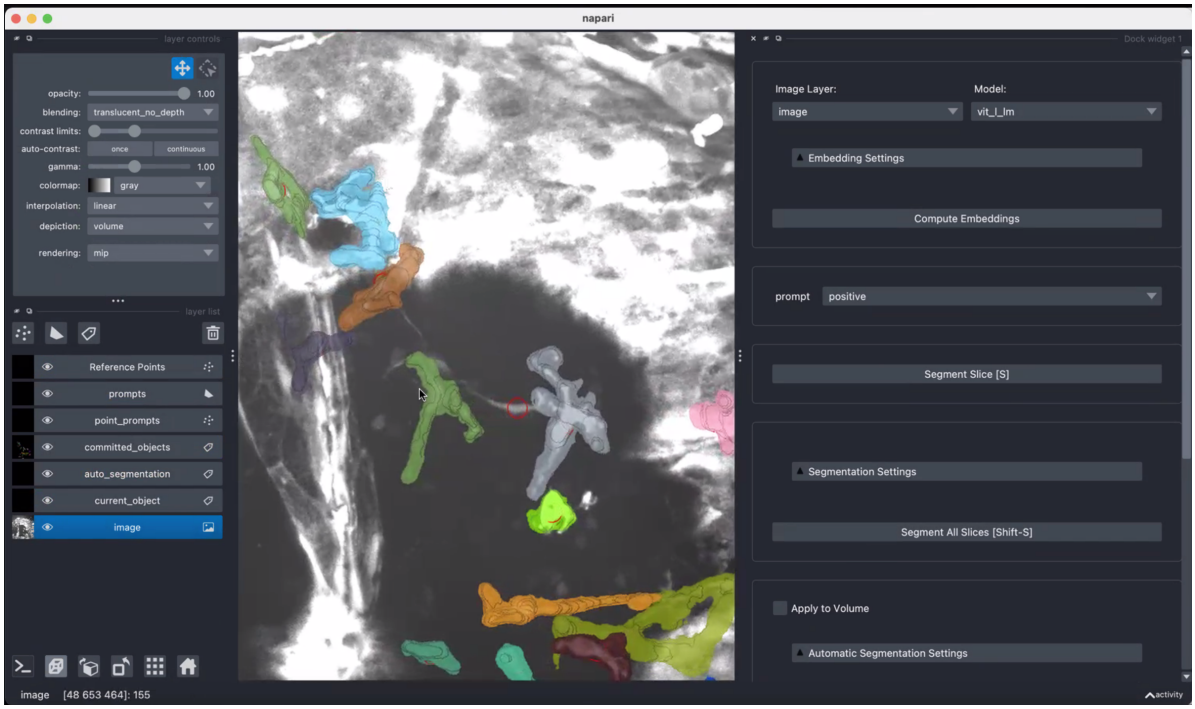


Figure 8.10: Visualizing 3D annotations in napari

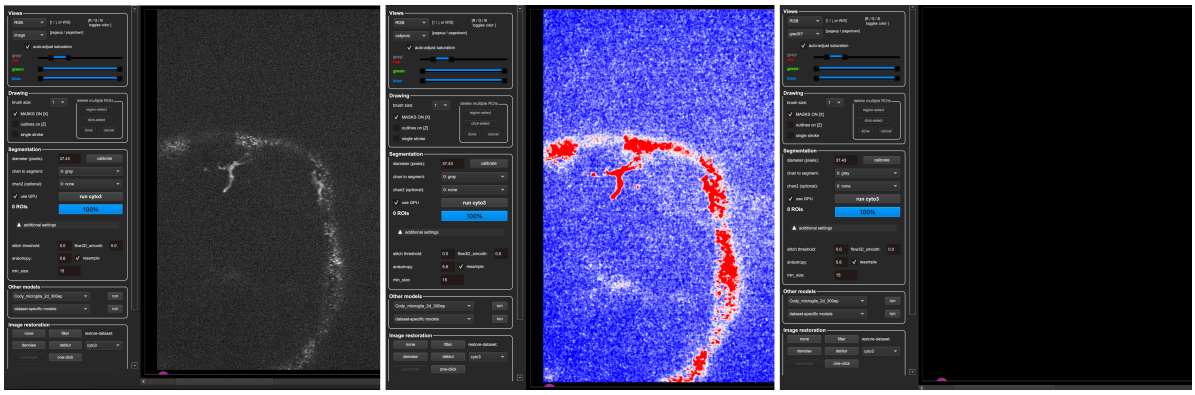


Figure 8.11: Cellpose predictions

Figure 8.12: ilastik 3 class predictions

Since neither a pre-trained model (µSAM) nor a fine-tuned model (Cellpose) worked on his images, Esteban decides to try pixel classification. He uses ilastik, first providing annotations for microglia and background. The classifier works quite well at distinguishing microglia from background, but classifies the skin autofluorescence as microglia. Esteban builds on this by using larger filters so that the classifier has more contextual information, training three classes (microglia, skin, and background) instead of two, and adding in autocontext [161], which improves classification by running it in multiple stages. Esteban's final three-class pixel classifier performs well at identifying microglia separately from skin across the images in his datasets. The image above (Figure 8.12) shows a zoom-in of one of his images with the three classes overlaid—red for background pixels, blue for skin pixels, and yellow for microglia pixels.

8.6.2.3 Conclusion

This case study starts with a scientist trying multiple state-of-the-art deep learning segmentation tools for a specific, challenging segmentation task. He ultimately discovers that a machine learning pixel classifier works better for this task, illustrating that 1) "less-advanced" tools can perform better than deep learning tools in some instances, and 2) biological expertise is always needed when inspecting any tool's outputs.

8.7 Conclusion

As we shift our focus from hardware to software, it is common and understandable to feel overwhelmed - the bioimage analysis tool space is a constantly changing array of best practices, and not all microscopists are highly computationally comfortable. This chapter therefore mostly talks about *how to make decisions*, as opposed to *use X tool for Y task*. Ultimately, the "right tool for your task" is the tool (or chain of tools) that allows you to be confident that your analysis has been performed **well enough**, helps you understand the answer to your biological question, and produces results with which you have sufficient confidence. Even if you try a tool and decide it isn't the right solution for this question, this time still isn't wasted: it has simply enlarged the toolbox you can use for answering future questions. In the subsequent chapters, we will teach you about how to train (Chapter 9) and evaluate (Chapter 10) models that could be important parts of your toolbox, but we suggest you keep our guidelines in mind as you finalize the decisions around when your model is "trained enough" and how you decide on your overall workflow.

9 How to Train and Use Deep Learning Models in Microscopy

Practical considerations through the lens of segmentation

Joanna W. Pylvänäinen, Iván Hidalgo-Cenalmor, & Guillaume Jacquemet
Turku Bioscience Centre, University of Turku and Åbo Akademi University
Faculty of Science and Engineering, Cell Biology, Åbo Akademi University
InFLAMES Research Flagship Center, University of Turku

The growing volume and complexity of image data necessitate increasingly advanced analytical tools. One example of a challenging task is image segmentation, the process of identifying and delineating structures of interest within images. Segmentation can be particularly difficult and time-consuming when dealing with large, multidimensional datasets, such as 3D volumes or time-lapse sequences, where manual annotation becomes impractical. Machine learning (ML), especially deep learning (DL), can provide effective solutions to these challenges [118].

ML algorithms learn patterns from data to perform tasks such as image classification and segmentation. Traditional ML methods, like random forest classifiers, depend on manually defined image features to classify pixels [9, 10, 25]. In contrast, DL algorithms can automatically discover and extract relevant features directly from image data using multilayer neural networks, which eliminates the need for manual feature selection. DL techniques are widely applied in complex image analysis tasks, including segmentation, object detection, feature extraction, denoising, and restoration [215, 247]. Due to their ability to automatically learn hierarchical features, DL methods usually achieve greater accuracy and efficiency than traditional ML techniques [162, 265].

Segmentation greatly benefits from ML and DL. This chapter offers practical guidance on preparing a segmentation project and emphasizes effective DL applications to tackle these challenges. While we focus on segmentation as a case study, the principles, workflows, and considerations discussed here are broadly applicable to other image analysis tasks, such as classification or denoising. Readers interested in these areas can adapt the described approaches to their specific needs.

9.1 Preparing for your project

9.1.1 Defining your task and success criteria

Every image analysis project should begin by clearly defining the scientific question you wish to answer, along with the criteria by which success will be measured. These decisions will fundamentally shape your entire workflow. Careful planning of your objectives ensures that the chosen approach closely aligns with your scientific goals and will guide critical decisions about data annotation, model selection, and performance evaluation. For more commentary on these ideas, see Chapter 8.

Since segmentation serves as the central example in this chapter, it is important to first understand the different types of segmentation tasks encountered in microscopy before discussing how deep learning methods can be applied. These tasks can typically be categorized into three main types (Figure 9.1). Each segmentation type presents distinct challenges and is suited to different biological questions:

Binary segmentation: This is the simplest form of segmentation that separates the foreground from the background. For example, in a microscopy image, this involves distinguishing cell nuclei (foreground) from the rest of the image (background). This method is useful for detecting whether a structure is present or absent without distinguishing individual objects.

Instance segmentation: This type of segmentation identifies and labels each object independently. For instance, each cell in an image receives a unique label. This method is crucial for tracking individual cells over time or measuring specific characteristics of each cell separately.

Semantic segmentation: This segmentation strategy involves labeling every pixel in an image according to its class, such as "nucleus," "cytoplasm," or "background." Unlike instance segmentation, semantic segmentation does not differentiate between individual objects within the same class. This method is beneficial for analyzing the spatial relationships and distribution of various cellular components.

Consider whether your segmentation solution is meant for a specific experiment or needs to generalize across various imaging techniques, sample types, or experimental conditions. Additionally, evaluate the volume of data to analyze, the feasibility of manual analysis, and the resources available to create a tailored image analysis pipeline. Avoid overengineering a solution when a simple analysis could provide the answer you seek.

Alongside task-specific considerations, it is equally important to clearly define the success criteria based on your objectives. For example, be prepared to answer the question, *"What do I need to accomplish for my analysis to be sufficient?"*. This is important because no analysis is ever 100% accurate. Establishing these criteria early streamlines both the development and evaluation processes, ensuring that your outcomes are scientifically meaningful and practically useful (see Chapter 10 for further discussion).

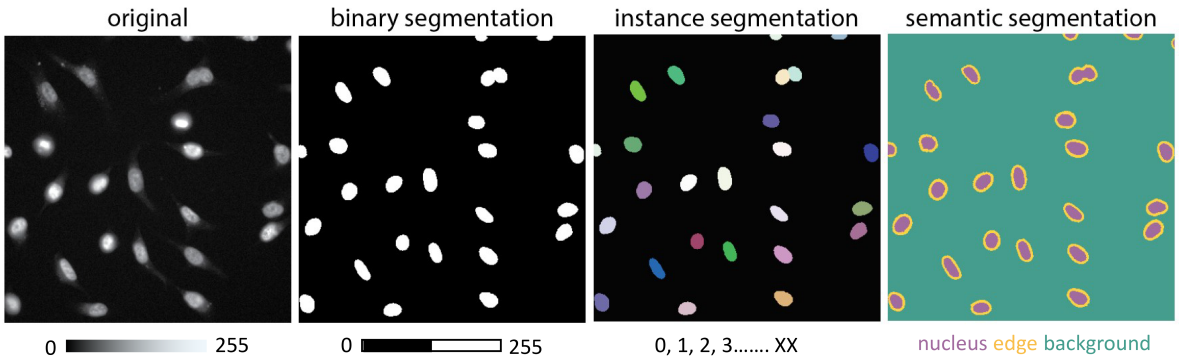


Figure 9.1: The three main types of segmentation in microscopy images. **original**: A raw grayscale fluorescence microscopy image showing cell nuclei stained with a nuclear marker. **binary segmentation**: Simplifies the image into two classes—foreground (white, nuclei) and background (black). **instance segmentation**: Assigns a unique label (shown in different colors) to each nucleus, facilitating individual object identification. **semantic segmentation**: Categorises each pixel into predefined classes—nucleus (purple), nucleus edge (yellow), and background (teal)—without distinguishing between individual objects.

9.1.2 Evaluating alternatives: Is DL the right choice?

Choosing the right computational method is essential for consistent and reproducible image analysis. For example, in segmentation tasks, while DL can deliver exceptional segmentation performance, traditional methods and pixel classifiers still offer straightforward and efficient solutions for most tasks (Figure 9.2).

Traditional image processing techniques—such as intensity-based thresholding, morphological operations, edge detection, and filtering—are ideal for objects with clear, distinguishable features. These methods are well-documented, easy to understand, and usually require minimal computing resources. Pixel classifiers, in particular, are user-friendly and can efficiently tackle many segmentation challenges with minimal manual annotation, making them highly effective for simpler analyzes or smaller datasets.

DL methods excel in complex scenarios where traditional approaches fail, especially when dealing with noisy or context-dependent data. When trained on large, annotated datasets, DL models can effectively generalize across diverse imaging conditions and sample types, rapidly processing significant volumes of images. However, in the absence of pre-trained models, DL methods rarely offer shortcuts for data analysis. DL methods generally take effort and time to implement.

If you are unsure which approach to use, we usually recommend first trying classical image processing methods and pixel classifiers (Figure 9.2). We typically initiate a DL project only if these methods fail to produce satisfactory results (see Section 9.2.3.2).

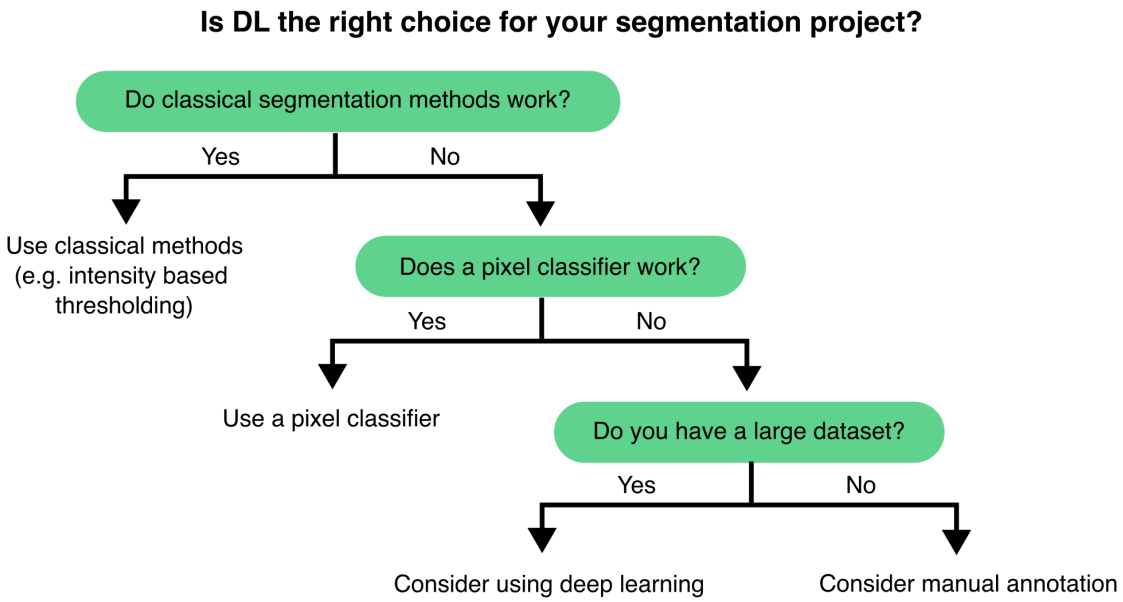


Figure 9.2: Is DL the right choice for your segmentation project? This decision tree guides the selection of appropriate segmentation approaches based on data complexity and project needs. Begin by testing classical image processing methods, such as intensity-based thresholding, which are efficient and easy to apply for well-defined features. If these methods prove insufficient, consider using a pixel classifier, which provides a user-friendly and effective solution for smaller datasets. Only consider DL if you possess a large annotated dataset and previous methods have failed. In the absence of suitable data or methods, manual annotation may be necessary.

9.2 Implementing a DL segmentation workflow

As before, although we use segmentation as our primary example, the workflow outlined in this section can be adapted to other deep learning tasks in microscopy and bioimage analysis.

9.2.1 Overview of a typical DL segmentation workflow

Once you decide to implement a DL approach for segmentation, the workflow can be divided into a series of steps (Figure 9.3).

The process starts by clearly defining your task (in this example, segmentation) and selecting the right DL approach (Step 1). Next, evaluate whether any existing pre-trained models can be used directly on your data or adapted for use (Step 2). If additional training is required—either from scratch or through transfer learning—prepare an appropriate training dataset that reflects your segmentation problem (Step 3). Then, train your model using the prepared dataset (Step 4) and thoroughly evaluate its performance using validation or test data (Step 5). Based on the results, you may need to refine the model by adjusting hyperparameters, improving annotations, or expanding the dataset. Once the model performs satisfactorily, it can be used to segment new, unseen data (Step 6). We will now discuss each step in more detail.

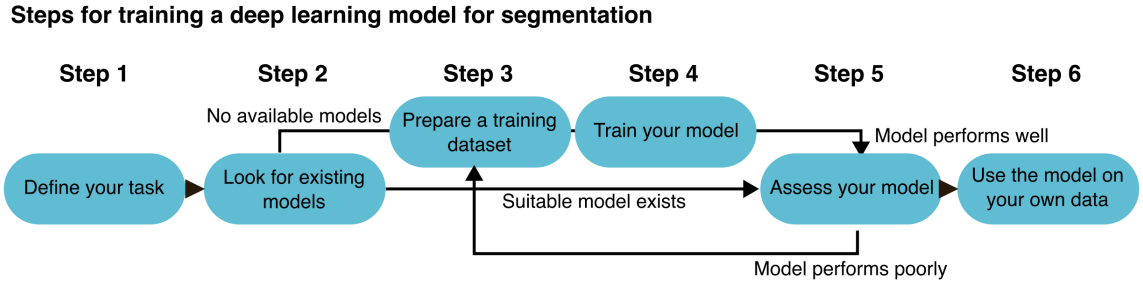


Figure 9.3: Conceptual workflow for training a DL segmentation model. The workflow begins with defining the segmentation task (Step 1), followed by searching for suitable pre-trained models (Step 2). If no such model exists, a training dataset must be prepared (Step 3), and the model is trained (Step 4). The trained model is then evaluated on validation or test data (Step 5). If it performs well, it can be applied to new data (Step 6); otherwise, the model is iteratively refined by returning to earlier steps. Feedback loops from evaluation to earlier stages help refine and improve model accuracy.

9.2.2 Selecting a suitable DL approach

The first step in choosing a DL approach for image segmentation is to clearly define your segmentation task, whether it is binary, semantic, or instance segmentation (Figure 9.1), and to determine if you require 2D or 3D segmentation. Next, you should consider whether the model or tool you plan to use makes assumptions about the shapes or structures of the objects you want to segment. Understanding these assumptions will aid in selecting a model that fits your specific biological problem (see Section 9.1.1). Additionally, consider the amount of data that needs annotation for a particular DL approach (see Section 9.2.4.5). Finally, take into account your available computational resources (see Section 9.3.2). More complex models typically demand more GPU memory, longer training times, and additional storage, especially for 3D data or large datasets.

For example, StarDist [287], a widely used tool for nuclei segmentation, assumes that objects are star-convex polygons: shapes for which straight lines drawn from the center to every boundary point are fully contained within the shapes. This assumption works well for round or oval shapes but makes StarDist less suitable for segmenting irregularly shaped or elongated structures. In contrast, Cellpose [305] uses spatial vector flows to direct pixels toward object centers. This approach enables Cellpose to segment objects of various shapes and sizes, including irregular, elongated, or non-convex forms.

Choosing the right DL strategy requires aligning your goal, object shape, data dimensionality, and computing capacity with the strengths and assumptions of the available DL architectures.

9.2.3 Deciding whether to train a new model

9.2.3.1 Leveraging pre-trained models

The increasing availability of already trained (pre-trained) DL models has greatly simplified image analysis. Many of these models can be directly applied to your data, removing the need to train your own model [21, 85]. This reduces the technical barrier and saves time, making advanced analysis more accessible. However, it is essential to evaluate the quality of any pre-trained model before relying on its results (Figure 9.4). A model that performs well in one context may not be as effective on your specific data. Always conduct quality control by visually inspecting the outputs and assessing performance with quantitative metrics such as Intersection over Union (IoU) or F1-score, using a small, representative test set. This step is vital when model predictions are used in downstream analyses (see Section 9.2.7 and Chapter 10).

Another significant benefit of pre-trained models is their adaptability. Instead of starting from scratch, you can often fine-tune an existing model (see Section 9.2.6). This method entails retraining the model with a smaller, task-specific dataset, enabling it to adjust to your images while requiring far fewer annotations.

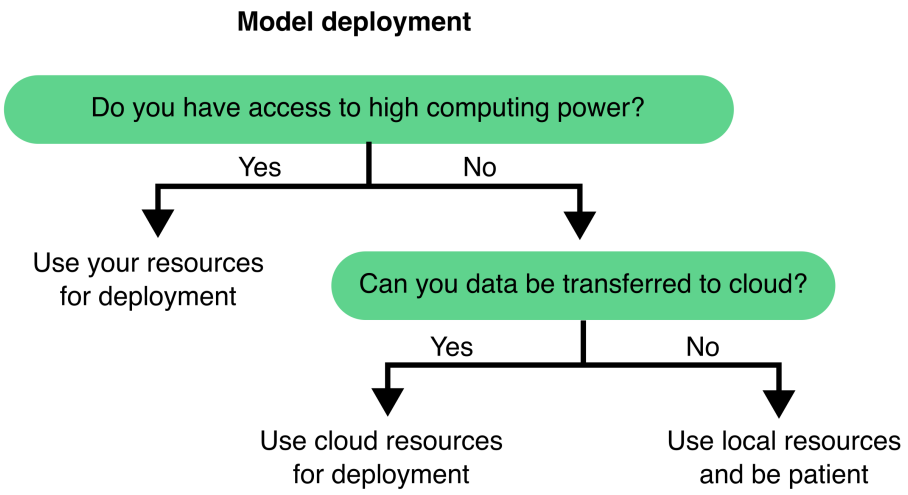


Figure 9.4: Decision workflow for selecting a DL model training approach. This flowchart outlines how to determine an appropriate training approach based on the availability and performance of pre-trained models.

Several excellent resources host pre-trained models suitable for microscopy. Researchers also increasingly share trained models alongside their datasets and publications, promoting open science. Platforms like Zenodo are commonly used for this purpose [83, 86], although deployment may require handling specific file formats or environments (see Chapter 8 for more information and resources).

9.2.3.2 When to train your model

Pre-trained models serve as an excellent starting point for various microscopy tasks. However, there are many scenarios where training a custom model becomes essential. Custom training enables the model to learn the specific characteristics of your dataset, experiment, or imaging modality, resulting in enhanced performance [8, 239, 51]. This is particularly crucial when your data differ significantly from the data used to train existing models. Thus, a model's performance should always be validated. If quality assessment metrics are poor or key features are not accurately segmented, consider training your own model.

Ultimately, always evaluate the model's performance against your defined success criteria (see Section 9.2.7). Custom training may be the best path forward if the current model does not meet your needs.

9.2.4 Preparing your dataset for training

A well-designed training dataset (see also Chapter 5) is essential for developing successful DL models for tasks such as segmentation. The number of images and the quality of annotations needed vary based on factors such as task complexity and the architecture of the intended model.

9.2.4.1 Types of model training

For segmentation, most DL models are trained using supervised learning, where each input image is paired with a manually annotated ground truth mask. In this context, all objects that need segmentation must be annotated in the training dataset. This approach enables the model to learn a direct mapping from raw images to segmentation outputs (Figure 9.7).

However, alternative approaches can help reduce the need for extensive manual annotations:

- **Unsupervised learning** trains models without paired input and output data. Instead, the network identifies patterns or similarities in unlabeled images [160].
- **Self-supervised learning** involves designing tasks in which the model learns useful features directly from the input data without needing explicit labels [184].
- **Weakly supervised learning** uses partial, noisy, or imprecise labels to guide training, which can significantly reduce annotation effort [44, 219].

9.2.4.2 Creating manual annotations

Creating accurate annotations manually is time-consuming, particularly for 3D datasets. Tools like Fiji [285], napari [4], and QuPath [16] are frequently employed for manual labeling. Typically, manual annotation involves drawing each object on the image and converting it into a mask or label.

Here is an example pipeline for manually annotating data using Fiji

See also [83].

1. **Open Fiji – activate the LOCI update site and restart Fiji.**
   LOCI tools are required for exporting ROI maps. To enable them, go to *Help* > *Update* > *Manage Update Sites*, look for 'LOCI' and check the *Active* checkbox. Then, click on *Apply and Close* and *Apply Changes*; this update site ensures the necessary plugins are installed. Finally, restart Fiji.
2. **Open the image you wish to annotate.**

Use *File > Open* to browse and load the microscopy image that you want to label manually. You can also drag and drop your image into Fiji.

3. **Select the Oval or Freehand selection tool.**
   These tools, found in the Fiji toolbar, allow you to manually outline the structures of interest in your image.

4. **Start drawing around each object (yes, each one!).**
   Carefully trace each cell or feature you want to annotate: precision is key to ensuring useful training data for DL.

5. **After drawing each object, press "t" on your keyboard → the selection will be stored in the ROI manager.**
   This adds the drawn region to the region of interest (ROI) list, keeping track of all annotated objects in the image.

6. **Repeat until all objects are in the ROI manager.**
   Continue drawing and pressing "t" until you have annotated every relevant object in the image.

7. **When finished, go to *Plugins > LOCI > ROI Map*.**
   This plugin converts all saved ROIs into a single labeled ROI map image, assigning unique values to each region.

8. **Save the generated ROI map with the same title as the original image in a separate folder.**
   Consistent naming ensures each annotated map can be correctly matched with its corresponding raw image during training or analysis.

9. **At the end, you will have one folder with the original images and another for the ROI maps.**
   This separation makes it easier to organize and use your data with image analysis or DL pipelines.

9.2.4.3 Accelerating annotation with automatic initial segmentations

Creating high-quality annotations often represents the most time-consuming aspect of training a DL model for segmentation. To alleviate this burden, you can start from automatically produced initial segmentations. For example, using simple thresholding methods such as Otsu's method to generate rough segmentations can decrease the total annotation time. Even more powerfully, pre-trained DL models such as those provided with StarDist [287] and Cellpose [305] can generate more accurate initial segmentations that users can manually refine. These annotations can then be used to retrain the model, establishing an iterative cycle that accelerates

both labeling and model refinement.

New tools are also pushing the boundaries of interactive annotation. For example, Segment Anything for Microscopy (μSAM) [8] facilitates automatic and user-guided segmentation and allows the model to be retrained on user-provided data. Similarly, Cellpose 2.0 [239] features a human-in-the-loop workflow, allowing users to edit DL-generated segmentations. This hybrid approach enhances accuracy while significantly reducing the time and effort required for manual annotation.

9.2.4.4 Expanding your dataset with augmentation and synthetic data

When the number of training samples is limited, augmentation techniques can enhance dataset diversity to improve the model's generalizability and performance on validation and testing data [170, 295]. Common augmentation strategies include image rotation, flipping, scaling, and contrast adjustment. However, it is important to apply augmentation carefully, as excessive or unrealistic augmentation can confuse the model or cause it to learn patterns that do not exist in real data.

In the absence of sufficient real data, synthetic data generated through simulations or domain randomization can help pre-train a model [180, 255]. These synthetic samples can expose the model to a broader range of scenarios early in training, before transitioning to fine-tuning with real, annotated data.

In summary, a successful segmentation pipeline relies on a careful balance between data quantity and annotation quality. Augmentation strategies can efficiently help to scale and balance training datasets.

9.2.4.5 Choosing the dataset size: specific vs. general models

In supervised training, it is crucial that each image in the training set is accompanied by a corresponding label image (see Section 9.2.4.1). The number of image-label pairs required depends on the number of labels per image, the complexity of the model, and the desired level of generalizability. Still, the key is having enough representative examples and corresponding annotations for the model to learn meaningful patterns.

Small and well-curated datasets consisting of tens of images may suffice for highly specific applications, such as segmenting cells or nuclei using a defined imaging modality [51]. In these scenarios, transfer learning can also be especially beneficial (see Section 9.2.6). Models designed to generalize across a wide range of conditions, tissue types, or imaging modalities typically require much larger and more diverse datasets (hundreds to thousands of annotated images) [305]. These datasets are essential for capturing the inherent variability in broader use cases.

9.2.5 Training a segmentation model from scratch

Once you have annotated your training dataset, the next steps are to organize your data for training, initialize your model by selecting appropriate hyperparameters, and start the training process (Figure 9.7).

9.2.5.1 Splitting your training data: training, validation, and test sets

A crucial part of preparing your dataset is dividing it into three subsets: training, validation, and test sets. Each subset should contain the original microscopy images paired with their corresponding ground truth segmentations. A common strategy is to allocate 70–80% of the data for training, 10–15% for validation, and the remainder for testing. To ensure unbiased evaluation, confirm that these subsets do not overlap in terms of fields of view, represent the variability of your entire dataset, and are randomly assigned to each set.

The **training set** is used to train the model to recognize relevant features. To enhance generalization, it must encompass a broad spectrum of scenarios and image conditions. Otherwise, the model risks overfitting—excelling with the training data but faltering with new images (Figure 9.6).

The **validation set** is used during training to provide feedback on the model's performance with unseen data. This feedback, conveyed as validation loss, assists in detecting overfitting (Figure 9.6), guiding hyperparameter tuning (see Section 9.2.5.3), and informing training decisions. Although a separate validation set is ideal, many workflows create one in practice by reserving a portion (typically 10% to 30%) of the training data.

The **test set**, which serves a separate role, evaluates the model's performance on entirely unseen data. Unlike the validation set, the test set is not utilized during training, ensuring an unbiased performance assessment. Test images should also include ground truth annotations to facilitate quantitative quality control. Reporting test set performance, using metrics such as accuracy, IoU, or F1-score, is crucial, especially when publishing or benchmarking your model [165].

9.2.5.2 Understanding the training process

A DL model is composed of multiple layers (Figure 9.5). Each layer contains tens to hundreds of image processing operations (typically multiplications or convolutions), each controlled by multiple adjustable parameters (called weights). Altogether, a DL model may contain millions of adjustable weights. When an input image is passed through a DL model, it is sequentially processed by each layer until an output is generated. Segmentation tasks typically involve converting input images into labeled outputs. During training, the model weights are modified as the model learns how to perform a specific task.

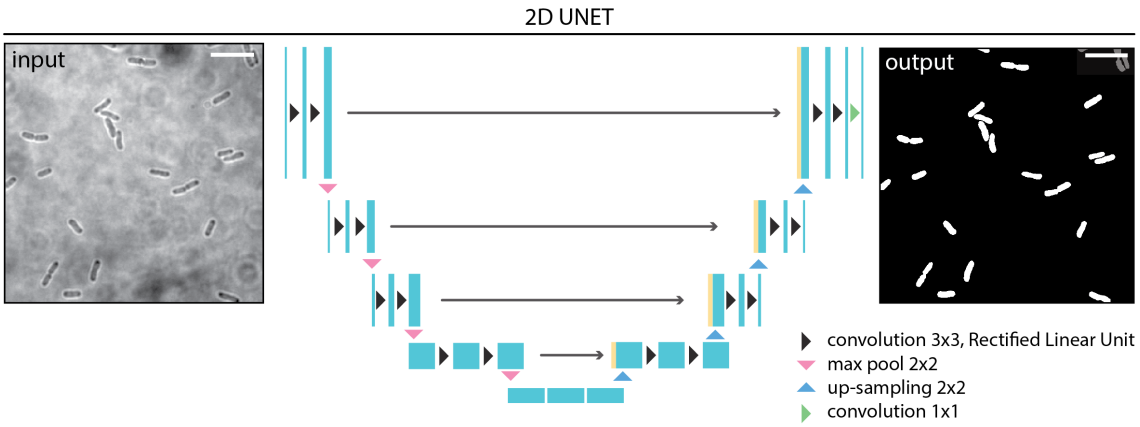


Figure 9.5: 2D U-Net architecture for image segmentation. It applies layers of convolutions, pooling, and upsampling to extract features and generate labeled segmentation masks. During training, model weights are iteratively adjusted based on the difference between predictions and ground truth labels, using a loss function and backpropagation.

Training begins with initializing these weights. When training from scratch, the initialization is often random. However, when using a pre-trained model, the weights are already optimized based on previous training, providing the model with a significant head start (see Section 9.2.6).

The training process is iterative (Figure 9.7). Each cycle of training is called an epoch. During each epoch, the model typically learns from every image in the training set. Since datasets are often too large to fit into memory all at once, each epoch is divided into steps or iterations, with each step processing a smaller subset of the data known as a batch. The batch size determines how many samples are processed simultaneously.

During each step, the model generates predictions for the current data batch. These predictions are compared to the ground truth labels using a loss function that calculates the difference between the predictions and the ground truth. This score is called the training loss. The model utilizes this feedback to adjust its weights through a process known as backpropagation, guided by an optimization algorithm, to improve its accuracy in future iterations.

At the end of each epoch, the model assesses its performance on the validation set, which comprises data it has not encountered during training. This produces the validation loss, indicating how well the model generalizes to new data.

Monitoring both training and validation losses during training helps determine whether the model is learning effectively. A consistently decreasing validation loss indicates that the model

is improving and generalizing well (see Section 9.2.5.4). To learn more on the intricacies of training models, see Chapter 4.

9.2.5.3 Choosing your model hyperparameters

Now that you understand the training process, the next step is to configure the model's hyperparameters, which are the settings that dictate how the model learns. While the model's parameters (its weights) are updated during training, hyperparameters are established beforehand, defining the structure and behavior of the training process. Below are some of the most common hyperparameters and their effects on training:

- **Batch size:** This refers to the number of images processed simultaneously in each training step. Smaller batch sizes are less demanding on memory and may enhance generalization, although they can result in slower training. In contrast, larger batch sizes accelerate training but necessitate more GPU memory.
- **Epochs:** An epoch refers to a training cycle in which the model processes the entire training dataset. Increasing the number of epochs allows the model to learn more, but also raises the risk of overfitting. More is not always better; it is essential to monitor performance on the validation set.
- **Learning rate:** It determines the extent to which the model's weights are adjusted during training. A high learning rate can result in quicker training but may overshoot the optimal solution. Conversely, a low learning rate provides more stable progress, although it may slow down convergence.
- **Optimizer:** An algorithm that updates weights to minimize the loss function. Common optimizers include stochastic gradient descent (SGD) and Adam (adaptive moment estimation)[156] , the latter being widely used for its adaptive learning rate and robust performance.
- **Learning rate scheduler:** Dynamically adjusts the learning rate during training, typically decreasing it after a specific number of epochs or when the validation loss plateaus. This approach helps balance rapid early learning with more refined convergence later on.
- **Patch size:** Instead of full-resolution images, smaller patches are often utilized for training to reduce memory usage and enhance training speed. The patch size is determined by both available resources and the scale of the structures to be segmented.
- **Patience (early stopping):** This parameter defines the number of epochs to wait before halting training if the validation loss does not improve. It helps prevent wasting resources on overfitting and overtraining.

Given the many possible configurations, tuning hyperparameters is often essential—especially when applying a model to new data. Start with the recommended values from the model's original publication, but you might need to conduct a hyperparameter search to optimize performance. This can range from a simple grid search to more advanced methods, such as Gaussian process-based, Bayesian optimization [137], or genetic algorithms [5].

9.2.5.4 Monitoring training and validation losses

Once your model begins training, it is helpful to evaluate its learning progress. The two key metrics for assessment are the training loss and the validation loss (Figure 9.6). Monitoring both throughout the training process offers insight into whether your model is improving and learning to generalize beyond the training data. The three main behaviors that you may encounter during training are:

- **Underfitting:** The model has been trained with insufficient data or for too few epochs, resulting in similar training and validation losses, which is far from optimal.
- **Good fitting:** Both training and validation losses decrease, with the validation loss slightly higher (worse) than the training loss, which is expected. This represents the ideal scenario.
- **Overfitting:** The model achieves an excellent training loss, but the validation loss does not improve and may in fact diverge. This may indicate overly similar training data or excessive training epochs, preventing the model from generalizing to new data.

9.2.6 Fine-tuning pre-existing models

Instead of training a model from scratch, fine-tuning an existing model is usually more efficient, especially when your data resembles the original dataset. This approach utilizes pre-trained weights and previously learned features, significantly decreasing the amount of required annotated data, training time, and computational resources.

9.2.6.1 Applying transfer learning

Transfer learning refers to the process of taking a pre-trained model and adapting it to a new but related task by providing task-specific training data, typically in the form of manually annotated image pairs (Figure 9.7). Transfer learning often involves freezing a part of the model (for instance, the initial layers or all layers except the last ones), so that these weights are not updated during training. Only the unfrozen layers are updated when the model is trained on new data. Since the base model already encodes many useful low-level features (e.g.,

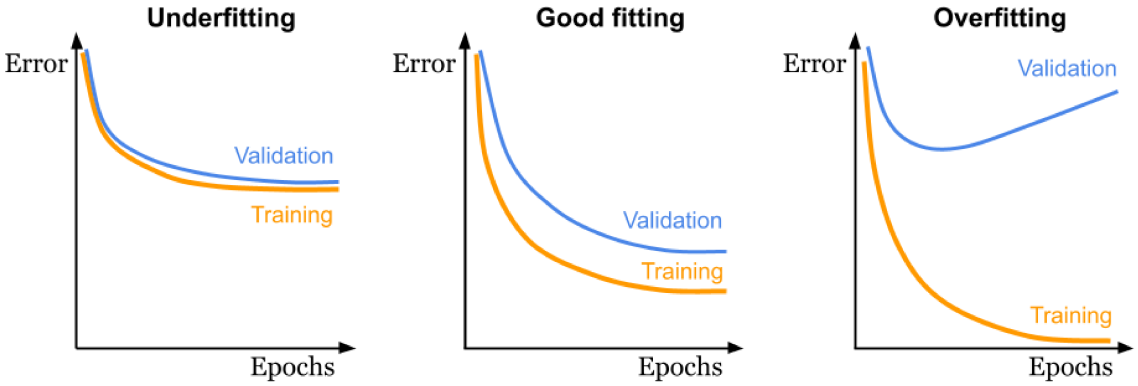


Figure 9.6: Monitoring training and validation losses during model training. The plot illustrates three typical learning behaviors: **underfitting** (both losses remain high and similar), **good fitting** (both losses decrease, with validation loss slightly higher), and **overfitting** (training loss continues to decrease while validation loss plateaus or increases), highlighting the importance of tracking these metrics to assess model performance and generalization.

edges, shapes, textures), this approach allows researchers to focus on refining the model for their specific biological structures or imaging modalities [179, 218].

This method is especially effective when:

- You have limited training data available.
- Your imaging conditions closely match those of the pre-trained model.
- You wish to quickly adapt a general model to a specific dataset.

9.2.6.2 Conducting fine-tuning

In classic fine-tuning, all layers of the pre-trained model are retrained, with their weights initialized from the original training (Figure 9.7) [8, 239]. Thus, you continue training the full model using the new data. This approach allows the model to adjust more comprehensively to new data while still preserving the advantages of pre-learned features.

Classic fine-tuning is ideal when:

- Your dataset is moderately different from the original training data (e.g., the same biological structure but different staining or modality).

- You expect that earlier layers may need to adapt, not just the final classifier or output layers. Early layers in deep networks typically learn to detect general features such as edges, textures, or simple shapes, while later layers capture more complex, task-specific patterns. If your new data differs in basic appearance or imaging modality, updating the early layers helps the model better extract relevant low-level features from your images.

- You have enough annotated data to avoid overfitting during full model training. Although this method is more computationally demanding than transfer learning, where only a subset of layers is retrained, it often leads to better results on diverse datasets.

9.2.6.3 Iterative training: keeping humans in the loop

Iterative fine-tuning is an interactive approach that combines model prediction with human annotation (see Section 9.2.4.3). The workflow typically starts with a pre-trained model predicting new images. A user then manually corrects or annotates these predictions, and the improved annotations update the model (Figure 9.7). This cycle continues, progressively enhancing the model's accuracy with each iteration until it performs as expected [57, 67, 239].

This method is particularly powerful when:

- Annotated data is scarce or expensive to generate.

- You work with rare structures, unusual imaging conditions, or new experimental systems.

- You want to efficiently build a custom model using feedback from domain experts.

9.2.7 Evaluating the performance of your model

With the rapid increase in DL tools and pre-trained models, it has become easier to use DL for image segmentation, but harder to determine which model will work best for your data. Regardless of how promising a model appears, you must always evaluate its performance before trusting its results. Evaluation is not optional; it is a critical step to ensure that the model meets the requirements of your specific task [165] (Figure 9.8).

There are two main ways to evaluate a model:

- Qualitative evaluation entails visually inspecting the model's predictions. This approach can help you quickly identify clear errors or failures. It is effective for a small number of images, but it becomes impractical for large datasets or for comparing similar-looking outputs across multiple models.

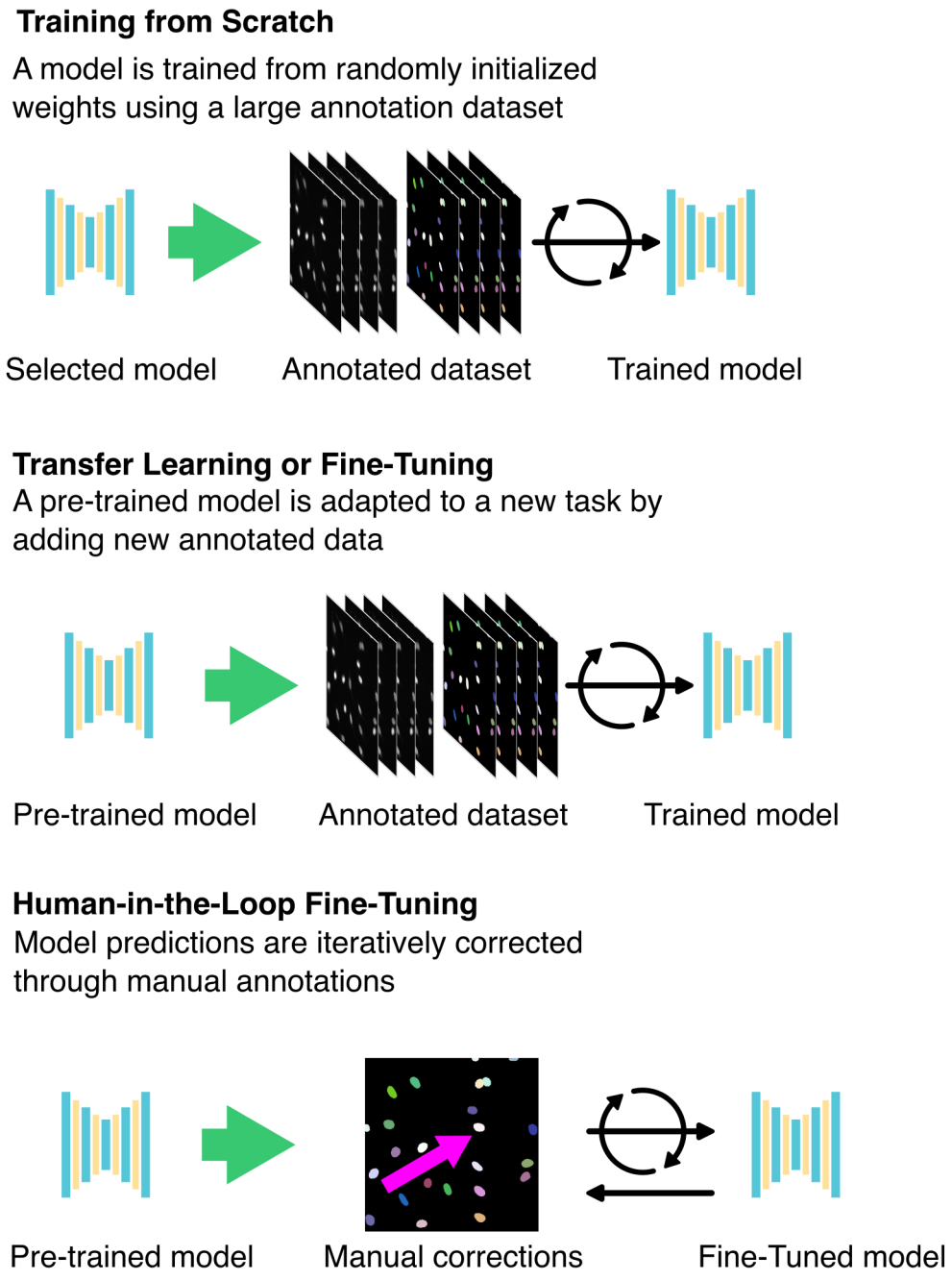


Figure 9.7: Strategies for training DL models for image segmentation. A model can be **trained from scratch** using a large annotated dataset, **fine-tuned** from a pre-trained model with task-specific data, or refined through **human-in-the-loop** workflows where model predictions are manually corrected and fed back for retraining. These approaches balance performance, data availability, and annotation effort.

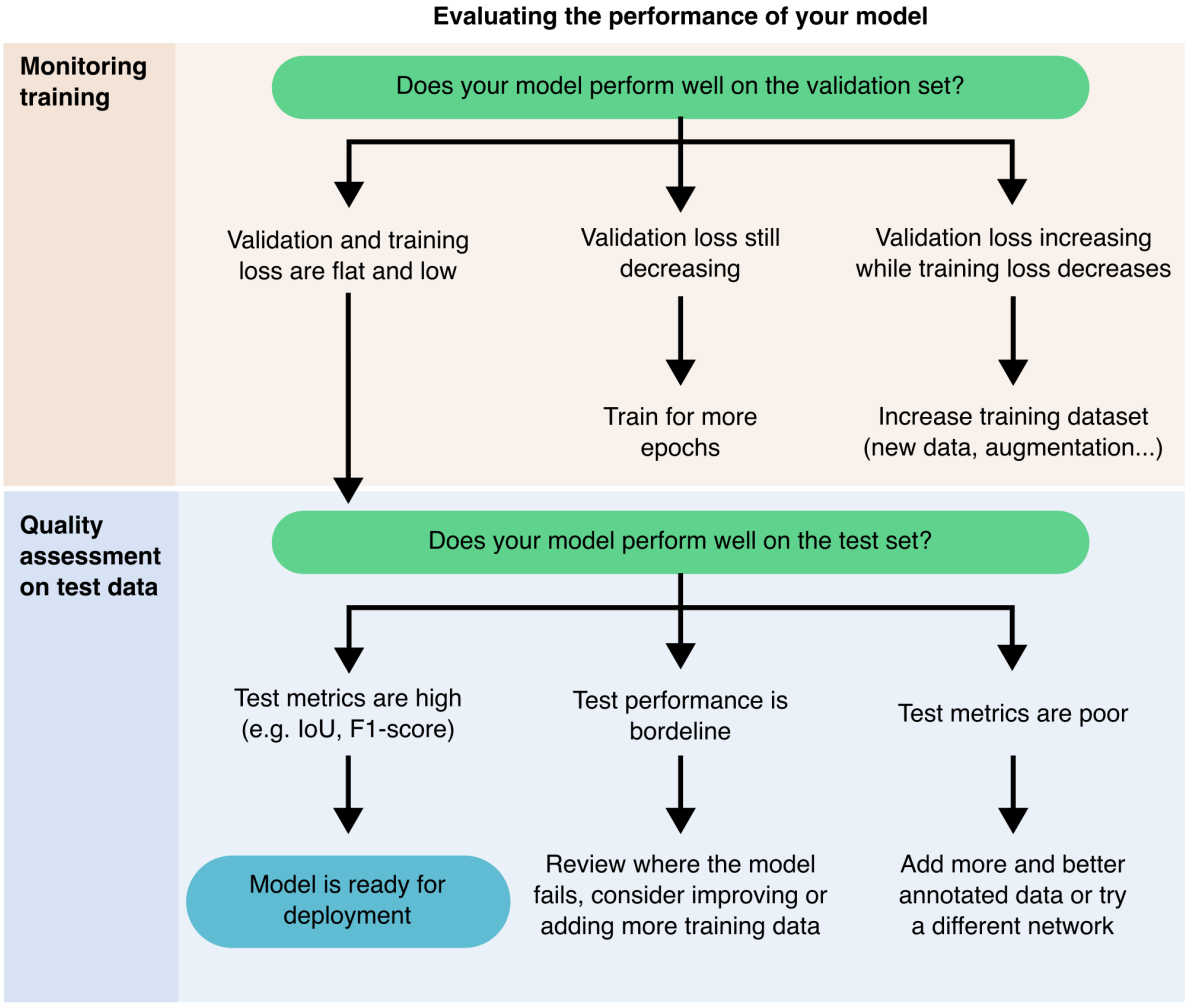


Figure 9.8: Workflow for evaluating DL model performance during training and on test data. Evaluating model performance is essential before deploying any DL model for image segmentation. This diagram outlines a two-stage process: assessment during training and on a separate test set. During training, validation and training losses (see Section 9.2.5.4) guide whether to continue training, stop, or expand the dataset. After training, performance is evaluated using a test set. High test metrics (e.g., IoU, F1-score) indicate readiness for deployment. Borderline or poor results suggest reviewing errors, refining training data, or trying a different model. This approach ensures model reliability and task-specific performance.

- Quantitative evaluation provides objective metrics for comparing models and tracking improvements. To achieve this, you need a small, labeled test set (typically 5 to 10 images with accurate ground truth segmentations). This test set must remain independent of your training and validation data to ensure an unbiased assessment.

Common metrics used in quantitative evaluation include:

- Intersection over Union (IoU): Also known as the Jaccard Index, this measures the overlap between the predicted segmentation and the ground truth.
- F1-score (Dice coefficient): This is especially valuable when the object of interest covers a small area in the image, as it balances precision and recall.
- True Positives (TP), False Positives (FP), and False Negatives (FN): These are particularly important in semantic segmentation and can be used to calculate the IoU or F1-score.

For more information on these metrics, see Chapter 10 and Laine, et al. [165].

If a model fails to produce reasonable results, even on simple examples, you can often reject it based solely on qualitative inspection. However, in these cases, quantitative metrics can still help you understand how and where the model fails.

If your evaluation metrics indicate weak performance, especially for certain structures or image types, you may need to fine-tune the model (see Section 9.2.6). Consistently strong scores across various test images suggest that a model could be dependable and ready for deployment. If no pre-trained model meets your expectations, the best course may be to train your model using your images (see Section 9.2.5).

In summary, **never skip evaluation**. Every model must be tested—both visually and quantitatively—to ensure it truly works for your data and provides results you can trust.

9.2.8 Deploying your model on new data

Once a segmentation model has been trained and validated, it can be used on new, unseen images. This step typically involves feeding new images into the model to generate segmentation predictions. The deployment approach depends on the computational resources (see Section 9.3.2) as well as the size and complexity of your dataset (Figure 9.9).

9.2.9 Troubleshooting common problems

I found a tool or DL model online, but it does not work. What should I do?

Figure 9.10 provides a visual guide for trouble-shooting a workflow using a given model.

When should I train a model or segment manually?

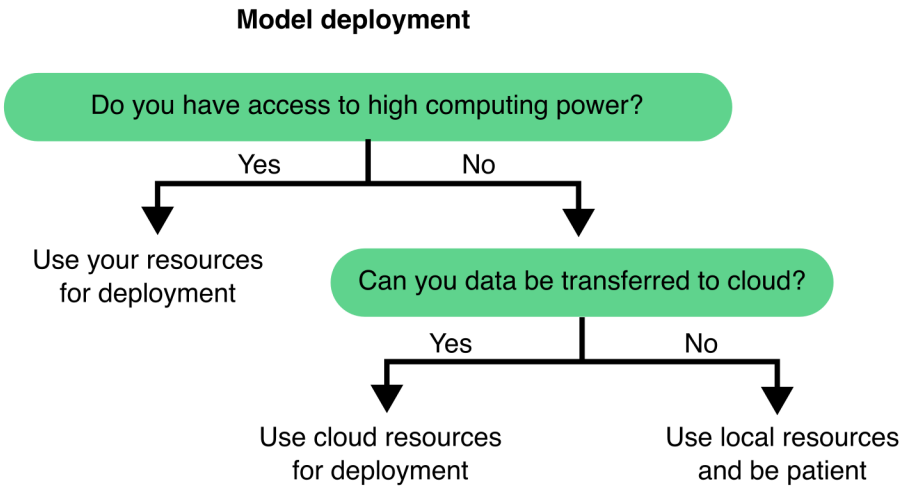


Figure 9.9: Decision workflow for model deployment strategy based on computational resources. The choice of deployment strategy depends on the availability of computational resources (see Section 9.3.2) and the sensitivity of the data. If high-performance computing resources are available locally, these should be used for deployment. In their absence, consider whether the data can be transferred to the cloud. If so, cloud-based resources offer an efficient solution. However, if data transfer is restricted—due to size or sensitivity—local deployment remains the only option, though it may require significantly more time.

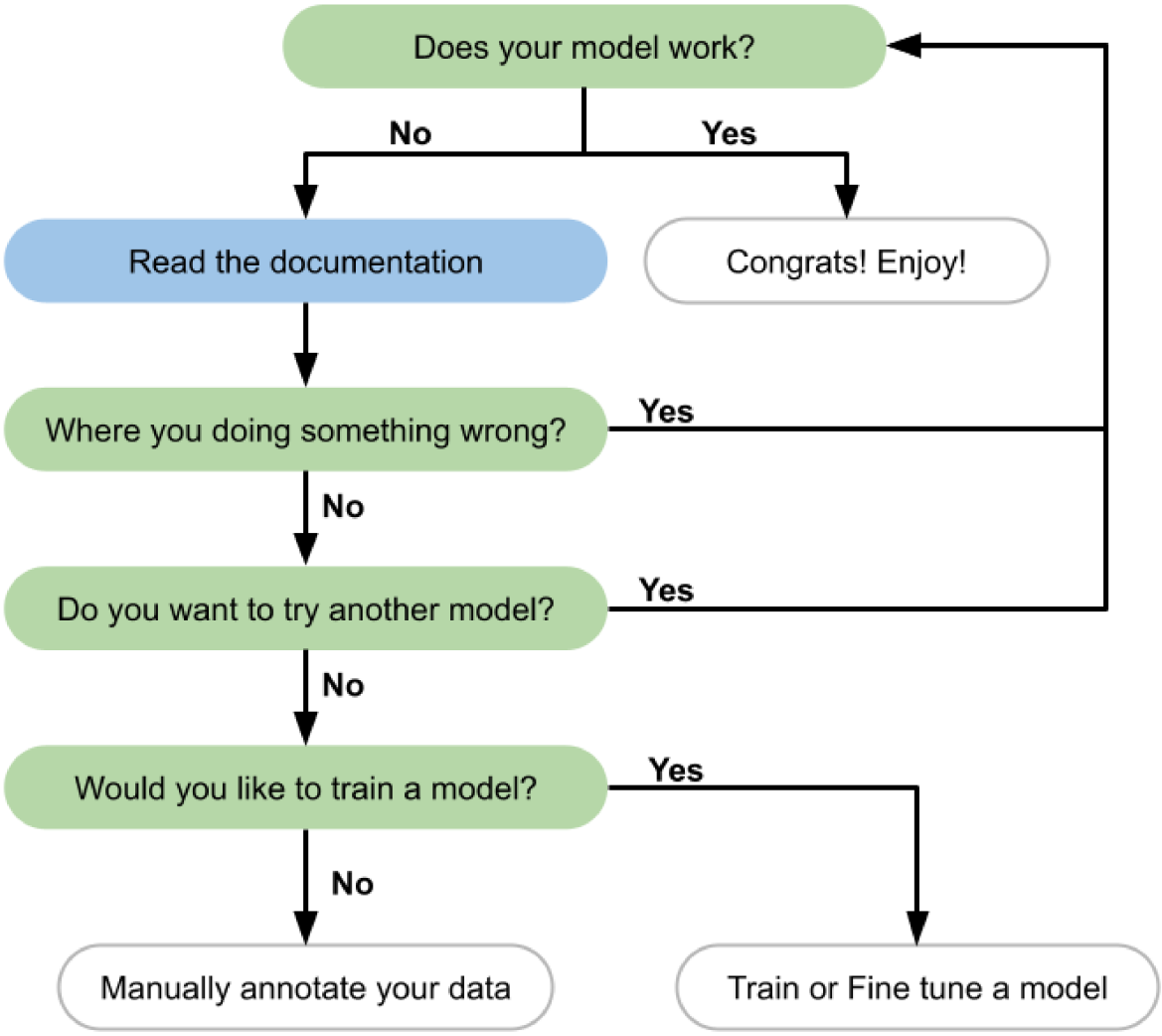


Figure 9.10: Common DL segmentation problems and troubleshooting tips.

Refer to Section 9.2.3.2 for more details, but generally, this decision depends on your dataset and the performance of existing pre-trained models (Figure 9.10). If you only need to segment a small number of images, manually segmenting them is often the quickest and simplest solution. However, if you are dealing with a large dataset, it may be more efficient to annotate a small subset and use it to train a deep learning model that can automate the segmentation of the rest.

I decided to train my DL model, but it is not performing correctly. What should I do?

First, ensure that you have trained the model for a sufficient number of epochs—this depends on the size of your dataset and the architecture of the model. Check the training loss: if it has plateaued, your model may be fully trained. If it is still decreasing, continue training.

If training is completed but results are poor, examine your data. Is the model missing specific features? Are there types of cells or structures that it consistently fails to segment? If so, ensure those examples are well represented and correctly annotated in your training data. You may need to enhance or expand your annotations.

If performance is poor, you may need additional annotated data to help the model generalize more effectively (Figure 9.8). Consider the following questions:

- Is my dataset balanced? Does it include sufficient examples of each structure or class I want to segment?
- Am I training on one experimental batch while validating or testing on another?

How many images should I have to train my model?

Refer to Section 9.1.1 for more details. There is no one-size-fits-all answer—it depends on the complexity of your task, your model architecture, and the variability in your data. More complex tasks typically require more data. Larger images can also be broken into more patches, effectively increasing your dataset size. While few-shot models are being developed for small datasets, most established DL models require a substantial amount of data.

Possible technical issues that you may encounter when training your DL model.

- The model predicts the same class for all pixels or segments in every cell. Your dataset might be unbalanced, containing too many similar examples. Adding more diverse or underrepresented examples can help the model learn to differentiate between classes.
- Out-of-memory errors during training: Consider reducing the batch size or the image patch size. If that doesn't resolve the issue, consider switching to a workstation or cloud service with greater computational capacity.

- The model performs well on training data but poorly on new images, suggesting overfitting (Figure 9.6). Implement data augmentation and increase dataset diversity to help the model generalize better.

- Inconsistent results across different computers: Differences in GPUs or environments can cause slight variations in outcomes. If the differences are significant, verify that all systems use consistent software versions and configurations. For further information on this topic, refer to Section 9.3.3.

9.3 Further considerations for DL segmentation

9.3.1 Choosing the right tools for DL

Selecting the right tools to train and use DL models depends mainly on your level of programming experience and comfort with technical interfaces.

If you prefer not to write code or use command-line tools, opt for platforms that offer graphical user interfaces (GUIs) or interactive notebooks with pre-configured workflows. These tools let you perform powerful segmentation tasks using intuitive interfaces and simple widgets.

GUI-based tools include, for instance (see Chapter 8 for more tools):

- Cellpose GUI
- Fiji with deepImageJ and StarDist plugins
- napari
- ilastik
- QuPath

Interactive Jupyter notebooks provide a flexible balance between code and GUI. They enable you to execute code in manageable steps (cells) and immediately see the results. Tools like BiaPy [87] and DL4MicEverywhere [121] leverage Jupyter notebooks, concealing complex code behind user-friendly interfaces. These platforms cater to users with little or no coding experience while still allowing advanced users to access and modify code as needed. DL4MicEverywhere, in particular, established a widely adopted framework for training DL models via notebooks, contributing to the standardization and simplification of the workflow.

If you are comfortable with programming, you will have even more flexibility. Languages such as Python, MATLAB, Julia, Java, and Rust provide options for building and customizing DL workflows. Python stands out as the most beginner-friendly and widely supported choice,

boasting a large ecosystem of libraries and community support. Popular Python libraries for DL include PyTorch, TensorFlow, Keras, and JAX.

While coding can involve a steeper learning curve, it allows you to create customized pipelines, integrate various tools, and troubleshoot intricate workflows, unlocking the full potential of DL for microscopy segmentation.

9.3.2 Managing computational resources

When using DL for microscopy, an important consideration is the availability and capacity of your computational resources (Figure 9.9). High-performance DL models, particularly those used for 3D image data, can be very demanding regarding memory and processing power.

When selecting or designing a DL model, evaluate your available infrastructure:

- GPU memory: Determines how large your model and batch size can be.
- Training time: Influences your ability to iterate quickly; simpler models train faster.
- Dataset size: Larger datasets benefit from more powerful hardware and longer training times.

A practical strategy involves starting with lightweight models that demand fewer resources and scaling up to more complex architectures only if performance improvements become necessary. Tools like StarDist and Cellpose, for example, provide efficient options that function effectively with relatively modest hardware.

Additionally, consider whether to train and deploy your model locally or in the cloud (Figure 9.11). Local training is often more feasible if you already have access to a compatible workstation and want full control over data and execution. However, cloud-based services like Google Colab or Amazon Web Services (AWS) offer access to more powerful hardware, removing the need for local infrastructure—this is especially beneficial when working with large models or 3D datasets.

There are four typical combinations of training and prediction workflows:

- **Training locally + prediction locally** is well suited for small to medium-sized datasets, especially when computational demands are moderate and data privacy is a priority. This approach also supports some user-friendly desktop applications, such as Cellpose 2.0 [239], which can be run locally without requiring cloud access or advanced technical setup.
- **Training locally + prediction in the cloud** may be useful when models are trained in-house but need to be deployed at scale or integrated into cloud-based pipelines.

- **Training in the cloud + prediction locally** enables researchers to take advantage of powerful cloud GPUs for model development, while keeping inference close to the data source (e.g., on a microscope workstation or when dealing with sensitive data).
- **Training in the cloud + prediction in the cloud** is well suited for collaborative projects or large-scale deployments, where access to centralized, scalable infrastructure is critical.

Choosing between these strategies depends on your data size, hardware access, choice of software, collaboration needs, and on whether your workflow prioritizes flexibility, scalability, or control.

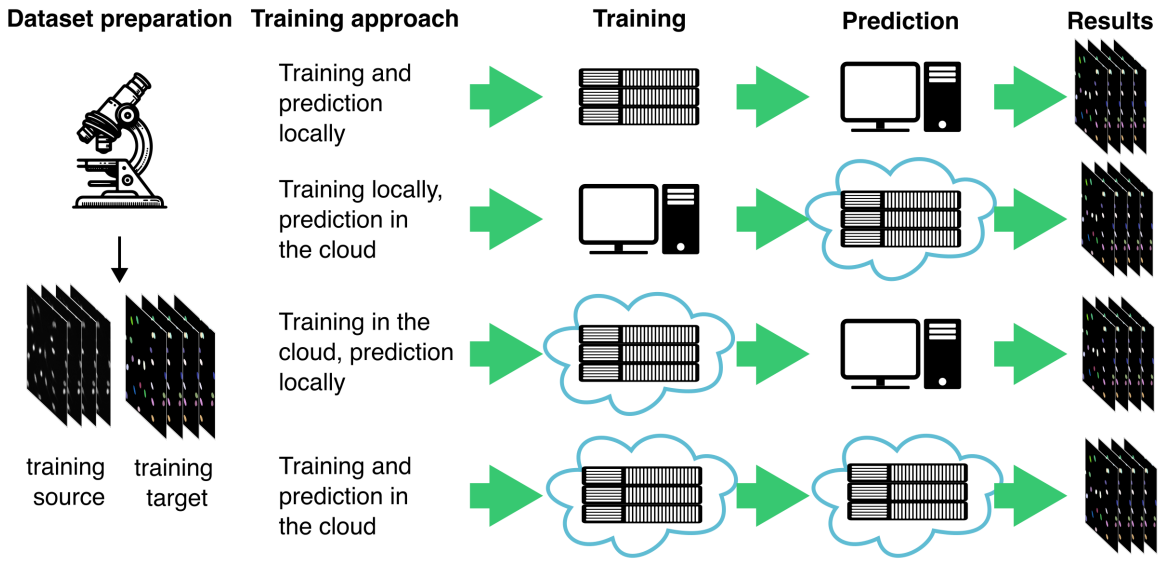


Figure 9.11: Training and deployment strategies for DL models in microscopy. Depending on the available tools and infrastructure, models can be trained and deployed locally or in the cloud. Modified from [51].

9.3.3 Ensuring reproducibility in DL

When sharing how you trained a DL model, two key elements often come to mind: the dataset used and the code that runs the model. However, in practice, reproducibility extends beyond just data and code. In programming environments like Python, which rely heavily on external libraries, ensuring reproducibility also requires capturing the exact configuration of the environment in which the model was trained.

DL models are sensitive to changes in library versions and dependencies. Even minor differences in the software stack can result in inconsistent outcomes or training failures. While sharing a

list of dependencies (e.g., a requirements.txt or a Conda environment file) is a constructive step, differences in operating systems or local setups can still lead to issues.

A robust and increasingly popular solution is containerization. Containers package software, dependencies, and environment settings into a portable and self-contained unit. One of the most widely used containerization tools is Docker. A Docker container behaves like a lightweight virtual machine that includes everything needed to run code, such as the libraries and runtime, ensuring applications run consistently across different machines.

Using containers ensures that your model training and inference processes remain consistent, no matter who executes them or where they are conducted. This makes it much easier for collaborators or reviewers to reproduce your results.

For researchers unfamiliar with software development, tools like DL4MicEverywhere [121] and bia-binder [272] simplify the use of containers by integrating them into user-friendly Jupyter notebook environments. These platforms enable researchers to benefit from the reproducibility of containers without needing to manage complex setups or command-line tools.

Reproducibility is crucial for establishing trust in computational results and facilitating long-term scientific collaboration. To ensure your DL workflows are reproducible, follow these best practices:

- Pin every software version used in your workflow.
- Document your environment setup thoroughly.
- Provide a containerized version of your training and inference pipeline when possible.

Taking these steps will make it easier for others to reproduce your results, build on your work, and apply your models in different research settings. For more information on best practices, consult Laine, et al. [165].

9.4 Summary and Outlook

Segmenting microscopy images remains a critical yet challenging task in bioimage analysis. In this chapter, we have used segmentation as a representative example to illustrate deep learning workflows and considerations. However, the strategies and best practices described here—such as data preparation, model selection, training, evaluation, and deployment—are relevant to a wide range of image analysis tasks, including classification, detection, and tracking. DL has undeniably transformed this field, offering robust solutions for segmenting complex and variable structures. Still, as this chapter emphasizes, DL is not always the fastest or the best approach. Classical image processing techniques or pixel classifiers often provide faster, simpler, and highly effective alternatives in many scenarios.

The decision to use DL should be driven by the complexity of the task, the availability of annotated data, and the specific goals of the segmentation project. Successful DL implementations often require significant investments in data curation, annotation, and computational resources. Furthermore, training from scratch is frequently avoidable thanks to the growing ecosystem of pre-trained models and resources shared by the community.

Notably, the landscape of DL segmentation is rapidly evolving. The emergence of foundation models, which are large, versatile networks pre-trained on vast and diverse datasets, promises to further lower the barriers to entry [8]. These models enable transfer learning, fine-tuning, and even zero-shot segmentation, where accurate predictions can be made on previously unseen data with minimal or no task-specific training. This shift opens exciting new avenues for researchers who previously lacked the resources or expertise to apply DL in their work.

The ongoing development and democratization of DL tools, along with enhancements in model generalizability, human-in-the-loop workflows, and reproducibility, are changing how microscopy data is analyzed. Still, the key to successful segmentation will always involve careful planning, quality control, and selecting the right tool for the task, whether it involves DL or not.

10 Output Quality

How to pick a model?

Morgan Schwartz[1], Diane Adjavon[2]
[1]HHMI Janelia Research Campus, [2]biohub,

You have trained a model. How do you know if it is any good? A model that looks convincing on a handful of images can quietly fail on the rest of your dataset, and a model that scores well on one measure of quality can perform poorly on another. Choosing the right way to measure quality is therefore just as important as the training itself.

In this chapter we will look at how metrics and losses differ, how to pick measures that actually reflect your scientific goal, and how to use held-out test data to get an honest estimate of how your model will behave on new images. We will focus on *segmentation* and *cell tracking* as concrete examples, but the principles apply broadly.

10.1 Metrics and Losses

In Chapter 9, you learned about how to train a machine learning model. Recall that, this requires selecting a loss function: a function that represents how far the model's output is from your desired ground truth. As training relies on gradient descent, the loss must be differentiable; the algorithm needs the slope of the loss with respect to every parameter to know which direction to step. The model's parameters are optimized to minimize the loss on a training dataset, and you evaluate the same loss on a held-out validation dataset to get an idea of how well your model would perform on new data.

Once you've trained a model, or potentially several, evaluating a diverse set of metrics on testing data allows you to get a better picture of the quality of the model. A metric is any function that scores your predictions. Unlike a loss, it does not have to be differentiable because it is not used for optimization. This means that you can get more specific which how you evaluate the quality of your model output, as we'll see below.

Consider the case of nuclear segmentation. Suppose you have fluorescently labelled nuclei, as in (Figure 10.1), that you want to segment for downstream analysis. Two standard losses for this task are binary cross entropy (BCE), and mean squared error (MSE). These functions are often optimized for the task of semantic segmentation, separating foreground from background. While both of these are differentiable functions, they have very different pros and cons when it comes to training a model.

```
import contextlib, io, logging
from cellpose import models

img = ski.data.cells3d()[21:]
image = img[18][1]  # nucleus channel

# Run Cellpose quietly for display
logging.getLogger('cellpose').setLevel(logging.ERROR)
with contextlib.redirect_stdout(io.StringIO()), contextlib.redirect_stderr(io.StringIO()):
    _model = models.CellposeModel(gpu=False, model_type='nuclei')
    _cp_labels, _, _ = _model.eval(image, diameter=None, channels=[0, 0])

# Use make_ground_truth for bw/filled/label_image so that failure modes
# (holes, split nuclei) are visible in the downstream figures
bw, filled, label_image = make_ground_truth(image)

image_label_overlay = label2rgb(_cp_labels, image=image, bg_label=0, alpha=0.2)
plt.imshow(image_label_overlay)
plt.axis("off")
plt.show()
```

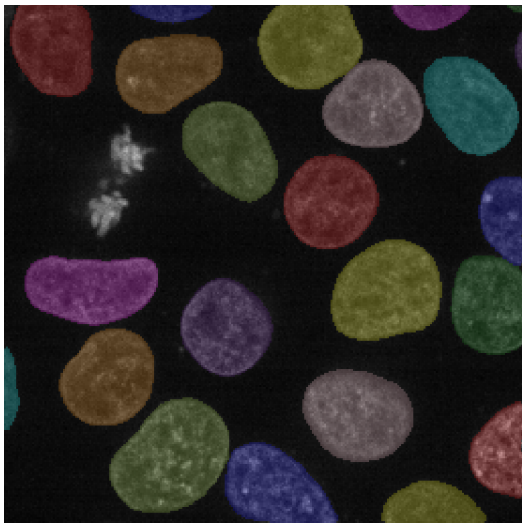


Figure 10.1: Our cells, with the ground truth overlaid. This ground truth is the output of a Cellpose[305] model, but in practice it should be manually annotated.

Binary Cross-Entropy (BCE)

Binary cross-entropy penalizes the model based on how confident it is in the wrong answer. This loss requires the output to be interpretable as a probability — it asks "is this pixel foreground or background?" and treats all wrongly-classified pixels the same way, whether they are at a cell boundary, inside a hole, or belong to an entirely missing cell. For ground truth label $y_i \in \{0, 1\}$ and predicted probability $\hat{y}_i \in [0, 1]$:

$$\text{BCE} = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$$

Mean Squared Error (MSE)

The mean squared error penalizes the model proportionally to the squared difference between its prediction and the ground truth:

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

MSE measures *how far* the prediction is from the target value, and does not require the output to be a probability — it works equally well when the model produces a continuous map such as a distance transform or a confidence score. Because it sums squared errors over all pixels, it is sensitive to the total *extent* of the error: predicting an entire missing cell as background accumulates far more MSE than a few wrongly-classified boundary

pixels.

Luckily, we can keep track of both of these things by using one of these functions as a metric. For example, as we optimize binary cross-entropy, we can compute the mean-squared error on the validation data. Watching multiple numbers evolve during training gives a fuller picture than any single number alone.

After training a model for segmantic, or foreground/background, segmentation, it is common to use a method such as watershed to obtain instance segmentations: segmentation masks for each individual nucleus. How well we can do this task, followed by what downstream analysis we intend for our instance segmentations, cannot be easily measured with BCE or MSE. One can imagine other functions, however, that could more accurately describe "good performance".

```
predictions = [
    make_undersegmented(filled),
    make_oversegmented(filled),
    make_missing_cells(label_image),
    make_holey(bw),
]

for pred in predictions:
    fig, ax = plt.subplots(figsize=(4, 4))
    ax.imshow(label2rgb(pred, image=image, bg_label=0))
    ax.axis("off")
    plt.show()
```

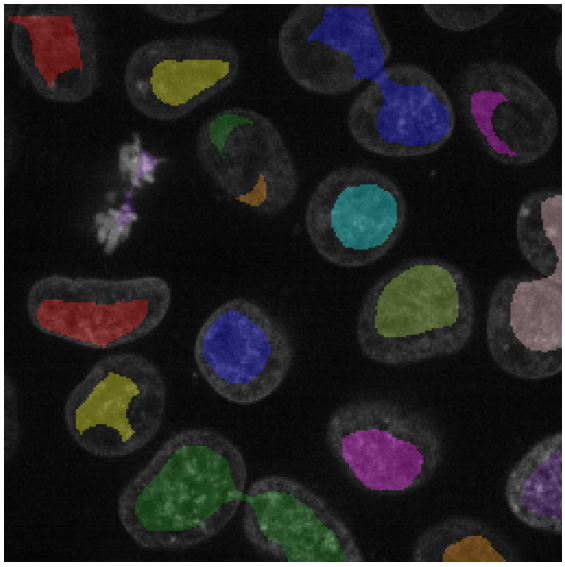
If we are interested in measuring the size of each nucleus, a model which under (Figure 10.2a) or over segments nuclei (Figure 10.2b) would create erroneous measurements of size. Instead a model that misses some detections (Figure 10.2c) might be preferable as long as the detections that exist correctly reflect the size of the nucleus. In this case an appropriate metric would be pixel-overlap metrics like Intersection over Union (IoU) or the Dice coefficient. A metric that captures this will look very different from BCE or MSE.

IoU and DICE are overlap metrics.

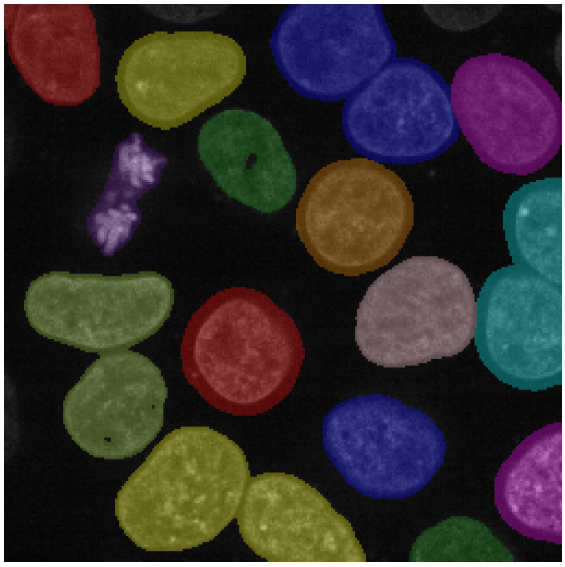
Both metrics quantify the overlap between a predicted region B and the ground truth A, by comparing the intersection of the two regions ($\cap$) and their union ($\cup$). Both metrics range from 0 (no overlap) to 1 (perfect overlap).

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} \qquad \text{DICE}(A, B) = \frac{2\,|A \cap B|}{|A| + |B|}$$

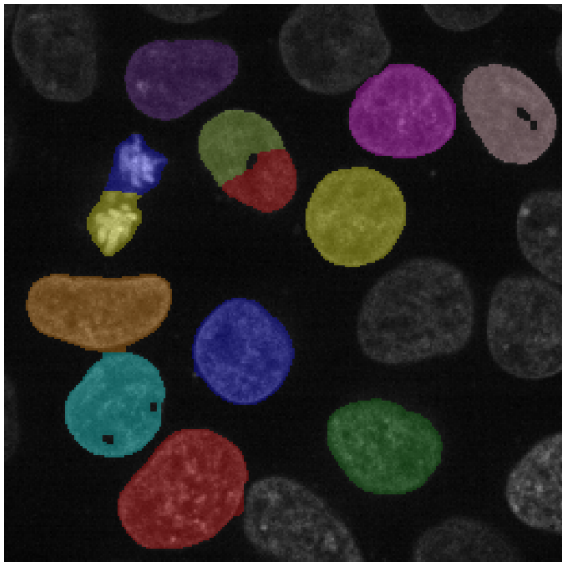
DICE is the harmonic mean of precision and recall: it balances false positives (predicted pixels outside A) and false negatives (pixels in A missed by B) equally. IoU is slightly



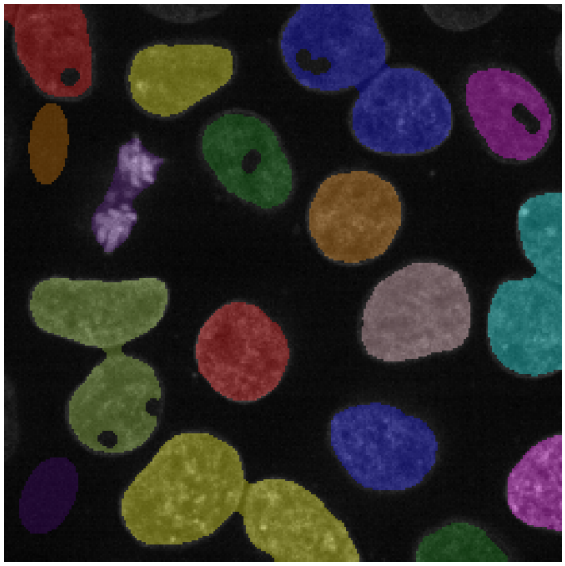
(a) Undersegmented: masks shrink away from cell borders



(b) Oversegmented: masks are too large, adjacent cells merge



(c) False negatives: missing cells and split nuclei.



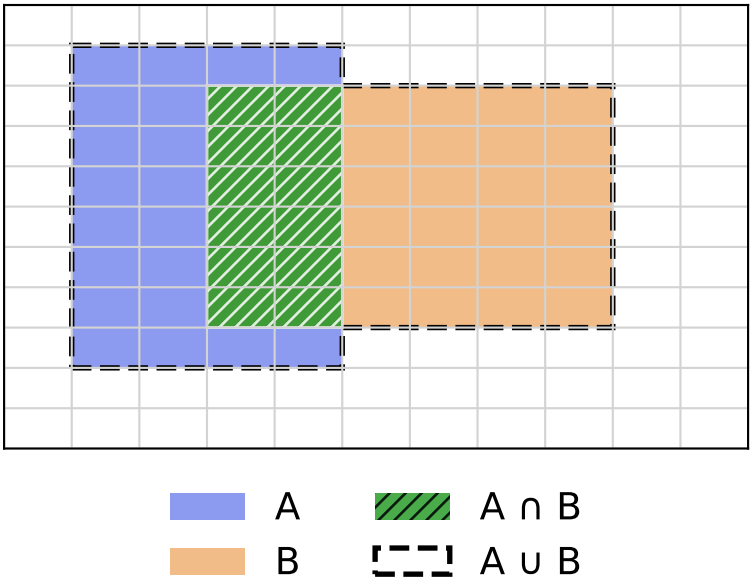
(d) Holes and false positives: gaps inside cells and spurious detections.

Figure 10.2: Four examples of segmentations from different models. Each has a distinct failure mode that is hard to spot at a glance.

stricter, because every falsely predicted pixel inflates the union without affecting the sum of the two areas. In practice the two metrics rank models in the same order.

> **Note**
>
> The DICE coefficient is the F1-score applied to pixel-level segmentation masks, and the two names are sometimes used interchangeably in the biomedical imaging literature.

```
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle, Patch, Polygon as MplPolygon
import numpy as np

navy, orange, green = "#1a35e0", "#e47a10", "#2a9d2a"

fig, ax = plt.subplots(figsize=(5, 3))
ax.set_xlim(0, 5)
ax.set_ylim(0, 5)
ax.set_xticks(np.arange(0, 6, 0.5))
ax.set_yticks(np.arange(0, 6, 0.5))
ax.grid(True, color="lightgray", lw=0.8)
ax.tick_params(labelbottom=False, labelleft=False, length=0)

A = (0.5, 1, 2, 4)      # x, y, width, height
B = (1.5, 1.5, 3, 3)

ax.add_patch(Rectangle(A[:2], A[2], A[3], facecolor=navy, alpha=0.5))
ax.add_patch(Rectangle(B[:2], B[2], B[3], facecolor=orange, alpha=0.5))

# Union: outline that follows the actual shape of A ∪ B
union_verts = [
    (A[0],      A[1]),           # bottom-left of A
    (A[0]+A[2], A[1]),           # bottom-right of A
    (A[0]+A[2], B[1]),           # down to bottom of B
    (B[0]+B[2], B[1]),           # bottom-right of B
    (B[0]+B[2], B[1]+B[3]),      # top-right of B
    (A[0]+A[2], B[1]+B[3]),      # back to A's right edge at top of B
    (A[0]+A[2], A[1]+A[3]),      # top-right of A
    (A[0],      A[1]+A[3]),      # top-left of A
]
ax.add_patch(MplPolygon(union_verts, closed=True,
                        facecolor="none", edgecolor="black", lw=2, linestyle="--"))

ix0, iy0 = max(A[0], B[0]), max(A[1], B[1])
ix1, iy1 = min(A[0]+A[2], B[0]+B[2]), min(A[1]+A[3], B[1]+B[3])
ax.add_patch(Rectangle((ix0, iy0), ix1-ix0, iy1-iy0,
                        facecolor=green, alpha=0.85, edgecolor="white", hatch="////"))

legend_elements = [
    Patch(facecolor=navy, alpha=0.5, label="A"),
    Patch(facecolor=orange, alpha=0.5, label="B"),
    Patch(facecolor=green, alpha=0.85, hatch="////", label="A ∩ B"),
    Patch(facecolor="none", edgecolor="black", lw=2, linestyle="--", label="A ∪ B"),
]
ax.legend(handles=legend_elements, loc="upper center", bbox_to_anchor=(0.5, -0.02),
          ncol=2, frameon=False, fontsize=14)
```



Alternatively, if we are interested in counting the number of nuclei, a model which changes the size of the nuclei should not impact the analysis. However a model that misses nuclei (Figure 10.2c), detects artifacts as nuclei (Figure 10.2d), merges multiple nuclei into one (Figure 10.2b), or splits a single nucleus into multiple (Figure 10.2c) would introduce errors into the final counts. In this case, a metric built around per-instance detection, such as precision and recall (or F1-score) after matching predicted masks to ground truth, captures the problems that matter to the analysis. It will also reveal the false positives and false negatives that would need correction. Different downstream uses call for different metrics.

Precision, recall, and F1-score are instance-level metrics.

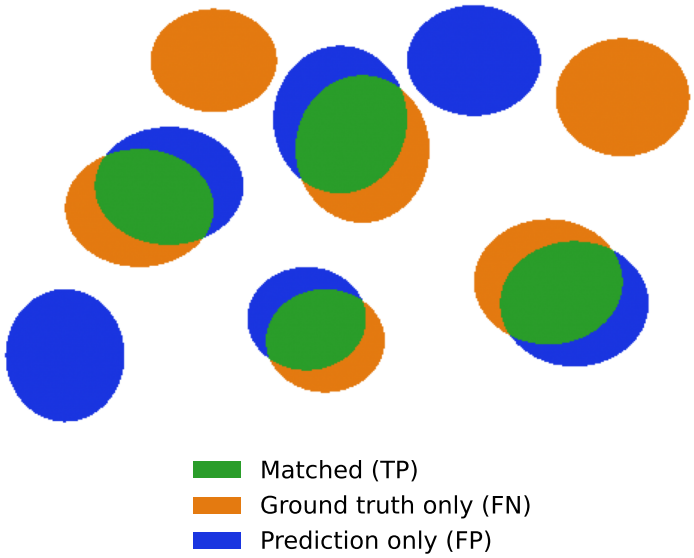
A predicted instance is a true positive (TP) if it sufficiently overlaps a ground-truth instance (e.g. IoU ≥ 0.5), a false positive (FP) if it has no matching ground truth, and a false negative (FN) if a ground-truth instance was never detected.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \qquad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$\text{F1} = \frac{2\,\text{TP}}{2\,\text{TP} + \text{FP} + \text{FN}}$$

Precision answers "of all predicted instances, how many were correct?"; recall answers "of all real instances, how many were found?"; F1 is their harmonic mean.

```
import matplotlib.pyplot as plt
from matplotlib.patches import Patch
from matplotlib.colors import ListedColormap
import numpy as np

navy, orange, green = "#1a35e0", "#e47a10", "#2a9d2a"

H, W = 300, 500  # raster grid

def ellipse_mask(cx, cy, rx, ry):
    """Boolean mask for an ellipse in axis coordinates [0,10] x [0,6]."""
    x = np.linspace(0, 10, W)
    y = np.linspace(0, 6,  H)
    xx, yy = np.meshgrid(x, y)
    return ((xx - cx) / rx)**2 + ((yy - cy) / ry)**2 <= 1

# Label image: 0=bg, 1=GT only (orange), 2=pred only (blue), 3=overlap (green)
img = np.zeros((H, W), dtype=int)

# True positives: GT and prediction ellipses slightly offset
tps = [  # gt_x, gt_y, gt_rx, gt_ry,  dx, dy  (prediction offset)
    (2.0, 3.0, 1.0, 0.8,   0.4,  0.3),
    (5.0, 3.8, 0.9, 1.0,  -0.3,  0.4),
    (7.5, 2.0, 1.0, 0.85,  0.35,-0.3),
    (4.5, 1.2, 0.8, 0.7,  -0.25, 0.3),
]
for cx, cy, rx, ry, dx, dy in tps:
    gt   = ellipse_mask(cx,    cy,    rx,   ry)
    pred = ellipse_mask(cx+dx, cy+dy, rx,   ry)
    img[gt   & ~pred] = 1
    img[pred & ~gt  ] = 2
    img[gt   &  pred] = 3

# False negatives - ground truth only (orange)
for cx, cy, rx, ry in [(3.0, 5.0, 0.85, 0.7), (8.5, 4.5, 0.9, 0.8)]:
    img[ellipse_mask(cx, cy, rx, ry)] = 1

# False positives - prediction only (blue)
for cx, cy, rx, ry in [(1.0, 1.0, 0.8, 0.9), (6.5, 5.0, 0.9, 0.75)]:
    img[ellipse_mask(cx, cy, rx, ry)] = 2

cmap = ListedColormap(["#ffffff", orange, navy, green])
fig, ax = plt.subplots(figsize=(5, 3))
ax.imshow(img, cmap=cmap, vmin=0, vmax=3,
          extent=[0, 10, 0, 6], origin="lower", aspect="auto")
ax.axis("off")

legend_elements = [
    Patch(facecolor=green,  label="Matched (TP)"),
    Patch(facecolor=orange, label="Ground truth only (FN)"),
    Patch(facecolor=navy,   label="Prediction only (FP)"),
]
ax.legend(handles=legend_elements, loc="upper center", bbox_to_anchor=(0.5, -0.02)
```



Open the callouts below for more metrics that are common in image analysis. You can find many other examples of metrics for various tasks, along with their pitfalls at MetricsReloaded.

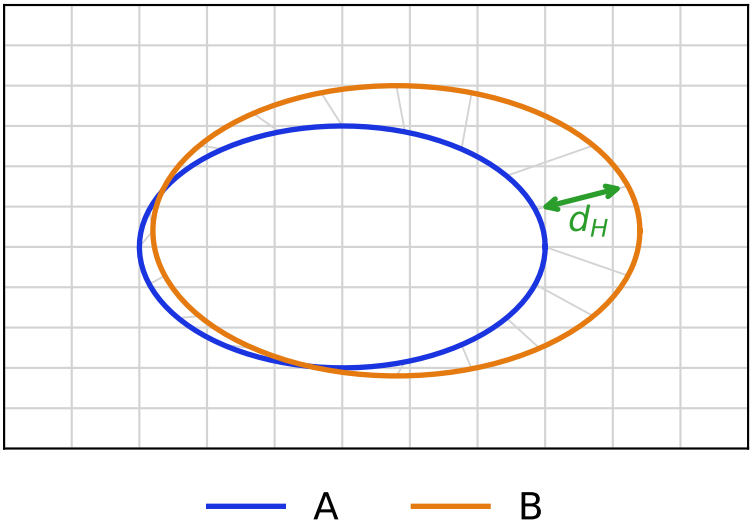
The Hausdorff distance focuses on boundaries.

The Hausdorff distance measures how far apart two sets of boundary points are at their worst-case point. For boundary point sets A and B:

$$d_H(A, B) = \max\left(\sup_{a \in A} \inf_{b \in B} d(a, b),\ \sup_{b \in B} \inf_{a \in A} d(a, b)\right)$$

A small Hausdorff distance means every point on one boundary has a close counterpart on the other.

```
import matplotlib.pyplot as plt
import numpy as np

navy, orange, green = "#1a35e0", "#e47a10", "#2a9d2a"

fig, ax = plt.subplots(figsize=(5, 3))
ax.set_xlim(0, 5)
ax.set_ylim(0, 5)
ax.set_xticks(np.arange(0, 6, 0.5))
ax.set_yticks(np.arange(0, 6, 0.5))
ax.grid(True, color="lightgray", lw=0.8)
ax.tick_params(labelbottom=False, labelleft=False, length=0)

theta_f = np.linspace(0, 2 * np.pi, 300)
theta_c = np.linspace(0, 2 * np.pi, 20, endpoint=False)

# A: ground truth boundary (circle)
A_line = np.stack([2.5 + 1.5 * np.cos(theta_f), 2.5 + 1.5 * np.sin(theta_f)], axis=1)
A_pts  = np.stack([2.5 + 1.5 * np.cos(theta_c), 2.5 + 1.5 * np.sin(theta_c)], axis=1)

# B: predicted boundary - slightly larger and offset
B_line = np.stack([2.9 + 1.8 * np.cos(theta_f), 2.7 + 1.8 * np.sin(theta_f)], axis=1)
B_pts  = np.stack([2.9 + 1.8 * np.cos(theta_c), 2.7 + 1.8 * np.sin(theta_c)], axis=1)

ax.plot(A_line[:, 0], A_line[:, 1], color=navy, lw=2, label="A")
ax.plot(B_line[:, 0], B_line[:, 1], color=orange, lw=2, label="B")

# Draw nearest-neighbour lines from each point in A to its closest point in B
dists = np.linalg.norm(A_pts[:, None] - B_pts[None], axis=2)
nn = np.argmin(dists, axis=1)
for i in range(len(A_pts)):
    ax.plot([A_pts[i, 0], B_pts[nn[i], 0]], [A_pts[i, 1], B_pts[nn[i], 1]],
            color="lightgray", lw=0.7, zorder=1)

# Highlight the worst-case pair (the Hausdorff point)
w = np.argmax(dists[np.arange(len(A_pts)), nn])
ax.annotate("", xy=(B_pts[nn[w], 0], B_pts[nn[w], 1]),
            xytext=(A_pts[w, 0], A_pts[w, 1]),
            arrowprops=dict(arrowstyle="<->", color=green, lw=2.0), zorder=3)
mid = (A_pts[w] + B_pts[nn[w]]) / 2
ax.text(mid[0] - 0.1, mid[1] - 0.3, "$d_H$", color=green, fontsize=13, va="center")

ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.02),
          ncol=2, frameon=False, fontsize=14)
```



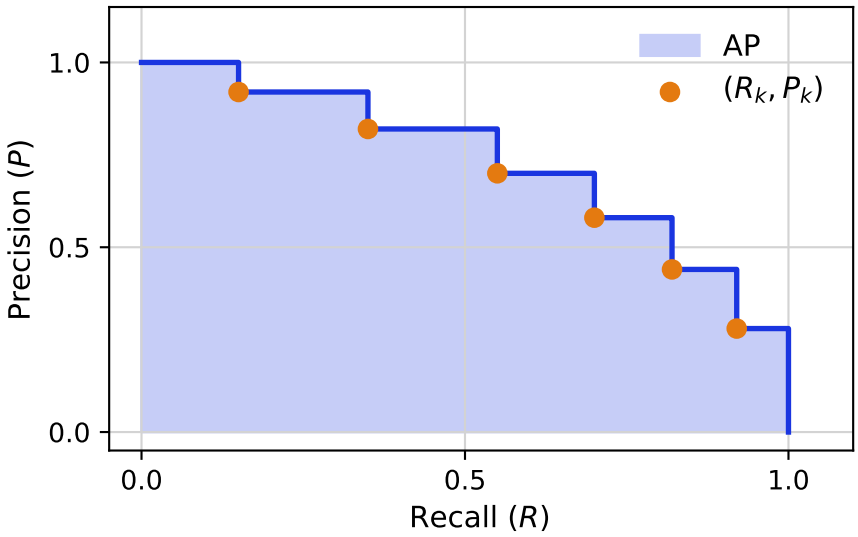
Average precision rewards finding the right instances.

Average Precision (AP) summarizes the precision–recall trade-off across confidence thresholds. A predicted instance counts as a true positive only if its IoU with a ground-truth instance exceeds a threshold x:

$$\mathrm{AP}_x = \sum_k (R_k - R_{k-1})\, P_k$$

where P_k and R_k are precision and recall at the k-th confidence threshold after matching predictions to ground-truth at IoU $\geq x$. Higher x demands more precise localization.

```
import matplotlib.pyplot as plt
import numpy as np

navy, orange = "#1a35e0", "#e47a10"

# Synthetic precision-recall curve
recall = np.array([0.0, 0.15, 0.35, 0.55, 0.70, 0.82, 0.92, 1.0])
prec   = np.array([1.0, 0.92, 0.82, 0.70, 0.58, 0.44, 0.28, 0.0])

fig, ax = plt.subplots(figsize=(5, 3))
ax.set_xlim(-0.05, 1.1)
ax.set_ylim(-0.05, 1.15)
ax.set_xlabel("Recall ($R$)", fontsize=11)
ax.set_ylabel("Precision ($P$)", fontsize=11)
ax.set_xticks([0.0, 0.5, 1.0])
ax.set_yticks([0.0, 0.5, 1.0])
ax.grid(True, color="lightgray", lw=0.8)

# Step function and shaded area under the curve = AP
ax.step(recall, prec, where="post", color=navy, lw=2)
ax.fill_between(recall, prec, step="post", alpha=0.25, color=navy, label="AP")

# Mark the individual (R_k, P_k) threshold points
ax.scatter(recall[1:-1], prec[1:-1], color=orange, s=45, zorder=5,
           label="$(R_k, P_k)$")

ax.legend(loc="upper right", frameon=False, fontsize=11)
```



Some of these may not be differentiable, and therefore cannot be directly optimized; they cannot be used as losses. The practical workflow is therefore: choose a differentiable loss to train the model, then evaluate a richer set of metrics on held-out data to judge whether the trained model actually behaves the way you need it to.

> **Tip**
>
> Explore this demo to see the direct impact of different segmentation errors on a variety of metrics.

As you've seen, there are many different metrics available for every task. Each will be more or less useful depending on what types of errors you can or cannot accept in your final predictions. So, how do you choose which metrics to measure and which ones to pay attention to?

10.2 What Metric to Pick?

When selecting a metric or metrics to use for evaluating a model, there are two key axes to consider. The first is the downstream analysis task that the data will be used for. The second is the time/cost needed to correct errors prior to that analysis task. In many cases the downstream analysis task will influence which types of errors can be tolerated.

This is why there is rarely one "correct" metric for a task like segmentation; the same set of predicted masks can look excellent by one measure and mediocre by another, and the right choice depends on tracing through how errors in the output propagate into errors in the analysis you actually care about. In some cases no single number can capture this tradeoff, and getting

a picture of quality you can act on requires reporting several complementary metrics side by side, or splitting evaluation into stages that reflect distinct sources of error. This tension is especially visible in cell tracking, where a solution can be scored separately on how well it detects cells in each frame and on how well it links them across frames, and both pieces have to be weighed together to judge whether the result is fit for the application at hand.

10.2.1 Cell Tracking Metrics

Cell tracking is typically approached as a multi-stage task. The first stage is detection where cells in each frame of the time series are detected using point detections, bounding boxes or segmentation masks. In the second stage, cells are connected between frames. For the purposes of evaluation, it is helpful to consider a tracking solution as a graph where each cell in a timepoint is a node and edges connect the cells (or nodes) across frames (Figure 10.3). A cell dividing into two daughters appears as a node with two outgoing edges, so a tracking solution is naturally represented as a set of lineage trees. Metrics that evaluate tracking solutions can consider both the performance of the detection phase (the nodes) and the performance of the linking (the edges).

With this framework, we can consider some of the common errors that appear in tracking solutions. Figure 10.4a illustrates basic node errors of a false positive detection (shown in red) and a missing blue detection. The missing node also leads to two missing edges. Often in biology when we are interested in cell tracking, there is a particular value placed an the detection of cell divisions so that cell lineages can be reconstructed over several rounds of divisions. As such we need to consider the specific types of errors that occur involving cell divisions. Figure 10.4b illustrates a missing edge that results in a missed division detection. Figure 10.4c instead shows a case where a division is detected, but one daughter is incorrectly assigned to the division leading to an incorrect lineage tree. Finally, Figure 10.4d illustrates an identity switch where edges are incorrectly assigned in a single frame leading to a swap of lineage identity.

There is a common inclination to use a single metric that is intended to capture overall performance on the task. In cell tracking, there are two common single metrics. The first is the TRA[203] score defined by the Cell Tracking Challenge. A second unified metric is CHOTA[151] (Cell-specific Higher Order Tracking Accuracy). Single metrics can be convenient; however, they are uninformative when the intention is to evaluate and then improve a tracking solution. A unified metric does not indicate what types of errors are present.

TRA[@mavska2014benchmark]

This score is a weighted sum of node and edge errors that is normalized between 0 and 1. It is derived from the Acyclic Oriented Graph Matching[205] (AOGM) measure which reflects the number of changes needed to turn the reference graph into the target graph.

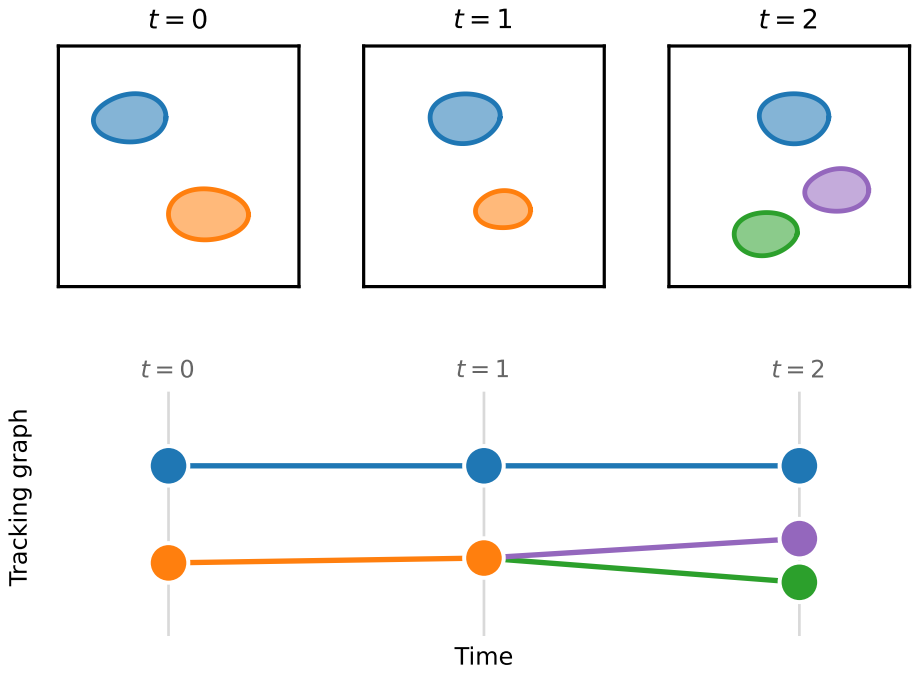


Figure 10.3: Cell tracking represented as a graph. (Top) Segmentation masks detect a few cells in each of three time points; the blue cell persists across all three frames while the orange cell divides into two daughters (green and purple) at $t = 2$. (Bottom) The corresponding tracking graph places one node per detected cell, with edges linking the same cell across frames. Node columns and colors align with the masks above, and the division appears as a node with two outgoing edges.

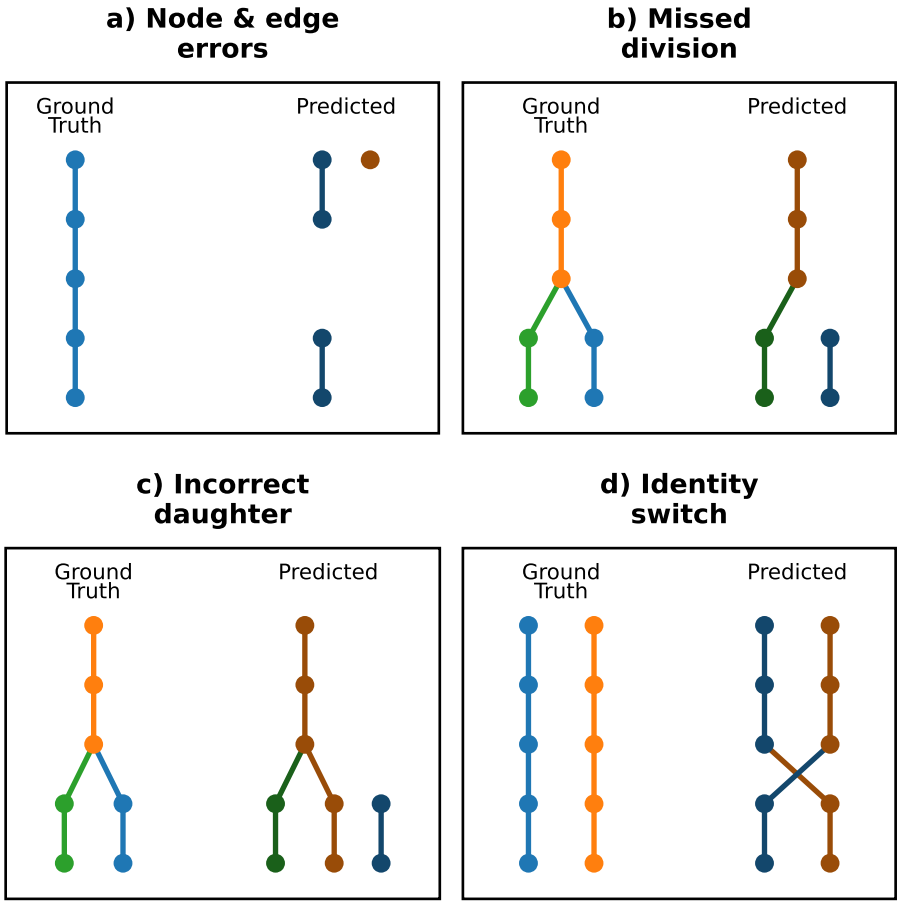


Figure 10.4: Common types of tracking errors. (a) This example prediction demonstrates multiple types of node and edge errors: false negative node, false negative edges and false positive node. (b) This example shows a missed division as the result of a missed edge between a parent and a daughter. (c) This example shows a division where the parent was correctly identified, but one of the daughters was incorrectly assigned to a different node. (d) This is an example of an identity switch where two nodes are swapped and assigned to the incorrect lineage part way through the lineage.

$$\mathrm{TRA} = 1 - \frac{\min(\mathrm{AOGM}, \mathrm{AOGM}_0)}{\mathrm{AOGM}_0}$$

CHOTA[@kaiser2024chota]

This metric captures local correctness, global coherence and aspects of lineage tracking. CHOTA is computed as

$$\mathrm{CHOTA} = \sqrt{\frac{\sum_{c \in \mathrm{TP}} A^{\sigma}(c)}{|\mathrm{TP}| + |\mathrm{FN}| + |\mathrm{FP}|}}$$

where $A^{\sigma}(c)$ is a lineage-oriented association score that measures how well predicted trajectories overlap with ground truth trajectories, accounting for all cells connected by lineage.

Figure 10.5 illustrates three tracking failure modes and the metric scores for each solution. As the discussion below shows, solutions that fail in very different ways can end up with similar overall scores, and the two unified metrics can even disagree with each other about which solution is preferable.

Consider the three failure modes in Figure 10.5. The two unified metrics produce very different rankings of solutions. The TRA score ranks no divisions (0.957) similarly to sparse edges (0.947) and far above missing nodes (0.732). In contrast, while all solutions score relatively low on CHOTA, the missing node solution ranks highest (0.560) followed by no divisions (0.333) and sparse edges (0.297). Because TRA and CHOTA weight node, edge and lineage errors differently, two established "single number" metrics can disagree about which solution is preferable. Neither score reveals whether the underlying problem is due to missed detections, broken links, or failure to find divisions. To identify the specific failure modes, we need to examine metrics for that we need to look at the node, edge and division metrics reported alongside them.

The per-component metrics reveal the distinct failure mode in each solution. The missing node solution (b) has the lowest node F1 (0.858) of the three because a quarter of detections are dropped; since each dropped node also severs the edges that pass through it, this single error mode depresses edge F1 (0.798) and division F1 (0.399) as well. The sparse edge (c) and no-division (d) solutions both keep every detection (node F1 = 1.000) and lose the same fraction of edges (edge F1 = 0.800), yet they are far from equivalent: sparse edges still recovers roughly half of the true divisions (division F1 = 0.505) because the deleted edges are scattered at random, while the no-division solution deterministically removes every division edge, so no division is ever proposed and division F1 is undefined.

```
import networkx as nx
```

```
def build_lineage_tree(
    n_starting_nodes: int = 10,
    n_frames: int = 30,
    division_every: int = 5,
) -> nx.DiGraph:
    """
    Build a lineage tree as a directed graph.

    - Starts with `n_starting_nodes` roots at frame 0
    - Every `division_every` frames, each live cell divides into 2 daughters
    - Between division frames, cells persist as a single descendant

    Nodes are labeled like:
        cell_id = "c0", "c1", ...
    and store:
        - frame
        - root
    """
    G = nx.DiGraph()

    cell_counter = 0
    live_cells = []

    # Create starting nodes at frame 0
    for root in range(n_starting_nodes):
        node = cell_counter
        cell_counter += 1
        G.add_node(node, frame=0, root=root)
        live_cells.append(node)

    # Advance through frames
    for frame in range(1, n_frames + 1):
        next_live_cells = []
        is_division_frame = (frame % division_every == 0)

        for parent in live_cells:
            root = G.nodes[parent]["root"]

            if is_division_frame:
                # Parent divides into two daughters
                for _ in range(2):
                    child = cell_counter
```

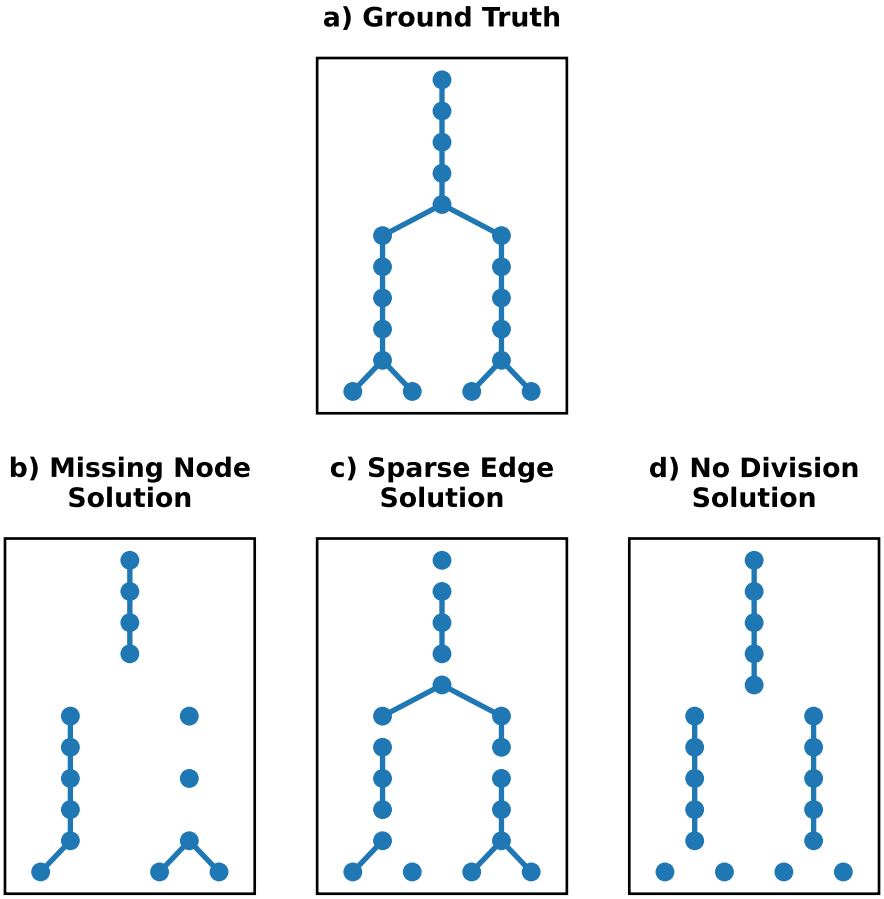
```
                    cell_counter += 1
                    G.add_node(child, frame=frame, root=root)
                    G.add_edge(parent, child)
                    next_live_cells.append(child)
            else:
                # Parent continues as one descendant
                child = cell_counter
                cell_counter += 1
                G.add_node(child, frame=frame, root=root)
                G.add_edge(parent, child)
                next_live_cells.append(child)

        live_cells = next_live_cells

    return G


def sparsify_edges(g, mod=3):
    del_edges = []
    for i, edge in enumerate(g.edges()):
        if i % mod == 0:
            del_edges.append(edge)
    g.remove_edges_from(del_edges)


def remove_divs(g):
    div_edges = []
    for node in g.nodes:
        if g.out_degree(node) == 2:
            div_edges.extend(g.edges(node))

    g.remove_edges_from(div_edges)


def remove_nodes(g, mod=4):
    del_nodes = []
    for i, node in enumerate(sorted(g.nodes)):
        if g.nodes[node]['frame'] > 0 and i % mod == 0:
            del_nodes.append(node)
    g.remove_nodes_from(del_nodes)
```

```
gt = build_lineage_tree(n_starting_nodes=10, n_frames=30, division_every=5)
```

```
from traccuracy.matchers import Matched
from traccuracy import TrackingGraph
from traccuracy.metrics import CTCMetrics, DivisionMetrics, BasicMetrics, CHOTAMetric

def compute_metrics(solution):
    """Score a solution against the full ground-truth tree with traccuracy."""
    matched = Matched(
        TrackingGraph(gt.copy(), frame_key='frame'),
        TrackingGraph(solution, frame_key='frame'),
        mapping=[(nid, nid) for nid in gt.nodes if nid in solution.nodes],
        matcher_info={},
    )
    results = {
        **CTCMetrics().compute(matched).results,
        **DivisionMetrics().compute(matched).results['Frame Buffer 0'],
        **BasicMetrics().compute(matched).results,
        **CHOTAMetric().compute(matched).results
    }
    return results
```

Whether an error is acceptable depends entirely on what the tracking result will be used for. If the downstream analysis measures single-cell metrics such as migration speed, the no-division solution is nearly as good as ground truth: every cell is detected, most links are correct, and the fact that no lineage relationships were ever proposed is irrelevant to the task. If instead the goal is to study clonal relationships or developmental lineages the no-division solution is worthless despite its high TRA, because it recovers zero divisions and can never support a lineage-based analysis. The missing node solution sits at the opposite end of this trade-off: it has the worst TRA of the three, but a downstream analysis focused on lineage structure might tolerate it better than that score suggests. Choosing a metric, or a small panel of metrics, should therefore start from the biological question at hand rather than from convenience.

A second, independent axis is how easy an error is to catch and fix once a human is proofreading the result, and it does not always line up with how tolerable the error is for the downstream analysis. Missing nodes and sparse edges leave a visible gap or an orphaned track segment, and the fix is local: add the missing node or edge back and reconnect two pieces that are otherwise correct. A systematically missed division is much harder to catch, because neither daughter track looks wrong on its own; the cells are simply tracked as if they had always been independent, so a curator scanning tracks has no local cue that a division was ever missed. The more tolerable error is therefore not necessarily the one with the better score on a given metric, but the one that is cheap to find and correct, particularly if missing detections are easy to flag and cheap to review manually.



Metric	Value
TRA	0.732
CHOTA	0.560
Node F1	0.858
Edge F1	0.798
Div. F1	0.399

Metric	Value
TRA	0.947
CHOTA	0.297
Node F1	1.000
Edge F1	0.800
Div. F1	0.505

Metric	Value
TRA	0.957
CHOTA	0.333
Node F1	1.000
Edge F1	0.800
Div. F1	n/a

Figure 10.5: Tracking solutions with common failure modes, each scored with a panel of tracking metrics. Trees show a single representative lineage truncated to 10 frames; the metric tables are computed over the full dataset (10 lineages, 30 frames). Panel b shows a solution where every fourth node (detection) is missing. Panel c shows a solution where every third edge is missing. Panel d shows a solution where no divisions were predicted. Division precision/F1 are undefined (n/a) for the no-division solution because it predicts no divisions.

10.3 Conclusion

We compute metrics to get a reasonable description of how well our model is expected to perform in the future, on unseen data. A single metric is rarely sufficient to fully evaluate a models performance as illustrated by Section 10.2.1. If multiple metrics are needed to fully evaluate a model, when do you run these metrics and on what data? As discussed in Section 9.2.5.4, you will minimally compute a loss both on your training and validation sets during the course of training. Some metrics can be easily computed on the validation dataset and used to select a checkpoint for further testing. These metrics should be computationally inexpensive to compute in order to avoid slowing down the training process. Typically these would be metrics that are computed on a pixel-pixel level. In contrast, consider metrics that are computed on an object level such as those discussed in Section 10.2.1. Each cell tracking metric requires a preliminary stage in which each ground truth cell is matched to a predicted cell prior to evaluation. While there are many different ways to perform the matching step, it is ultimately a slow and expensive process. Metrics that rely on a matching stage are best reserved for running on the final test data split. Table 10.1 elaborates on the stages of model evaluation and illustrates how metrics might be selected for a segmentation model.

Table 10.1: **A comparison of stages of mode** lights specific metrics that would be pipeline.

Stage	Speed of Computation	Data Split	Purpose
Training loss	Fast	Train	Guides gradient descent during training
Validation loss	Fast	Val	Enables monitoring of over/under fitting during trai
Validation metrics	Fast	Val	Additional metrics that provide more insight into tr
Test metrics	Slow	Test	Final metrics that are run on test data in order to c

Biological data varies from experiment to experiment, however, and whether the model will be robust to these variations remains somewhat of an open question. In the ideal case, we would like the models to be able to output a level of certainty along with its predicted segmentations or tracks. Unfortunately, uncertainty estimation remains an unsolved problem, methods such as test-time augmentation [220, 324] can give you an estimate if you have a good model of your data's sources of noise.

Test-time augmentation as a measure of uncertainty

Test-time augmentation is one way of quantifying the uncertainty of a model, by estimating the robustness of its outputs with regards to noise.
To apply TTA, you run the model multiple times on the same image, each time with a perturbation that reflects a realistic source of variation in your imaging system. For fluorescence microscopy, this might include: adding Poisson or Gaussian noise to simulate

photon shot noise or detector read noise, shifting the intensity or contrast to mimic staining variability between experiments, or applying a slight blur to simulate focus drift. You then aggregate the predictions to get your final output.
The spread across the individual predictions acts as a proxy for uncertainty. Where every perturbed version produces nearly the same thing, we can be confident in the model is output. Regions Where the predictions disagree, however, may warrant manual review.
Note that TTA only captures uncertainty relative to the noise sources you have modelled. It will not detect failure modes shared by all augmentations, such as a systematic bias introduced during training.

The best measure of your model's output, current and future, still remains a good testing dataset. Make sure to fully annotate a testing set that is diverse enough to be representative of future experiments. Compute a variety of metrics to understand the different types of failures you can expect, and to what extent. And finally, make sure to look at your model outputs in conjunction with your data! For most tasks, the best judge of quality is you.

AI Disclosure

AI language models (Anthropic's Claude, specifically Claude Sonnet 5) were used during the preparation of this chapter. Their roles included drafting prose, generating code examples, and producing demonstration figures. All content was reviewed, revised, and verified by the human authors, who take full responsibility for the accuracy and scientific integrity of the final text. Selection of references was the work of the human authors.

11 Outlook

What can AI enable for biological imaging?

Anna Kreshuk
European Molecular Biology Laboratory

With this book, we hope to have set you on a journey of exploration into the use of AI methods in microscopy. In this final chapter, we reflect on several early examples in which AI has already changed what can be observed, measured, and inferred from microscopy data, and then look ahead to the questions and capabilities now coming into view. Finally, we turn to the path before you—how you can build on what you have learned to not only answer specific biological questions, but contribute to the broader advancement of this rapidly evolving domain.

11.1 Early successes: AI across biological imaging

11.1.1 In situ cryo-Electron Tomography

Cryogenic electron tomography (cryo-ET) images biological samples preserved in vitreous ice by acquiring a series of transmission electron micrographs at different tilt angles and reconstructing them into a three-dimensional volume. Combined with computational post-processing [59], cryo-ET offers the highest spatial resolution available for imaging cellular structures in their near-native context, and can resolve repeated macromolecular complexes at ångström-level resolution [26]. The data is, however, not easy to analyze: the electron dose must be limited to avoid damaging the sample, resulting in a very low signal-to-noise ratio, while the restricted range of achievable tilt angles produces a characteristic missing-wedge artefact. The frontier application of cryo-ET to intact microorganisms and thin cellular lamellae adds a further challenge of macromolecule identification in an extremely crowded and heterogeneous environment[346]. AI is becoming integral across the workflow: helping to select suitable acquisition regions [80], improve reconstructions [254], segment cellular structures, and locate large numbers of macromolecular particles for structural averaging [309, 168, 262, 185]. These capabilities are beginning to turn cellular tomograms into molecular maps in which complexes can be identified and compared in situ [42, 208, 244], opening the way to studying

their organization, interactions, and structural states inside cells rather than in isolation. As acquisition and analysis become increasingly automated, the longer-term vision is a form of visual proteomics that can systematically chart the molecular architecture of cells in near-native conditions [226, 229, 332].

11.1.2 Connectomics

The field of connectomics aims to reconstruct and analyze complete neural circuit diagrams. Microscopy for connectomics faces the combined challenge of requiring a very high resolution for individual synapse detection and a very large field of view for complete neuron tracing. Up until the very recent advances in expansion microscopy [308], electron microscopy remained the only modality suitable for connectomics studies, routinely generating tens to hundreds to thousands of terabytes of image data at nanoscale resolution [345, 310]. As the infeasibility of fully manual analysis became obvious with the first significant volume acquisitions (the first *c. elegans* connectome from White, et al. White et al. [337] required a decade of manual tracing!), neuroscientists sought collaboration with machine learning and computer vision experts to speed up and automate neuron segmentation and synapse detection. Since then, deep learning has enabled increasingly complex pipelines [142, 40, 12], including AI-based error detection and correction [110, 49]. More recent models are beginning to infer properties that were previously inaccessible from morphology alone, such as neurotransmitter identity prediction directly from electron microscopy [79]. As connectomics expands towards whole mammalian brains and substantial volumes of the human brain [291, 120], progress in AI-based analysis will remain inseparable from progress in the field itself [141].

11.1.3 Image restoration and virtual staining

As discussed in Chapter 6, AI-based image restoration is becoming an increasingly important part of modern microscopy. Restoration models can suppress noise, compensate for optical degradation, improve spatial or temporal resolution, and reconstruct useful images from measurements acquired with fewer photons or less sampling [22, 297]. This can enable faster, deeper, and gentler imaging, extending the duration of live-cell experiments and revealing structures or dynamics that would otherwise be difficult to observe [334, 252, 102]. Related virtual labelling approaches can predict fluorescence-like representations from label-free or sparsely labelled images [232, 63, 189]. However, since these models generate image content rather than simply measure it, their outputs may contain convincing-looking structures that are weakly supported by the acquired data [133]. Trustworthiness therefore depends on careful validation against independent measurements, appropriate controls, and biological expectations [81]. Emerging approaches are beginning to complement generated images with calibrated, spatially resolved uncertainty information, helping researchers identify regions that require closer inspection [11, 187, 248]. Together, these developments are shifting AI to become an integrated—and increasingly transparent—part of the imaging process itself [297, 122, 256].

11.1.4 Imaging development

Development transforms a single cell into a complex organism through coordinated cell divisions, movements, shape changes, and differentiation. Modern light-sheet microscopes can capture these processes across entire living embryos with high spatial and temporal resolution, producing volumetric time-lapse recordings in which tens of thousands of cells must be segmented and tracked over hundreds or thousands of frames [154, 164, 342]. Both tasks are challenging at this scale: cells divide, deform, and become densely packed, while tracking algorithms must preserve identities, detect divisions, and remain robust to segmentation errors that can propagate through complete lineage trees [50]. Continued progress in AI-based segmentation is improving developmental analysis pipelines. Increasingly sophisticated approaches are extending machine learning to tracking itself, with recent methods addressing learned cell association, segmentation uncertainty, and iterative error correction at whole-embryo scale[199, 35, 90, 326]. Together, these advances bring us closer to turning whole-embryo movies into quantitative records of every cell's behaviour and history [199, 169], ultimately enabling systematic comparisons across embryos, perturbations, and species [7, 134].

11.1.5 Multimodal integration in large-scale experiments

High-throughput microscopy experiments are reaching an extraordinary scale, producing hundreds of terabytes of data and images of billions of cells across tens to hundreds of thousands of chemical and genetic perturbations [335, 52]. At the same time, new experimental platforms are beginning to measure several complementary aspects of cellular state, pairing morphological imaging with readouts such as gene expression, protein abundance, and spatial organisation [333, 103, 280]. Integrating these modalities is challenging as they differ in dimensionality, resolution, and experimental biases, while each captures only a partial view of the biological response. Early AI-based studies have shown that imaging, transcriptomic, and chemical representations can be combined to improve the identification of related perturbations and mechanisms of action[219, 277], while newer models are beginning to predict one modality from another, including transcriptome-guided prediction of cellular morphology[352, 327, 24]. The longer-term ambition of these efforts is the development of Virtual Cells[41]: computational models that integrate diverse measurements of cellular state and predict how cells will respond to previously unseen perturbations [2, 116, 77]. Although such models remain at an early stage [224], they point towards a future in which large parts of cellular experimentation can first be explored in silico, guiding which hypotheses and interventions should be tested in the laboratory.

Across these examples, AI succeeds not simply by replacing manual analysis, but by changing the scale, resolution, or complexity at which biological systems can be studied. In microscopy, AI does not act on images in isolation: it operates on measurements shaped by the sample, labels, optics, and the acquisition process. Its greatest impact will therefore come when models,

experiments, and biological questions are designed together rather than treated as separate stages.

11.2 Why is now a good time to start with deep learning for microscopy?

As the examples above illustrate, deep learning can provide a decisive advantage when analyzing data that is large, volumetric, high-throughput, or time-resolved. In other applications (e.g., screening, image-based cell sorting, or targeted imaging), the accuracy of the analysis pipeline determines how reliably regions and events are identified, and therefore directly influences the success of the experiment. Many such closed-loop applications, in which analysis actively shapes data acquisition, are discussed further in Chapter 7 on smart microscopy. Automated quality control, detection of rare or transient phenotypes, adaptive acquisition focused on particular structures, classification of cells before downstream manipulation, and prioritisation of candidates for follow-up experiments: all of these can directly benefit and have already been made target of the development of AI for microscopy. The scalability of deep learning analysis also allows these workflows to be extended to datasets sufficiently large to reveal subtle phenotypes and dynamic patterns. More broadly, advances in accuracy, robustness, and computational speed do more than save time: they practically redefine the experiments that can be attempted and the biological questions that can be answered.

Over the years of the explosive growth of deep-learning-based technology, the field has accumulated substantial practical knowledge. Established architectures discussed in Chapter 4 already perform well across many common tasks, including classification, segmentation, detection, tracking, and image restoration, while newer approaches continue to expand what is possible. Many microscopy problems no longer require inventing an entirely new method from scratch; instead, the challenge is often to select an appropriate model, adapt it to the data, and validate that its outputs are biologically meaningful. Such adaptation is more accessible than ever, and even developing a new method from the ground up is becoming easier. As detailed in Chapter 3, emerging conversational tools [175, 268] allow users to describe an analysis problem in natural language and receive guidance, executable code, or the beginnings of a complete workflow, making it possible to assemble useful pipelines with little or no initial coding. From there, general AI-assisted coding tools can support a gradual progression towards greater customisation, helping you understand, modify, and debug the code behind your analyses. Pretrained and increasingly general-purpose models also make it possible to transfer knowledge between datasets and tasks, often reducing the amount of annotation needed for a new experiment. Once validated, automated pipelines can apply the same analysis consistently across large studies, improving reproducibility as well as throughput. At the same time, microscopy-specific software, public datasets, tutorials, and educational resources—including this book—are making this growing body of expertise easier to find and reuse.

The existing tools discussed in Chapter 8 are already powerful, but they are most useful when applied with an understanding of how they work and where they can fail. It is tempting to treat a model as a black box and immediately pass its output to the next analysis step, but even small systematic errors can distort downstream biological conclusions. Here, you have a distinct advantage as a domain expert: you understand the experimental context, know which structures and behaviours are biologically plausible, and can recognise when an output scores well on standard benchmarks but remains scientifically implausible (as discussed in Chapter 10). Understanding the basic principles behind a method will help you combine this biological judgement with appropriate validation, recognise unreliable outputs, and design better experiments. Used in this way, deep learning becomes a powerful means of amplifying your expertise.

11.3 What's next?

Deep learning is developing rapidly and in ways that remain difficult to predict. Few researchers asked in 2016—the year before the Transformer architecture was introduced—would have anticipated today's generative models, natural-language interfaces, and foundation models, or the accuracy and generality now achieved on established microscopy tasks such as cell and nuclei segmentation. Rather than trying to forecast individual architectures, let us consider the broader directions in which microscopy image analysis is beginning to move. Across all of them, we believe that continued progress will depend on shared datasets, rigorous evaluation standards, and close collaboration between biologists, microscopists, and AI researchers.

One such direction is the increasing transfer of models from natural image computer vision to microscopy. Foundation models trained on natural images, such as DINOv2 and the newer DINOv3, are beginning to provide useful representations for cellular morphology and microscopy tasks, sometimes with limited adaptation to the target data [230, 298, 221]. This is perhaps less surprising than it first appears: humans also approach microscopy using a visual system developed through experience with the natural world, and gradually learn to recognise domain-specific structures and patterns. Future workflows may similarly span a continuum, from using general-purpose vision models directly, through lightweight adaptation and more extensive supervised fine-tuning, to full self-supervised retraining on large collections of microscopy images [8, 202, 243, 155].

For data-driven generative models that produce microscopy images from acquired measurements, the next challenge will be to make uncertainty operational. A model may already indicate that several outputs are compatible with the available data, but this information is useful only if it can propagate down the analysis pipeline. Future segmentation, tracking, and quantification methods will need to handle uncertainty in their inputs and report how ambiguity in generated images affects their results. Uncertainty could also guide experimental decisions, e.g., by triggering reacquisition, directing higher-resolution imaging towards ambiguous regions, or

identifying cases that require expert review. The goal is therefore to build workflows that respond appropriately when the data does not support a single confident interpretation.

Another emerging direction is the development of agentic analysis systems that move beyond generating individual pieces of code towards assembling and evaluating complete microscopy workflows, including additional experimental steps (Chapter 3). Their success will depend on strict validation at every stage, because an error introduced early in a workflow can propagate unnoticed into the final biological conclusion. Here again, your domain expertise is a critical asset: it provides the experimental context needed to define meaningful checks, recognise implausible outputs, and decide when closer human inspection is needed. Although current examples remain sparse, they point towards a future in which natural-language interaction becomes an interface not only to individual models, but to increasingly autonomous and reproducible experimental pipelines.

A deeper conceptual shift will be the expansion of AI from processing images to interpreting the biology measured by them. Representation-learning approaches are already being used to discover cell types and unusual phenotypes without predefined labels, or to organize proteome-scale patterns of cellular morphology and subcellular protein distribution [361, 107]. Other models can infer properties that are not directly visible in the input image, including cellular forces and mechanical behaviour [136, 307, 240], or reconstruct temporal processes from collections of static observations [34]. These early examples will give rise to analysis systems that identify phenotypes, infer hidden physical or molecular states, and propose relationships within complex biological data. The long-term opportunity is for AI to become not only a tool for extracting measurements from images, but a partner in turning those measurements into biological understanding.

11.4 How to continue from here?

One of the best ways to continue learning is to engage with the open-source and open-science ecosystem around bioimage analysis. Public model repositories such as the BioImage Model Zoo and—for deep learning more broadly—HuggingFace can show you how others preprocess data, build their models, and evaluate intermediate and final performance. Reproducing or adapting an existing workflow is often more instructive than starting from an empty notebook, as it exposes the many practical decisions that sit between an architecture and a reliable biological result. Similarly, your own models can become useful long before your final biological results are ready, so consider contributing them to model repositories with complete metadata, descriptions of their intended use and limitations, and reproducible training and validation code. Sharing the full workflow rather than only the final weights makes it far easier for others to build upon your work. Open communities such as the Scientific Community Image Forum (Image.sc) give a further opportunity to learn directly from tool developers and experienced users. As your own expertise grows, answering questions and sharing solutions is a great way to strengthen your understanding and help others solve similar problems.

Finally, the central importance of data in deep learning cannot be emphasised enough. As a domain expert, you hold the key to the future capabilities of AI models for microscopy: architectures will continue to evolve, but the quality, diversity, and documentation of the data remain the foundation of what any model can learn. Following the FAIR principles—making data findable, accessible, interoperable and reusable—helps preserve the data value beyond the original study, allowing it to support new models and biological questions. AI-ready microscopy data should therefore be treated as a scientific product in its own right: organised in standard formats, accompanied by rich metadata, and linked to clear annotations and provenance. Researchers acquiring such data can contribute it to public repositories such as the BioImage Archive, which provides a permanent home for biological imaging datasets and makes them available for reuse by the wider community. Community resources such as this book aim to provide the experience with training AI models, enabling domain experts to structure their data into training, validation, and test sets that avoid hidden leakage and provide trustworthy estimates of model performance. Designed with both biological and computational expertise, such datasets can support not only individual experiments, but also future benchmarks that attract AI researchers to the field and help address its outstanding problems. You do not need to wait for a perfect model or a complete benchmark: a documented dataset, a reproducible notebook, or a well-described failure case can already be valuable to others. Share your models, data, and expertise generously! In doing so, you join the community shaping the future of AI in microscopy and help make the next biological discoveries possible.

Glossary

A-posteriori Analysis

An approach in which images are processed after the complete dataset has been acquired and stored.

AI agent

AI agents extend models into systems that can take actions (running code, calling APIs, controlling hardware), observe the results, and adapt their approach iteratively. At its core, an AI agent is a system built around a language model that operates in a loop: it observes the current state (an image, a dataset, an error message), reasons about what to do next, acts by invoking tools or executing code, and then observes the outcome — repeating until the task is complete. This observe-reason-act cycle is what distinguishes an agent from a chatbot.

Aberration

A distortion of the light's wavefront caused by imperfections in the optical path or by refractive-index variations in the sample. It spreads the light and makes images blurrier than the ideal diffraction-limited image.

Activation Function

An activation function is a mathematical formula that calculates the output of a node. These functions are nonlinear, which allows the network to solve nontrivial problems. They are so named because the output of the function decides to what extent each node in the network is "activated," an analogy to the way in which biological neurons can be activated.

Activations

The values of elements in neural network units, often specifically the values of the output of an activation function.

Adaptive Optics

Technology that senses distortions in the wavefront of light and cancels them, thereby suppressing optical aberrations to enhance image clarity.

Agentic Workflow

Agentic workflows are hand-designed sequences of tool calls with human-specified decision points. A developer would define: "first call the segmentation tool, then if the count is above 100 switch to algorithm B." These workflows are effective but rigid: every decision point must be anticipated in advance.

Artificial intelligence

Artificial intelligence (AI) refers to a computational system that is capable of performing a task that would normally require human intelligence.

Attribution map

A per-pixel image of how much each part of the input contributed to a model's output. Some widely used variants are insensitive to the trained model and should be checked before they are trusted.

Autoencoder

A deep learning architecture used to learn efficient coding of unlabeled data. An autoencoder learns two functions: an encoding function that transforms the input data, and a decoding function that recreates the input data from the encoded representation. It is an unsupervised learning version of the more general encoder-decoder architecture.

Auxiliary Variables

Auxiliary variables are variables added to a linear programming problem to simplify mathematics, improve precision, or handle missing data. An example of an auxiliary variable is distance to the boundary. These or other shape descriptors can be used as part of of a loss function.

Average Precision

Average Precision (AP) summarizes the precision-recall trade-off across confidence thresholds. A predicted instance counts as a true positive only if its IoU with a ground-truth instance exceeds a threshold x:

$$\mathrm{AP}_x = \sum_k (R_k - R_{k-1})\, P_k$$

where P_k and R_k are precision and recall at the k-th confidence threshold after matching predictions to ground-truth at IoU $\geq x$. Higher x demands more precise localization.

Backpropagation

The method that computes, for every parameter of a neural network, how a small change in that parameter would change the error measured by the loss function. It applies the chain rule of calculus backwards, from the error through all layers of the network. Backpropagation itself does not change the network; gradient descent uses the values it computes to adjust the parameters so that the error decreases.

Balanced Dataset

In a balanced dataset, each class or type of data is equally represented in the training data. In an unbalanced dataset, the model may become biased towards the classes that have more examples.

Batch

A small group of data that is processed together at the same time. For example, when training a machine learning model, a batch is a group of data that is given to the model for learning. Batches are commonly used to make the processes more efficient.

Batch Effects

Systematic technical differences between groups of samples that arise from preparation, handling, acquisition, or processing rather than from the biological variable of interest.

Bayesian Optimization

A strategy that allows the optimization of black-box functions such as deep neural networks. It creates a surrogate model, which is a probabilistic representation of the objective function, using only a few example points.

Bias Vector

The bias vector is a set of values (one for each node in the layer) that is added to the weighted input of the layer before the activation function is calculated. Note that the use of "bias" here is mathematical and not related to any potential biases in the model's decision making.

Binary Cross-Entropy

Binary cross-entropy (BCE) penalizes the model based on how confident it is in the wrong answer. BCE requires the output to be interpretable as a probability — it asks "is this pixel foreground or background?" and treats all wrongly-classified pixels the same way, whether they are at a cell boundary, inside a hole, or belong to an entirely missing cell. For ground truth label $y_i \in \{0, 1\}$ and predicted probability $\hat{y}_i \in [0, 1]$:

$$\text{BCE} = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$$

Binary Segmentation

A type of image segmentation where each pixel is classified into one of two categories — typically "foreground" (e.g., cell) or "background." The output is a binary mask distinguishing objects (set to a value of 1) from their background (0).

Boosting algorithm

An ensemble method that builds a strong classifier from many simple ones, typically small decision trees, trained one after another so that each corrects the errors its predecessors left behind.

Bounding region

A geometric shape, typically a box, drawn around an object in an image to indicate its spatial extent.

CARE (Content-aware image restoration)

A deep learning-based method for image restoration that leverages content-specific features to enhance degraded images. See https://github.com/CSBDeep/CSBDeep for more information.

Capacity

A model's capacity reflects how much information a network can store. It corresponds to a network's ability to model an arbitrary function.

Centroid coordinate

An object's geometric center point, used as a simple way to describe its location in an image.

Channels

In natural color images there are often three channels (red, green, and blue). In microscopy images, the number of channels can refer to the number of fluorescent labels or image settings used while imaging. The term is used similarly in neural networks where channels refers to the number of layers in an image. These channels can come from convolutional operations, leading to images with hundreds of channels corresponding to each filter used.

Class

A class describes the group or category an image, pixel, or object falls into. This includes distinguishing dataset types such as “control” or “mutant,” as well as distinguishing “foreground” and “background” pixels.

Class label

A descriptive name selected from a predefined list of categories, assigned to a detected object or event to describe its identity.

Classification

The process of assigning a data point to one of a set of predefined categories, based on its measured features.

Clustering

Clustering groups unlabeled points within a dataset based on similarities or patterns in the data.

Coding Agent

Coding agents are AI systems that operate directly in development environments with access to terminals, file systems, and web browsers. These agents go well beyond earlier LLM assistants that could only suggest code snippets.

Computer Vision

A field of computer science wherein computers extract information from images. It often involves object detection within images and can involve classification of the images and/or objects.

Computing Cluster

A computing cluster connects individual computers (nodes) into a fast network that allows them to act as a more powerful computer.

Confidence Score

A continuous value, often between 0 and 1, that reflects how certain a model is about a given prediction. Unlike a distance transform, which encodes geometric information such as position within an object, a confidence score expresses the model's belief that a prediction is correct — for example, that a pixel belongs to the foreground class or that a detected object was correctly identified during object detection. Confidence scores are useful for filtering predictions, such as discarding low-confidence detections, but a high confidence score does not guarantee a correct prediction, particularly when a model is poorly calibrated.

Context Window

A context window is the size of the largest input that a model can consider at once. Making the context window larger allows the model to incorporate more information into the output.

Contrastive loss

A contrastive loss function attempts to minimize the distance between input pairs that are similar while simultaneously maximizing the distance between input pairs that are dissimilar.

Convolution

A mathematical process where a kernel (small matrix) slides over input data (e.g., images) to compute feature maps, highlighting patterns like edges or textures.

Convolutional Neural Networks (CNNs)

A deep learning architecture that applies convolutions to automatically learn features from images for computer vision tasks like classification and detection.

Coordinate transformation

A mathematical process for expressing the position of a point from one coordinate system to another. For example, knowing how many pixels were removed from a cropped image allows for calculating the location of a cropped pixel in the original image.

Cytoprofile

A numerical feature vector describing a biological cell's phenotype, derived from measurements of shape, intensity, and texture extracted from microscopy images.

Data Augmentation

A strategy to artificially increase the diversity of a dataset prior to training by applying transformations such as rotation, flipping, or brightness adjustment. It helps improve model robustness and generalisation.

Data Leakage

Information from validation or test examples that influences model training or selection, often because related samples, duplicates, or preprocessing statistics cross dataset splits.

Data Provenance

The record of how each file in a dataset came to be and what produced it, forming a graph in which every file is a node and every processing step an edge back to the files it was derived from. Acquisition is the first node, holding the specimen, instrument, and settings that produced the raw image. Later nodes cover processing steps with their order and parameters, targets with the annotator, model, or script that generated them and the review they received, and the assignment of examples to splits. Provenance is what allows a result to be traced back to the raw data, corrected, and reproduced.

Data loader

The data loader is a software component that connects the raw training data files with the training pipeline. It can manage tasks such as batching and efficiently feed samples to the model.

Decision tree

Decision trees are flow-chart-like models that make simple yes/no decisions in a series of conditional steps. The first root node splits the entire dataset into two groups, after which internal nodes and branches continue to split the data through conditional questions, finally terminating in a leaf node that provides the answer, class, or prediction.

Deconvolution

A mathematical process to partially reverse the blurring effect caused by the microscope's PSF, increasing contrast and resolution over the raw image data if performed carefully.

Deep learning

Deep learning is a type of machine learning that uses multi-layer neural networks to perform tasks. The term "deep" refers to the use of many layers in the network.

Denoising

Denoising is a process in which unwanted noise or distortions are removed from an image or dataset. The goal of denoising is to preserve the underlying signal while doing so.

Detection

Detection finds where the objects are without defining their full shape. This requires simpler annotations (e.g., a point or box) than full segmentation.

Differentiable Function

A differentiable function is a function whose derivative exists at every point within the function's domain.

Dimensionality reduction

Taking a dataset that is in a high-dimensional space and reducing it to a lower-dimensional representation that tries to retain as much of the important information and underlying properties of the high-dimensional data as possible.

Distance Transform

A transformation that assigns each foreground pixel a value representing its distance to the nearest background pixel or object boundary, producing an image that peaks at object centers and falls to zero at their edges. In instance segmentation, distance transforms are commonly used to help separate touching or overlapping objectsâ€"for example, as a target the model learns to regress during training, or as an input to watershed-based post-processing that splits a binary mask into individual instances.

Domain Randomization

Using simulations or synthetic training data, domain randomization applies random and exaggerated variations to background, lighting, shapes, or textures in the synthetic dataset. This strategy helps the model learn domain-invariant features and is usually used for pretraining a neural network or to enable simulation-to-real transfer.

Domain shift

Domain shift occurs when a model is trained and tested on one type of data, but later applied to a related but different data distribution. In microscopy, an example of this is training a model on data from one microscope, but then applying it to data acquired using a different system or technique.

Downsampling

Downsampling is a technique to reduce the sampling of input data. For an image, this involves descreasing the number of pixels, and thus decreasing the resolution of the image.

Effect Size

How "strong" a phenotype is, or how mathematically possible it is to distinguish a given population from the control population.

Embedding Space

The space of lists of numbers (embeddings) that a model uses to describe its inputs. Also known as latent space or latent feature space.

Encoder-Decoder

An encoder-decoder architecture refers to a network with two parts. The "encoder" component of the network is a series of layers that take the input and map it to an abstract representation. The second "decoder" component of the network is trained to recreate input data from the abstract representation. This type of architecture learns an efficient way to represent the input data and is often used for dimensionality reduction.

End-to-end learning

A deep learning approach in which all stages of a task (e.g., locating and identifying objects) are optimized jointly within a single model, rather than being handled as separate, independent steps.

Ensemble

A machine-learning approach that combines predictions from multiple individual models to produce a single result that can be more robust than any single model alone.

Epoch

One complete pass through the entire training dataset during the training process.

Event

A set of spatial features that change or transitions over time, temporally defined by a beginning and an end.

Event detection

The algorithmic identification of temporal changes or transitions in a system, requiring analysis of data across time.

F1 Score

A classification metric that gives the harmonic mean of precision and recall. This metric was originally designed for binary classification but can be adapted to multiclass classification by calculating the F1 score per class.

$$\mathrm{F1} = \frac{2\,\mathrm{TP}}{2\,\mathrm{TP} + \mathrm{FP} + \mathrm{FN}}$$

This metric is also known as the DICE coefficient.

False Negatives

In a scenario where you have two classes "positive" and "negative", you try to predict cases as one of those classes. False negatives are the cases that you incorrectly predicted as negative and were really positive.

False Positives

In a scenario where you have two classes "positive" and "negative", you try to predict cases as one of those classes. False positives are the cases that you incorrectly predicted as positive and were really negative.

Feature vector

A set of numerical measurements (such as size, shape, intensity, or texture) that together provide a quantitative description of an object.

Fiji

An image processing platform that comes bundled with many plugins for scientific image analysis. See https://imagej.net/software/fiji/ for more information.

Fine-Tuning

Fine-tuning takes a pre-trained model and adapts it to a new task by providing task-specific training data. In classic fine-tuning, all layers of the pre-trained model are retrained, with their weights initialised from the original training. Thus, you continue training the full model using the new data. This approach allows the model to adjust more comprehensively to new data while still preserving the advantages of pre-learned features.

Foundation Model

A foundation model is a model that was trained on a very large and very diverse dataset. Because of this diversity in training data, foundation models can be used for many downstream tasks or can be fine-tuned for specific purposes.

Frequency Domain

The representation of an image as a function of spatial frequency, obtained by transforming an image from the spatial domain using the Fourier transform.

Function Calling

Function calling is also called tool use. This is the ability for LLMs to invoke external tools, APIs, and software through structured interfaces rather than generating plain text.

Gaussian Process

A common surrogate model for optimization strategies such as Bayesian Optimization. A Gaussian Process is a non-parametric model that represents a probability distribution over possible functions. In the hyperparameter search scenario, the Gaussian Process models the probability of getting an objective function value based on some hyperparameters.

General Agency

General agency gives AI agents access to broad capabilities — a terminal, a file system, APIs, analysis libraries — and lets them determine the right sequence of actions for each task. Rather than pre-scripting workflows, the agent receives a goal ("segment and count the nuclei in this dataset") and works out what tools to use, in what order, handling errors as they arise.

Generative Adversarial Networks (GANs)

A deep learning architecture where two neural networks, a generator and a discriminator, are trained in an adversarial process, enabling the generator to create synthetic data, such as realistic images, by learning to deceive the discriminator.

Generative Model

A model trained to draw samples from the distribution of possible outputs, rather than to return their mean. Running it twice on the same input can give two different answers. Each sample has the texture and contrast of real data, unlike the smooth mean, but no single sample represents the whole distribution. Variational autoencoders, generative adversarial networks, diffusion models, and flow matching are common examples.

Genetic Algorithms

An optimisation method inspired by the principles of natural selection and genetics. It starts with a population of solutions. These solutions are combined through a process called crossover to produce new solutions (offspring). During this process, random changes or mutations may occur to introduce diversity. After crossover and mutation, a selection step chooses the best solutions from both the parent and offspring populations to form the next generation. This cycle repeats for a set number of generations or until a predefined goal or stopping criterion is met.

Gradient Descent

Gradient descent computes the gradient of the loss with respect to each weight. Moving the weights in the negative direction of the gradient reduces the loss for the given images or data points over which the loss is computed.

Ground Truth

Accurate data against which a model can be evaluated. Ground truth data is often manually annotated. The data type itself will vary depending on the task and evaluation. e.g. instance segmentation may be compared to ground truth object counts or masks.

Hallucinations

Outputs from a model that do not have a basis in the input data and may contain false or misleading information.

Harmonic Mean

A type of average of a set of numbers, calculated as the number of values divided by the sum of their reciprocals. Unlike the arithmetic mean, the harmonic mean is closer to the smaller of the values being averaged, so it penalizes cases where one value is much lower than the other. This makes it useful for combining two metrics, such as precision and recall, into a single score that only looks good when both metrics are good.

Hausdorff Distance

The Hausdorff distance measures how far apart two sets of boundary points are at their worst-case point. For boundary point sets A and B:

$$d_H(A, B) = \max\left(\sup_{a \in A} \inf_{b \in B} d(a, b),\ \sup_{b \in B} \inf_{a \in A} d(a, b)\right)$$

A small Hausdorff distance means every point on one boundary has a close counterpart on the other.

Human-in-the-loop

Human-in-the-loop workflows use model predictions that are manually corrected for retraining. This iterative approach can be used to supervise the model output and improve accuracy.

Hyperparameters

The options you choose when training a machine learning model that affect the training process or the architecture of the model (e.g., learning rate, batch size, number of layers, training loss, etc.) are called hyperparameters. This term is used to differentiate them from the parameters (also known as weights) of the machine learning model.

Image Classification

A computer vision task where each image is associated with one class and the goal of this task is to correctly predict that class.

Image Restoration

The process of recovering clear, high-quality images from degraded raw data contaminated by blur, noise, or other distortions.

Image Translation

Image-to-image translation (or style transfer) is a generative AI technique which transforms an input image into the style of another image. In the microscopy context, this could manifest as a network which transforms an image of one modality (e.g. brightfield) into a different modality (e.g. DAPI staining of nuclei) in the same field of view.

Instance Segmentation

A segmentation task that not only separates objects from the background but also distinguishes between individual objects of the same type (e.g., separating touching cells one by one).

IoU

"Intersection over Union". A segmentation metric that calculates the area of overlap between two segmentation masks divided by the area of their union.

Kernel

Convolutional kernels are also known as filters. They are small matrices that define the function for taking input pixels and creating an output image. They are often used for tasks like blurring or edge detection.

Large Language Model (LLM)

A foundation model for text, pre-trained on a vast amount of text to predict the next word, and usually further trained to follow instructions. Users interact with LLMs in natural language to accomplish a variety of tasks.

Learning Rate

The amount weights are updated at every iteration during gradient descent. Metaphorically, this represents how fast the network "learns" from the current state at each iteration.

Learning Rate Schedule

A programmatic change in the learning rate over the course of an entire training run. This often involves reducing the learning rate near the end of training to enable smaller steps down the gradient, fine-tuning the weights near the loss minimum.

Linear Layer

Linear layers are also known as fully-connected layers or dense layers. They are a set of input nodes (or neurons) that are each connected to every output node of the layer.

Localization

The task of determining the spatial position of an object or event within an image.

Logprobs

Logprobs is short for log probabilities. Taking the logarithm of a probability has practical computational advantages. In the AI context, logprobs refers to the probability of each token, given the previous token, and can be used as a measure of model confidence.

Long-short term memory (LSTM)

A type of recurrent neural network designed to retain information over longer sequences, addressing limitations of standard RNNs in capturing long-range temporal dependencies.

Loss function

A loss function (or cost function) is a formula for quantifying how much a model's prediction deviates from the actual ground truth value. The loss function returns a single scalar number to quantify the loss. As training relies on gradient descent, the loss must be differentiable; the algorithm needs the slope of the loss with respect to every parameter to know which direction to step.

Machine learning

A type of artificial intelligence in which a person designs a family of functions with free parameters, and an optimization procedure fits the parameters to a set of examples. The rules are learned from data rather than written down by hand, and the goal is that they generalize to new data.

Manual Annotation

The process of manually labeling specific structures or objects in an image using drawing tools. Typically done in software like Fiji or Napari, this step is essential for creating ground truth data to train or evaluate machine learning models.

Mean Squared Error

Mean-squared error (MSE) penalizes the model proportionally to the squared difference between its prediction and the ground truth:

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

MSE measures *how far* the prediction is from the target value, and does not require the output to be a probability — it works equally well when the model produces a continuous map such as a distance transform or a confidence score. Because it sums squared errors over all pixels, it is sensitive to the total *extent* of the error: predicting an entire missing cell as background accumulates far more MSE than a few wrongly-classified boundary pixels.

Metadata

Any data that provides additional information about other data. In bioimaging, examples include information about sample preparation, the imaging instrument, and image acquisition parameters.

Model Context Protocol (MCP)

The Model Context Protocol (MCP) is a standardized way for AI systems to connect to external tools and data sources. MCP gives any AI model a universal way to discover and use any compatible tool — analogous to how USB standardized device connectivity. For microscopy, MCP means that a single agent could potentially access a microscope control system, an image analysis library, a literature database, and a model repository through the same protocol.

Model collapse

Model collapse occurs when performance degrades due to training on uncurated or synthetic data, especially training on data from another model (including the same model at prior training steps).

Momentum

In the context of learning rate, momentum speeds up learning when the gradient is consistent over multiple iterations, which allows the training to avoid local minima.

Multilayer Perceptron

A sequence of fully-connected layers that are applied in series, each on the output of the previous one, is called a multilayer perceptron (MLP). This is the simplest example of a deep neural network.

N2N (Noise2Noise)

A denoising method that trains a neural network on pairs of independently noisy images of the same scene, one as input and the other as target. It needs no clean reference image, but it needs the pairs. See https://github.com/NVlabs/noise2noise for more information.

N2S (Noise2Self)

A self-supervised denoising method that trains a neural network assuming statistically independent noise across the image, requiring only single noisy images without paired clean data. See https://github.com/czbiohub-sf/noise2self for more information.

N2V (Noise2Void)

A self-supervised denoising method that trains a neural network to predict pixel values from noisy images by masking input pixels, requiring only single noisy images without paired clean data. See https://github.com/juglab/n2v for more information.

NaN

Not-a-number (NaN) values are used where an operation results in something that is not a real number (such as dividing zero by zero).

Natural Image

Natural images are images of the environment. This most commonly refers to color photographs of people, nature, objects, etc. Many machine learning researchers focus on natural images for tasks like self-driving cars, but microscopy images pose additional challenges for AI.

Network Architecture

The architecture of a machine learning model refers to the design and structure of the model, including choices in how individual components (such as preprocessing or feature extraction) are connected to each other.

Neural Network

A function built from many simple units, each adding up weighted inputs and passing the result through a simple nonlinear response, arranged in layers. The weights are the parameters learned during training. The name is a historical analogy to networks of biological neurons.

Neural Network Unit

A neural unit is the processing element of a neural network. It also sometimes called a node or artificial neuron.

Nonlinear

If a system or function is non-linear, changes in its output are not proportional to the changes that are made to the input. Non-linear functions are incorporated into machine learning networks to allow for the solution of complicated problems because it allows for non-linear decision boundaries.

Nonlinear Problem

A mathematical problem where the governing equations or operations are nonlinear, meaning outputs are not linearly proportional to inputs.

Normalization

Data normalization is a pre-processing step to prepare input images (or features) such that their values are in a common range. Values are often rescaled to [0,1] or [-1, 1] to avoid models learning about absolute values of intensity rather than image features.

Nyquist sampling

The Nyquist sampling rule states that recovering a continous signal requires sampling it at twice its highest frequency to accurately reconstruct it. In microscopy, Nyquist sampling thus requires that pixel sizes should be smaller than half the relevant optical resolution.

OME-Zarr

OME-Zarr is a next-generation file format (NGFF) developed by the Open Microscopy Environment. The Zarr format stores images in smaller chunked image arrays, which allows for efficient access, storage, and processing of large datasets, while OME provides standards for accessing microscopy metadata.

Object

A discrete spatial feature within an image, spatially defined by properties such as size, shape, intensity, and location.

Object Detection

A computer vision task that identifies and locates individual objects within an image, typically by drawing bounding boxes around them. It provides both the category (what) and position (where) of each object.

Object storage

Object stoarge is a computer storage architecture that views data as blobs or objects, rather than a traditional file system file hierarchy. This has advanatges for indexing and metadata management.

Optimizer

An optimizer is a function that is used to adjust model parameters during training. By adjusting weights and biases, the optimizer attempts to minimize the loss function and thus make the model more accurate.

Overfitting

When a model fits its training data so closely that it memorizes noise and incidental details, and no longer generalizes well to new data.

Padding

Padding involves adding extra layers of pixels around an image to avoid edge effects. Padding can be performed by simply setting the extra pixels to zero, or it can take into account the image itself (such as in mirror padding or replicate padding).

Panoptic Segmentation

A computer vision technique that is a combination of semantic segmentation and instance segmentation. It separates an image into regions while also detecting individual object instances within those regions.

Patches

In image processing, a patch refers to a small region or subset of pixels that are extracted from the larger image.

Perceptron

Perceptrons are a specific type of an artificial neuron or node. They perform a calculation that is a linear weighted sum of the inputs to the node followed by a nonlinear activation function. They are used for binary classification, and can be combined to create more complex network architectures.

Perceptual Loss

Instead of comparing pixel intensity, perceptual loss functions compare high-level features — such as those defined in the embedding space — of images between model outputs and ground truth. This allows models to produce outputs that match the high-level structure and texture of the ground truth that human perception encodes.

Pixel Classifiers

Machine learning models that classify each pixel in an image based on features such as intensity, texture, or local neighborhood. Commonly used in traditional workflows for segmentation or classification tasks.

Point Spread Function (PSF)

A mathematical function that describes how an imaging system blurs a point source.

Pooling Layer

A pooling layer aggregates information from many vectors into fewer vectors, i.e., it downsamples the image. The pooling layer does this using sliding windows across the image of maximum or average operations. This removes redundant information and reduces the size of the image for the following calculations.

Pre-training

The first and most expensive stage of training, in which a model learns a general-purpose representation from a large dataset, usually without labels by self-supervised learning. The pre-trained model is then adapted to a specific task with far fewer labeled examples, by fine-tuning or semi-supervised learning. Foundation models are pre-trained models.

Precision

A classification metric that measures the proportion of predicted positives that are correct: true positives divided by the sum of true positives and false positives. Precision answers "of everything the model called positive, how much actually was?" A model can achieve high precision by only making positive predictions it is very confident about, at the cost of missing some true positives (i.e., lower recall).

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Probability Vector

A probability vector is composed of values that indicate the probability of a particular variable (e.g., the probability that an image is a dog or cat). The values of a probability vector sum to one.

Pseudo-label

Unlike a manually annotated label, a pseudo-label is created by a model. In semi-supervised learning, this allows a model to learn from a few manually annotated labels and large amount of unsupervised data. However, pseudo-labels can be biased by mistakes in the model and should validated in the same manner as manual annotations.

Quality Control Metric

Any metric that can be used to evaluate quality. It will vary depending on the task and data type. It can be binary (e.g. an image doesn't have debris) or continuous (e.g. annotated object centroids are within 5 pixels of the ground truth centroids).

RCAN (residual channel attention network)

A deep learning-based method using residual learning and channel attention to improve image restoration tasks. See https://github.com/AiviaCommunity/3D-RCAN for more information.

Random forest

An ensemble of decision trees, each trained on a random subset of the examples and features, whose votes are combined into a single prediction. Widely used for interactive pixel classification, e.g. in ilastik and Labkit.

ReLU

An activation function common in deep learning that outputs the input directly if it is positive, and outputs zero otherwise. That is, it is the ramp function where it equals x if x>0 and 0 if x<=0. This characteristic helps introduce non-linearity into the model and mitigate the vanishing gradient problem. ReLU stands for rectified linear unit.

Real-time analysis

An approach in which images are processed as the data is acquired. This is in contrast to a-posterior analysis, where the data is acquired before processing.

Recall

A classification metric that measures the proportion of actual positives that were correctly identified: true positives divided by the sum of true positives and false negatives. Recall answers "of everything that was actually positive, how much did the model find?" A model can achieve high recall by predicting positive more liberally, at the cost of more false positives (i.e., lower precision). Also known as sensitivity or the true positive rate.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Receptive Field

The receptive field size of a convolutional layer is the total spatial extent of pixels that influence the activations of the layer. The receptive field of the entire network describes the size of the region in the input image that influences the output image.

Recurrent neural networks (RNNs)

Neural network architectures designed to process sequential data by maintaining an internal memory of previous inputs, used to model temporal dependencies.

Region of interest (ROI)

A defined area within an image or specimen selected for closer analysis, imaging, or targeted intervention.

Regularization

Regularization adds a penalty term to the loss function to prevent overfitting. This term discourages overly complex models.

Reinforcement learning

In reinforcement learning, AI agents learn how to make decisions by acting and receiving rewards or penalties for the consequences, adjusting their behavior to collect more reward. No examples of correct behavior are needed.

Representation

The internal description of an input that a network computes on the way to its output: a list of numbers that summarizes what the network has learned to consider relevant about an image or a text. A good representation can be reused for tasks the network was not trained on, which is what makes foundation models useful.

Representation collapse

A failure of self-supervised learning in which the network describes every input with the same, or nearly the same, embedding. This trivially satisfies objectives that only reward agreement between views, but the network learns nothing. Not to be confused with model collapse.

ResNet

A residual neural network (ResNet) is a deep learning architecture in which skip connections between layers are used to learn residual mappings instead of direct transformations, increasing training efficacy and deeper networks.

Resampling

In the context of CNNs, resampling describes changing the spatial dimensions of the image or features maps through either downsampling or upsampling. For example, an input 2D image could be resampled to be half the size in x and y.

Residual Connection

A specific instance of a skip connection, in which the input of a transformation block is added to the output.

Resolution enhancement

Deep learning methods that attempt to predict higher resolution images given lower resolution input data.

Retrieval-Augmented Generation

A technique that allows LLMs to incorporate new supplmental information from outside the training set. This increases the flexibility of LLMs to respond to domain-specific queries.

Segmentation mask

A segmentation mask is an image where each pixel is given a corresponding class. For example, this could be a black and white image where cells are white and the background is black.

Self-attention

Attention allows a model to determine the importance of each component of an input sequence. Self-attention is a type of attention mechanism that is commonly used in transformer architectures.

Self-supervised learning

A form of machine learning that turns unlabeled data into a supervised problem, typically by hiding part of the data and training the model to fill it in. The model either is used directly, as in self-supervised denoising, or provides a representation that is later adapted to other tasks.

Semantic Segmentation

A form of segmentation where each pixel in an image is assigned to a class (e.g., nucleus, cytoplasm, background), but it does not distinguish between separate instances of the same class.

Semi-Supervised Learning

Training on a small amount of labeled data combined with a large amount of unlabeled data, for example by letting the model label the unlabeled examples itself (pseudo-labels) and learning from those as well. It often starts from a model after pre-training without labels. Not to be confused with weak supervision, which trains on coarse or noisy labels.

Shallow decision trees

Simple, low-complexity decision tree models that make classification decisions using only a few splits, often used as building blocks within ensemble methods.

Shape descriptors

A set of quantitative measurements that characterize the geometric form of an object, such as circularity, elongation, or perimeter.

Shortcut learning

Shortcut learning occurs when a network learns about superficial or trivial features in the data rather than learning the important underlying concepts. For example, a classifier whose control wells all sit at the edge of a plate may learn to recognize the edge position rather than the phenotype, and networks trained on chest radiographs can pick up which hospital acquired an image.

Sigmoid Function

An activation function common in deep learning that non-linearly maps real inputs to outputs between 0 and 1, being most sensitive to changes in inputs around zero and increasingly compressing extreme positive or negative inputs as they approach 1 or 0 respectively; this characteristic enables it to model probabilities for binary classification and introduce smooth non-linearity.

Signal-to-noise ratio

The signal-to-noise ratio (SNR) measures the level of the desired signal to the unwanted noise or background. In fluorescence microscopy, this often relates the intensity of the labeled object to the intensity of the image background.

Silent error

A silent error occurs when a model outputs the wrong answer or otherwise fails without generating any warning or message.

Single-shot detection

An object detection approach in which localization and classification are performed simultaneously in a single pass through the model, rather than through separate sequential stages.

Skills Atrophy

Skills atrophy is the decrease or loss of human expertise due to underuse. In this context, skills atrophy could be caused by an over-reliance on AI tools. This gradual erosion of the community's ability would decrease the field's ability to evaluate AI outputs.

Skip Connection

Skip connections are also sometimes know as residual connections. These connections are made when the output of a layer of the model is added to or concatenated with a later layer in the network while bypassing any layers that may be in between. For example, the layers in a U-Net architecture connect across image resolutions.

Smart microscopy

An imaging approach in which acquisition parameters are automatically adjusted based on features or events recognized in the data, rather than relying solely on manual operator control.

Spatial Domain

The representation of an image as a function of spatial coordinates.

Star-convex Polygon

A geometric shape used in segmentation algorithms like StarDist. Imagine drawing straight lines (rays) from the centre of an object out toward its edges — if you can see the edge from the centre in all directions, the object is considered star-convex. This method works well for blob-like structures such as nuclei, because their general shape can be captured by measuring how far each ray travels from the centre to the boundary.

Stochastic Gradient Descent (SGD)

As most modern problems require large training datasets to effectively capture underlying signals, attempting to calculate the gradient over the entire dataset is often computationally prohibative. This is overcome by batching the training data into smaller subsets. This naturally injects some stochasticty (randomness) into the gradient descent procedure based on how the batches are randomly determined.

Stride

When performing a convolution or cross-correlation, the kernel (aka the filter) moves across the entire image. When the stride is 1, this corresponds to moving the filter across every pixel in the image. For larger values of the stride, the kernel is moved more than one element at a time. For example, a stride of 2 would skip every other pixel in the image.

Structural Similarity Index (SSIM)

The structural similarity index measure (SSIM) is a metric which measures the similarity between two images. It attempts to improve upon MSE by taking image texture into account. It is calculated using the following formula.

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2\mu_y^2 + c_1)(\sigma_x^2\sigma_y^2 + c_2)}$$

where x, y are two images, μ is the average intensity, σ is the variance of the intensity, and c_1, c_2 are variables determined by the dynamic range of the image ($c_i = (k_i L)^2$ for $k_1 = 0.01, k_2 = 0.03$).

Supervised learning

A form of machine learning in which every training input comes paired with the correct answer, the target, and the model learns to produce that answer from the input. The target can be a class, a position, a mask, or another image.

Support vector machine (SVM)

A supervised machine-learning algorithm that learns the decision boundary that best separates data points of different classes by maximizing the margin between them.

Tensor

Tensors are multi-dimensional matrices that are used in many machine learning contexts as a way of organizing data.

Test Data

Data held out from both the training data and validation data, used only after training is complete to give an unbiased estimate of how a model will perform on new, unseen data. Because it plays no role in training or hyperparameter tuning, test data should only be used once a model is finalized.

Texture

A quantitative descriptor of the spatial pattern of intensity variation within a region of interest, capturing properties like smoothness or granularity.

Tokens

Tokens are units of data processed by an AI model. For an LLM, these tokens are chunks that may be whole words or subword pieces. For instance, “microscope” might be a single token, while “fluorescence” might be split into “fluor” + “escence.”

Tool Definitions

Tool definitions are structured descriptions of available functions, their parameters, and expected inputs. When a user makes a request, the model reasons about which tools are needed, generates structured calls with correct parameters, receives results, and decides whether to take further action or report findings.

Tracking

Tracking links objects over time. It requires both finding objects and determining how they are associated over time.

Training Data

Data used to teach a machine learning model to perform a task, such as classification, segmentation, restoration, or generation. Depending on the method, training data may be paired with target labels or ground truth (as in supervised learning) or used on its own without labels (as in self-supervised or unsupervised learning).

Training Target

A target is the feature or outcome that a model should output. This could be a label for an image, a segmentation, or even another image. In supervised learning, the model is trained on input images or features paired with a corresponding target.

Transfer Learning

A deep learning technique that reuses a model pre-trained on one task as the starting point for a new, related task, leveraging its learned knowledge to improve performance or reduce training requirements. In practice, part of a pretrained neural network (usually the initial layers, responsible for feature extraction) is frozen and reused in a new model. These frozen layers, with the knowledge from a previous dataset, are combined with untrained layers tailored for a specific bioimaging task. During training, only the new layers will be updated, allowing the model to adapt to the new task with limited data.

Transformer Models

A deep learning architecture, originally developed for language, in which every part of the input is compared with every other part (attention), so that distant regions can inform each other. A vision transformer (ViT) cuts an image into small patches and treats them like the words of a sentence.

Transposed Convolution

As in a normal convolution, a transposed convolution involves an image and a kernel. However, in this case, the image slides over the kernel (as opposed to the other way around). This naturally results in a spatially upsampled output, relative to the input. The stride and padding determines the upsampling factor. In particular, if the input is of size (I_h, I_w), then a transposed convolution with kernel size (k_h, k_w), padding size (p_h, p_w), and stride (s_h, s_w) will result in an output of size (O_h, O_w):

$$O_h = (I_h - 1)s_h + k_h - 2p_h$$
$$O_w = (I_w - 1)s_w + k_w - 2p_w$$

This is often used in CNN architectures such as the U-Net to produce outputs at the correct spatial scale. See this example for more information.

True Negatives

In a scenario where you have two classes "positive" and "negative", you try to predict cases as one of those classes. True negatives are the cases that you predicted as negative and were really negative.

True Positives

In a scenario where you have two classes "positive" and "negative", you try to predict cases as one of those classes. True positives are the cases that you predicted as positive and were really positive.

U-Net

A U-Net is a type of convolutional neural network that was specifically developed for the task of image segmentation. It has since been used for a variety of tasks. The model is so named because downsampling layers are followed by upsampling layers, which are each connected by skip connections, and together this gives diagrams of the network a U shape.

UUID

A Universally Unique Identifier (UUID) is a number used to identify a file or information. When generated using UUID standards, these 128-bit numbers are unlikely to be duplicated even across independent users and thus can be used without worrying about confusion with other files.

Unsupervised learning

Unsupervised learning is a type of machine learning in which the machine works only with the raw, unlabeled data. It uses algorithms to find structure or patterns in the data without user guidance or predetermined categories.

Upsampling

Upsampling is a technique to increase the sampling of input data. For an image, this involves increasing the number of pixels and interpolating values for the newly sampled pixels.

Validation Data

Data set aside from the training data and used to evaluate a model's performance during training, without being used to update the model's weights. It is typically used to tune hyperparameters and monitor for overfitting, for example by deciding when to stop training.

Virtual Machine

On a physical computer, you install an operating system (e.g., Windows or Ubuntu) that you interact with. A virtual machine is a program that simulates a complete computer with its own operating system. This lets you run a "computer inside your computer" (e.g., using Linux inside Windows or the other way around). As this simulated computer is separate from your physical one, it adds an extra layer of security, because unless the user specifically allows it, the virtual machine cannot access or connect to your real computer.

Vision-Language Model (VLM)

A Vision-language Model (VLM) is a type of model that can process both images and text with a single architecture. Because they can accept both natural language and visual inputs, they can include context in their analysis.

Visual Question Answering

Visual Question Answering (VQA) is when a model is asked an open-ended question based on an image input. The result is a natural language response to a natural language question associated with an image.

Weakly Supervised Learning

Weakly supervised learning takes partial, noisy, or imprecise labels to guide training. These labels are often combined with unlabeled data which the model learns from in an unsupervised manner. This can be significantly reduce the annotation time and speed up deployment of a model.

Wiener Filter

Wiener filters are used to estimate an unknown underlying signal from am observed noisy response. For an example use-case of this filter in image processing, see this Matlab implementation.

Zernike Modes

A set of orthogonal polynomials used to describe and correct wavefront aberrations in optical systems.

References

[1] Cyril Achard et al. "CellSeg3D, Self-supervised 3D cell segmentation for fluorescence microscopy". In: *eLife* 13 (2025), RP99848. DOI: 10.7554/eLife.99848.4.

[2] Abhinav K. Adduri et al. "Predicting cellular responses to perturbation across diverse contexts with State". In: *bioRxiv* (2025). Preprint, version 2. DOI: 10.1101/2025.06.26.661135.

[3] Julius Adebayo et al. "Sanity Checks for Saliency Maps". In: *Advances in Neural Information Processing Systems*. Vol. 31. 2018. DOI: 10.48550/arXiv.1810.03292. arXiv: 1810.03292.

[4] Jannis Ahlers et al. *napari: a multi-dimensional image viewer for Python*. 2023. DOI: 10.5281/zenodo.8115575. URL: https://zenodo.org/record/8115575 (visited on 10/05/2023).

[5] Hussain Alibrahim and Simone A. Ludwig. "Hyperparameter Optimization: Comparing Genetic Algorithm against Grid Search and Bayesian Optimization". In: *2021 IEEE Congress on Evolutionary Computation (CEC)*. 2021, pp. 1551–1559. DOI: 10.1109/CEC45853.2021.9504761. URL: https://ieeexplore.ieee.org/document/9504761.

[6] Jonatan Alvelid et al. "Event-triggered STED imaging". In: *Nature Methods* 19.10 (2022), pp. 1268–1275. ISSN: 1548-7091. DOI: 10.1038/s41592-022-01588-y.

[7] Fernando Amat and Philipp J. Keller. "Towards comprehensive cell lineage reconstructions in complex organisms using light-sheet microscopy". In: *Development, Growth & Differentiation* 55.4 (2013), pp. 563–578. DOI: https://doi.org/10.1111/dgd.12063. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1111/dgd.12063. URL: https://onlinelibrary.wiley.com/doi/abs/10.1111/dgd.12063.

[8] Anwai Archit et al. "Segment Anything for Microscopy". en. In: *Nature Methods* 22.3 (2025). Publisher: Nature Publishing Group, pp. 579–591. ISSN: 1548-7105. DOI: 10.1038/s41592-024-02580-4. URL: https://www.nature.com/articles/s41592-024-02580-4 (visited on 04/17/2025).

[9] Ignacio Arganda-Carreras et al. "Trainable Weka Segmentation: a machine learning tool for microscopy pixel classification". In: *Bioinformatics* 33.15 (2017), pp. 2424–2426. ISSN: 1367-4803. DOI: 10.1093/bioinformatics/btx180. URL: https://doi.org/10.1093/bioinformatics/btx180 (visited on 08/22/2023).

[10] Matthias Arzt et al. "LABKIT: Labeling and segmentation toolkit for big image data". In: *Front. Comput. Sci.* 4 (2022). ISSN: 2624-9898. URL: https://www.frontiersin.org/articles/10.3389/fcomp.2022.777728 (visited on 08/30/2022).

[11] Ashesh Ashesh et al. “Micro plit: : semantic unmixing of fluorescent microscopy data”. In: *Nature Methods* 23.5 (2026), pp. 1047–1057. ISSN: 1548-7105. DOI: 10.1038/s41592-026-03082-1. URL: https://www.nature.com/articles/s41592-026-03082-1.

[12] Anthony Azevedo et al. “Connectomic reconstruction of a female Drosophila ventral nerve cord”. In: *Nature* 631.8020 (2024), pp. 360–368. ISSN: 0028-0836. DOI: 10.1038/s41586-024-07389-x.

[13] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. “Layer Normalization”. In: (2016). DOI: /10.48550/arXiv.1607.06450. arXiv: 1607.06450 [stat.ML]. URL: https://arxiv.org/abs/1607.06450.

[14] Harikrushnan Balasubramanian et al. “Imagining the future of optical microscopy: everything, everywhere, all at once”. en. In: *Communications Biology* 6.1 (2023). Publisher: Nature Publishing Group, p. 1096. ISSN: 2399-3642. DOI: 10.1038/s42003-023-05468-9. URL: https://www.nature.com/articles/s42003-023-05468-9 (visited on 12/03/2025).

[15] Randall Balestriero and Yann LeCun. *LeJEPA: Provable and Scalable Self-Supervised Learning Without the Heuristics*. 2025. DOI: 10.48550/arXiv.2511.08544. arXiv: 2511.08544 [cs.LG]. URL: https://arxiv.org/abs/2511.08544.

[16] Peter Bankhead et al. “QuPath: Open source software for digital pathology image analysis”. en. In: *Sci. Rep.* 7.1 (2017). Number: 1 Publisher: Nature Publishing Group, p. 16878. ISSN: 2045-2322. DOI: 10.1038/s41598-017-17204-5. URL: https://www.nature.com/articles/s41598-017-17204-5 (visited on 10/11/2023).

[17] Adrien Bardes et al. “Revisiting Feature Prediction for Learning Visual Representations from Video”. In: *Transactions on Machine Learning Research* (2024). ISSN: 2835-8856. URL: https://openreview.net/forum?id=QaCCuDfBk2.

[18] Joshua Batson and Loic Royer. “Noise2Self: Blind Denoising by Self-Supervision”. In: *Proceedings of the 36th International Conference on Machine Learning*. Ed. by Kamalika Chaudhuri and Ruslan Salakhutdinov. Vol. 97. Proceedings of Machine Learning Research. PMLR, 2019, pp. 524–533. URL: https://proceedings.mlr.press/v97/batson19a.html.

[19] Pierre-Louis Bazin et al. “Volumetric neuroimage analysis extensions for the MIPAV software package”. In: *Journal of Neuroscience Methods* 165.1 (2007), pp. 111–121. ISSN: 0165-0270. DOI: https://doi.org/10.1016/j.jneumeth.2007.05.024. URL: https://www.sciencedirect.com/science/article/pii/S0165027007002270.

[20] Amir Beck and Marc Teboulle. “A Fast Iterative Shrinkage-Thresholding Algorithm for Linear Inverse Problems”. In: *SIAM Journal on Imaging Sciences* 2.1 (2009), pp. 183–202. DOI: 10.1137/080716542. eprint: https://doi.org/10.1137/080716542. URL: https://doi.org/10.1137/080716542.

[21] Leire Bejarano et al. “Interrogation of endothelial and mural cells in brain metastasis reveals key immune-regulatory mechanisms”. In: *Cancer Cell* 42.3 (2024), 378–395.e10. ISSN: 1535-6108. DOI: 10.1016/j.ccell.2023.12.018. URL: https://www.sciencedirect.com/science/article/pii/S1535610823004464 (visited on 04/22/2025).

[22] Chinmay Belthangady and Loic A. Royer. "Applications, promises, and pitfalls of deep learning for fluorescence image reconstruction". In: *Nature Methods* 16.12 (2019), pp. 1215–1225. ISSN: 1548-7091. DOI: 10.1038/s41592-019-0458-z.

[23] Loubna Ben Allal et al. *SmolLM2*. Hugging Face. 2024. URL: https://huggingface.co/HuggingFaceTB/SmolLM2-135M.

[24] Ihab Bendidi et al. "A Cross Modal Knowledge Distillation & Data Augmentation Recipe for Improving Transcriptomics Representations through Morphological Features". In: *Proceedings of the 42nd International Conference on Machine Learning*. Ed. by Aarti Singh et al. Vol. 267. Proceedings of Machine Learning Research. PMLR, 2025, pp. 3712–3730. URL: https://proceedings.mlr.press/v267/bendidi25a.html.

[25] Stuart Berg et al. "Ilastik: Interactive machine learning for (bio)image analysis". en. In: *Nat. Methods* 16.12 (2019). Number: 12 Publisher: Nature Publishing Group, pp. 1226–1232. ISSN: 1548-7105. DOI: 10.1038/s41592-019-0582-9. URL: https://www.nature.com/articles/s41592-019-0582-9 (visited on 08/22/2023).

[26] Casper Berger et al. "Cryo-electron tomography on focused ion beam lamellae transforms structural cell biology". In: *Nature Methods* 20.4 (2023), pp. 499–511. ISSN: 1548-7091. DOI: 10.1038/s41592-023-01783-5.

[27] Christof A. Bertram et al. "Data Set Creation for Supervised Deep Learning-Based Analysis of Microscopic Images: Review of Important Considerations and Recommendations". In: *Veterinary Pathology* (2026), p. 03009858261457959. DOI: 10.1177/03009858261457959. URL: https://doi.org/10.1177/03009858261457959.

[28] Anthony Bilodeau et al. "Development of AI-assisted microscopy frameworks through realistic simulation with pySTED". In: *Nature Machine Intelligence* 6 (2024), pp. 1197–1215. DOI: 10.1038/s42256-024-00903-w. URL: https://www.nature.com/articles/s42256-024-00903-w.

[29] Christopher Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006. URL: https://www.microsoft.com/en-us/research/publication/pattern-recognition-machine-learning/.

[30] Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao. *YOLOv4: Optimal Speed and Accuracy of Object Detection*. 2020. arXiv: 2004.10934 [cs.CV]. URL: https://arxiv.org/abs/2004.10934.

[31] Michael V. Boland and Robert F. Murphy. "A neural network classifier capable of recognizing the patterns of all major subcellular structures in fluorescence microscope images of HeLa cells". In: *Bioinformatics* 17.12 (2001), pp. 1213–1223. ISSN: 1367-4811. DOI: 10.1093/bioinformatics/17.12.1213.

[32] Martin J. Booth. "Adaptive Optics in Microscopy". In: *Philosophical Transactions: Mathematical, Physical and Engineering Sciences* 365.1861 (2007), pp. 2829–2843. ISSN: 1364503X. URL: http://www.jstor.org/stable/25190627 (visited on 08/19/2025).

[33] Catherine Bouchard et al. “Resolution enhancement with a task-assisted GAN to guide optical nanoscopy image analysis and acquisition”. In: *Nature Machine Intelligence* 5.8 (2023), pp. 830–844. ISSN: 2522-5839. DOI: 10.1038/s42256-023-00689-3. URL: https://doi.org/10.1038/s42256-023-00689-3.

[34] Thomas Boyer et al. “Static2Dynamic: Reconstructing videos of unobservable cellular, developmental, and disease processes”. In: *bioRxiv* (2026). Preprint, posted May 20, 2026. DOI: 10.64898/2026.05.18.725860.

[35] Jordão Bragantini et al. “Ultrack: pushing the limits of cell tracking across biological scales”. In: *Nature Methods* 22.11 (2025), pp. 2423–2436. DOI: 10.1038/s41592-025-02778-0.

[36] Mark-Anthony Bray et al. “Cell Painting, a high-content image-based assay for morphological profiling using multiplexed fluorescent dyes”. In: *Nature Protocols* 11.9 (2016), pp. 1757–1774. DOI: 10.1038/nprot.2016.105.

[37] Leo Breiman. “Random Forests”. In: *Machine Learning* 45.1 (2001), pp. 5–32. DOI: 10.1023/A:1010933404324.

[38] Sergey Brin and Lawrence Page. “The anatomy of a large-scale hypertextual web search engine”. In: *Computer Networks and ISDN Systems* 30.1-7 (1998), pp. 107–117. DOI: 10.1016/S0169-7552(98)00110-X.

[39] Coleman Broaddus et al. “Removing Structured Noise with Self-Supervised Blind-Spot Networks”. In: *2020 IEEE 17th International Symposium on Biomedical Imaging (ISBI)*. 2020, pp. 159–163. DOI: 10.1109/ISBI45749.2020.9098336.

[40] Julia Buhmann et al. “Automatic detection of synaptic partners in a whole-brain Drosophila electron microscopy data set”. In: *Nature Methods* 18.7 (2021), pp. 771–774. ISSN: 1548-7091. DOI: 10.1038/s41592-021-01183-7.

[41] Charlotte Bunne et al. “How to build the virtual cell with artificial intelligence: Priorities and opportunities”. In: *Cell* 187.25 (2024), pp. 7045–7063. DOI: 10.1016/j.cell.2024.11.015.

[42] Garth Lawrence Burn et al. “Myeloperoxidase transforms chromatin into neutrophil extracellular traps”. In: *Nature* 647.8090 (2025), pp. 747–756. ISSN: 0028-0836. DOI: 10.1038/s41586-025-09523-9.

[43] Juan C. Caicedo et al. “Data-analysis strategies for image-based cell profiling”. In: *Nature Methods* 14.9 (2017), pp. 849–863. DOI: 10.1038/nmeth.4397.

[44] Juan C. Caicedo et al. “Weakly Supervised Learning of Single-Cell Feature Embeddings”. In: *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. ISSN: 2575-7075. 2018, pp. 9309–9318. DOI: 10.1109/CVPR.2018.00970. URL: https://ieeexplore.ieee.org/document/8579068 (visited on 04/17/2025).

[45] Nicolas Carion et al. “End-to-End Object Detection with Transformers”. In: *Lecture Notes in Computer Science*. Cham: Springer International Publishing, 2020, pp. 213–229. ISBN: 9783030584511. DOI: 10.1007/978-3-030-58452-8_13.

[46] Nicolas Carion et al. “SAM 3: Segment Anything with Concepts”. In: *International Conference on Learning Representations (ICLR)*. 2026. DOI: 10.48550/arXiv.2511.16719. arXiv: 2511.16719.

[47] Mathilde Caron et al. “Emerging Properties in Self-Supervised Vision Transformers”. In: *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*. 2021, pp. 9630–9640. DOI: 10.1109/ICCV48922.2021.00951. URL: https://ieeexplore.ieee.org/document/9709990.

[48] Anne E Carpenter et al. “CellProfiler: image analysis software for identifying and quantifying cell phenotypes”. In: *Genome Biology* 7.10 (2006). ISSN: 1474-760X. DOI: 10.1186/gb-2006-7-10-r100.

[49] Brendan Celii et al. “NEURD offers automated proofreading and feature extraction for connectomics”. In: *Nature* 640.8058 (2025), pp. 487–496. ISSN: 0028-0836. DOI: 10.1038/s41586-025-08660-5.

[50] Binghao Chai et al. “Opportunities and challenges for deep learning in cell dynamics research”. In: *Trends in Cell Biology* 34.11 (2024), pp. 955–967. ISSN: 0962-8924. DOI: 10.1016/j.tcb.2023.10.010.

[51] Lucas von Chamier et al. “Democratising deep learning for microscopy with ZeroCostDL4Mic”. en. In: *Nature Communications* 12.1 (2021), p. 2276. ISSN: 2041-1723. DOI: 10.1038/s41467-021-22518-0. URL: https://www.nature.com/articles/s41467-021-22518-0 (visited on 06/24/2022).

[52] Srinivas Niranj Chandrasekaran et al. “Three million images and morphological profiles of cells treated with matched chemical and genetic perturbations”. In: *Nature Methods* 21 (2024), pp. 1114–1121. DOI: 10.1038/s41592-024-02241-6.

[53] Fabrice de Chaumont et al. “Icy: an open bioimage informatics platform for extended reproducible research”. en. In: *Nat. Methods* 9.7 (2012), pp. 690–696. URL: https://doi.org/10.1038/nmeth.2075.

[54] Mikaël Chelli et al. “Hallucination Rates and Reference Accuracy of ChatGPT and Bard for Systematic Reviews: Comparative Analysis”. In: *Journal of Medical Internet Research* 26.1 (2024), e53164. DOI: 10.2196/53164. URL: https://www.jmir.org/2024/1/e53164.

[55] Fei Chen, Paul W. Tillberg, and Edward S. Boyden. “Expansion microscopy”. In: *Science* 347.6221 (2015), pp. 543–548. DOI: 10.1126/science.1260088. eprint: https://www.science.org/doi/pdf/10.1126/science.1260088. URL: https://www.science.org/doi/abs/10.1126/science.1260088.

[56] Huanhuan Chen et al. “Advancements and Practical Considerations for Biophysical Research: Navigating the Challenges and Future of Super-resolution Microscopy”. In: *Chemical & Biomedical Imaging* 2.5 (2024), pp. 331–344. DOI: 10.1021/cbmi.4c00019. URL: https://doi.org/10.1021/cbmi.4c00019.

[57] Jianxu Chen et al. *The Allen Cell and Structure Segmenter: a new open source toolkit for segmenting 3D intracellular structures in fluorescence microscopy images*. en. Pages: 491035 Section: New Results. 2020. DOI: 10.1101/491035. URL: https://www.biorxiv.org/content/10.1101/491035v2 (visited on 04/22/2025).

[58] Jiji Chen et al. "Three-dimensional residual channel attention networks denoise and sharpen fluorescence microscopy image volumes". In: *Nature Methods* 18.6 (2021), pp. 678–687. ISSN: 1548-7105. DOI: 10.1038/s41592-021-01155-x. URL: https://doi.org/10.1038/s41592-021-01155-x.

[59] Muyuan Chen et al. "A complete data processing workflow for cryo-ET and subtomogram averaging". In: *Nature Methods* 16.11 (2019), pp. 1161–1168. ISSN: 1548-7091. DOI: 10.1038/s41592-019-0591-8.

[60] Ting Chen et al. "A simple framework for contrastive learning of visual representations". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. ICML'20. JMLR.org, 2020, pp. 1597–1607. URL: https://dl.acm.org/doi/10.5555/3524938.3525087.

[61] Ting Chen et al. "Big self-supervised models are strong semi-supervised learners". In: *Proceedings of the 34th International Conference on Neural Information Processing Systems*. NIPS '20. Red Hook, NY, USA: Curran Associates Inc., 2020, pp. 22243–22255. ISBN: 978-1-7138-2954-6. URL: https://dl.acm.org/doi/10.5555/3495724.3497589.

[62] Xiang Chen, Meel Velliste, and Robert F. Murphy. "Automated interpretation of subcellular patterns in fluorescence microscope images for location proteomics". In: *Cytometry Part A* 69A.7 (2006), pp. 631–640. ISSN: 1552-4922. DOI: 10.1002/cyto.a.20280.

[63] Eric M. Christiansen et al. "In silico labeling: Predicting fluorescent labels in unlabeled images". In: *Cell* 173.3 (2018), 792–803.e19. DOI: 10.1016/j.cell.2018.03.040.

[64] Beth A Cimini. *When to say 'good enough'*. 2019. URL: https://carpenter-singh-lab.broadinstitute.org/blog/when-to-say-good-enough.

[65] Dan C. Cireşan et al. "Mitosis Detection in Breast Cancer Histology Images with Deep Neural Networks". In: *Lecture Notes in Computer Science*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 411–418. ISBN: 9783642407628. DOI: 10.1007/978-3-642-40763-5_51.

[66] Christian Conrad et al. "Micropilot: automation of fluorescence microscopy–based imaging for systems biology". In: *Nature Methods* 8.3 (2011), pp. 246–249. ISSN: 1548-7091. DOI: 10.1038/nmeth.1558.

[67] Ryan Conrad and Kedar Narayan. "Instance segmentation of mitochondria in electron microscopy images with a generalist deep learning model trained on a diverse dataset". In: *Cell Systems* 14.1 (2023), 58–71.e5. ISSN: 2405-4712. DOI: 10.1016/j.cels.2022.12.006. URL: https://www.sciencedirect.com/science/article/pii/S240547122200494X (visited on 04/22/2025).

[68] Corinna Cortes and Vladimir Vapnik. "Support-vector networks". In: *Machine Learning* 20.3 (1995), pp. 273–297. ISSN: 0885-6125. DOI: 10.1007/bf00994018.

[69] George Cybenko. “Approximation by superpositions of a sigmoidal function”. In: *Mathematics of control, signals and systems* 2.4 (1989), pp. 303–314. DOI: 10.1007/BF02551274.

[70] Kostadin Dabov et al. “Image Denoising by Sparse 3-D Transform-Domain Collaborative Filtering”. In: *IEEE Transactions on Image Processing* 16.8 (2007), pp. 2080–2095. DOI: 10.1109/TIP.2007.901238. URL: https://ieeexplore.ieee.org/document/4271520.

[71] Stephan Daetwyler and Reto Paul Fiolka. “Light-sheets and smart microscopy, an exciting future is dawning”. In: *Communications Biology* 6.1 (2023). ISSN: 2399-3642. DOI: 10.1038/s42003-023-04857-4. URL: http://dx.doi.org/10.1038/s42003-023-04857-4.

[72] Stephan Daetwyler et al. “Imaging of cellular dynamics from a whole organism to subcellular scale with self-driving, multiscale microscopy”. In: *Nature Methods* 22.3 (2025), pp. 569–578. ISSN: 1548-7091. DOI: 10.1038/s41592-025-02598-2.

[73] Aram Danielyan et al. “Denoising of two-photon fluorescence images with Block-Matching 3D filtering”. In: *Methods* 68.2 (2014). Tools and Methods for Cellular Localization and Measurements, pp. 308–316. ISSN: 1046-2023. DOI: https://doi.org/10.1016/j.ymeth.2014.03.010. URL: https://www.sciencedirect.com/science/article/pii/S1046202314001030.

[74] David Dao et al. “CellProfiler Analyst: interactive data exploration, analysis and classification of large biological image sets”. In: *Bioinformatics* 32.20 (2016), pp. 3210–3212. ISSN: 1367-4811. DOI: 10.1093/bioinformatics/btw390.

[75] Jia Deng et al. “ImageNet: A large-scale hierarchical image database”. In: *2009 IEEE Conference on Computer Vision and Pattern Recognition*. 2009, pp. 248–255. DOI: 10.1109/cvpr.2009.5206848.

[76] Jacob Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Minneapolis, Minnesota: Association for Computational Linguistics, 2019, pp. 4171–4186. DOI: 10.18653/v1/N19-1423. URL: https://aclanthology.org/N19-1423/.

[77] Mingze Dong et al. “Stack: In-context learning of single-cell biology”. In: *bioRxiv* (2026). Preprint, version 2. DOI: 10.64898/2026.01.09.698608.

[78] Alexey Dosovitskiy et al. “An image is worth 16x16 words: Transformers for image recognition at scale”. In: *International Conference on Learning Representations (ICLR)*. 2021. DOI: 10.48550/arXiv.2010.11929.

[79] Nils Eckstein et al. “Neurotransmitter classification from electron microscopy images at synaptic sites in Drosophila melanogaster”. In: *Cell* 187.10 (2024), 2574–2594.e23. ISSN: 0092-8674. DOI: 10.1016/j.cell.2024.03.016.

[80] Fabian Eisenstein, Yoshiyuki Fukuda, and Radostin Danev. “Smart parallel automated cryo-electron tomography”. In: *Nature Methods* 21.9 (2024), pp. 1612–1615. ISSN: 1548-7091. DOI: 10.1038/s41592-024-02373-9.

[81] Nitsan Elmalam and Assaf Zaritsky. "Cell context-dependent in silico organelle localization in label-free microscopy images". In: *Nature Methods* 23.2 (2026), pp. 405–416. DOI: 10.1038/s41592-025-02960-4.

[82] Dmitry Ershov et al. "TrackMate 7: integrating state-of-the-art segmentation algorithms into tracking pipelines". In: *Nature Methods* 19.7 (2022), pp. 829–832. DOI: 10.1038/s41592-022-01507-1.

[83] Elnaz Fazeli et al. "Automated cell tracking using StarDist and TrackMate". eng. In: *F1000Research* 9 (2020), p. 1279. ISSN: 2046-1402. DOI: 10.12688/f1000research.27019.1. URL: https://f1000research.com/articles/9-1279/v1.

[84] Francesco Fersini et al. "Wavefront estimation through structured detection in laser scanning microscopy". In: *Biomed. Opt. Express* 16.5 (2025), pp. 2135–2155. DOI: 10.1364/BOE.559899. URL: https://opg.optica.org/boe/abstract.cfm?URI=boe-16-5-2135.

[85] Daniel Fisch et al. "Molecular definition of the endogenous Toll-like receptor signalling pathways". en. In: *Nature* 631.8021 (2024). Publisher: Nature Publishing Group, pp. 635–644. ISSN: 1476-4687. DOI: 10.1038/s41586-024-07614-7. URL: https://www.nature.com/articles/s41586-024-07614-7 (visited on 04/22/2025).

[86] Gautier Follain et al. *Fast label-free live imaging reveals key roles of flow dynamics and CD44-HA interaction in cancer cell arrest on endothelial monolayers*. en. Pages: 2024.09.30.615654 Section: New Results. 2024. DOI: 10.1101/2024.09.30.615654. URL: https://www.biorxiv.org/content/10.1101/2024.09.30.615654v1 (visited on 03/21/2025).

[87] Daniel Franco-Barranco et al. "BiaPy: accessible deep learning on bioimages". en. In: *Nat. Methods* 22.6 (2025), pp. 1124–1126. DOI: 10.1038/s41592-025-02699-y.

[88] Jerome Friedman, Trevor Hastie, and Robert Tibshirani. "Additive logistic regression: a statistical view of boosting (With discussion and a rejoinder by the authors)". In: *The Annals of Statistics* 28.2 (2000). ISSN: 0090-5364. DOI: 10.1214/aos/1016218223.

[89] Jan Funke et al. "Large Scale Image Segmentation with Structured Loss Based Deep Learning for Connectome Reconstruction". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 41.7 (2019), pp. 1669–1680. DOI: 10.1109/TPAMI.2018.2835450.

[90] Benjamin Gallusser and Martin Weigert. "Trackastra: Transformer-based cell tracking for live-cell microscopy". In: *Computer Vision – ECCV 2024*. Lecture Notes in Computer Science. Cham: Springer Nature Switzerland, 2024. DOI: 10.1007/978-3-031-73116-7_27.

[91] Robert Geirhos et al. "Shortcut learning in deep neural networks". In: *Nature Machine Intelligence* 2.11 (2020), pp. 665–673. DOI: 10.1038/s42256-020-00257-z.

[92] Ross Girshick. "Fast R-CNN". In: *2015 IEEE International Conference on Computer Vision (ICCV)*. 2015, pp. 1440–1448. DOI: 10.1109/iccv.2015.169.

[93] Ross Girshick et al. "Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation". In: *2014 IEEE Conference on Computer Vision and Pattern Recognition*. 2014, pp. 580–587. DOI: 10.1109/cvpr.2014.81.

[94] Johannes Girstmair et al. *Mastodon: The Command Center for Large-Scale Lineage-Tracing Microscopy Datasets*. 2025. DOI: 10.64898/2025.12.10.693416. URL: https://doi.org/10.64898/2025.12.10.693416.

[95] Thibaut Goldsborough et al. "InstanSeg: an embedding-based instance segmentation algorithm optimized for accurate, efficient and portable cell segmentation". In: *arXiv [cs.CV]* (2024). URL: https://doi.org/10.48550/arXiv.2408.15954.

[96] Estibaliz Gómez-de-Mariscal et al. "DeepImageJ: A user-friendly environment to run deep learning models in ImageJ". en. In: *Nat. Methods* 18.10 (2021), pp. 1192–1195. URL: https://doi.org/10.1038/s41592-021-01262-9.

[97] R.C. Gonzalez and R.E. Woods. *Digital Image Processing*. Prentice Hall, 2008. ISBN: 9780131687288. URL: https://books.google.com/books?id=8uGOnjRGEzoC.

[98] Ian J. Goodfellow et al. "Generative Adversarial Networks". In: *Advances in Neural Information Processing Systems*. Vol. 27. 2014. DOI: 10.48550/arXiv.1406.2661. arXiv: 1406.2661.

[99] Paul C. Goodwin. "Chapter 10 - Quantitative deconvolution microscopy". In: *Quantitative Imaging in Cell Biology*. Ed. by Jennifer C. Waters and Torsten Wittman. Vol. 123. Methods in Cell Biology. Academic Press, 2014, pp. 177–192. DOI: https://doi.org/10.1016/B978-0-12-420138-5.00010-0. URL: https://www.sciencedirect.com/science/article/pii/B9780124201385000100.

[100] Google. *Neuroglancer: WebGL-Based Viewer for Volumetric Data*. URL: https://github.com/google/neuroglancer (visited on 09/03/2026).

[101] Matthias Griebel et al. "Deep learning-enabled segmentation of ambiguous bioimages with deepflash2". In: *Nature Communications* 14.1 (2023). ISSN: 2041-1723. DOI: 10.1038/s41467-023-36960-9.

[102] Nicola Gritti et al. "Image restoration of degraded time-lapse microscopy data mediated by near-infrared imaging". In: *Nature Methods* 21.2 (2024), pp. 311–321. DOI: 10.1038/s41592-023-02127-z.

[103] Jiacheng Gu et al. "Mapping multimodal phenotypes to perturbations in cells and tissue with CRISPRmap". In: *Nature Biotechnology* 43.7 (2024), pp. 1101–1115. DOI: 10.1038/s41587-024-02386-x.

[104] Chuan Guo et al. "On Calibration of Modern Neural Networks". In: *Proceedings of the 34th International Conference on Machine Learning*. Vol. 70. Proceedings of Machine Learning Research. 2017, pp. 1321–1330. DOI: 10.48550/arXiv.1706.04599. arXiv: 1706.04599.

[105] Min Guo et al. "Deep learning-based aberration compensation improves contrast and resolution in fluorescence microscopy". In: *Nature Communications* 16.1 (2025), p. 313. ISSN: 2041-1723. DOI: 10.1038/s41467-024-55267-x. URL: https://doi.org/10.1038/s41467-024-55267-x.

[106] Min Guo et al. “Rapid image deconvolution and multiview fusion for optical microscopy”. In: *Nature Biotechnology* 38.11 (2020), pp. 1337–1346. ISSN: 1546-1696. DOI: 10.1038/s41587-020-0560-x. URL: https://doi.org/10.1038/s41587-020-0560-x.

[107] Ankit Gupta et al. “SubCell: Proteome-aware vision foundation models for microscopy capture single-cell biology”. In: *bioRxiv* (2026). Preprint, version 3, posted June 8, 2026. DOI: 10.1101/2024.12.06.627299.

[108] Robert Haase et al. “A call for FAIR and open-access training materials to advance BioImage Analysis”. In: (2024). URL: https://osf.io/preprints/osf/2zgmc_v1.

[109] Robert Haase et al. “Benchmarking Large Language Models for Bio-Image Analysis Code Generation”. In: *bioRxiv* (2024). DOI: 10.1101/2024.04.19.590278. URL: https://www.biorxiv.org/content/10.1101/2024.04.19.590278v3.

[110] Daniel Haehn et al. “Guided Proofreading of Automatic Segmentations for Connectomics”. In: *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2018, pp. 9319–9328. DOI: 10.1109/CVPR.2018.00971.

[111] Guy M Hagen et al. “Fluorescence microscopy datasets for training deep neural networks”. In: *GigaScience* 10.5 (2021), giab032. ISSN: 2047-217X. DOI: 10.1093/gigascience/giab032. eprint: https://academic.oup.com/gigascience/article-pdf/10/5/giab032/60688420/giab032.pdf. URL: https://doi.org/10.1093/gigascience/giab032.

[112] Karen M. Hampson et al. “Adaptive optics for high-resolution imaging”. In: *Nature Reviews Methods Primers* 1.1 (2021), p. 68. ISSN: 2662-8449. DOI: 10.1038/s43586-021-00066-7. URL: https://doi.org/10.1038/s43586-021-00066-7.

[113] Kaiming He et al. “Deep Residual Learning for Image Recognition”. In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 770–778. DOI: 10.1109/cvpr.2016.90.

[114] Kaiming He et al. “Mask R-CNN”. In: *2017 IEEE International Conference on Computer Vision (ICCV)*. 2017, pp. 2980–2988. DOI: 10.1109/iccv.2017.322.

[115] Kaiming He et al. “Masked autoencoders are scalable vision learners”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2022, pp. 16000–16009. DOI: 10.1109/CVPR52688.2022.01553.

[116] Siyu He et al. “Squidiff: predicting cellular development and responses to perturbations using a diffusion model”. In: *Nature Methods* 23.1 (2026), pp. 65–77. DOI: 10.1038/s41592-025-02877-y.

[117] Larissa Heinrich et al. “Synaptic Cleft Segmentation in Non-isotropic Volume Electron Microscopy of the Complete Drosophila Brain”. In: *Medical Image Computing and Computer Assisted Intervention - MICCAI 2018: 21st International Conference, Granada, Spain, September 16-20, 2018, Proceedings, Part II*. Granada, Spain: Springer-Verlag, 2018, pp. 317–325. ISBN: 978-3-030-00933-5. DOI: 10.1007/978-3-030-00934-2_36. URL: https://doi.org/10.1007/978-3-030-00934-2_36.

[118] Larissa Heinrich et al. “Whole-cell organelle segmentation in volume electron microscopy”. en. In: *Nature* 599.7883 (2021). Publisher: Nature Publishing Group, pp. 141–146. ISSN: 1476-4687. DOI: 10.1038/s41586-021-03977-3. URL: https://www.nature.com/articles/s41586-021-03977-3 (visited on 04/17/2025).

[119] Stefan W. Hell. “Far-Field Optical Nanoscopy”. In: *Science* 316.5828 (2007), pp. 1153–1158. DOI: 10.1126/science.1137395. eprint: https://www.science.org/doi/pdf/10.1126/science.1137395. URL: https://www.science.org/doi/abs/10.1126/science.1137395.

[120] Moritz Helmstaedter. “Synaptic-resolution connectomics: towards large brains and connectomic screening”. In: *Nature Reviews Neuroscience* 27.2 (2026), pp. 101–120. ISSN: 1471-003X. DOI: 10.1038/s41583-025-00998-z.

[121] Iván Hidalgo-Cenalmor et al. “DL4MicEverywhere: deep learning for microscopy made flexible, shareable and reproducible”. en. In: *Nat. Methods* 21.6 (2024), pp. 925–927. ISSN: 1548-7091, 1548-7105. DOI: 10.1038/s41592-024-02295-6. URL: https://www.nature.com/articles/s41592-024-02295-6 (visited on 05/20/2024).

[122] Lucien Hinderling et al. “Smart microscopy: current implementations and a roadmap for interoperability”. In: *Methods in Microscopy* 3.1 (2026), pp. 71–91. ISSN: 2942-3899. DOI: 10.1515/mim-2025-0029.

[123] Geoffrey E Hinton and Ruslan R Salakhutdinov. “Reducing the dimensionality of data with neural networks”. In: *Science* 313.5786 (2006), pp. 504–507. DOI: 10.1126/science.1127647.

[124] Geoffrey E. Hinton et al. “Improving neural networks by preventing co-adaptation of feature detectors”. In: (2012). DOI: 10.48550/arXiv.1207.0580. arXiv: 1207.0580 [cs.NE]. URL: https://arxiv.org/abs/1207.0580.

[125] Jonathan Ho, Ajay Jain, and Pieter Abbeel. “Denoising Diffusion Probabilistic Models”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020. DOI: 10.48550/arXiv.2006.11239. arXiv: 2006.11239.

[126] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780. ISSN: 0899-7667. DOI: 10.1162/neco.1997.9.8.1735.

[127] Max A. Hockenberry and Wesley R. Legant. “Cells in the mechanical spotlight”. In: *Biophysical Journal* 121.19 (2022), pp. 3571–3572. ISSN: 0006-3495. DOI: 10.1016/j.bpj.2022.09.003.

[128] Xuekai Hou et al. “HD2Net: a deep learning framework for simultaneous denoising and deaberration in fluorescence microscopy”. In: *Opt. Express* 33.13 (2025), pp. 27317–27333. DOI: 10.1364/OE.554927. URL: https://opg.optica.org/oe/abstract.cfm?URI=oe-33-13-27317.

[129] Lejia Hu et al. “Image enhancement for fluorescence microscopy based on deep learning with prior knowledge of aberration”. In: *Opt. Lett.* 46.9 (2021), pp. 2055–2058. DOI: 10.1364/OL.418997. URL: https://opg.optica.org/ol/abstract.cfm?URI=ol-46-9-2055.

[130] Gang Huang et al. “Embodied Intelligence Unlocks Autonomous Microscopy”. In: *bioRxiv* (2025). DOI: 10.1101/2025.08.13.670210. URL: https://www.biorxiv.org/content/10.1101/2025.08.13.670210v1.

[131] Gao Huang et al. “Deep networks with stochastic depth”. In: *European conference on computer vision*. Springer. 2016, pp. 646–661. DOI: 10.1007/978-3-319-46493-0_39.

[132] Kai Huang and Robert F. Murphy. “From quantitative microscopy to automated image understanding”. In: *Journal of Biomedical Optics* 9.5 (2004), p. 893. ISSN: 1083-3668. DOI: 10.1117/1.1779233.

[133] Luzhe Huang et al. “A robust and scalable framework for hallucination detection in virtual tissue staining and digital pathology”. In: *Nature Biomedical Engineering* 9.12 (2025), pp. 2196–2214. ISSN: 2157-846X. DOI: 10.1038/s41551-025-01421-9.

[134] Teun A. P. M. Huijben et al. “inTRACKtive: a web-based tool for interactive cell tracking visualization”. In: *Nature Methods* 22.11 (2025), pp. 2229–2231. ISSN: 1548-7091. DOI: 10.1038/s41592-025-02777-1.

[135] Khalid A. Ibrahim et al. “Self-driving microscopy detects the onset of protein aggregation and enables intelligent Brillouin imaging”. In: *Nature Communications* 16.1 (2025). ISSN: 2041-1723. DOI: 10.1038/s41467-025-60912-0.

[136] Sacha Ichbiah et al. “Embryo mechanics cartography: inference of 3D force atlases from fluorescence microscopy”. In: *Nature Methods* 20 (2023), pp. 1989–1999. DOI: 10.1038/s41592-023-02084-7.

[137] Ilija Ilievski et al. “Efficient Hyperparameter Optimization for Deep Learning Algorithms Using Deterministic RBF Surrogates”. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 31.1 (2017). DOI: 10.1609/aaai.v31i1.10647. URL: https://ojs.aaai.org/index.php/AAAI/article/view/10647.

[138] Sergey Ioffe and Christian Szegedy. “Batch normalization: accelerating deep network training by reducing internal covariate shift”. In: *Proceedings of the 32nd International Conference on Machine Learning - Volume 37*. ICML’15. Lille, France: JMLR.org, 2015, pp. 448–456. DOI: 10.5555/3045118.3045167.

[139] Mostafa Jahanifar et al. “Domain Generalization in Computational Pathology: Survey and Guidelines”. In: *ACM Computing Surveys* 57.11 (Nov. 30, 2025), pp. 1–37. ISSN: 0360-0300, 1557-7341. DOI: 10.1145/3724391. URL: https://dl.acm.org/doi/10.1145/3724391 (visited on 09/04/2026).

[140] Nasim Jamali et al. “2020 BioImage Analysis Survey: Community experiences and needs for the future”. In: *Biological Imaging* 1 (2021), e4. DOI: 10.1017/S2633903X21000039. URL: https://www.cambridge.org/core/journals/biological-imaging/article/2020-bioimage-analysis-survey-community-experiences-and-needs-for-the-future/9E824DC0C27568FE5B9D12FB59B1BB90.

[141] Michał Januszewski and Viren Jain. “Next-generation AI for connectomics”. In: *Nature Methods* 21.8 (2024), pp. 1398–1399. ISSN: 1548-7091. DOI: 10.1038/s41592-024-02336-0.

[142] Michał Januszewski et al. “High-precision automated reconstruction of neurons with flood-filling networks”. In: *Nature Methods* 15.8 (2018), pp. 605–610. ISSN: 1548-7091. DOI: 10.1038/s41592-018-0049-4.

[143] Khuloud Jaqaman et al. “Robust single-particle tracking in live-cell time-lapse sequences”. In: *Nature Methods* 5.8 (2008), pp. 695–702. DOI: 10.1038/nmeth.1237.

[144] Na Ji. “Adaptive optical fluorescence microscopy”. In: *Nature Methods* 14.4 (2017), pp. 374–380. ISSN: 1548-7105. DOI: 10.1038/nmeth.4218. URL: https://doi.org/10.1038/nmeth.4218.

[145] Ziwei Ji, Justin D. Li, and Matus Telgarsky. “Early-stopped neural networks are consistent”. English (US). In: *Advances in Neural Information Processing Systems 34 - 35th Conference on Neural Information Processing Systems, NeurIPS 2021*. Ed. by Marc’Aurelio Ranzato et al. Advances in Neural Information Processing Systems. Publisher Copyright: © 2021 Neural information processing systems foundation. All rights reserved.; 35th Conference on Neural Information Processing Systems, NeurIPS 2021 ; Conference date: 06-12-2021 Through 14-12-2021. Neural information processing systems foundation, 2021, pp. 1805–1817. URL: https://dl.acm.org/doi/10.5555/3540261.3540400.

[146] Carlos E. Jimenez et al. “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” In: *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*. 2024. URL: https://arxiv.org/abs/2310.06770.

[147] Glenn Jocher et al. *Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models*. 2026. arXiv: 2606.03748 [cs.CV]. URL: https://arxiv.org/abs/2606.03748.

[148] Thouis R Jones et al. “CellProfiler Analyst: data exploration and analysis software for complex image-based screens”. In: *BMC Bioinformatics* 9.1 (2008). ISSN: 1471-2105. DOI: 10.1186/1471-2105-9-482.

[149] Thouis R. Jones et al. “Scoring diverse cellular morphologies in image-based screens with iterative feedback and machine learning”. In: *Proceedings of the National Academy of Sciences* 106.6 (2009), pp. 1826–1831. ISSN: 0027-8424. DOI: 10.1073/pnas.0808843106.

[150] James Jonkman et al. “Tutorial: Guidance for Quantitative Confocal Microscopy”. In: *Nature Protocols* 15.5 (May 2020), pp. 1585–1611. ISSN: 1750-2799. DOI: 10.1038/s41596-020-0313-9. URL: https://www.nature.com/articles/s41596-020-0313-9 (visited on 09/05/2026).

[151] Timo Kaiser, Vladimír Ulman, and Bodo Rosenhahn. “Chota: A higher order accuracy metric for cell tracking”. In: *European Conference on Computer Vision*. Springer. 2024, pp. 122–138. DOI: 10.1007/978-3-031-91721-9_8.

[152] Iksung Kang et al. “Adaptive optical correction in in vivo two-photon fluorescence microscopy with neural fields”. In: *bioRxiv* (2024). DOI: 10.1101/2024.10.20.619284. eprint: https://www.biorxiv.org/content/early/2024/10/22/2024.10.20.619284.full.pdf. URL: https://www.biorxiv.org/content/early/2024/10/22/2024.10.20.619284.

[153] Iksung Kang et al. “Coordinate-based neural representations for computational adaptive optics in widefield microscopy”. In: *Nature Machine Intelligence* 6.6 (2024), pp. 714–725. ISSN: 2522-5839. DOI: 10.1038/s42256-024-00853-3. URL: http://dx.doi.org/10.1038/s42256-024-00853-3.

[154] Philipp J. Keller et al. “Reconstruction of zebrafish early embryonic development by scanned light sheet microscopy”. In: *Science* 322.5904 (2008), pp. 1065–1069. DOI: 10.1126/science.1162493.

[155] Kian Kenyon-Dean et al. “ViTally Consistent: Scaling Biological Representation Learning for Cell Microscopy”. In: *Proceedings of the 42nd International Conference on Machine Learning*. Ed. by Aarti Singh et al. Vol. 267. Proceedings of Machine Learning Research. PMLR, 2025, pp. 29735–29752. URL: https://proceedings.mlr.press/v267/kenyon-dean25a.html.

[156] Diederik P. Kingma and Jimmy Ba. “Adam: A Method for Stochastic Optimization”. In: (2017). DOI: 10.48550/arXiv.1412.6980. arXiv: 1412.6980 [cs.LG]. URL: https://arxiv.org/abs/1412.6980.

[157] Diederik P. Kingma and Max Welling. “Auto-Encoding Variational Bayes”. In: *International Conference on Learning Representations (ICLR)*. 2014. DOI: 10.48550/arXiv.1312.6114. arXiv: 1312.6114.

[158] Alexander Kirillov et al. “Segment Anything”. In: *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*. 2023, pp. 3992–4003. DOI: 10.1109/ICCV51070.2023.00371.

[159] Thomas Kluyver et al. “Jupyter Notebooks - a publishing format for reproducible computational workflows”. In: *Positioning and Power in Academic Publishing: Players, Agents and Agendas*. Ed. by Fernando Loizides and Birgit Scmidt. Netherlands: IOS Press, 2016, pp. 87–90. URL: https://eprints.soton.ac.uk/403913/.

[160] Bogdan Kochetov et al. “UNSEG: unsupervised segmentation of cells and their nuclei in complex tissue samples”. en. In: *Communications Biology* 7.1 (2024). Publisher: Nature Publishing Group, pp. 1–14. ISSN: 2399-3642. DOI: 10.1038/s42003-024-06714-4. URL: https://www.nature.com/articles/s42003-024-06714-4 (visited on 04/22/2025).

[161] Anna Kreshuk and Chong Zhang. “Machine Learning: Advanced Image Segmentation Using ilastik”. en. In: *Methods Mol. Biol.* 2040 (2019), pp. 449–463. URL: https://link.springer.com/protocol/10.1007/978-1-4939-9686-5_21.

[162] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. “ImageNet Classification with Deep Convolutional Neural Networks”. In: *Advances in Neural Information Processing Systems*. Vol. 25. Curran Associates, Inc., 2012. DOI: 10.1145/3065386. URL: https://proceedings.neurips.cc/paper_files/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html (visited on 10/05/2023).

[163] Alexander Krull, Tim-Oliver Buchholz, and Florian Jug. "Noise2Void - Learning Denoising From Single Noisy Images". In: *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019, pp. 2124–2132. DOI: 10.1109/CVPR.2019.00223. URL: https://ieeexplore.ieee.org/document/8954066.

[164] Uros Krzic et al. "Multiview light-sheet microscope for rapid in toto imaging". In: *Nature Methods* 9.7 (2012), pp. 730–733. DOI: 10.1038/nmeth.2064.

[165] Romain F. Laine et al. "Avoiding a replication crisis in deep-learning-based bioimage analysis". In: *Nature methods* 18.10 (2021), pp. 1136–1144. ISSN: 1548-7091. DOI: 10.1038/s41592-021-01284-3. URL: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC7611896/ (visited on 10/06/2023).

[166] P. Philippe Laissue et al. "Assessing Phototoxicity in Live Fluorescence Imaging". In: *Nature Methods* 14.7 (July 2017), pp. 657–661. ISSN: 1548-7105. DOI: 10.1038/nmeth.4344. URL: https://www.nature.com/articles/nmeth.4344 (visited on 09/05/2026).

[167] Manan Lalit, Pavel Tomancak, and Florian Jug. "EmbedSeg: Embedding-based Instance Segmentation for Biomedical Microscopy Data". In: *Medical Image Analysis* 81 (2022), p. 102523. DOI: 10.1016/j.media.2022.102523.

[168] Lorenz Lamm et al. "MemBrain v2: an end-to-end tool for the analysis of membranes in cryo-electron tomography". In: *bioRxiv* (2025). DOI: 10.1101/2024.01.05.574336. eprint: https://www.biorxiv.org/content/early/2025/04/22/2024.01.05.574336.full.pdf. URL: https://www.biorxiv.org/content/early/2025/04/22/2024.01.05.574336.

[169] Merlin Lange et al. "A multimodal zebrafish developmental atlas reveals the state-transition dynamics of late-vertebrate pluripotent axial progenitors". In: *Cell* 187.23 (2024), 6742–6759.e17. ISSN: 0092-8674. DOI: 10.1016/j.cell.2024.09.047.

[170] Y. Lecun et al. "Gradient-based learning applied to document recognition". In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324. ISSN: 1558-2256. DOI: 10.1109/5.726791. URL: https://ieeexplore.ieee.org/document/726791 (visited on 04/22/2025).

[171] Yann LeCun. "The MNIST database of handwritten digits". In: *http://yann.lecun.com/exdb/mnist/* (1998). URL: https://huggingface.co/datasets/ylecun/mnist.

[172] Yann LeCun, Yoshua Bengio, et al. "Convolutional networks for images, speech, and time series". In: *The handbook of brain theory and neural networks* 3361.10 (1995), pp. 255–258. DOI: 10.5555/303568.303704.

[173] Rachel M. Lee et al. "Believing is seeing – the deceptive influence of bias in quantitative microscopy". In: *Journal of Cell Science* 137.1 (2024), jcs261567. ISSN: 0021-9533. DOI: 10.1242/jcs.261567. URL: https://doi.org/10.1242/jcs.261567 (visited on 12/03/2025).

[174] Jaakko Lehtinen et al. "Noise2Noise: Learning Image Restoration without Clean Data". In: *Proceedings of the 35th International Conference on Machine Learning*. Ed. by Jennifer Dy and Andreas Krause. Vol. 80. Proceedings of Machine Learning Research. PMLR, 2018, pp. 2965–2974. URL: https://proceedings.mlr.press/v80/lehtinen18a.html.

[175] Wanlu Lei et al. “BioImage.IO Chatbot: a community-driven AI assistant for integrative computational bioimaging”. In: *Nature Methods* 21 (2024), pp. 1368–1370. DOI: 10.1038/s41592-024-02370-y. URL: https://www.nature.com/articles/s41592-024-02370-y.

[176] Binglan Li, Anil K. Saini, Jessica G. Hernandez, et al. “Agentic AI and the rise of in silico team science in biomedical research”. In: *Nature Biotechnology* (2026). DOI: 10.1038/s41587-026-03035-1. URL: https://www.nature.com/articles/s41587-026-03035-1.

[177] Jizhou Li, Florian Luisier, and Thierry Blu. “Pure-let deconvolution of 3D fluorescence microscopy images”. In: *2017 IEEE 14th International Symposium on Biomedical Imaging (ISBI 2017)*. 2017, pp. 723–727. DOI: 10.1109/ISBI.2017.7950621. URL: https://ieeexplore.ieee.org/document/7950621.

[178] Yue Li et al. “Incorporating the image formation process into deep learning improves network performance”. In: *Nature Methods* 19.11 (2022), pp. 1427–1437. ISSN: 1548-7105. DOI: 10.1038/s41592-022-01652-7. URL: https://doi.org/10.1038/s41592-022-01652-7.

[179] Yuexiang Li and Linlin Shen. “cC-GAN: A Robust Transfer-Learning Framework for HEp-2 Specimen Image Segmentation”. In: *IEEE Access* 6 (2018), pp. 14048–14058. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2018.2808938. URL: https://ieeexplore.ieee.org/document/8301400 (visited on 04/17/2025).

[180] Binbin Lin et al. “A deep learned nanowire segmentation model using synthetic data augmentation”. en. In: *npj Computational Materials* 8.1 (2022). Publisher: Nature Publishing Group, pp. 1–12. ISSN: 2057-3960. DOI: 10.1038/s41524-022-00767-x. URL: https://www.nature.com/articles/s41524-022-00767-x (visited on 04/22/2025).

[181] Tsung-Yi Lin et al. “Feature Pyramid Networks for Object Detection”. In: *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017, pp. 936–944. DOI: 10.1109/CVPR.2017.106.

[182] Tsung-Yi Lin et al. “Focal Loss for Dense Object Detection”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.2 (2020), pp. 318–327. ISSN: 0162-8828. DOI: 10.1109/tpami.2018.2858826.

[183] Yaron Lipman et al. “Flow Matching for Generative Modeling”. In: *International Conference on Learning Representations (ICLR)*. 2023. DOI: 10.48550/arXiv.2210.02747. arXiv: 2210.02747.

[184] Bojing Liu et al. “Self-supervised learning reveals clinically relevant histomorphological patterns for therapeutic strategies in colon cancer”. en. In: *Nature Communications* 16.1 (2025). Publisher: Nature Publishing Group, p. 2328. ISSN: 2041-1723. DOI: 10.1038/s41467-025-57541-y. URL: https://www.nature.com/articles/s41467-025-57541-y (visited on 04/17/2025).

[185] Guole Liu et al. “DeepETPicker: Fast and accurate 3D particle picking for cryo-electron tomography using weakly supervised deep learning”. In: *Nature Communications* 15.1 (2024). ISSN: 2041-1723. DOI: 10.1038/s41467-024-46041-0.

[186] Jihong Liu et al. “A Saturation Artifacts Inpainting Method Based on Two-Stage GAN for Fluorescence Microscope Images”. In: *Micromachines* 15.7 (2024). ISSN: 2072-666X. DOI: 10.3390/mi15070928. URL: https://www.mdpi.com/2072-666X/15/7/928.

[187] Tao Liu et al. “Bayesian deep-learning structured illumination microscopy enables reliable super-resolution imaging with uncertainty quantification”. In: *Nature Communications* 16.1 (2025). ISSN: 2041-1723. DOI: 10.1038/s41467-025-60093-w.

[188] Ze Liu et al. “Swin transformer: Hierarchical vision transformer using shifted windows”. In: *Proceedings of the IEEE/CVF international conference on computer vision*. 2021, pp. 10012–10022. DOI: 10.1109/ICCV48922.2021.00986.

[189] Ziwen Liu et al. “Robust virtual staining of landmark organelles with Cytoland”. In: *Nature Machine Intelligence* 7 (2025), pp. 901–915. DOI: 10.1038/s42256-025-01046-2.

[190] Vebjorn Ljosa, Katherine L Sokolnicki, and Anne E Carpenter. “Annotated high-throughput microscopy image sets for validation”. en. In: *Nat. Methods* 9.7 (2012), p. 637. URL: https://doi.org/10.1038/nmeth.2083.

[191] Katharina Löffler and Ralf Mikut. “EmbedTrack – Simultaneous Cell Segmentation and Tracking Through Learning Offsets and Clustering Bandwidths”. In: *IEEE Access* 10 (2022), pp. 77147–77157. DOI: 10.1109/ACCESS.2022.3192880.

[192] Ilya Loshchilov and Frank Hutter. “SGDR: Stochastic Gradient Descent with Warm Restarts”. In: (2017). DOI: 10.48550/arXiv.1608.03983. arXiv: 1608.03983 [cs.LG]. URL: https://arxiv.org/abs/1608.03983.

[193] David G. Lowe. “Distinctive image features from scale-invariant keypoints”. In: *International Journal of Computer Vision* 60.2 (2004), pp. 91–110. DOI: 10.1023/B:VISI.0000029664.99615.94.

[194] L. B. Lucy. “An iterative technique for the rectification of observed distributions”. In: *Astronomical Journal* 79 (1974), p. 745. DOI: 10.1086/111605.

[195] Florian Luisier et al. “Fast interscale wavelet denoising of Poisson-corrupted images”. In: *Signal Processing* 90.2 (2010), pp. 415–427. ISSN: 0165-1684. DOI: https://doi.org/10.1016/j.sigpro.2009.07.009. URL: https://www.sciencedirect.com/science/article/pii/S0165168409003016.

[196] Dora Mahecic et al. “Event-driven acquisition for content-enriched microscopy”. In: *Nature Methods* 19.10 (2022), pp. 1262–1267. ISSN: 1548-7091. DOI: 10.1038/s41592-022-01589-x.

[197] Lisa Mais, Peter Hirsch, and Dagmar Kainmueller. “PatchPerPix for Instance Segmentation”. In: *Computer Vision – ECCV 2020*. Lecture Notes in Computer Science. Cham: Springer International Publishing, 2020, pp. 288–304. DOI: 10.1007/978-3-030-58595-2_18.

[198] Markku Makitalo and Alessandro Foi. "Optimal Inversion of the Generalized Anscombe Transformation for Poisson-Gaussian Noise". In: *IEEE Transactions on Image Processing* 22.1 (2013), pp. 91–103. DOI: 10.1109/TIP.2012.2202675. URL: https://ieeexplore.ieee.org/document/6212354.

[199] Caroline Malin-Mayor et al. "Automated reconstruction of whole-embryo cell lineages by learning from sparse annotations". In: *Nature Biotechnology* 41.1 (2023), pp. 44–49. DOI: 10.1038/s41587-022-01427-7.

[200] Indrajeet Mandal et al. "Evaluating large language model agents for automation of atomic force microscopy". In: *Nature Communications* 16 (2025), p. 9104. DOI: 10.1038/s41467-025-64105-7. URL: https://www.nature.com/articles/s41467-025-64105-7.

[201] Trevor Manz et al. "Viv: Multiscale Visualization of High-Resolution Multiplexed Bioimaging Data on the Web". In: *Nature Methods* 19.5 (2022), pp. 515–516. DOI: 10.1038/s41592-022-01482-7. URL: https://doi.org/10.1038/s41592-022-01482-7.

[202] Markus Marks et al. "CellSAM: a foundation model for cell segmentation". In: *Nature Methods* 22.12 (2025), pp. 2585–2593. ISSN: 1548-7105. DOI: 10.1038/s41592-025-02879-w. URL: http://dx.doi.org/10.1038/s41592-025-02879-w.

[203] Martin Maška et al. "A benchmark for comparison of cell tracking algorithms". In: *Bioinformatics* 30.11 (2014), pp. 1609–1617. DOI: 10.1093/bioinformatics/btu080.

[204] Martin Maška et al. "The Cell Tracking Challenge: 10 years of objective benchmarking". In: *Nature Methods* 20.7 (2023), pp. 1010–1020. DOI: 10.1038/s41592-023-01879-y.

[205] Pavel Matula et al. "Cell tracking accuracy measurement based on comparison of acyclic oriented graphs". In: *PloS one* 10.12 (2015), e0144959. DOI: 10.1371/journal.pone.0144959.

[206] John McCarthy et al. "A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence, August 31, 1955". In: *AI Magazine* 27.4 (2006). Original proposal written in 1955, pp. 12–14.

[207] Leland McInnes, John Healy, and James Melville. *UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction*. 2018. DOI: 10.48550/arXiv.1802.03426. arXiv: 1802.03426.

[208] Bridget M. McVeigh et al. "Visualization of lysosomal membrane proteins by cryo electron tomography". In: *Nature Communications* 16.1 (2025). ISSN: 2041-1723. DOI: 10.1038/s41467-025-64314-0.

[209] Erik Meijering. "A bird's-eye view of deep learning in bioimage analysis". In: *Computational and Structural Biotechnology Journal* 18 (2020), pp. 2312–2325. ISSN: 2001-0370. DOI: 10.1016/j.csbj.2020.08.003.

[210] Erik Meijering. "Cell Segmentation: 50 Years Down the Road [Life Sciences]". In: *IEEE Signal Processing Magazine* 29.5 (2012), pp. 140–145. ISSN: 1053-5888. DOI: 10.1109/msp.2012.2204190. URL: http://dx.doi.org/10.1109/MSP.2012.2204190.

[211] Yaron Meirovitch et al. “SmartEM: machine learning-guided electron microscopy”. In: *Nature Methods* 23 (2026), pp. 193–204. DOI: 10.1038/s41592-025-02929-3. URL: https://www.nature.com/articles/s41592-025-02929-3.

[212] Keith Miller. “Least Squares Methods for Ill-Posed Problems with a Prescribed Bound”. In: *SIAM Journal on Mathematical Analysis* 1.1 (1970), pp. 52–74. DOI: 10.1137/0501006. eprint: https://doi.org/10.1137/0501006. URL: https://doi.org/10.1137/0501006.

[213] Tamás Miseta, Attila Fodor, and Ágnes Vathy-Fogarassy. “Surpassing early stopping: A novel correlation-based stopping criterion for neural networks”. In: *Neurocomputing* 567 (2024), p. 127028. ISSN: 0925-2312. DOI: https://doi.org/10.1016/j.neucom.2023.127028. URL: https://www.sciencedirect.com/science/article/pii/S0925231223011517.

[214] Volodymyr Mnih et al. “Human-level control through deep reinforcement learning”. In: *Nature* 518.7540 (2015), pp. 529–533. DOI: 10.1038/nature14236.

[215] Erick Moen et al. “Deep learning for cellular image analysis”. en. In: *Nature Methods* 16.12 (2019). Number: 12 Publisher: Nature Publishing Group, pp. 1233–1246. ISSN: 1548-7105. DOI: 10.1038/s41592-019-0403-1. URL: https://www.nature.com/articles/s41592-019-0403-1 (visited on 11/30/2023).

[216] Josh Moore et al. “OME-NGFF: A next-Generation File Format for Expanding Bioimaging Data-Access Strategies”. In: *Nature Methods* 18.12 (Dec. 2021), pp. 1496–1498. ISSN: 1548-7105. DOI: 10.1038/s41592-021-01326-w. URL: https://www.nature.com/articles/s41592-021-01326-w (visited on 08/12/2026).

[217] Leonor Morgado et al. “The rise of data-driven microscopy powered by machine learning”. In: *Journal of Microscopy* 295.2 (2024), pp. 85–92. ISSN: 0022-2720. DOI: 10.1111/jmi.13282.

[218] Mohammad Amin Morid, Alireza Borjali, and Guilherme Del Fiol. “A scoping review of transfer learning research on medical image analysis using ImageNet”. In: *Computers in Biology and Medicine* 128 (2021), p. 104115. ISSN: 0010-4825. DOI: 10.1016/j.compbiomed.2020.104115. URL: https://www.sciencedirect.com/science/article/pii/S0010482520304467 (visited on 04/17/2025).

[219] Nikita Moshkov et al. “Learning representations for image-based profiling of perturbations”. en. In: *Nature Communications* 15.1 (2024). Publisher: Nature Publishing Group, p. 1594. ISSN: 2041-1723. DOI: 10.1038/s41467-024-45999-1. URL: https://www.nature.com/articles/s41467-024-45999-1 (visited on 04/17/2025).

[220] Nikita Moshkov et al. “Test-time augmentation for deep learning-based cell segmentation on microscopy images”. In: *Scientific reports* 10.1 (2020), p. 5068. DOI: 10.1038/s41598-020-61808-3.

[221] Théo Moutakanni et al. “Cell-DINO: Self-supervised image-based embeddings for cell fluorescent microscopy”. In: *PLOS Computational Biology* 21.12 (2025), e1013828. DOI: 10.1371/journal.pcbi.1013828.

[222] R.F. Murphy, M. Velliste, and G. Porreca. "Robust classification of subcellular location patterns in fluorescence microscope images". In: *Proceedings of the 12th IEEE Workshop on Neural Networks for Signal Processing*. 2022, pp. 67–76. DOI: 10.1109/nnsp.2002.1030018.

[223] *napari: a multi-dimensional image viewer for Python*. DOI: 10.5281/zenodo.3555620. URL: https://zenodo.org/records/16883660.

[224] Nature Biotechnology. "Minimal life by computer". In: *Nature Biotechnology* 44 (2026). Editorial, pp. 493–494. DOI: 10.1038/s41587-026-03110-7.

[225] Fnu Neha et al. *From classical techniques to convolution-based models: A review of object detection algorithms*. 2024. arXiv: 2412.05252 [cs.CV]. URL: https://arxiv.org/abs/2412.05252.

[226] Stephan Nickell et al. "A visual approach to proteomics". In: *Nature Reviews Molecular Cell Biology* 7.3 (2006), pp. 225–230. ISSN: 1471-0072. DOI: 10.1038/nrm1861.

[227] Wei-Zhi Nie et al. "3D Convolutional Networks-Based Mitotic Event Detection in Time-Lapse Phase Contrast Microscopy Image Sequences of Stem Cell Populations". In: *2016 IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. 2016, pp. 1359–1366. DOI: 10.1109/cvprw.2016.171.

[228] Kefu Ning et al. "Deep self-learning enables fast, high-fidelity isotropic resolution restoration for volumetric fluorescence microscopy". In: *Light: Science & Applications* 12.1 (2023), p. 204. ISSN: 2047-7538. DOI: 10.1038/s41377-023-01230-2. URL: https://doi.org/10.1038/s41377-023-01230-2.

[229] Eva Nogales and Julia Mahamid. "Bridging structural and cell biology with cryo-electron microscopy". In: *Nature* 628.8006 (2024), pp. 47–56. ISSN: 0028-0836. DOI: 10.1038/s41586-024-07198-2.

[230] Maxime Oquab et al. "DINOv2: Learning Robust Visual Features without Supervision". In: (2024). arXiv: 2304.07193 [cs.CV]. URL: https://arxiv.org/abs/2304.07193.

[231] Pamela Osuna-Vargas et al. "Denoising Diffusion Models for High-Resolution Microscopy Image Restoration". In: *2025 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. 2025, pp. 4320–4330. DOI: 10.1109/WACV61041.2025.00424. URL: https://ieeexplore.ieee.org/document/10943369.

[232] Chawin Ounkomol et al. "Label-free prediction of three-dimensional fluorescence images from transmitted-light microscopy". In: *Nature Methods* 15.11 (2018), pp. 917–920. ISSN: 1548-7105. DOI: 10.1038/s41592-018-0111-2.

[233] Long Ouyang et al. "Training language models to follow instructions with human feedback". In: *Advances in Neural Information Processing Systems* 35 (2022), pp. 27730–27744.

[234] Wei Ouyang, Kevin W Eliceiri, and Beth A Cimini. "Moving beyond the desktop: prospects for practical bioimage analysis via the web". In: *Frontiers in Bioinformatics* 3 (2023), p. 1233748. DOI: 10.3389/fbinf.2023.1233748. URL: https://www.frontiersin.org/journals/bioinformatics/articles/10.3389/fbinf.2023.1233748/full.

[235] Wei Ouyang et al. "BioImage Model Zoo: A Community-Driven Resource for Accessible Deep Learning in BioImage Analysis". en. In: *bioRxiv* (2022). Pages: 2022.06.07.495102 Section: New Results, p. 2022.06.07.495102. DOI: 10.1101/2022.06.07.495102. URL: https://www.biorxiv.org/content/10.1101/2022.06.07.495102v1 (visited on 10/05/2023).

[236] Wei Ouyang et al. "ImJoy: an open-source computational platform for the deep learning era". en. In: *Nat. Methods* 16.12 (2019), pp. 1199–1200. URL: https://doi.org/10.1038/s41592-019-0627-0.

[237] Yaniv Ovadia et al. "Can You Trust Your Model's Uncertainty? Evaluating Predictive Uncertainty Under Dataset Shift". In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019, pp. 13969–13980. DOI: 10.48550/arXiv.1906.02530. arXiv: 1906.02530.

[238] Marius Pachitariu, Michael Rariden, and Carsen Stringer. "Cellpose-SAM: superhuman generalization for cellular segmentation". In: *bioRxiv* (2025), pp. 2025–04. DOI: 10.1101/2025.04.28.651001.

[239] Marius Pachitariu and Carsen Stringer. "Cellpose 2.0: how to train your own model". en. In: *Nat. Methods* 19.12 (2022). Publisher: Nature Publishing Group, pp. 1634–1641. ISSN: 1548-7105. DOI: 10.1038/s41592-022-01663-4. URL: http://www.nature.com/articles/s41592-022-01663-4 (visited on 11/09/2022).

[240] Abinash Padhi et al. "Deep learning reveals how cells pull, buckle, and navigate fibrous environments". In: *Proceedings of the National Academy of Sciences* 122.47 (2025), e2424047122. DOI: 10.1073/pnas.2424047122.

[241] Eunwoo Park et al. "Unsupervised inter-domain transformation for virtually stained high-resolution mid-infrared photoacoustic microscopy using explainable deep learning". In: *Nature Communications* 15.1 (2024), p. 10892. ISSN: 2041-1723. DOI: 10.1038/s41467-024-55262-2. URL: https://doi.org/10.1038/s41467-024-55262-2.

[242] Hyoungjun Park et al. "Deep learning enables reference-free isotropic super-resolution for volumetric fluorescence microscopy". In: *Nature Communications* 13.1 (2022), p. 3297. ISSN: 2041-1723. DOI: 10.1038/s41467-022-30949-6. URL: https://doi.org/10.1038/s41467-022-30949-6.

[243] Chau Pham, Juan C. Caicedo, and Bryan A. Plummer. "ChA-MAEViT: Unifying Channel-Aware Masked Autoencoders and Multi-Channel Vision Transformers for Improved Cross-Channel Learning". In: (2025). DOI: 10.48550/arXiv.2503.19331. arXiv: 2503.19331 [cs.CV].

[244] Barrett M. Powell et al. "Rapid structural analysis of bacterial ribosomes in situ". In: *Communications Biology* 8.1 (2025). ISSN: 2399-3642. DOI: 10.1038/s42003-025-07586-y.

[245] Mangal Prakash, Alexander Krull, and Florian Jug. "Fully Unsupervised Diversity Denoising with Convolutional Variational Autoencoders". In: *International Conference on Learning Representations (ICLR)*. 2021. DOI: 10.48550/arXiv.2006.06072. arXiv: 2006.06072.

[246] Lutz Prechelt. "Automatic early stopping using cross validation: quantifying the criteria". In: *Neural Networks* 11.4 (1998), pp. 761–767. DOI: 10.1016/S0893-6080(98)00010-0.

[247] Joanna W. Pylvänäinen et al. "Live-cell imaging in the deep learning era". en. In: *Current Opinion in Cell Biology* 85 (2023), p. 102271. ISSN: 09550674. DOI: 10.1016/j.ceb.2023.102271. URL: https://linkinghub.elsevier.com/retrieve/pii/S0955067423001205 (visited on 11/07/2023).

[248] Chang Qiao et al. "A neural network for long-term super-resolution imaging of live cells with reliable confidence quantification". In: *Nature Biotechnology* 44.1 (2026), pp. 110–119. DOI: 10.1038/s41587-025-02553-8.

[249] Chang Qiao et al. "Deep learning-based optical aberration estimation enables offline digital adaptive optics and super-resolution imaging". In: *Photon. Res.* 12.3 (2024), pp. 474–484. DOI: 10.1364/PRJ.506778. URL: https://opg.optica.org/prj/abstract.cfm?URI=prj-12-3-474.

[250] Chang Qiao et al. "Evaluation and development of deep neural networks for image super-resolution in optical microscopy". In: *Nature Methods* 18.2 (2021), pp. 194–202. ISSN: 1548-7105. DOI: 10.1038/s41592-020-01048-5. URL: https://doi.org/10.1038/s41592-020-01048-5.

[251] Chang Qiao et al. "Rationalized deep learning super-resolution microscopy for sustained live imaging of rapid subcellular processes". In: *Nature Biotechnology* 41.3 (2023), pp. 367–377. ISSN: 1546-1696. DOI: 10.1038/s41587-022-01471-3. URL: https://doi.org/10.1038/s41587-022-01471-3.

[252] Liying Qu et al. "Self-inspired learning for denoising live-cell super-resolution microscopy". In: *Nature Methods* 21.10 (2024), pp. 1895–1908. DOI: 10.1038/s41592-024-02400-9.

[253] Yuanhao Qu et al. "CRISPR-GPT for agentic automation of gene-editing experiments". In: *Nature Biomedical Engineering* 10.2 (2026), pp. 245–258. DOI: 10.1038/s41551-025-01463-z. URL: https://www.nature.com/articles/s41551-025-01463-z.

[254] Ramya Rangan et al. "CryoDRGN-ET: deep reconstructing generative networks for visualizing dynamic biomolecules inside cells". In: *Nature Methods* 21.8 (2024), pp. 1537–1545. ISSN: 1548-7091. DOI: 10.1038/s41592-024-02340-4.

[255] Luis Rangel DaCosta et al. "A robust synthetic data generation framework for machine learning in high-resolution transmission electron microscopy (HRTEM)". en. In: *npj Computational Materials* 10.1 (2024). Publisher: Nature Publishing Group, pp. 1–11. ISSN: 2057-3960. DOI: 10.1038/s41524-024-01336-0. URL: https://www.nature.com/articles/s41524-024-01336-0 (visited on 04/22/2025).

[256] Alfredo Rates et al. “Smart microscopy: adaptive microscope control to improve the way we see life”. In: *npj Imaging* 4.1 (2026). ISSN: 2948-197X. DOI: 10.1038/s44303-026-00145-y.

[257] Nikhila Ravi et al. “SAM 2: Segment Anything in Images and Videos”. In: *International Conference on Learning Representations (ICLR)*. 2025. DOI: 10.48550/arXiv.2408.00714. arXiv: 2408.00714.

[258] Joseph Redmon and Ali Farhadi. *YOLOv3: An Incremental Improvement*. 2018. arXiv: 1804.02767 [cs.CV]. URL: https://arxiv.org/abs/1804.02767.

[259] Joseph Redmon et al. “You Only Look Once: Unified, Real-Time Object Detection”. In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 779–788. DOI: 10.1109/cvpr.2016.91.

[260] Shaoqing Ren et al. “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 39.6 (2017), pp. 1137–1149. ISSN: 0162-8828. DOI: 10.1109/tpami.2016.2577031.

[261] Hamid Rezatofighi et al. “Generalized Intersection Over Union: A Metric and a Loss for Bounding Box Regression”. In: *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019, pp. 658–666. DOI: 10.1109/cvpr.2019.00075.

[262] Gavin Rice et al. “TomoTwin: generalized 3D localization of macromolecules in cryo-electron tomograms with structural data mining”. In: *Nature Methods* 20.6 (2023), pp. 871–880. ISSN: 1548-7091. DOI: 10.1038/s41592-023-01878-z.

[263] William Hadley Richardson. “Bayesian-Based Iterative Method of Image Restoration∗”. In: *J. Opt. Soc. Am.* 62.1 (1972), pp. 55–59. DOI: 10.1364/JOSA.62.000055. URL: https://opg.optica.org/abstract.cfm?URI=josa-62-1-55.

[264] Mohammad Hossein Rohban et al. “Systematic morphological profiling of human gene and allele function via Cell Painting”. In: *eLife* 6 (2017). ISSN: 2050-084X. DOI: 10.7554/elife.24060.

[265] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. “U-Net: Convolutional Networks for Biomedical Image Segmentation”. en. In: *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*. Ed. by Nassir Navab et al. Lecture Notes in Computer Science. Cham: Springer International Publishing, 2015, pp. 234–241. ISBN: 978-3-319-24574-4. DOI: 10.1007/978-3-319-24574-4_28. URL: https://link.springer.com/chapter/10.1007/978-3-319-24574-4_28.

[266] F. Rosenblatt. “The perceptron: A probabilistic model for information storage and organization in the brain”. In: *Psychological Review* 65.6 (1958), pp. 386–408. ISSN: 1939-1471. DOI: 10.1037/h0042519.

[267] H.A. Rowley, S. Baluja, and T. Kanade. “Neural network-based face detection”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 20.1 (1998), pp. 23–38. ISSN: 0162-8828. DOI: 10.1109/34.655647.

[268] Loïc A. Royer. "Omega – harnessing the power of large language models for bioimage analysis". In: *Nature Methods* 21 (2024), pp. 1371–1373. DOI: 10.1038/s41592-024-02310-w. URL: https://www.nature.com/articles/s41592-024-02310-w.

[269] Curtis T Rueden et al. "Scientific Community Image Forum: A discussion forum for scientific image software". en. In: *PLoS Biol.* 17.6 (2019), e3000340. URL: https://doi.org/10.1371/journal.pbio.3000340.

[270] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. "Learning representations by back-propagating errors". In: *Nature* 323.6088 (1986), pp. 533–536. ISSN: 0028-0836. DOI: 10.1038/323533a0.

[271] Olga Russakovsky et al. "ImageNet Large Scale Visual Recognition Challenge". In: *International Journal of Computer Vision* 115.3 (2015), pp. 211–252. ISSN: 0920-5691. DOI: 10.1007/s11263-015-0816-y.

[272] Craig T. Russell et al. *bia-binder: A web-native cloud compute service for the bioimage analysis community*. arXiv:2411.12662 [q-bio]. 2024. DOI: 10.48550/arXiv.2411.12662. URL: http://arxiv.org/abs/2411.12662 (visited on 04/09/2025).

[273] Daniel Sage et al. "DeconvolutionLab2: An open-source software for deconvolution microscopy". In: *Methods* 115 (2017). Image Processing for Biologists, pp. 28–41. ISSN: 1046-2023. DOI: https://doi.org/10.1016/j.ymeth.2016.12.015. URL: https://www.sciencedirect.com/science/article/pii/S1046202316305096.

[274] Debayan Saha et al. "Practical sensorless aberration estimation for 3D microscopy with deep learning". In: *Opt. Express* 28.20 (2020), pp. 29044–29053. DOI: 10.1364/OE.401933. URL: https://opg.optica.org/oe/abstract.cfm?URI=oe-28-20-29044.

[275] Steffen J. Sahl, Stefan W. Hell, and Stefan Jakobs. "Fluorescence nanoscopy in cell biology". In: *Nature Reviews Molecular Cell Biology* 18.11 (2017), pp. 685–701. ISSN: 1471-0080. DOI: 10.1038/nrm.2017.71. URL: https://doi.org/10.1038/nrm.2017.71.

[276] Asako Sakaue-Sawano et al. "Visualizing Spatiotemporal Dynamics of Multicellular Cell-Cycle Progression". In: *Cell* 132.3 (2008), pp. 487–498. DOI: 10.1016/j.cell.2007.12.033. URL: https://doi.org/10.1016/j.cell.2007.12.033.

[277] Ana Sanchez-Fernandez et al. "CLOOME: contrastive learning unlocks bioimaging databases for queries with chemical structures". In: *Nature Communications* 14 (2023), p. 7339. DOI: 10.1038/s41467-023-42328-w.

[278] Claudio Filipi Gonçalves Dos Santos and João Paulo Papa. "Avoiding Overfitting: A Survey on Regularization Methods for Convolutional Neural Networks". In: *ACM Comput. Surv.* 54.10s (2022). ISSN: 0360-0300. DOI: 10.1145/3510413. URL: https://doi.org/10.1145/3510413.

[279] P. Sarder and A. Nehorai. "Deconvolution methods for 3-D fluorescence microscopy images". In: *IEEE Signal Processing Magazine* 23.3 (2006), pp. 32–45. DOI: 10.1109/MSP.2006.1628876. URL: https://ieeexplore.ieee.org/document/1628876.

[280] Reuben A. Saunders et al. “Perturb-Multimodal: A platform for pooled genetic screens with imaging and sequencing in intact mammalian tissue”. In: *Cell* 188.17 (2025), 4790–4809.e22. DOI: 10.1016/j.cell.2025.05.022.

[281] Scalable Minds. *webKnossos: Visualize, Share and Annotate Your Large 3D Images Online*. URL: https://home.webknossos.org/ (visited on 09/03/2026).

[282] Robert E. Schapire. “The strength of weak learnability”. In: *Machine Learning* 5.2 (1990), pp. 197–227. DOI: 10.1007/BF00116037.

[283] Lothar Schermelleh, Rainer Heintzmann, and Heinrich Leonhardt. “A guide to super-resolution fluorescence microscopy”. In: *Journal of Cell Biology* 190.2 (2010), pp. 165–175. ISSN: 0021-9525. DOI: 10.1083/jcb.201002018. eprint: https://rupress.org/jcb/article-pdf/190/2/165/1568325/jcb_201002018.pdf. URL: https://doi.org/10.1083/jcb.201002018.

[284] Lothar Schermelleh et al. “Super-resolution microscopy demystified”. In: *Nature Cell Biology* 21.1 (2019), pp. 72–84. ISSN: 1476-4679. DOI: 10.1038/s41556-018-0251-8. URL: https://doi.org/10.1038/s41556-018-0251-8.

[285] Johannes Schindelin et al. “Fiji: an open-source platform for biological-image analysis”. en. In: *Nature Methods* 9.7 (2012). ISBN: 1548-7105, pp. 676–682. ISSN: 1548-7105. DOI: 10.1038/nmeth.2019. URL: https://doi.org/10.1038/nmeth.2019.

[286] Johannes Schindelin et al. “The ImageJ ecosystem: An open platform for biomedical image analysis”. en. In: *Mol. Reprod. Dev.* 82.7-8 (2015), pp. 518–529. URL: https://doi.org/10.1002/mrd.22489.

[287] Uwe Schmidt et al. “Cell Detection with Star-Convex Polygons”. en. In: *Medical Image Computing and Computer Assisted Intervention - MICCAI 2018*. Ed. by Alejandro F. Frangi et al. Lecture Notes in Computer Science. Cham: Springer International Publishing, 2018, pp. 265–273. ISBN: 978-3-030-00934-2. DOI: 10.1007/978-3-030-00934-2_30. URL: https://link.springer.com/chapter/10.1007/978-3-030-00934-2_30.

[288] Rajavi Shah, Nodar Gogoberidze, and Beth Cimini. *Bilayers*. URL: https://github.com/bilayer-containers/bilayers.

[289] Zafran Hussain Shah et al. “Image restoration in frequency space using complex-valued CNNs”. In: *Frontiers in Artificial Intelligence* Volume 7 - 2024 (2024). ISSN: 2624-8212. DOI: 10.3389/frai.2024.1353873. URL: https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2024.1353873.

[290] Natan T. Shaked et al. “Label-free biomedical optical imaging”. In: *Nature Photonics* 17.12 (2023), pp. 1031–1041. ISSN: 1749-4893. DOI: 10.1038/s41566-023-01299-6. URL: http://dx.doi.org/10.1038/s41566-023-01299-6.

[291] Alexander Shapson-Coe et al. “A petavoxel fragment of human cerebral cortex reconstructed at nanoscale resolution”. In: *Science* 384.6696 (2024), eadk4858. DOI: 10.1126/science.adk4858. eprint: https://www.science.org/doi/pdf/10.1126/science.adk4858. URL: https://www.science.org/doi/abs/10.1126/science.adk4858.

[292] Aabid Shariff et al. “Automated Image Analysis for High-Content Screening and Analysis”. In: *SLAS Discovery* 15.7 (2010), pp. 726–734. ISSN: 2472-5552. DOI: 10.1177/1087057110370894.

[293] Arlo Sheridan et al. “Local shape descriptors for neuron segmentation”. In: *Nature methods* 20.2 (2023), pp. 295–303. DOI: 10.1038/s41592-022-01711-z.

[294] Yu Shi et al. “Smart lattice light-sheet microscopy for imaging rare and complex cellular events”. In: *Nature Methods* 21.2 (2024), pp. 301–310. ISSN: 1548-7091. DOI: 10.1038/s41592-023-02126-0.

[295] Connor Shorten and Taghi M. Khoshgoftaar. “A survey on Image Data Augmentation for Deep Learning”. In: *Journal of Big Data* 6.1 (2019), p. 60. ISSN: 2196-1115. DOI: 10.1186/s40537-019-0197-0. URL: https://doi.org/10.1186/s40537-019-0197-0 (visited on 04/22/2025).

[296] Edward H. Shortliffe and Bruce G. Buchanan. “A model of inexact reasoning in medicine”. In: *Mathematical Biosciences* 23.3-4 (1975), pp. 351–379. DOI: 10.1016/0025-5564(75)90047-4.

[297] Hari Shroff et al. “Live-cell imaging powered by computation”. In: *Nature Reviews Molecular Cell Biology* 25.6 (2024), pp. 443–463. ISSN: 1471-0080. DOI: 10.1038/s41580-024-00702-6. URL: https://doi.org/10.1038/s41580-024-00702-6.

[298] Oriane Siméoni et al. “DINOv3”. In: (2025). DOI: 10.48550/arXiv.2508.10104. arXiv: 2508.10104 [cs.CV]. URL: https://arxiv.org/abs/2508.10104.

[299] Suganya Sivagurunathan et al. “Bridging imaging users to imaging analysis - A community survey”. en. In: *J. Microsc.* (2023). URL: https://doi.org/10.1111/jmi.13229.

[300] Nicholas Sofroniew et al. *napari: a multi-dimensional image viewer for Python*. Version v0.8.0. 2026. DOI: 10.5281/zenodo.21361474. URL: https://doi.org/10.5281/zenodo.21361474.

[301] Fabio Alexandre Spanhol et al. “Breast cancer histopathological image classification using convolutional neural networks”. In: *2016 international joint conference on neural networks (IJCNN)*. IEEE. 2016, pp. 2560–2567. DOI: 10.1109/IJCNN.2016.7727519.

[302] Willi L. Stepp et al. “Smart hybrid microscopy for cell-friendly detection of rare events”. In: *Nature Communications* 17.1 (2026). ISSN: 2041-1723. DOI: 10.1038/s41467-025-68168-4.

[303] David R Stirling et al. “CellProfiler 4: improvements in speed, utility and usability”. en. In: *BMC Bioinformatics* 22.1 (2021), p. 433. URL: https://doi.org/10.1186/s12859-021-04344-9.

[304] Carsen Stringer and Marius Pachitariu. “Cellpose3: one-click image restoration for improved cellular segmentation”. In: *Nature Methods* 22 (2025), pp. 592–599. DOI: 10.1038/s41592-025-02595-5. URL: https://www.nature.com/articles/s41592-025-02595-5.

[305] Carsen Stringer et al. “Cellpose: a generalist algorithm for cellular segmentation”. en. In: *Nature Methods* 18.1 (2021), pp. 100–106. ISSN: 1548-7105. DOI: 10.1038/s41592-020-01018-x. URL: https://www.nature.com/articles/s41592-020-01018-x (visited on 06/03/2024).

[306] K.-K. Sung and T. Poggio. “Example-based learning for view-based human face detection”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 20.1 (1998), pp. 39–51. ISSN: 0162-8828. DOI: 10.1109/34.655648.

[307] Yuanyuan Tao et al. “Inferring cellular contractile forces and work using deep morphology traction microscopy”. In: *Biophysical Journal* 123.18 (2024), pp. 3217–3230. DOI: 10.1016/j.bpj.2024.07.020.

[308] Mojtaba R. Tavakoli et al. “Light-microscopy-based connectomic reconstruction of mammalian brain tissue”. In: *Nature* 642.8067 (2025), pp. 398–410. ISSN: 0028-0836. DOI: 10.1038/s41586-025-08985-1.

[309] Irene de Teresa-Trueba et al. “Convolutional networks for supervised mining of molecular patterns within cellular context”. In: *Nature Methods* 20.2 (2023), pp. 284–294. ISSN: 1548-7091. DOI: 10.1038/s41592-022-01746-2.

[310] The MICrONS Consortium et al. “Functional connectomics spanning multiple areas of mouse visual cortex”. In: *Nature* 640.8058 (2025), pp. 435–447. ISSN: 0028-0836. DOI: 10.1038/s41586-025-08790-w.

[311] Yunjie Tian, Qixiang Ye, and David Doermann. *YOLOv12: Attention-Centric Real-Time Object Detectors*. 2025. arXiv: 2502.12524 [cs.CV]. URL: https://arxiv.org/abs/2502.12524.

[312] A. N. Tikhonov. “Solution of incorrectly formulated problems and the regularization method”. In: *Soviet Math. Dokl.* 4 (1963), pp. 1035–1038.

[313] Zhan Tong et al. “VideoMAE: masked autoencoders are data-efficient learners for self-supervised video pre-training”. In: *Proceedings of the 36th International Conference on Neural Information Processing Systems*. NIPS ’22. Red Hook, NY, USA: Curran Associates Inc., 2022, pp. 10078–10093. ISBN: 978-1-7138-7108-8.

[314] Andrew Trask et al. “Neural arithmetic logic units”. In: *Advances in Neural Information Processing Systems*. Vol. 31. Curran Associates, Inc., 2018, pp. 8035–8044. URL: https://papers.nips.cc/paper/2018/file/0e64a7b1e8e0f1c1e8e0f1c1e8e0f1c1-Paper.pdf.

[315] Srinivas C. Turaga et al. “Convolutional Networks Can Learn to Generate Affinity Graphs for Image Segmentation”. In: *Neural Computation* 22.2 (2010), pp. 511–538. DOI: 10.1162/neco.2009.10-08-881.

[316] Jessica Valli et al. “Seeing beyond the limit: A guide to choosing the right super-resolution microscopy technique”. In: *Journal of Biological Chemistry* 297.1 (2021), p. 100791. ISSN: 0021-9258. DOI: https://doi.org/10.1016/j.jbc.2021.100791. URL: https://www.sciencedirect.com/science/article/pii/S0021925821005846.

[317] Ashish Vaswani et al. “Attention is All you Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc., 2017. URL: https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html (visited on 08/05/2026).

[318] Manasij Venkatesh, Kavya Mohan, and Chandra Sekhar Seelamantula. “Directional bilateral filters for smoothing fluorescence microscopy images”. In: *AIP Advances* 5.8 (2015), p. 084805. ISSN: 2158-3226. DOI: 10.1063/1.4930029. eprint: https://pubs.aip.org/aip/adv/article-pdf/doi/10.1063/1.4930029/12895317/084805_1_online.pdf. URL: https://doi.org/10.1063/1.4930029.

[319] Giuseppe Vicidomini, Paolo Bianchini, and Alberto Diaspro. “STED super-resolved microscopy”. In: *Nature Methods* 15.3 (2018), pp. 173–182. ISSN: 1548-7105. DOI: 10.1038/nmeth.4593. URL: https://doi.org/10.1038/nmeth.4593.

[320] P. Viola and M. Jones. “Rapid object detection using a boosted cascade of simple features”. In: *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. CVPR 2001*. Vol. 1, pp. I-511-I–518. DOI: 10.1109/cvpr.2001.990517.

[321] Eric C. Wait, Michael A. Reiche, and Teng-Leong Chew. “Hypothesis-driven quantitative fluorescence microscopy – the importance of reverse-thinking in experimental design”. In: *Journal of Cell Science* 133.21 (2020), jcs250027. ISSN: 0021-9533. DOI: 10.1242/jcs.250027. URL: https://doi.org/10.1242/jcs.250027 (visited on 12/03/2025).

[322] Stéfan van der Walt et al. “scikit-image: image processing in Python”. en. In: *PeerJ* 2 (2014), e453. URL: https://peerj.com/articles/453/.

[323] Ao Wang et al. “YOLOv10: Real-Time End-to-End Object Detection”. In: *Advances in Neural Information Processing Systems 37*. 2024, pp. 107984–108011. DOI: 10.52202/079017-3429.

[324] Guotai Wang et al. “Aleatoric uncertainty estimation with test-time augmentation for medical image segmentation with convolutional neural networks”. In: *Neurocomputing* 338 (2019), pp. 34–45. DOI: 10.1016/j.neucom.2019.01.103.

[325] Hongda Wang et al. “Deep learning enables cross-modality super-resolution in fluorescence microscopy”. In: *Nature Methods* 16.1 (2019), pp. 103–110. ISSN: 1548-7105. DOI: 10.1038/s41592-018-0239-0. URL: https://doi.org/10.1038/s41592-018-0239-0.

[326] Mengfan Wang et al. “High-Fidelity Long-term Whole-embryo Lineage and Fate Reconstruction by Iterative Tracking with Error Correction”. In: *bioRxiv* (2026). DOI: 10.64898/2026.03.12.711203. eprint: https://www.biorxiv.org/content/early/2026/03/16/2026.03.12.711203.full.pdf. URL: https://www.biorxiv.org/content/early/2026/03/16/2026.03.12.711203.

[327] Xuesong Wang et al. “Prediction of cellular morphology changes under perturbations with a transcriptome-guided diffusion model”. In: *Nature Communications* 16 (2025), p. 8210. DOI: 10.1038/s41467-025-63478-z.

[328] Zhou Wang et al. “Image quality assessment: from error visibility to structural similarity”. In: *IEEE Transactions on Image Processing* 13.4 (2004), pp. 600–612. DOI: 10.1109/TIP.2003.819861. URL: https://ieeexplore.ieee.org/document/1284395.

[329] Zifeng Wang et al. “Making large language models reliable data science programming copilots for biomedical research”. In: *Nature Biomedical Engineering* (2026). DOI: 10.1038/s41551-025-01587-2. URL: https://www.nature.com/articles/s41551-025-01587-2.

[330] Zijie J. Wang et al. “CNN Explainer: Learning Convolutional Neural Networks with Interactive Visualization”. In: *IEEE Transactions on Visualization and Computer Graphics* 27.2 (2021), pp. 1396–1406. DOI: 10.1109/TVCG.2020.3030418.

[331] Asmamaw T. Wassie, Yongxin Zhao, and Edward S. Boyden. *Expansion microscopy: principles and uses in biological research*. 2019. DOI: 10.1038/s41592-018-0219-4. URL: https://doi.org/10.1038/s41592-018-0219-4.

[332] Abigail J.I. Watson and Alberto Bartesaghi. “Advances in cryo-ET data processing: meeting the demands of visual proteomics”. In: *Current Opinion in Structural Biology* 87 (2024), p. 102861. ISSN: 0959-440X. DOI: https://doi.org/10.1016/j.sbi.2024.102861. URL: https://www.sciencedirect.com/science/article/pii/S0959440X24000885.

[333] Gregory P. Way et al. “Morphology and gene expression profiling provide complementary information for mapping cell state”. In: *Cell Systems* 13.11 (2022), 911–923.e9. DOI: 10.1016/j.cels.2022.10.001.

[334] Martin Weigert et al. “Content-aware image restoration: pushing the limits of fluorescence microscopy”. In: *Nature Methods* 15.12 (2018), pp. 1090–1097. ISSN: 1548-7105. DOI: 10.1038/s41592-018-0216-7. URL: https://doi.org/10.1038/s41592-018-0216-7.

[335] Erin Weisbart et al. “Cell Painting Gallery: an open resource for image-based profiling”. In: *Nature Methods* 21.10 (2024), pp. 1775–1777. DOI: 10.1038/s41592-024-02399-z.

[336] Joseph Weizenbaum. “ELIZA—a computer program for the study of natural language communication between man and machine”. In: *Communications of the ACM* 9.1 (1966), pp. 36–45. DOI: 10.1145/365153.365168.

[337] John Graham White et al. “The structure of the nervous system of the nematode Caenorhabditis elegans”. In: *Philosophical Transactions of the Royal Society of London. B, Biological Sciences* 314.1165 (1986), pp. 1–340. ISSN: 0080-4622. DOI: 10.1098/rstb.1986.0056. eprint: https://royalsocietypublishing.org/rstb/article-pdf/314/1165/1/330977/rstb.1986.0056.pdf. URL: https://doi.org/10.1098/rstb.1986.0056.

[338] Norbert Wiener. *Extrapolation, Interpolation, and Smoothing of Stationary Time Series: With Engineering Applications*. The MIT Press, 1949. ISBN: 9780262257190. DOI: 10.7551/mitpress/2946.001.0001. eprint: https://direct.mit.edu/book-pdf/2313079/book_9780262257190.pdf. URL: https://doi.org/10.7551/mitpress/2946.001.0001.

[339] Steffen Wolf et al. “The Mutex Watershed and its Objective: Efficient, Parameter-Free Graph Partitioning”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 43.10 (2021), pp. 3724–3738. DOI: 10.1109/TPAMI.2020.2980827.

[340] Yicong Wu and Hari Shroff. "Faster, sharper, and deeper: structured illumination microscopy for biological imaging". In: *Nature Methods* 15.12 (2018), pp. 1011–1019. ISSN: 1548-7105. DOI: 10.1038/s41592-018-0211-z. URL: https://doi.org/10.1038/s41592-018-0211-z.

[341] Yicong Wu and Hari Shroff. "Multiscale fluorescence imaging of living samples". In: *Histochemistry and Cell Biology* 158.4 (2022), pp. 301–323. ISSN: 1432-119X. DOI: 10.1007/s00418-022-02147-4. URL: https://doi.org/10.1007/s00418-022-02147-4.

[342] Bin Yang et al. "DaXi—high-resolution, large imaging volume and multi-view single-objective light-sheet microscopy". In: *Nature Methods* 19.4 (2022), pp. 461–469. DOI: 10.1038/s41592-022-01417-2.

[343] Kyrollos Yanny et al. "Deep learning for fast spatially varying deconvolution". In: *Optica* 9.1 (2022), pp. 96–99. DOI: 10.1364/OPTICA.442438. URL: https://opg.optica.org/optica/abstract.cfm?URI=optica-9-1-96.

[344] Shunyu Yao et al. "ReAct: Synergizing Reasoning and Acting in Language Models". In: *arXiv preprint arXiv:2210.03629* (2022). DOI: 10.48550/arXiv.2210.03629. URL: https://arxiv.org/abs/2210.03629.

[345] Wenjing Yin et al. "A petascale automated imaging pipeline for mapping neuronal circuits with high-throughput transmission electron microscopy". In: *Nature Communications* 11.1 (2020). ISSN: 2041-1723. DOI: 10.1038/s41467-020-18659-3.

[346] Lindsey N. Young and Elizabeth Villa. "Bringing Structure to Cell Biology with Cryo-Electron Tomography". In: *Annual Review of Biophysics* 52.Volume 52, 2023 (2023), pp. 573–595. ISSN: 1936-1238. DOI: https://doi.org/10.1146/annurev-biophys-111622-091327. URL: https://www.annualreviews.org/content/journals/10.1146/annurev-biophys-111622-091327.

[347] Jiahui Yu et al. "UnitBox: An Advanced Object Detection Network". In: *Proceedings of the 24th ACM international conference on Multimedia*. 2016, pp. 516–520. DOI: 10.1145/2964284.2967274.

[348] Guo-Xun Yuan, Chia-Hua Ho, and Chih-Jen Lin. "Recent Advances of Large-Scale Linear Classification". In: *Proceedings of the IEEE* 100.9 (2012), pp. 2584–2603. ISSN: 0018-9219. DOI: 10.1109/jproc.2012.2188013.

[349] John R. Zech et al. "Variable generalization performance of a deep learning model to detect pneumonia in chest radiographs: A cross-sectional study". In: *PLOS Medicine* 15.11 (2018), e1002683. DOI: 10.1371/journal.pmed.1002683.

[350] Chong Zhang et al. "Bio-Image Informatics Index BIII: A unique database of image analysis tools and workflows for and by the bioimaging community". In: *arXiv [q-bio.QM]* (2023). URL: https://doi.org/10.48550/arXiv.2312.11256.

[351] Kai Zhang et al. "Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising". In: *IEEE Transactions on Image Processing* 26.7 (2017), pp. 3142–3155. DOI: 10.1109/TIP.2017.2662206. URL: https://ieeexplore.ieee.org/document/7839189.

[352] Peng Zhang et al. “Systematically decoding pathological morphologies and molecular profiles with unified multimodal embedding”. In: *Nature Methods* 23 (2026), pp. 903–908. DOI: 10.1038/s41592-026-03070-5.

[353] Xiao Zhang et al. “SwinCell: a 3D transformer and flow-based framework for improved cell segmentation”. In: *Communications Biology* 8.1 (2025), p. 962. DOI: 10.1038/s42003-025-08397-x.

[354] Yanzhao Zhang et al. *Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models*. 2025. DOI: 10.48550/arXiv.2506.05176. arXiv: 2506.05176 [cs.CL]. URL: https://arxiv.org/abs/2506.05176.

[355] Yide Zhang et al. “A Poisson-Gaussian Denoising Dataset With Real Fluorescence Microscopy Images”. In: *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Optica Publishing Group, 2019, pp. 11702–11710. DOI: 10.1109/CVPR.2019.01198. URL: https://ieeexplore.ieee.org/document/8953965.

[356] Yulun Zhang et al. *Image Super-Resolution Using Very Deep Residual Channel Attention Networks*. 2018. arXiv: 1807.02758 [cs.CV]. URL: https://arxiv.org/abs/1807.02758.

[357] Yian Zhao et al. “DETRs Beat YOLOs on Real-time Object Detection”. In: *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2024, pp. 16965–16974. DOI: 10.1109/cvpr52733.2024.01605.

[358] Liqun Zhong, Guole Liu, and Ge Yang. “Blind Denoising of Fluorescence Microscopy Images Using GAN-Based Global Noise Modeling”. In: *ISBI*. 2021, pp. 863–867. URL: https://doi.org/10.1109/ISBI48211.2021.9434150.

[359] You Zhou et al. “Aberration Modeling in Deep Learning for Volumetric Reconstruction of Light-Field Microscopy”. In: *Laser & Photonics Reviews* 17.10 (2023), p. 2300154. DOI: https://doi.org/10.1002/lpor.202300154. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/lpor.202300154. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/lpor.202300154.

[360] Zhi-Hua Zhou. “A brief introduction to weakly supervised learning”. In: *National Science Review* 5.1 (2018), pp. 44–53. ISSN: 2095-5138. DOI: 10.1093/nsr/nwx106.

[361] Valentyna Zinchenko et al. “MorphoFeatures for unsupervised exploration of cell types, tissues, and organs in volume electron microscopy”. In: *eLife* 12 (2023), e80918. DOI: 10.7554/eLife.80918.

A 3D-RCAN for Image Restoration

A Manual for Using AI for Image Restoration

Yue Li[1], Jun Zhu[1], Mingzhe Wei[1], Hari Shroff[2], & Min Guo[1]
[1]Zhejiang University, [2]HHMI Janelia Research Campus

A.1 Brief Introduction to the Model

Residual Channel Attention Networks (RCAN [356]), are deep learning architectures initially designed for 2D super-resolution images, leveraging residual learning and channel attention mechanisms to prioritize high-frequency details. Their hierarchical "residual-in-residual" structure and adaptive channel-wise feature scaling enable superior recovery of fine textures in natural images. Researchers adapted RCAN for 3D fluorescence microscopy data to address challenges in volumetric imaging. This 3D-RCAN [58] implementation modifies the original architecture by replacing 2D convolutions with 3D operations, optimizing parameters (e.g., residual groups, channel dimensions) for GPU efficiency, and processing spatiotemporal (4D) data. This adaptation enables denoising and resolution enhancement of volumetric time-lapse microscopy, achieving prolonged live-cell imaging with minimal photo bleaching. By training on paired low/high-quality datasets (e.g., confocal ↔ STED or iSIM ↔ expansion microscopy), 3D-RCAN restores sub cellular structures, improves lateral and axial resolution, and outperforms existing networks (e.g., CARE [334]) in fidelity metrics (e.g., SSIM [328] or peak signal to noise ratio). Its applications span fixed and live-cell studies, offering a versatile tool to overcome trade offs between spatiotemporal resolution and phototoxicity in fluorescence microscopy.

Several features of this neural network enable it to handle complex image restoration tasks effectively, as described in the paragraphs below. In later sections, we will employ this model as an example to show you how to undertake AI restoration in fluorescence microscopy.

A.1.1 Residual Learning

3D-RCAN uses a residual learning approach, which means that the network learns the difference (or residual) between the input image and the desired output. Instead of learning the entire image transformation directly, the network focuses on predicting the fine details that need improvement. This makes it easier for the network to capture high-frequency details (like edges and textures) while ignoring unnecessary background information. The network architecture is built using a residual-in-residual structure. This means that within each "residual block" (a unit of processing), there are even smaller "residual blocks." This hierarchical design improves the network's ability to learn intricate patterns in the data, allowing for better restoration of fine image details. The network essentially learns to focus on increasingly refined features at different levels of abstraction.

3D-RCAN incorporates skip connections in its architecture. Skip connections are shortcuts that allow the input image to bypass certain network layers and pass directly to later layers. There are long skip connections (which bypass multiple layers) and short skip connections (which skip only a few layers). These connections enable the network to focus on high-frequency information and avoid losing important details that may be "washed out" in deeper layers.

A.1.2 Channel Attention Mechanism

One of the distinctive features of 3D-RCAN is its channel attention mechanism. In any image, different features (such as color or depth in the case of microscopy images) have different levels of influence in the image. The channel attention mechanism allows 3D-RCAN to focus on the most important channels by adaptively adjusting the weight given to each feature channel. This helps the network to enhance high-resolution details in relevant channels while minimizing the influence of less important features.

A.1.3 3D Adaptation for Image Volumes

3D-RCAN is designed to handle not only 2D images but also 3D image volumes. In many microscopy techniques, images are captured as a series of 2D planes stacked together to form a 3D volume. RCAN adapts its structure to process these multi-dimensional data efficiently. By working in 3D, the network can improve both the lateral (XY) and axial (Z) resolution of the image volumes, which is especially important in applications like fluorescence microscopy.

A.1.4 Channel Reduction for Efficient 3D Processing

By reducing channel dimensions before processing and restoring them afterward, the network minimizes redundant computations while retaining essential spatial and structural details. This mechanism enables deeper architectures without overwhelming GPU memory, particularly

crucial for large 3D microscopy volumes. Coupled with channel attention, the compressed features are dynamically recalibrated to prioritize high-frequency details, ensuring robust restoration of fine structures like microtubules or mitochondrial membranes. The approach optimizes both training speed and inference performance, validated across synthetic and experimental datasets.

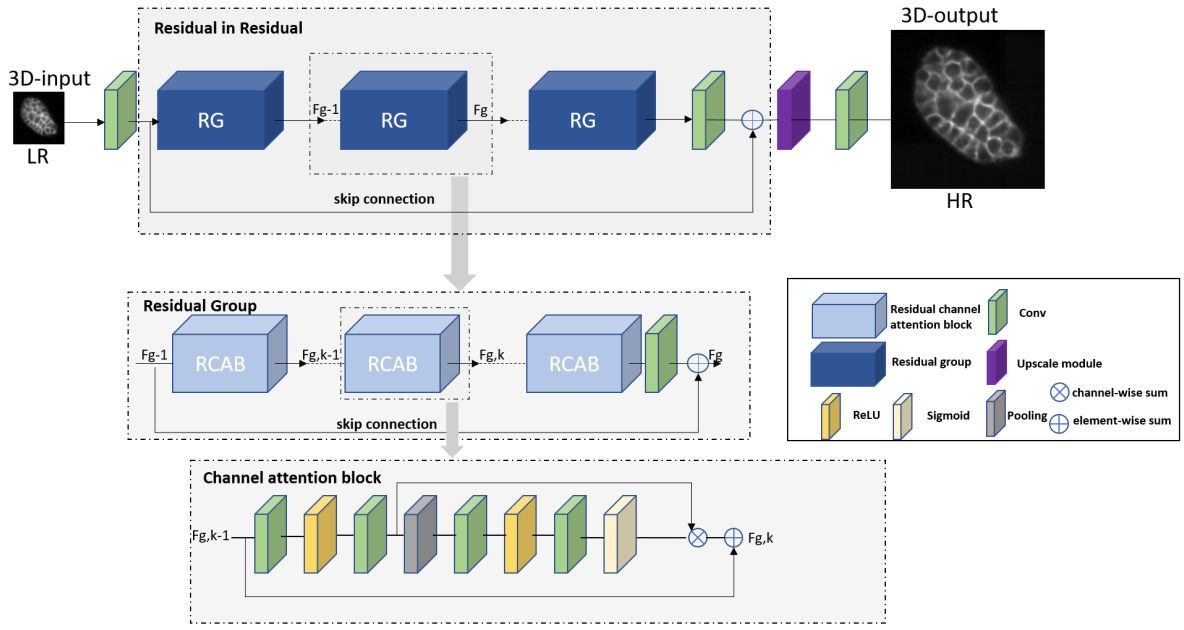


Figure A.1: **The architecture of 3D-RCAN.** This diagram illustrates the 3D Residual Channel Attention Network (RCAN) architecture for volumetric resolution enhancement. The low-resolution (LR) 3D input progresses through cascaded Residual Groups (RG), each containing Residual Channel Attention Blocks (RCAB) with channel attention to prioritize critical features. Skip connections preserve structural details across groups. The upscale module progressively enhances spatial resolution, culminating in the high-resolution (HR) 3D output.

A.2 Overview of the Code and Data

A.2.1 System Requirements

The basic system requirements are:

- Operating System: Windows 10, Linux, or Mac OS
- Python 3.7
- GPU: NVIDIA GPU with CUDA 10.0 and cuDNN 7.6.5

For more information about system requirements, please refer to the 3D-RCAN README.

A.2.2 Installation of Dependencies

3D-RCAN itself does not require installation and installation of the required the dependencies takes a few seconds on a typical PC. The installation process involves three main steps:

1. Check system requirements: Ensure your system meets the basic requirements listed above, particularly Python 3.7 and CUDA compatibility.

2. Create a new virtual environment: Set up an isolated Python environment to avoid conflicts with other projects and ensure reproducibility.

3. Install dependencies: Download the repository and install required Python packages using the requirements.txt file provided in the 3D-RCAN respository.

A.2.3 Quick Setup Steps

1. **Clone the repository**:

```
git clone https://github.com/AiviaCommunity/3D-RCAN.git
```

2. **Create and activate virtual environment**:

```
# Create virtual environment
python -m venv RCAN3D

# Activate (Windows)
.\\RCAN3D\\Scripts\\activate

# Activate (macOS/Linux)
source RCAN3D/bin/activate
```

3. **Install dependencies**:

```
pip install -r requirements.txt
```

For comprehensive installation instructions, please refer to the *Dependencies Installation* section in the 3D-RCAN README.

A.2.4 Dataset

Custom Data: 3D-RCAN supports 3D data with strict low-quality/high-quality paired training (e.g., noisy/blurred inputs vs. high-SNR/clean GT) . Performance depends on domain alignment between training data (intensity distribution, resolution, noise characteristics) and target applications. The data should follow the format of 3D TIFF stacks (Z-Y-X order).

Example datasets available: You can leverage open-source datasets to obtain paired low-quality and high-quality data, such as the dataset from the 3D-RCAN paper. The 3D RCAN dataset is publicly available on Zenodo. It includes paired low-quality and ground truth (GT) data for diverse applications: denoising mulitple biological structures (actin, ER, Golgi, lysosomes, microtubules, and mitochondria), synthetic blurred phantom spheres with sharp GT, Confocal-to-STED microscopy modality transfer (microtubules, nuclear pores, DNA, and live-cell nuclei), and expansion microscopy (microtubules and mitochondria, pairing synthetic raw images with deconvolved iSIM images). Live-cell test data cover U2OS cells (mitochondria and lysosomes) and Jurkat T cells (EMTB-GFP). Raw data represent inputs (e.g., confocal images, noisy/blurred volumes), while GT corresponds to high-quality outputs (e.g., STED, deconvolved iSIM).

A.2.5 Training

Training a 3D-RCAN model requires a `config.json` file to specify parameters and data locations.

To initiate training, use the command:

```
python train.py -c config.json -o /path/to/training/output/dir
```

Training data can be specified in the `config.json` file by either:

1. Providing paths to folders containing raw and ground truth images (`training_data_dir`).
2. Listing specific pairs of raw and ground truth image files (training_image_pairs).

Numerous other options can be configured in the JSON file, including validation data, model architecture parameters (like `num_residual_blocks`, `num_residual_groups`), data augmentation settings, learning rate, loss functions, and metrics. The defaults for `num_residual_blocks` (3) and `num_residual_groups` (5) are set to balance performance and hardware constraints, aiming for optimal accuracy on standard GPUs (16-24GB VRAM) without causing memory overflow.

The expected runtime is approximately 5-10 minutes per epoch on a system similar to the tested environment (NVIDIA GeForce GTX 1080 Ti - 11GB) using the example `config.json`. Training progress and loss values can be monitored using TensorBoard.

For comprehensive details on configuring the `config.json` file, all available training options, and further instructions, please refer to the Training section in the 3D-RCAN README.

A.2.6 Applying the Model

Trained 3D-RCAN models can be applied using the `apply.py` script. The script will automatically select the model with the lowest validation loss from the specified model directory.

There are two primary ways to apply a model:

1. To a single image

   ```
   python apply.py -m /path/to/model_dir -i input_raw_image.tif -o output.tif
   ```

2. To a folder of images (batch mode)

   ```
   python apply.py -m /path/to/model_dir
   -i /path/to/input_image_dir
   -o /path/to/output_image_dir
   ```

   When processing a folder, ground truth images can also be specified using the `-g` argument for comparison.

Key command-line arguments include:

- `-m` or `--model_dir`: Path to the folder containing the trained model (required).
- `-i` or `--input`: Path to the input raw image or folder (required).
- `-o` or `--output`: Path for the output processed image or folder (required).
- `-g` or `--ground_truth`: Path to a reference ground truth image or folder (optional).
- `-b` or `--bpp`: Bit depth of the output image (e.g., 8, 16, 32).
- `-B` or `--block_shape`: Dimensions (Z,Y,X) for processing large images in blocks.
- `-O` or `--block_overlap_shape`: Overlap size (Z,Y,X) between blocks.
- `--normalize_output_range_between_zero_and_one`: Normalizes output intensity to the [0, 1] range, or to the full bit depth range (e.g., [0, 65535] for 16-bit) when combined with `-b`.
- `--rescale`: Performs affine rescaling to minimize MSE between restored and ground truth images, useful for comparisons similar to CARE methodology.
- `-f` or `--output_tiff_format`: Sets the output TIFF format (e.g., “imagej” or “ome”).

For a complete list of all options and detailed explanations, please refer to the Model Apply section in the 3D-RCAN README.

A.3 Denoising Tutorial

A.3.1 Data Preparation

Input Format: 3D TIFF stacks (Z-Y-X order)

Data Pairs: The example image pairs we employ are shown in Figure A.2 and are available in the denoising section of the data for the 3D-RCAN paper.

- **Low-SNR Data**: Noisy images. Here we use low signal-to-noise ratio data obtained by imaging mitochondria labeled with mEmerald-Tomm20-C-10 in living U2OS cells using instant structured illumination microscopy under low illumination intensities.
- **High-SNR Ground Truth**: Corresponding clean images. Here we use high signal-to-noise ratio and high-resolution data obtained by imaging mitochondria labeled with mEmerald-Tomm20-C-10 in living U2OS cells using instant structured illumination microscopy under high illumination intensities.

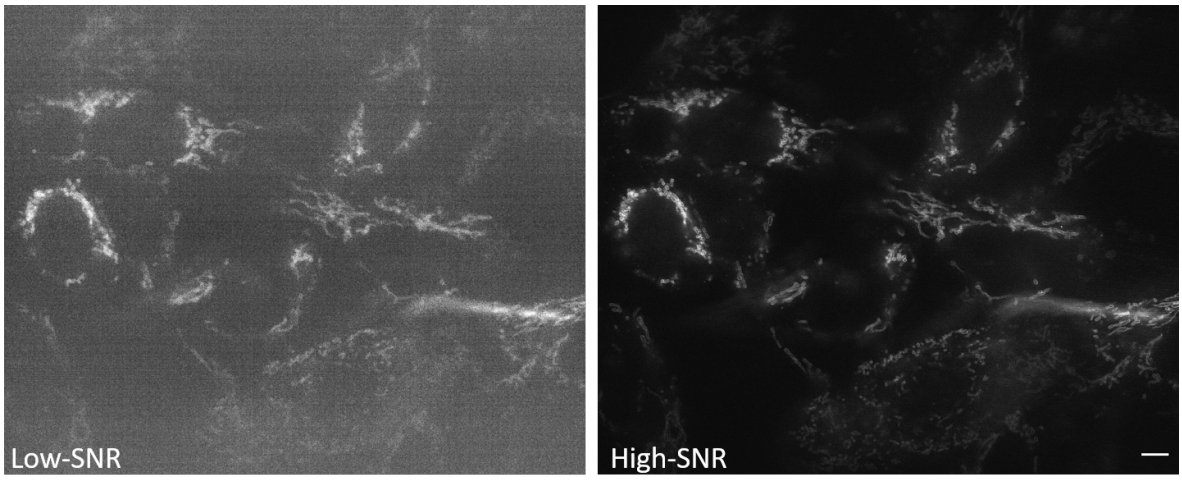


Figure A.2: **Dataset for denoising tasks.** (left) Low signal-to-noise ratio (SNR) image and (right) high-SNR image of mitochondria in U2OS cells. Images were acquired using instant structured illumination microscopy under low and high illumination intensities, respectively. Scale bar: 5 m.

Directory Structure: Organize the paired image data as follows.

```
dataset/
  train/
     raw/   # Training noisy data
     gt/    # Training clean data
```

```
  val/
      raw/   # Validation noisy data
      gt/    # Validation clean data
```

The validation data is not necessary for model training, but is an important part of assessing the model quality after training.

A.3.2 Training a Denoising Model

A.3.2.1 Configuration File

Configure the settings file (`config_denoise.json`) to define the data locations for the training/validation sets and the initial network hyperparameters. An example configuration file is shown below.

```
{
  "training_data_dir": {
      "raw": "E:\\Denoising\\Tomm20_Mitochondria\\Training\\Raw",
      "gt": "E:\\Denoising\\Tomm20_Mitochondria\\Training\\GT"
  },
  "validation_data_dir": {
      "raw": "E:\\Denoising\\Tomm20_Mitochondria\\Val\\raw",
      "gt": "E:\\Denoising\\Tomm20_Mitochondria\\Val\\gt"
  },
  "num_channels": 32,
  "num_residual_blocks": 3,
  "num_residual_groups": 5,
  "epochs": 100,
  "steps_per_epoch": 256,
  "initial_learning_rate": 1e-4
}
```

A.3.2.2 Training Command

Run the training using the following command, updating the path to where you have stored the example images as appropriate.

```
python train.py -c config_denoise.json
-o " E:\\Denoising\\Tomm20_Mitochondria\\output"
```

> **Tip**
>
> If you are using a Mac or Linux system, replace the \ \ in the example paths with /.

A.3.2.3 Training Outputs

The output directory will save the model parameters during the training process. For example, the file weights_092_0.06313289.hdf5 represents the model parameters saved at the 92nd training epoch, with a loss value of 0.06313289.

A.3.3 Applying the Denoising Model

A.3.3.1 Apply Command

To apply the model trainined in the previous section, provide the model (`-m`), path for the input images (`-i`), and path for the output denoised images (`-o`).

```
python apply.py
  -m "E:\\Denoising\\Tomm20_Mitochondria\\output"
  -i "E:\Denoising\\Tomm20_Mitochondria\\Test\\Raw"
  -o "E:\\Denoising\\Tomm20_Mitochondria\\Test\\Denoised"
```

A.3.3.2 Output Analysis

Since the output TIFF file is a two-channel ImageJ Hyperstack containing raw and restored images, we can easily compare the denoising effects.

3D-RCAN reconstructs continuous organelle membranes (e.g., mitochondrial cristae) in noisy volumetric data by suppressing stochastic noise, achieving sub-diffraction structural integrity without amplifying high-frequency artifacts or distorting fine textures.

When evaluating the quality of denoised results generated by 3D-RCAN, PSNR (Peak Signal-to-Noise Ratio) and MSE (Mean Squared Error) serve as critical quantitative metrics. PSNR measures the logarithmic relationship between the maximum possible pixel intensity and the distortion introduced during restoration, with higher values indicating better alignment between the restored image and the ground truth (GT). Conversely, MSE calculates the average squared difference between corresponding pixels, where lower values reflect smaller pixel-level errors.

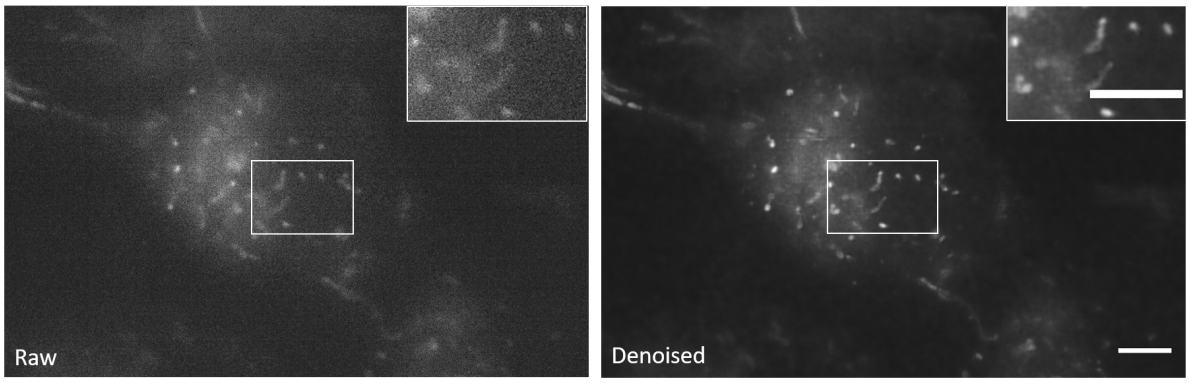


Figure A.3: **Denoising Performance.** The images show raw data input (left) and denoised image after using 3D-RCAN (right). Scale bars: 5 m for main image and 1 m for inset.

For meaningful comparisons, the restored image should exhibit a higher PSNR and lower MSE relative to the raw input when both are evaluated against the GT. This directly quantifies the model's ability to enhance resolution while minimizing deviations from the true signal.

You can calculate PSNR and MSE with the formulas below, where m, n, and p are the dimensions of the 3D image. $I_{\text{raw}}(i,j,k)$ is the intensity value of the raw image at position (i,j,k) and $I_{\text{GT}}(i,j,k)$ is the intensity value of the ground truth image at position (i,j,k).

$$\text{MSE} = \frac{1}{mnp}\sum_{i=0}^{m-1}\sum_{j=0}^{n-1}\sum_{k=0}^{p-1}[I_{\text{raw}}(i,j,k) - I_{\text{GT}}(i,j,k)]^2 \tag{A.1}$$

$$\text{PSNR} = 10 \cdot \log_{10}\left(\frac{\text{MAX}_I^2}{\text{MSE}}\right) \tag{A.2}$$

We recommend intensity normalization across raw, restored, and GT images (e.g., scaling to $[0,1]$) to avoid metric distortions. For heterogeneous samples, compute localized metrics within regions of interest (e.g., cell membranes vs. cytoplasm) to identify performance variations. Note that while high PSNR/MSE improvements generally correlate with perceptual quality, they should complement visual inspection and task-specific validations (e.g., downstream analysis accuracy). Always verify that the model's output preserves biological relevance—metrics alone cannot capture context-specific artifacts or over-smoothing.

A.3.4 Further Denoising Guidance

To improve the performance of the 3D-RCAN model when unsatisfied with denoising results, we recommend starting by tuning hyperparameters in the `config.json` file. Start by ad-

justing the model architecture parameters to balance computational constraints and feature extraction capabilities. For instance, increasing `num_channels` (default:32) enhances the network's capacity to learn complex features but requires more GPU memory. If limited by hardware, reducing it to 16-24 while moderately increasing `num_residual_groups` (default:5) or `num_residual_blocks` per group (default:3) could maintain depth without overwhelming resources. The `channel_reduction` ratio (default:8) for attention mechanisms can be lowered to 4-6 to amplify the model's sensitivity to channel-wise dependencies, though this should be validated using metrics to avoid overfitting.

Training dynamics can be optimized by revisiting the `initial_learning_rate` (default:1e-4). If training loss plateaus or fluctuates, gradually lowering it to 1e-5 stabilizes convergence, while aggressive learning rates (e.g., 5e-4) might accelerate early training but risk divergence. Switching the loss function from MAE (robust to noise) to MSE could prioritize pixel-level accuracy for high-frequency details, especially when ground truth data has sharp structures. Extending training epochs beyond the default 300—coupled with early stopping based on validation loss—helps the model capture subtle patterns in volumetric data.

Data-related parameters require careful calibration. Enabling `data_augmentation` (default:True) remains vital for small datasets to simulate variations in microscopy imaging, but it can be disabled for large, diverse datasets to speed up training. The `intensity_threshold` (default:0.25) and `area_ratio_threshold` (default:0.5) act as filters for low-quality patches. Raising the intensity threshold to 0.3-0.4 suppresses noisy backgrounds, while lowering the area ratio to 0.3-0.4 accommodates sparse biological structures like microtubules. For validation, always specify a dedicated `validation_data_dir` to monitor generalization and prevent overfitting.

Finally, hardware limitations can be mitigated by reducing `block_shape` dimensions during inference or simplifying the model architecture. Experiment iteratively: establish a baseline with default settings, then progressively scale model complexity while tracking validation metrics like PSNR/SSIM. Consider hybrid strategies, such as combining MAE loss with SSIM regularization (a loss penalty using SSIM to preserve structural fidelity and prevent over-smoothing) or dynamic learning rate schedules, to adapt to specific data characteristics. Always validate changes against biologically relevant structures in your test set, as quantitative metrics alone may not reflect practical image quality for microscopy analysis.

A.4 Deconvolution Tutorial

A.4.1 Data Preparation

Input Format: 3D TIFF stacks (Z-Y-X order)

Data Pairs: The example image pairs we employ here are shown in Figure A.4 and are available in the 'DL Decon' section of DeAbePlusData.

- **Input (Raw**): Low-quality, blurry 3D images with PSF such as single-view volumetric images acquired via diSPIM (0.8 NA × 0.8 NA optics), with voxel size 0.1625 × 0.1625 × 1.0 µm
- **Ground Truth (GT):** High-quality, non-blurry 3D images such as dual-view joint deconvolution-reconstructed volumes (fusion of both diSPIM views), with isotropic resolution.

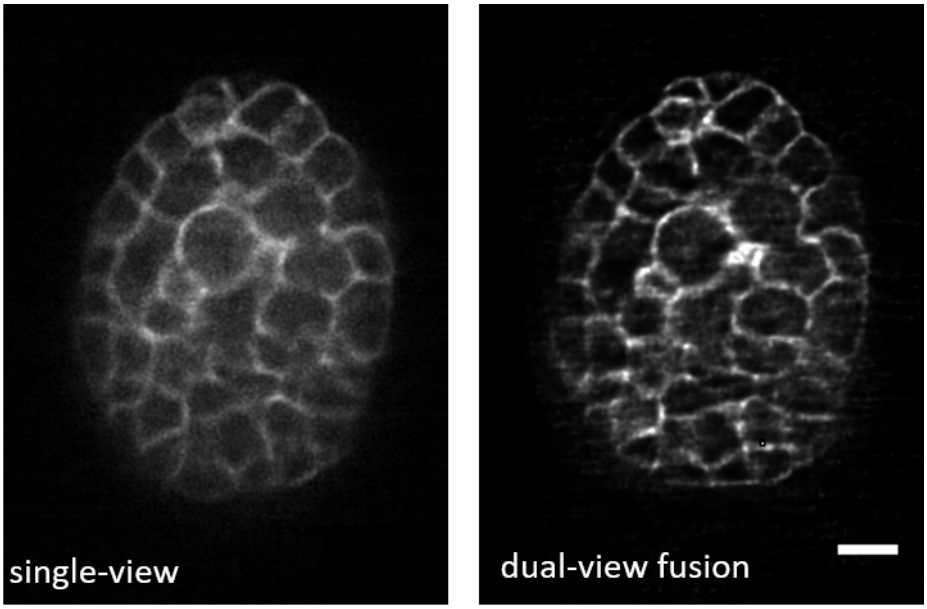


Figure A.4: **Dataset for Deconvolution.** The images show the single-view image (network input, left) and dual-view image (right) of C. elegans embryos. Scale bar: 5 m.

Directory Structure: Organize data as follows.

```
dataset/
  train/
      raw/   # Training raw data
      gt/    # Training label data
  val/
       raw/   # Validation raw data
       gt/    # Validation label data
```

The validation data is not necessary for model training, but is an important part of assessing the model quality after training.

A.4.2 Training a Deconvolution Model

A.4.2.1 Configuration File

Configure the settings file (`config_decon.json`) to define the data locations for the training/validation sets and the initial network hyper-parameters.

```
{
  "training_data_dir": {
      "raw": "E:\\data_decon\\Decon_Training_Data\\Raw",
      "gt":"E:\\data_decon\\Decon_Training_Data\\gt"
  },
  "validation_data_dir": {
      "raw":"E:\\data_decon\\Decon_val_Data\\Raw",
      "gt": "E:\\data_decon\\Decon_val_Data\\gt"
  },
  "num_channels": 32,
  "num_residual_blocks": 3,
  "num_residual_groups": 5,
  "epochs": 200,
  "steps_per_epoch": 256,
  "initial_learning_rate": 1e-4
}
```

A.4.2.2 Training Command

Run the training using the following command, updating the path to where you have stored the example images as appropriate.

```
python train.py -c config_decon.json -o  "E:\\data_decon\\model_output"
```

> **Tip**
>
> If you are using a Mac or Linux system, replace the \ \ in the example paths with /.

A.4.2.3 Training Outputs:

The output directory will save the model parameters during the training process. For example, the file weights_092_0.06313289.hdf5 represents the model parameters saved at the 92nd training epoch, with a loss value of 0.06313289.

A.4.3 Applying the Deconvolution Model

A.4.3.1 Apply Command

To apply the model trainined in the previous section, provide the model (`-m`), path for the input images (`-i`), and path for the output denoised images (`-o`).

```
python apply.py
  -m "E:\\data_decon\\model_output"
  -i "E:\\data_decon\\Decon_val_Data\\Raw"
  -o "E:\\data_decon\\Decon_val_Data\\deconvolved"
```

A.4.3.2 Output Analysis

Since the output TIFF file is a two-channel ImageJ Hyperstack containing raw and restored images, we can easily compare the deconvolved effects.

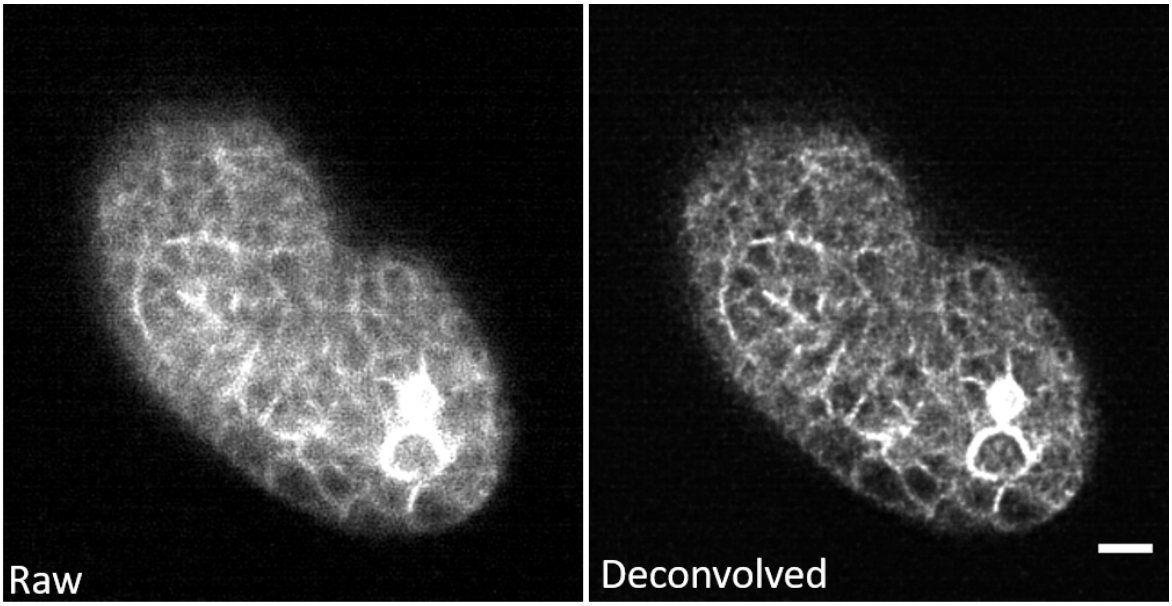


Figure A.5: **Network Prediction for Deconvolution.** The images show the raw data from C. elegans embryos (left) and 3D-RCAN prediction (right), demonstrating deconvolution. Scale bar: 5 m.

3D-RCAN corrects out-of-focus blur in GFP-labeled *C. elegans* embryos, resolving densely packed microtubule arrays within mitotic spindles and restoring cortical membrane contours

obscured by optical scattering in thick embryonic samples, without introducing grid-like artifacts typical of Fourier-based deconvolution.

Refer to Section A.3.3.2 for baseline validation metrics. Compare with classical methods (e.g., Richardson [263] -Lucy [194] Deconvolution).

A.4.4 Further Deconvolution Guidance

Refer to Section A.3.4 for general guidance. For deconvolution, use a lightweight attention mechanism (≤ 3 channel reduction steps) to avoid over-smoothing high-frequency structures.

A.5 Aberration Correction Tutorial

A.5.1 Data Preparation

Input Format: 3D TIFF stacks (Z-Y-X order)

Data Pairs: The sample image pairs employed here were *C. elegans* embryos expressing a GFP marker , as shown in Figure A.6. You can find these examples in the 'DL DeAbe' section of the DeAbePlusData.

- **Aberrated Input Data**: Artificially degraded images, corrupted by synthetic aberrations modeled via Zernike polynomials (modes 1–7, defocus coefficient ≤ 1.5 rad, other modes ≤ 0.5 rad) and Poisson noise. Degradation is applied to shallow-layer ground truth using the physical forward model.
- **High-Quality Label Data**: Directly acquired shallow-layer images serve as ground truth, validated by their proximity to the objective lens (minimal inherent aberrations).

Directory Structure: Organize data as follows.

```
dataset/
  train/
      raw/   # Training aberrated data
      gt/    # Training label data
  val/
      raw/   # Validation aberrated data
      gt/    # Validation label data
```

The validation data is not necessary for model training, but is an important part of assessing the model quality after training.

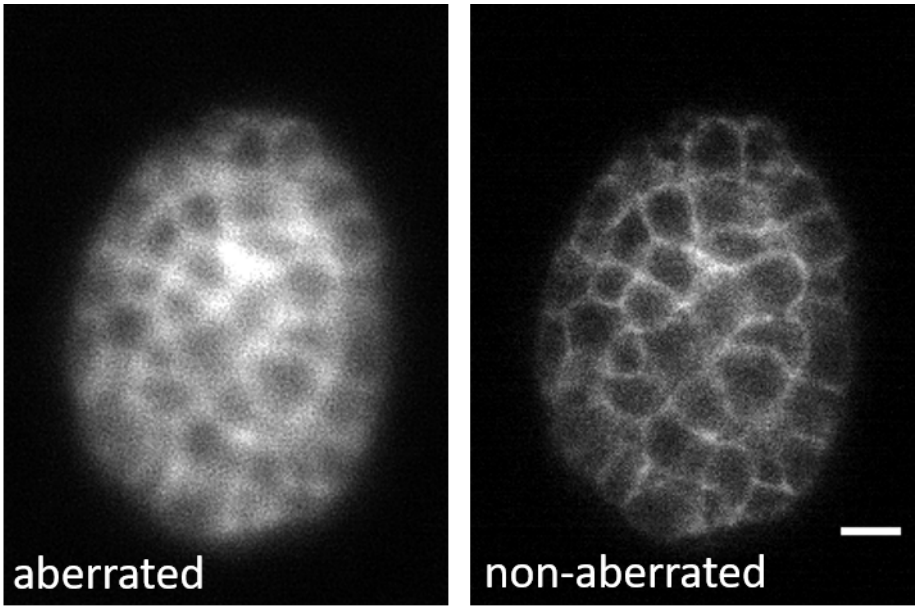


Figure A.6: **Dataset for Aberration Correction.** The images show the aberrated data (network input, left) and shallow-layer data (right) of C. elegans embryos expressing a GFP marker. Scale bar: 5 m.

A.5.2 Training an Aberration Correction Model

A.5.2.1 Configuration File

Configure the settings file (`config_deabe.json`) to define the data locations for the training/validation sets and the initial network hyperparameters.

```
{
  "training_data_dir": {
      "raw": "E:\\data_aberration\\CropForTraining_ab\\Aberrated",
      "gt":"E:\\data_aberration\\CropForTraining_ab\\GT"
  },
  "validation_data_dir": {
      "raw":"E:\\data_aberration\\CropForTraining_ab\\Validation\\Aberrated",
      "gt": "E:\\data_aberration\\CropForTraining_ab\\Validation\\GT"
  },
  "num_channels": 32,
  "num_residual_blocks": 3,
  "num_residual_groups": 5,
  "epochs": 100,
```

```
  "steps_per_epoch": 100,
  "initial_learning_rate": 1e-4
}
```

A.5.2.2 Training Command

Run the training using the following command, updating the path to where you have stored the example images as appropriate.

```
python train.py -c config_deabe.json
-o " E:\\DeAbe\\Train\\output"
```

> **Tip**
>
> If you are using a Mac or Linux system, replace the \ \ in the example paths with /.

A.5.2.3 Training Outputs:

The output directory will save the model parameters during the training process. For example, the file weights_092_0.06313289.hdf5 represents the model parameters saved at the 92nd training epoch, with a loss value of 0.06313289.

A.5.3 Applying the Aberration Correction Model

A.5.3.1 Apply Command

To apply the model trainined in the previous section, provide the model (`-m`), path for the input images (`-i`), and path for the output denoised images (`-o`).

```
python apply.py
  -m "E:\\DeAbe\\Train\\output"
  -i "E:\DeAbe\\Test\\Raw"
  -o "E:\\DeAbe\\Test\\Deabrrated"
```

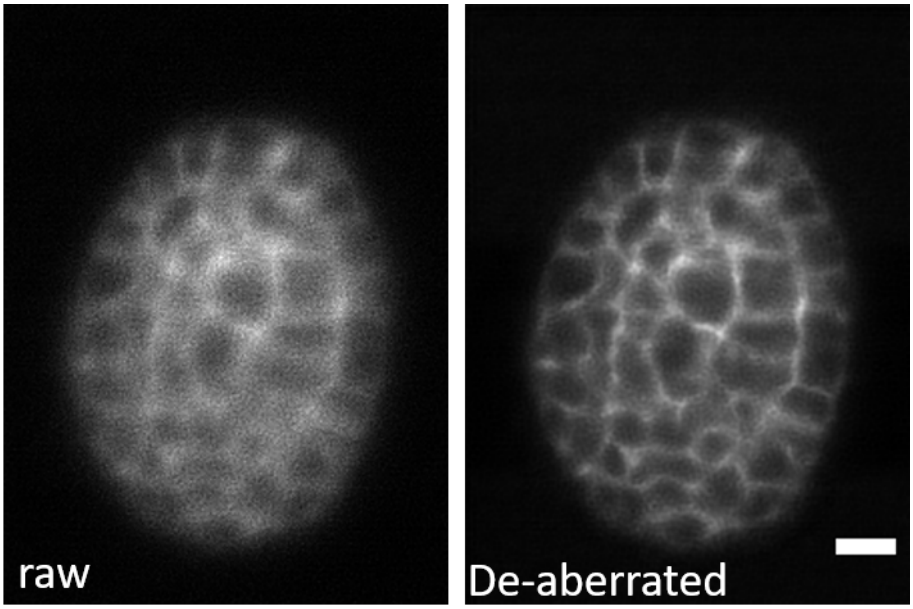


Figure A.7: **3D-RCAN Deaberration prediction.** The images show the raw data input to the network (left) and 3D-RCAN prediction (right), demonstrating aberration correction. Scale bar: 5 m.

A.5.3.2 Output Analysis

Since the output TIFF file is a two-channel ImageJ Hyperstack containing raw and restored images, we can easily compare the de-aberration effects.

3D-RCAN effectively corrects system-induced optical aberrations in GFP-labeled *C. elegans* embryos. The de-aberrated image restores spatially ordered structures obscured by asymmetric blur in the raw data, while suppressing background granularity and enhancing signal-to-background contrast.

Refer to Section A.3.3.2 for baseline validation metrics. For aberration correction validation, analyze Fourier spectra to assess restored spatial symmetry and high-frequency energy, quantify wavefront errors via phase retrieval or bead-based PSF measurements, and evaluate intensity uniformity/edge sharpness across volumes.

A.5.4 Further Aberration Correction Guidance

Refer to Section A.3.4 for basic guidance. Prioritize RCAB layers with wider receptive fields (kernel size $\leq 5^3$) to capture large-scale distortion patterns.

A.6 Resolution Enhancement Tutorial

A.6.1 Data Preparation

Input Format: 3D TIFF stacks (Z-Y-X order)

Data Pairs: The example image pairs we employ are shown in Figure A.8. You can find these examples in the 'confocal_2_STED' section of the data for the 3D-RCAN paper.

- **Raw data:** The "raw" data here refers to the original confocal microscopy images of the fixed mouse embryonic fibroblast (MEF) cells, specifically focusing on the microtubules. These images are acquired directly from the confocal microscope without any further processing or enhancement. They represent the initial, unaltered state of the microtubule structures as captured by the confocal microscope.
- **Ground Truth (GT) data**: The "ground truth" (GT) data consists of the high-resolution STED (Stimulated Emission Depletion) microscopy images of the same fixed MEF cells, again focusing on the microtubules.

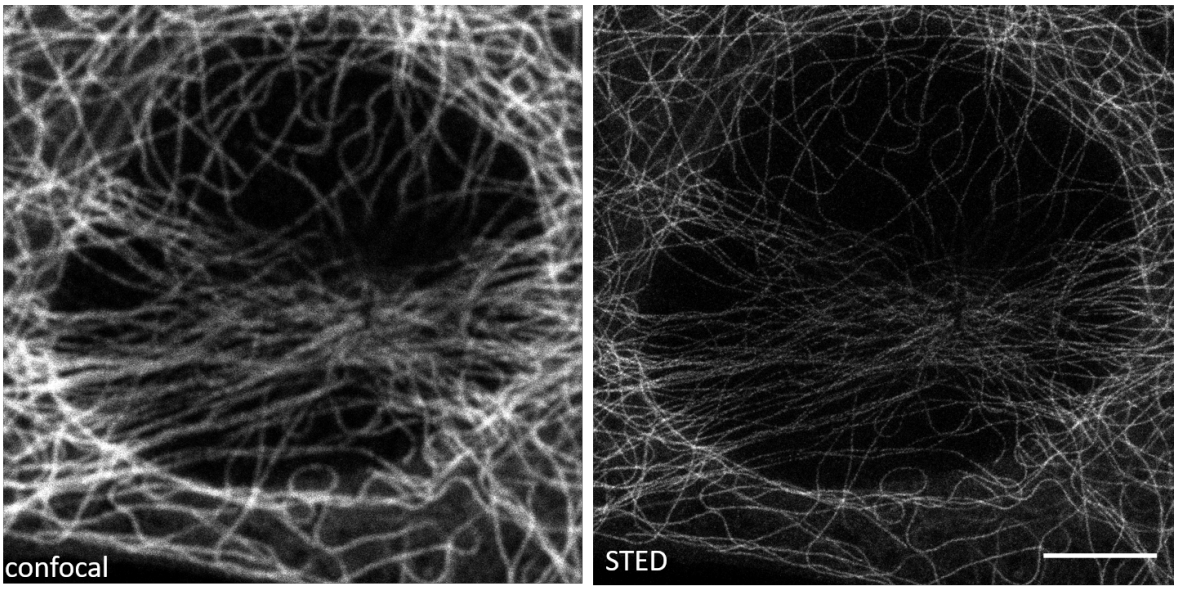


Figure A.8: **Dataset for resolution enhancement.** Image captured by confocal microscope (network input, left) and image captured by STED microscopy (ground truth, right). Both images show microtubules in mouse embryonic fibroblast cells. Scale bar: 5 m.

Directory Structure: Organize data as follows.

```
dataset/
   train/
       raw/   #confocal data
       gt/    #STED data
   val/
        raw/   # confocal data
        gt/    # STED data
```

The validation data is not necessary for model training, but is an important part of assessing the model quality after training.

A.6.2 Training a Resolution Enhancement Model

A.6.2.1 Configuration File

Configure the settings file (`config_sr.json`) to define the data locations for the training/validation sets and the initial network hyper-parameters.

```
{
  "training_data_dir": {
      "raw": "E:\\Confocal_2_STED\\Microtubule\\Training\\raw",
      "gt": "E:\\Confocal_2_STED\\Microtubule\\Training\\gt1"

  },

  "num_channels": 32,
  "num_residual_blocks": 3,
  "num_residual_groups": 5,
  "epochs": 100,
  "steps_per_epoch": 256,
  "initial_learning_rate": 1e-4
}
```

A.6.2.2 Training Command

Run the training using the following command, updating the path to where you have stored the example images as appropriate.

```
python train.py
-c config_ex.json
-o "E:\\Confocal_2_STED\\Microtubule\\Training\\output"
```

> **Tip**
>
> If you are using a Mac or Linux system, replace the \ \ in the example paths with /.

A.6.3 Applying the Resolution Enhancement Model

A.6.3.1 Apply Command

To apply the model trainined in the previous section, provide the model (`-m`), path for the input images (`-i`), and path for the output denoised images (`-o`).

```
python apply.py
-m "E:\\Confocal_2_STED\\Microtubule\\Training\\output"
-i "E:\DATA_for_3dRCAN\Confocal_2_STED\Microtubule\test\"
-o "E:\\Confocal_2_STED\\Microtubule\\test\output\\"
```

A.6.3.2 Output Analysis

Since the output TIFF file is a two-channel ImageJ Hyperstack containing raw and restored images, we can easily investigate the extent of resolution enhancement.

The model resolves densely packed microtubule intersections and eliminates axial blur in confocal-derived volumes, restoring filament topology to near-STED resolution.

Refer to Section A.3.3.2 for baseline validation metrics. Check for artifacts in Z-stack transitions and consistency in organelle morphology.

A.6.4 Further Resolution Enhancement Guidance

Refer to Section A.3.4 for basic guidance. Optimize residual groups (e.g., increase from 5 to 10) and channel dimensions (≤ 64) to handle spatially variant upsampling.

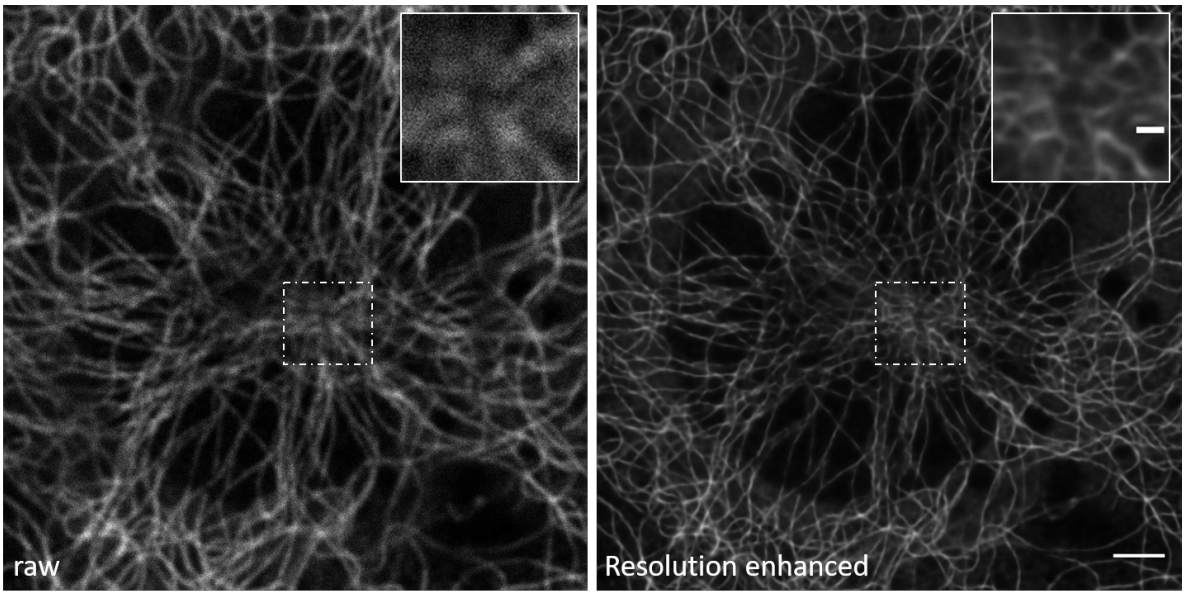


Figure A.9: **Network Prediction for Enhanced Resolution.** The images show the raw data of immunolabeled microtubules from mouse embryonic fibroblast cells (left) and 3D-RCAN predictions (right), demonstrating enhanced resolution. Scale bars: 5 m for main figure and 1 m for inset.